

VCS® PrimeSim™ AMS User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	19
Related Products, Publications, and Trademarks	19
Conventions	20
Customer Support	21
Statement on Inclusivity and Diversity	21
1. Getting Started With Mixed-Signal Simulation	22
Overview of the Mixed-Signal Simulation Flows	22
Verilog-SPICE Flow	23
VHDL/Verilog-SPICE Flow	23
Verilog-AMS-SPICE Flow	23
Analog and Digital Domains	24
Introducing the VCS PrimeSim AMS Tool	24
Running Cosimulation	24
Running Cosimulation From the Command Line	25
Concurrent Simulations	26
Change XA Setup at Runtime	26
Running Cosimulation With Verdi	26
Mixed-Signal Setup Checklist	27
Netlist-Related Checks	28
Identical Module and Subcircuit Names	28
Case-Sensitivity	29
Connecting Power Supplies to SPICE Subcircuits With Verilog Top	29
Method #1: No Supply Pins	29
Method #2: SPICE and Verilog Supply Pins	30
Method #3: Mixed Supply Pin Definitions	31
Netlist Statements	32
Simulation Time	33
Port-Related Checks	33
Port Mapping	33
Duplicate Ports	36
Parameterized Bus Ports	36

Version Compatibility Between Analog and Digital Engines	37
Required UNIX Paths and Variable Settings	39
Using the 64-bit Binaries	40
Reassigning Warning/Error Messages	40
Full Hierarchical Path	40
Known Problems	41
2. Mixed-Signal Simulation Feature Highlights	42
Interface A/D and D/A Signal Conversions	42
Cases Where A/D and D/A Interface Elements are Not Inserted	44
Case #1	44
Case #2	45
Controlling Through-net Optimization	46
Signal Conversion From Verilog-to-SPICE and SPICE-to-Verilog	47
Converting Signal Values	47
Modeling Dynamic Supply in Mixed-Signal Simulation	49
Converting Signal Strength	50
Resistance Calculation for Digital-to-Analog Strength Conversion	51
Strength Calculation for Analog-to-Digital Conversion	53
Creating a Resistance Map File	53
Mapping Resistance Values Separately for A2D and D2A Conversion	54
Verilog-Top/SPICE-Top Flows	54
Donut Configurations	55
Multiple Cell Views in the Nettype Flow	58
Specifying a Voltage Reference for Interface Elements	58
Using ie_reference_voltage With Internal Supplies	59
Supply IEs Connected to ie_reference_voltage nodes	60
Interface Element Supply Precedence	60
Report Files	61
Using Library-Based Interface Element Options	61
Library-Based Interface Element Example	62
Synchronizing Analog and Digital Events	64
Verilog-A Model Instantiation	65
Parameter Passing Rules	65
3. Using Advanced Features in Mixed-Signal Simulation	67

Save and Restore Feature in VCS PrimeSim AMS	67
Performing Sweeps and Alters for Efficient Mixed-Signal Simulation	68
Reading and Driving Analog Signals from the Digital Domain	68
Changing the Analog Configuration in the Middle of a Simulation	68
ace reread	69
Changes Allowed in the Configuration File	70
Changes in the SPICE Files Allowed With the ace reread Command	70
Postlayout Simulation Through Back-Annotation	70
Using an SDF File	71
SDF Annotation at the Mixed-Signal Boundary	72
Meta-Encrypted SPICE Netlists in Mixed-Signal Design	72
Resolving Instance Name Conflicts for Testbench Reuse	72
Prerequisites	74
Using resolve_x_inst_prefix	74
Running Concurrent Mixed-Signal Simulations From the Same Compiled Code ...	74
Enabling Concurrent Mixed-Signal Simulations	75
Creating Separate Output Files During Concurrent Mixed-Signal Simulation ...	75
Changing the Configuration File or SPICE Netlist File for Concurrent Mixed-Signal Simulation	77
Controlling the Number of Time Points at the Analog-Digital Boundary	77
Running Monte Carlo Analysis in Mixed-Signal Simulation	79
Running Sequential Monte Carlo Analysis	80
Output Files and Directories for PrimeSim XA	80
Output Files and Directories for FineSim	81
Output Files and Directories for PrimeSim	83
UCLI Support	84
Limitations	85
Running Distributed Monte Carlo Analysis	85
Running AC Analysis During Transient Analysis	86
Running AC Analysis	86
Adding Netlist Commands to the Control File	87
Adding PrimeSim XA Commands at Runtime	88
Skipping Analog Simulation for Specific Time Windows	88
Running Transient Noise Analysis	92
 4. Saving and Restoring Simulation State	 93

Basic Save and Restore Usage	93
Running Multiple Simulations With Save and Restore	94
5. Cross Module Reference	97
Accessing, Forcing, and Releasing Analog Signals From the Digital Domain	97
Accessing SPICE Nodes Through the Verilog Language XMR	98
Accessing SPICE Nodes Through hdl_xmr, hdl_xmr_force, and hdl_xmr_release	99
\$hdl_xmr System Task and hdl_xmr() Procedure	100
\$hdl_xmr_force System Task and hdl_xmr_force() Procedure	101
\$hdl_xmr_release System Task and hdl_xmr_release() Procedure	102
Accessing SPICE Nodes Through Proprietary System Tasks and Functions	104
\$snps_force_volt()	105
\$snps_inject_current()	106
\$snps_release_volt()	106
\$snps_get_volt()	107
\$snps_get_port_current()	107
\$snps_get_areal()	108
\$snps_get_aint()	109
\$snps_get_dreal()	109
\$snps_get_dint()	110
snps_above()	111
snps_absdelta()	111
snps_cross()	112
Accessing SPICE Nodes Through UCLI XMRs	113
UCLI force and release Commands	113
Support of Verilog Force and Release Assignments on wreal Nets	115
Usage Example	115
Limitations	116
Using Bus Notation With an XMR Task	116
Using form_spice_bus for SPICE Ports Appearing in the Form of a Bus	117
6. Sweep/Alter in Mixed-Signal Simulation	118
Types of Sweeps	118
Parameter Sweeps	119
Data Sweeps	119

Temperature Sweeps	120
Alter Blocks	120
DC Sweeps	121
Combining Sweeps	122
Running Distributed Sweeps and Alters	122
Output Files and Directories	122
PrimeSim XA-Generated Files	123
FineSim-Generated Files	123
PrimeSim-Generated Files	123
VCS-Generated Output Files	124
Examples of a Parameter Sweep of Two Values	124
Placing Waveform Files in One Directory	125
Merged FSDB Files	126
Group Files	126
UCLI Support	127
Sweep and Alter With Save/Restore or ACE-Reread	128
Known Limitations	128
7. Mixed-Signal Simulation in the Verilog-SPICE Flow	129
Required Input Files for the Verilog-SPICE Flow	129
Creating a Mixed-Signal Simulation Control File	129
Using a Mixed-Signal Initialization File	131
Compiling the Design for a Verilog-SPICE Simulation	132
Recompiling the Design	132
Automatic Verilog Dummy Module Generation	133
Running a Verilog-SPICE Simulation	133
8. Mixed-Signal Simulation in the VHDL/Verilog-SPICE Flow	134
Required Input Files for the VHDL/Verilog-SPICE Flow	135
Creating a Mixed-signal Simulation Control File for VHDL/Verilog-SPICE	135
Selecting Multiple Views	136
View Selection for Cells Under a VHDL Parent	136
Using a VHDL Setup File	137
Using a Verilog Wrapper	138
VHDL and Verilog Descriptions	139

Transistor-Level Descriptions	140
Using the VHDL/Verilog-SPICE Autowrapper Utility	140
Using the Autowrapper Utility	142
Running a VHDL/Verilog-SPICE Simulation	145
Known Limitations	146
9. Mixed-Signal Simulation in the Verilog-AMS-SPICE Flow	147
Required Input Files for the Verilog-AMS-SPICE Flow	148
Preparing a Mixed-Signal Simulation in Verilog-AMS-SPICE	148
Mixed-Signal Simulation Control File	148
Selecting Multiple Views	149
Files Containing Connect Rule and Connect Module Definitions	150
Understanding Analog and Digital Blocks in Verilog-AMS	150
Nets and Disciplines	151
Converting Signals With Interface A/D and D/A Connect Modules	153
Identifying the Correct Connect Module	154
Understanding Connect Rules	154
Files Containing Connect Rule and Connect Module Definitions	156
Compiling the Design for a Verilog-AMS-SPICE Simulation	157
-ams	157
-ams_discipline logic	157
-ams_dresolution	158
-ams_iereport	158
Verilog Netlist Files	158
Recompiling the Design	159
Running a Verilog-AMS-SPICE Simulation	159
Resolving Keyword Conflicts between SystemVerilog and Verilog-AMS	159
Support for Wreal Nets in Verilog-AMS Flow	160
Usage Example	160
Unsupported Features in Verilog-AMS-SPICE	162
Support for SystemC Designs in Verilog-AMS	163
Use Model	163
Usage Example	163
10. Output Files	169

Contents

Capturing Analog and Digital Signals in Output Files	169
Generating an Analog Output File	169
Generating a Digital Output File	171
Generating Merged FSDB Files	172
Generating Merged FSDB	172
Unified Dump of Voltage and Current Signals	173
Setting the Scope of Dumping Current Signals	174
Writing Out Both Sides of an Interface Element	175

11. Report Files	177
connectmodule.rpt	178
hierarchy.rpt	179
interface_activity.rpt	179
interface_connectivity.rpt	180
interface_element.rpt	181
interface_element_temporal.rpt	185
interface_tracing.rpt	185
mview.rpt	186
port.rpt	186
runtime_interface_element.rpt	188
through_net.rpt	189
use_cell_view.rpt	190

12. Interactive Debug	192
Invoking the Interactive Mode With the UCLI Debugging Feature With Verilog-SPICE or VHDL/Verilog-SPICE	192
UCLI Ace Analog Interactive Commands	193
Pausing and Resuming Simulations	193
Debugging Current Consumption	194
Viewing and Modifying Interface Element Options at Runtime	195
Viewing Interface Element Details at Runtime	195
ace show_ie	195
Modifying Interface Element Options at Runtime	199

A. Mixed-Signal Control Commands	201
a2d	204
ams_cdef_inst	211
ams_cdef_net	212
ams_cm	213
ams_set_discipline	216
ams_supply	218
bus_format	219
choose	220
d2a	221
disable_ie	226
downgrade_to_warn	227
duplicate_net_inst_name	229
dynamic_supply_filter	230
e2n for current_r Nettype	234
e2n for i_wire Nettype	236
e2n for th_wire Nettype	238
e2n for voltage_r Nettype	240
e2n for v_wire_avg, v_wire_sum, and v_wire_one Nettypes	242
e2r	244
e2u	246
form_spice_bus	249
gen_spice_wrapper	251
ie_activity_rpt	254
ie_connect_rpt	255
ie_reference_voltage	256
ie_tracing_rpt	257
insert_cell	259
map_by_node	260
n2e for current_r Nettype	261
n2e for i_wire Nettype	263
n2e for th_wire Nettype	265

Contents

n2e for voltage_r Nettype	266
n2e for v_wire_avg, v_wire_sum, and v_wire_one Nettypes	269
netlist_commands_begin	271
netlist_commands_end	272
optimize_shadowfile	272
param_pass	273
port_connect	273
port_dir	275
print_ie_res	276
r2e	280
remove_d2a	281
report_option	282
resolve_x_inst_prefix	290
rmap_file	290
rt_a2d	293
rt_d2a	294
rt_e2n	295
rt_e2r	296
rt_n2e	297
rt_r2e	298
shadow_file	298
shadow_file_dir	299
skip_xmr_name_check	300
spice_top	300
transient_analysis	301
u2e	303
udn_bidir	306
udn_e2n	307
udn_n2e	309
upf_port_connect	310
upgrade_to_error	312
use_spice	313

use_verilog	319
use_veriloga	322
use_vhdl	326
B. Using Hierarchical Aliases for Interface Elements in Mixed-Signal Commands	330
Limitation	331
C. Mixed-Signal Simulation for Three-Dimensional Integrated Circuits (3DIC) ..	333
Introduction to the 3DIC Mixed-Signal Flow	333
How 3DIC Works in the Standalone PrimeSim XA Tool	333
Basic Mixed-Signal 3DIC Implementation	334
Enhanced Mixed-Signal 3DIC Implementation	334
Specifying a Verilog-Top With SPICE Leaf	334
Specifying a SPICE-Top With Verilog or VHDL Leaf	335
Cell-Based Mixed-Signal Commands Affected by 3DIC Scope	336
Support for Verilog-A	336
Wildcard Support	337
Standalone 3DIC Global Supplies	337
Direct Supply Connections Through the Mixed-signal Interface	337
D. Unified Power Format (UPF) in Mixed-Signal Simulation	339
Overview of UPF in Mixed-Signal Simulation	339
Preparing a Mixed-Signal Simulation With UPF	340
Running a Mixed-Signal Simulation With UPF	340
SPICE/HDL Boundary	341
SPICE/HDL Boundary for Signals	341
SPICE/HDL Boundary for Supplies	342
Example Design	344
Tunneling SPICE Supply Ports Through the UPF Supply Network	346
Tunneling Use Model	346
Tunneling Limitations	348
UPF Power-Aware Interface Elements	348
Boundary SPICE Port With set_port_attributes Specified in the UPF	349
Boundary SPICE Port With set_related_supply_net Specified in the UPF	349

SPICE Cell With Liberty Information	349
SPICE Cell Without set_port_attributes, set_related_supply_net, and Liberty Information	350
Examples	350
Example 1	350
Example 2	352
Limitations	354
E. Support for Nettypes in Mixed-Signal Simulation	355
Overview of Nettypes in the VCS PrimeSim AMS Tool	355
Predefined Nettypes	356
current_r Nettype	357
i_wire Nettype	357
e2n Interface Element for i_wire and current_r	357
n2e Interface Element for i_wire and current_r	359
voltage_r Nettype	359
n2e Interface Element for v_wire_avg, v_wire_sum, v_wire_one, and voltage_r	360
e2n Interface Element for v_wire_avg, v_wire_sum, v_wire_one, and voltage_r	360
th_wire Nettype	361
n2e Interface Element for th_wire	361
e2n Interface Element for th_wire	362
v_wire_avg Nettype	362
v_wire_one Nettype	363
n2e Interface Element for v_wire_avg, v_wire_sum, and v_wire_one	363
e2n Interface Element for v_wire_avg, v_wire_sum, and v_wire_one	364
v_wire_sum Nettype	364
Use Model for Predefined Nettypes	365
Verilog File - test.v	365
SPICE File - test.spi	366
Mixed-Signal Simulation Control File - vcsAD.init	366
PrimeSim XA Control File - xa.cfg	366
vcs Run Script - run	366
VCS ucli Control File - xaVcs.ucli	367
User-Defined Nettypes	367
User-Define Nettypes With Predefined Interface Elements	368
01_test.v	369
01_test.spi	370

01_test.init	370
Command Line	370
User-Defined Nettypes With Custom Interface Elements	371
Use Model	371
Example	374
02_test.v	374
02_test.spi	376
vt.va	376
Command Line	377
Limitations	377
Arrays of SystemVerilog Nettypes	377
Connecting Bits	378
Rules and Limitations	379
F. Verilog/VHDL/PrimeSim XA VPI Mixed-Signal Simulation	380
Running a Verilog/VHDL/PrimeSim XA VPI Mixed-Signal Simulation	381
Setting Up System Environment Variables for Mixed-Signal Simulation	382
Mixed-Signal Simulation With Verilog as the Top Instance	382
High-Level Mixed-Signal Simulation Instructions	382
Detailed Mixed-Signal Simulation Instructions	383
Instance-Based Instantiation With Verilog Configuration	387
Mixed-Signal Simulation With VHDL as the Top Instance	389
Mixed-Signal Simulation With SPICE as the Top Instance	394
Donut Partitioning With Verilog as the Top Instance (V-S-V)	397
Donut Partitioning With SPICE as the Top Instance (S-V-S)	397
Using SPICE-on-Top Partitioning	397
First Run Example	398
Second Run Example	398
Verilog and SPICE Files	399
Save-Restart in Mixed-Signal Simulation	400
Configuration File Commands	400
analog_cell	402
define_print_variable	402
define_strength	403
digital_cell	404
digital_cell_inst	404
dump_interface	404

Contents

dump_port_prop	405
dump_setting	405
map_subckt_name	405
map_unfound_port	406
set_args	406
set_intr_mode	406
set_port_prop	407
verilog_file	411
Automatic Voltage Level Detection	412
Voltage Setting Rules	412
Rule 1	412
Rule 2	412
Mixed-Signal Simulation Interactive Mode	412
List Interface Nodes	413
csl	413
Print Global Interface History in Time	414
csh	414
Print Interface Node History	415
csnh, csinh	415
Set the Number of Entries Printed by csnh and csinh	415
csnph	415
Set Watchpoint to Interface Node	416
csnw, csinw	416
Delete Watchpoint	417
csdnw, csdinw	417
Verilog System Tasks for Mixed-Signal Simulation	417
Mixed-Signal Simulation Setup Guidelines	419
Map Correct Port Voltages	419
Define Clear Port Direction	419
Set Input Ports As Voltage Sources If Possible	419
Define SPICE Netlist Bus Notation	419
Handle Bidirectional Ports	420
Partitioning Guidelines	420
Partition Boundary With Clear Digital Behavior	420
Avoid Partitioning at Timing Sensitive Signals	420
Avoid Reach-in Signals in Analog Partitions	421
Avoid Partitioning at Bidirectional Signals Involved Strength Fighting and Pass Switches	421

Avoid Fine Grain Partitioning	421
Strength Table Setup Guidelines	421
Mixed-Signal Simulation With ModelSim	423
ModelSim/PrimeSim XA Integration	424
Running ModelSim/PrimeSim XA Mixed-Signal Simulation With Standalone ModelSim	424
Running ModelSim/PrimeSim XA Mixed-Signal Simulation Under the ADMS Environment	425
PrimeSim XA Features Not Supported by Mixed-Signal Simulation	425
References	426
G. FineSim VPI Cosimulation	427
Mixed-Mode Simulation	427
Verilog Cosimulation	427
Running Verilog Cosimulation	428
Verilog Simulator Commands	429
Examples	429
Verilog Offset Time in the FineSim Cosimulation Flow	435
FineSim Pro Tasks	435
\$ finesim_config	436
\$ finesim_input	437
\$ finesim_output	437
\$ finesim_inout	438
\$ finesim_module	438
\$ finesim_instance	439
Configuration Commands	440
.RESISTANCE	440
.A2D	442
.D2A	443
.SCOPE	444
.INPUT	444
.OUTPUT	445
.INOUT	446
.OPTION	446
.FINESIM	447
finesim_a2d / finesim_d2a Parameters	448
Automatic Verilog Instance Generation	448

Contents

-genv	449
finesim_bus_format	449
finesim_port_map_by_name	449
finesim_verilog_file	450
finesim_verilog_instance	450
finesim_verilog_module	450
finesim_verilog_module_file	451
finesim_verilog_subckt_file	451
Circuit Example: (test.sp)	451
Parallel Cosimulation	452
Common Cosimulation Problems	452
Error while reading shared library symbols, cannot find new threads: generic error	453
ERROR: VPI NOFORCB: vpi_put_value() cannot force a bit of an unexpanded vector net	453
H. PrimeSim VPI Cosimulation	454
Mixed-Mode Simulation	454
Verilog Cosimulation	454
Running Verilog Cosimulation	455
Verilog Simulator Commands	456
Examples	456
Verilog Offset Time in the PrimeSim Cosimulation Flow	462
PrimeSim Pro Tasks	462
\$primesim_config	463
\$primesim_input	464
\$primesim_output	464
\$primesim_inout	465
\$primesim_module	465
\$primesim_instance	466
Configuration Commands	467
.RESISTANCE	467
.A2D	469
.D2A	470
.SCOPE	471
.INPUT	471
.OUTPUT	472

Contents

.INOUT	472
.OPTION	473
.PRIMESIM	474
primesim_a2d / primesim_d2a Parameters	475
Automatic Verilog Instance Generation	475
-genv	476
primesim_bus_format	476
primesim_port_map_by_name	476
primesim_verilog_file	477
primesim_verilog_instance	477
primesim_verilog_module	477
primesim_verilog_module_file	478
primesim_verilog_subckt_file	478
Circuit Example: (test.sp)	478
Parallel Cosimulation	479
Common Cosimulation Problems	479
Error while reading shared library symbols, cannot find new threads: generic error	479
ERROR: VPI NOFORCB: vpi_put_value() cannot force a bit of an unexpanded vector net	480
I. Reserved Keywords	481
Reserved Keywords for Verilog-SPICE and VHDL/Verilog-SPICE	481
Reserved Keywords for Verilog-AMS-SPICE	484
Glossary	486

About the VCS PrimeSim AMS User Guide

The *VCS PrimeSim AMS User Guide* describes how to set up and run mixed-signal simulations using the VCS PrimeSim AMS tool with one of the following analog simulation engines:

- PrimeSim™ XA (formerly CustomSim™)
- FineSim® (FineSim® Pro and FineSim® SPICE)
- PrimeSim™ (PrimeSim™ Pro and PrimeSim™ SPICE)

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new VCS PrimeSim AMS features, enhancements, and changes is available in *PrimeSim XA: What's New*.

Related Products, Publications, and Trademarks

For additional information about mixed-signal simulation, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

- PrimeSim™ XA
- FineSim® (FineSim® Pro and FineSim® SPICE)
- PrimeSim™ (PrimeSim™ Pro and PrimeSim™ SPICE)
- PrimeSim™ Continuum

- VCS®
- PrimeSim™ Custom Fault (formerly TestMAX™ CustomFault™)
- Custom Compiler™ and the Simulation and Analysis Environment (SAE)
- PrimeWave™ Design Environment

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Getting Started With Mixed-Signal Simulation

This chapter provides an overview of the mixed-signal simulation flows.

This chapter describes:

- [Overview of the Mixed-Signal Simulation Flows](#)
- [Introducing the VCS PrimeSim AMS Tool](#)
- [Running Cosimulation](#)
- [Mixed-Signal Setup Checklist](#)
- [Netlist-Related Checks](#)
- [Port-Related Checks](#)
- [Version Compatibility Between Analog and Digital Engines](#)
- [Reassigning Warning/Error Messages](#)
- [Full Hierarchical Path](#)
- [Known Problems](#)

Overview of the Mixed-Signal Simulation Flows

A mixed-signal simulation enables simulating a design partly modeled in analog and partly modeled in digital.

Mixed-signal simulation is possible through different solutions, using the VCS PrimeSim AMS tool with one of the following analog simulation engines:

- PrimeSim XA
- FineSim
- PrimeSim

These solutions support the following flows:

- [Verilog-SPICE Flow](#)
- [VHDL/Verilog-SPICE Flow](#)
- [Verilog-AMS-SPICE Flow](#)

You can use any of these flows for simulating a mixed-signal design, depending on the description language used to model the netlists (SPICE, VHDL, or Verilog):

Note:

Throughout this manual, any reference to Verilog implies SystemVerilog as well. For example, wherever Verilog is supported in the mixed-signal flows, SystemVerilog is supported.

Before starting a mixed-signal simulation, you should verify the SPICE subcircuits and Verilog modules individually to ensure that they are error-free.

You must only install and use the VCS binaries for mixed-signal simulation, regardless of the flow or the VCS features (such as VHDL support) used in the simulation.

Verilog-SPICE Flow

The Verilog-SPICE flow is required when the analog parts of the design are modeled in one of the SPICE formats supported by the analog engine or behavioral analog (Verilog-A), and the digital parts are modeled in Verilog.

This flow is supported using the VCS PrimeSim AMS tool with the following analog simulation engines: PrimeSim XA, FineSim, and PrimeSim.

VHDL/Verilog-SPICE Flow

The VHDL/Verilog-SPICE flow is required when some or all of the digital portion of the design is modeled in VHDL. In this flow, the VCS tool must be used as the digital engine, because this tool supports VHDL as well as Verilog. This flow is almost identical to the Verilog-SPICE flow, but includes support for VHDL blocks in the digital netlist.

This flow is supported using the VCS PrimeSim AMS tool with the following analog simulation engines: PrimeSim XA, FineSim, and PrimeSim.

Verilog-AMS-SPICE Flow

The Verilog-AMS-SPICE flow is required when some or all of the design is modeled in the Verilog-AMS language. This flow also supports blocks or subcircuits that are modeled in SPICE or conventional digital Verilog.

This flow is supported using the VCS PrimeSim AMS tool with the following analog simulation engines: PrimeSim XA, FineSim, and PrimeSim.

Note:

VHDL is not currently supported in this flow.

Analog and Digital Domains

Depending on the computational methods used to calculate the values of a signal or variable, the signal or variable can belong to the analog domain (also known as *continuous domain*) or digital domain (also known as *discrete domain*).

Voltage and current values are calculated in the analog domain, while the contents of registers and states of gate primitives are calculated in the digital domain. Integer and real variables can belong to either the analog or digital domain, depending on how their values are assigned.

If a value is assigned within an analog block, the domain is considered to be *analog*. If a value is assigned in a digital block, the domain is considered to be *digital*. The assignment to real and integer variables can occur only in one domain.

Values calculated in the digital domain change values in a discrete and non-continuous manner. As a result, the derivative—with respect to the time of a digital value—is always zero (0). Values calculated in the analog domain, however, vary continuously and their derivative—with respect to the time—varies (as the value varies).

Introducing the VCS PrimeSim AMS Tool

The VCS PrimeSim AMS simulates mixed-signal designs by employing two distinct tools. VCS simulates digital blocks, while PrimeSim simulates analog blocks. These tools interact to generate final outputs that can be either in split form or merged depending on the options used.

Furthermore, the VCS PrimeSim AMS flow simplifies switching and interaction between domains. Advanced features accelerate analysis and reduce turnaround time. This flow offers a comprehensive, automated, unified environment.

Running Cosimulation

This section explains how to execute cosimulations.

- [Running Cosimulation From the Command Line](#)
- [Concurrent Simulations](#)

- [Change XA Setup at Runtime](#)
- [Running Cosimulation With Verdi](#)

Running Cosimulation From the Command Line

This topic describes running mixed-signal cosimulation using VCS PrimeSim AMS through the command line. The process involves three steps:

1. Analyzing the Design:

VCS provides you with the `vhdlan` and `vlogan` executables to analyze your VHDL and Verilog code and stores the intermediate files in the design or a work library. For example,

```
vlogan my_top.v -full64 -kdb -l vlogan.log
```

2. Elaborating the Design:

VCS provides you with the `vcs` executable to compile and elaborate the design. This executable compiles/elaborates your design using the intermediate files and combines them to generate a binary simulation executable called `simv`. For example,

```
vcs -ad=vcsAD.init -debug_access+all -l vcs.log
```

3. Simulating the Design:

Simulate your design by executing the binary simulation executable, `simv`. For example,

```
simv -ucli -i dump.do -l simv.log
```

Understanding vcsAD.init and dump.do (UCLI Input File)

This `vcsAD.init` file defines settings for the waveform output. The first line in the below sample script specifies the format as merged FSDB (combining analog and digital waveforms).

The second line configures the XA tool, specifying its configuration file location and output directory.

Sample Script

```
set_waveform_option -format fsdb -file merge  
choose xa -hspice top.spi -c xa.cfg -o xaOut
```

Following is the example of the UCLI input file `dump.do` that contains following commands to control the simulation:

```
dump -file OUTPUT/out.fsdb -type fsdb  
dump -add / -depth 10 -fid fsdb0
```

```
run 10 us
exit
```

Concurrent Simulations

VCS PrimeSim AMS supports running multiple simulations concurrently, either within the same directory or separate directories. To prevent overwriting results and simulation failures when using the same directory, specify different output directory names for each simulation.

Alternatively, copy the `simv` executable to different directories and execute concurrent simulations independently in each location. For detailed instructions, refer to [Running Concurrent Mixed-Signal Simulations From the Same Compiled Code](#) chapter.

Change XA Setup at Runtime

To modify the analog setup while a simulation is running, pause the simulation at the desired point, reload the updated analog configuration file, and then resume the simulation with the new settings.

Sample Script

```
%simv -ucli
ucli% run 10u
ucli% ace reread -c xa_at_10us.cfg
ucli% run 10u
ucli% ace reread -c xa_at_20us.cfg
ucli% run 1ms
ucli % quit
```

The configuration files `xa_at_10us.cfg` and `xa_at_20us.cfg` files can contain different XA options for analog blocks.

For more details on adding PrimeSim XA commands during runtime, refer to [Adding PrimeSim XA Commands at Runtime](#) topic.

Running Cosimulation With Verdi

Running cosimulation with Verdi includes two step flow namely [Interactive Simulation](#) and [Post Processing](#).

Interactive Simulation

1. Run the simulation through Verdi: Use the `simv -verdi` command to start the simulation.
2. Control simulation from graphical user interface (GUI) or Unified Command-Line Interface (UCLI): Use the Verdi GUI or UCLI mode to perform actions such as adding waveforms, stop or continuing the simulation, and forcing signal values.

Note:

`-kdb` and `-debug_access+all` options are mandatory for interactive simulation.

Example 1 Interactive simulation

- VCS Compilation with `-kdb` option

```
%vlogan -kdb / vhdlan -kdb ...
```

- VCS elaboration

```
%vcs -kdb -debug_access+all ..
```

- Simulation (either in GUI or UCLI mode)

```
%simv -gui -do vcs.do
```

or

```
%simv -ucli -do vcs.do
```

Post Processing

1. Run simulation in batch mode: `verdi -ssf dump.fsdb`.
2. Dump FSDB with predefined set of signals.

Example 2 Post Processing

Open Verdi for debugging

```
verdi -ssf dump.fsdb &
```

Refer to Verdi documentation *Verification Continuum™ Verdi® User Guide and Tutorial* for detailed information.

Mixed-Signal Setup Checklist

Before using a mixed-signal flow, there are several tasks that must be completed. Verilog netlist files or SPICE netlist files might require modification for a successful netlist compilation. Check the following guidelines before simulating a design.

The following is a list of issues to investigate before beginning a mixed-signal simulation.

- [Netlist-Related Checks](#)
 - Identical module and subcircuit names
 - Case-sensitivity
 - Power supplies
 - Netlist statements
 - Simulation time
- [Port-Related Checks](#)
 - Port mapping
 - Duplicate ports
 - Parameterized bus ports

These issues are described in detail in the following sections. When running a mixed-signal simulation, you must create a mixed-signal simulation control file the contains commands that specify which view is used for blocks in the hierarchy, how ports are connected, and so on. See [Creating a Mixed-Signal Simulation Control File](#) for more information on creating the file.

Netlist-Related Checks

This topic describes netlist-related issues that should be investigated before beginning a mixed-signal simulation.

- [Identical Module and Subcircuit Names](#)
- [Case-Sensitivity](#)
- [Connecting Power Supplies to SPICE Subcircuits With Verilog Top](#)
- [Netlist Statements](#)
- [Simulation Time](#)

Identical Module and Subcircuit Names

For a multi-view cell, the Verilog module name and the SPICE subcircuit names must be identical. The number of ports and the names of the ports must also match.

Case-Sensitivity

For a multi-view cell, the case used for the Verilog module name and the SPICE subcircuit name must be identical. The cases used for port names must also be identical between the SPICE and Verilog views. Also, be aware that every name in PrimeSim HSPICE netlists is treated as lowercase (by default). Because Verilog is case-sensitive, and the VCS tool processes everything as-is (by default), you must be aware of any possible case discrepancies between the two views.

Connecting Power Supplies to SPICE Subcircuits With Verilog Top

Power supplies must correctly connect to SPICE subcircuits for the simulation to work properly. If the SPICE subcircuit is instantiated within a Verilog netlist or testbench, the supply nets can connect by using one of the following methods:

- [Method #1: No Supply Pins](#)
- [Method #2: SPICE and Verilog Supply Pins](#)
- [Method #3: Mixed Supply Pin Definitions](#)

Method #1: No Supply Pins

If the SPICE subcircuit definitions and the Verilog instances do not contain supply pins, you must connect the power supplies to the SPICE subcircuits by using `.global` statements in the SPICE netlist. The following example uses this method to provide supply net connectivity to designs where the VDD and VSS pins do not appear in the subcircuit port list. [Example 3](#) shows the SPICE netlist and [Example 4](#) shows the corresponding Verilog netlist.

Example 3 SPICE Netlist With .global Statement for Power and Ground

```
* Power supply nets are propagated
* by using a .global statement

v_vdd vdd 0 1.8
v_vss vss 0 0
.global vdd vss

.subckt inv in out
m1 out in vdd vdd p_tran l=0.25u w=10.0u
m2 out in vss vss n_tran l=0.25u w=10.0u
.ends
```

The following example shows the Verilog netlist that instantiates the SPICE inverter subcircuit from [Example 3](#). The VDD and VSS power supply nets are defined as global nets in the SPICE netlist and are not included in the port list for the inverter.

Example 4 Verilog *inv* Instance Without Power and Ground Pins

```
module verilog_top(...);  
...  
// Power pins do not appear in the port list  
inv i1 (.in(d_in), .out(d_out));  
...  
endmodule
```

Method #2: SPICE and Verilog Supply Pins

If both the subcircuit definitions and their instantiations in the Verilog netlist contain supply pins, there are two options for connecting the supplies to instances in the SPICE netlist:

- Use the `d2a` command with the `-powernet` option

A Verilog-driven power net connects to a SPICE subcircuit through a `d2a` element at the subcircuit boundary. By default, `d2a` interface elements include a series resistance map resistor, which is undesirable for power nets. To remove the resistor, specify the `-powernet` option with the mixed-signal `d2a` command for both VDD and VSS.

For example:

```
// vcsAD.init  
d2a -powernet -hiv 1.2 -lov 0 -node top.vdd;  
d2a -powernet -hiv 1.2 -lov 0 -node top.vss;
```

The hierarchical paths to the power nets can be viewed in the `simv.msv/interface_element.rpt` report file.

Note:

If the Verilog nets that drive the SPICE supplies are defined as Verilog predefined `supply0` or `supply1` nets, the tool automatically treats those nets as if the `d2a -powernet` command was applied to them. In this case, no `rmap` resistor is placed at the `d2a` interface for those nets and they are treated as ideal voltage sources on the analog side.

- Create a new two-port SPICE subcircuit

In this methodology, you create a new two-port SPICE subcircuit to provide power for the SPICE instances. By instantiating this new subcircuit under Verilog, the two ports are connected to the supply pins for other SPICE cells under Verilog.

Example 5 defines a two-port SPICE subcircuit named `spice_pwr_supply`. The SPICE inverter connects to power and ground by creating `vdd` and `vss` ports in the inverter subcircuit definition. **Example 6** shows the corresponding Verilog netlist that instantiates the power supply subcircuit from SPICE and shows an equivalent inverter instance that contains `vdd` and `vss` ports.

Example 5 Two-port SPICE Subcircuit for Generating VDD and VSS

```
* two_port.spi
* Power pins specified in subckt port list
.subckt inv in out vdd vss
m1 out in vdd vdd p_tran l=0.25u w=10.0u
m2 out in vss vss n_tran l=0.25u w=10.0u
.ends

* The spice_pwr_supply subcircuit supplies
* the vdd and vss power supply signals
.subckt spice_pwr_supply vdd vss
v_vdd vdd 0 1.8
v_vss vss 0 0
.ends
```

Example 6 Verilog Netlist for VDD and VSS Generated by SPICE

```
// verilog_top.v
module verilog_top (...);
...
// Power pins included in the inverter instance
inv i1 (.in(d_in), .out(d_out), .vdd(vdd_wire), .vss(vss_wire));
...
// spice_pwr_supply supplies VDD and VSS for the design
spice_pwr_supply s1 (.vdd(vdd_wire), .vss(vss_wire));
...
endmodule
```

Method #3: Mixed Supply Pin Definitions

This method applies when the following conditions are true:

- The SPICE subcircuit is instantiated under a Verilog or VHDL parent module
- The SPICE subcircuit *contains* power ports in the subcircuit definition
- The Verilog instance *does not contain* power pins

[Example 7](#) shows the Verilog netlist instantiation of an inverter. The SPICE definition of the inverter is shown in [Example 8](#), note that the port definitions do not match because the Verilog instance does not contain definitions for ports `vdd` and `vss` as defined in the SPICE subcircuit.

Example 7 Verilog Netlist With SPICE Inverter Instance (No Power Pins)

```
inv1 i1 (.y(y_wire), .a(a_wire));
```

Example 8 SPICE Subcircuit With Power Ports

```
* subckt definition contains power ports
.subckt inv1 y a vdd vss
```

```
...
.ends

* Ideal power supply definitions
v3 vdd! 0 DC=3.0
v2 vss! 0 DC=0.0
```

In this example, the power nets are not connected from the HDL to the SPICE subcircuit. To connect the power, use the mixed-signal `port_connect` command to connect those ports to any analog net that is connected to an ideal supply as shown in the following example:

Example 9 *port_connect Command Used to Connect Power Nets*

```
// vcsAD.init
port_connect -cell inv1 ( vdd => vdd!, vss => vss! );
```

In the following example, the `port_connect` statement connects power to two ideal supplies that are defined in subcircuit at the hierarchical path `top.pwrblk`:

Example 10 *port_connect Command Used to Connect to top.pwr_blk.vdd*

```
// vcsAD.init
port_connect -cell inv1 (vdd => top.pwrblk.vdd , vss => top.pwrblk.vss);
```

For more information, see the [port_connect](#) command description.

Netlist Statements

Any statements, keywords, or elements that are not supported by the analog simulator in a standalone analog simulation are also not supported in mixed-signal.

All other options and statements that are required for correct analog simulation setup, such as `.options scale=1e-6`, must also be included in the netlist for mixed-signal simulation. The following example shows a SPICE netlist file sample.

```
* test.spi
.lib 'models' TT
.inc 'cells.spi'
.options scale=1e-6
.temp 27
.global vdd gnd
.subckt test a b c d
x1 a b n1 vdd cell1
x2 c n1 d ref cell2
vref ref gnd 2.0
.ends
vvdd vdd 0 dc 3.3
vgnd gnd 0 dc 0
.end
```

Simulation Time

In mixed-signal simulation, the simulation time can be determined by either the analog domain or the digital domain. In the analog domain, the time is usually defined by the end time in the `.tran` statement. In the digital domain, time is usually determined by a `$finish` or `$stop` system task in the Verilog code.

In the digital domain, if the simulation end time is specified by a `$finish` system task in Verilog, the simulation exits when `$finish` is executed. If the simulation end time is specified by a `$stop` system task in Verilog, then the simulation stops and enters the Unified Command-Line Interface (UCLI) interactive mode.

In the analog domain, if the simulation end time is specified by the end time in the SPICE `.tran` statement, by default, the simulation exits at the specified time. You can use the [transient_analysis](#) command (currently only supported in the PrimeSim XA simulation engine) to change the default behavior and specify that the simulation enter UCLI interactive mode at the simulation end time as defined by the `.tran` statement.

In the case when different end times are specified in the analog and digital domains, the smaller of the two times determines the simulation end time.

Note:

For the PrimeSim XA, FineSim, and PrimeSim simulation engines to report the completed percentage of simulation time, there must be a `.tran` statement in the SPICE code. The analog engine bases its percentage calculation on the runtime given by the `.tran` statement.

Port-Related Checks

The section contains information about:

- [Port Mapping](#)
- [Duplicate Ports](#)
- [Parameterized Bus Ports](#)

Port Mapping

By default, mixed-signal simulation requires that a cell with both Verilog and SPICE views has the same number of ports and matching port names in both views. If the number of ports or the names of the ports differ between the two views, the compilation exits with an error message. To resolve the port inconsistency problem between different views, you must use the `port_map` option of the `use_spice`, `use_verilog` or `use_vhdl` commands. To handle a discrepancy in the number of ports (for example if one view has extra ports),

you can use the `port_connect` command. If there is no port mismatch between different views of a cell, ports from different views are mapped as described below.

For a SPICE parent instantiating a Verilog child, by default, ports are mapped by position. You can enforce *name-based port mapping* by creating a SPICE view for the Verilog cell. In name-based port mapping, the tool automatically maps ports by name for the child Verilog cell by comparing the port order between the SPICE and Verilog views and rearranging the Verilog port order to match the SPICE view.

For a Verilog parent instantiating a SPICE child, either name-based or position-based port mapping can be used. When using port mapping by name, the Verilog module port name begins with a period character (`.`), and must be identical to the SPICE port name. The cases for port names at the instantiation must be identical to the case of ports in the subcircuit definition. [Figure 1](#) demonstrates the Verilog syntax for name-based port mapping. [Figure 2](#) is an example of port mapping by name.

Figure 1 Verilog Syntax for a nor Gate Instance

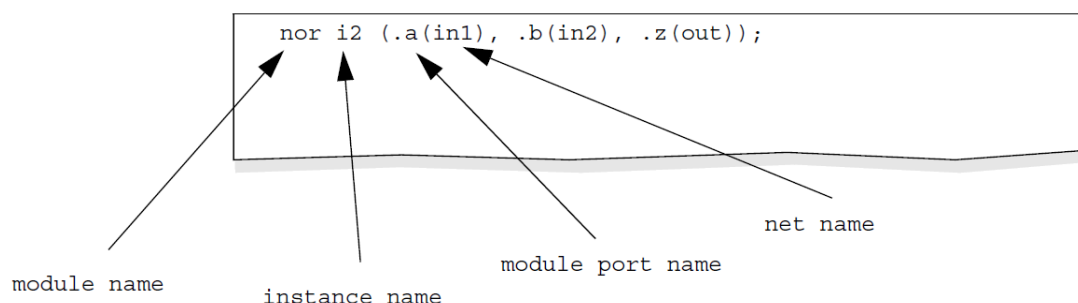
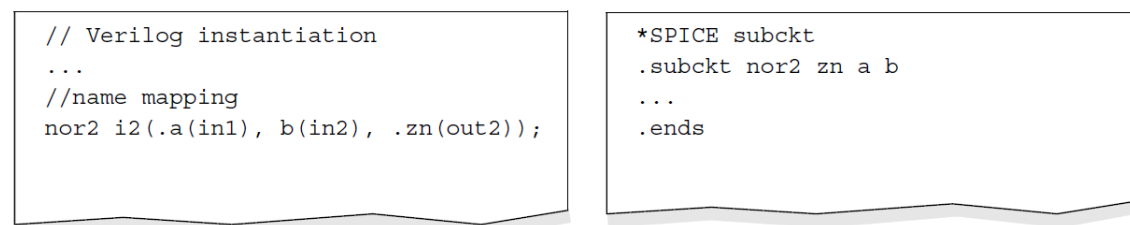


Figure 2 Port Mapping by Name



Port mapping at the Verilog instantiation can also be implemented by port position. In [Figure 3](#), the port `zn` of the SPICE subcircuit `nor1` is connected to the wire `out1` at the instantiation of that subcircuit in Verilog. And ports `a` and `b` of the SPICE subcircuit `nor1` are connected to Verilog wires `in1` and `in2`, respectively.

Figure 3 Port Mapping by Position

<pre>// Verilog instantiation ... //position mapping nor1 i1 (out1, in1, in2);</pre>	<pre>*SPICE subckt .subckt nor1 zn a bends</pre>
--	--

Port mapping between Verilog and SPICE can also be achieved for bus ports. By default, mixed-signal simulation requires that the bus members defined in SPICE must be contiguous in the subcircuit definition (that is, no other port defined between any two members of the same bus port), and the ports must be sequentially numbered, either ascending or descending. If these conditions are not met, the compilation exits with an error message. In addition, the ascending or descending order of the bus members in SPICE must be identical to the order defined by HDL, otherwise, the port connections are incorrect. If the SPICE block has both an HDL view and a SPICE view, the tool uses the HDL declaration to rearrange the bus orders in SPICE so that they match the HDL.

[Figure 4](#) shows an example of a SPICE subcircuit (addr) with bus ports and the instantiation of addr in a Verilog parent block. In the SPICE subcircuit, the port names a[3], a[2], a[1], and a[0] are contiguous, and the sequencing of the bus members meets the expectation of the parent Verilog (both are descending). As a result, compilation should proceed without error due to bus order issues and port name resolution in SPICE. In addition, the port connections for this bus are correct since both the bus port a in SPICE and the wire ai in Verilog are in a descending order.

By default, the tool assumes that the SPICE port names that connect to a mixed-net and contain closed bracket characters [] are members of a bus. If the bus notation in the SPICE subcircuit is a string different from [], use the mixed-signal `bus_format` command to override the default SPICE bus notation. For more information, see [bus_format](#).

Figure 4 Port Mapping for the Verilog Vector Port and SPICE Scalar Port

<pre>// Verilog instantiation ... addr i3(.a(ai[3:0]), .b(bi[3:0]), .cin(ci), .s(su[3:0]), .cout(co)); ...</pre>	<pre>*SPICE subckt .subckt addr a[3] a[2] a[1] a[0] +b[3] b[2] b[1] b[0] +cin s[3] s[2] s[1] s[0] +cout .ends</pre>
--	---

If the number of ports in the SPICE or HDL definition of the child block is more than the number of ports referenced in the instantiation by the parent, the extra ports of the child remain dangling. One usual example of this case is when the child block has power pins

but the instantiation in the parent does not refer to those pins. As a result, the tool does not connect the pins to any net in the parent. You can use the [port_connect](#) command to connect those unconnected ports to any net in the design.

If the number of ports in the child subcircuit or HDL definition is less than the number of ports referenced in the instantiation, the compilation fails. This can happen when VDD and VSS pins are referenced in the instantiation but are not found in the child port list. To resolve this problem, use the `-port_map` option of the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands to treat those extra ports in the instantiation as `snps_open` and ignore them.

Duplicate Ports

Duplicate port names in the SPICE subcircuit cannot be used, except for power supply nodes.

Parameterized Bus Ports

Parameterized bus ports are ports with variable width, where the width is determined by a user-defined parameter. Parameterized bus ports at the analog/digital boundary are supported for Verilog modules with some limitations, but are not supported for VHDL. A Verilog or VHDL cell is defined to be at the analog/digital boundary if its SPICE view is instantiated under a digital parent, or if the Verilog or VHDL cell itself is instantiated under a SPICE parent.

Parameterized ports for Verilog or VHDL cells anywhere in the design, other than at the analog/digital boundary, are supported and are not subject to the rules and limitations laid out in this manual.

The following example shows a parameterized bus in Verilog:

Example 11 Parametrized Bus Width

```
// top.v
`define BUS_WIDTH 8
...
module proc ( din, dout);
input  [ (`BUS_WIDTH-1) : 0 ] din;
output [ (`BUS_WIDTH-1) : 0 ] dout;
...
```

Here the bus width for ports `din` and `dout` is determined by the `BUS_WIDTH` parameter and is set to 8. If the bus widths for these ports in the SPICE definition of the cell are also 8, the compilation proceeds without error. However, if the bus width of the Verilog bus (defined by the `BUS_WIDTH` parameter) is different from the SPICE bus width, an error message is issued during compilation.

Version Compatibility Between Analog and Digital Engines

Mixed-signal simulation requires that the versions of the analog engine (the PrimeSim XA, FineSim, and PrimeSim simulation engines) and the VCS tool are compatible; otherwise, the compilation might fail. In addition, the operating system utilities (such as cc, gcc, and ld) used during the VCS compilation must meet the mixed-signal simulation version requirements.

All the version compatibility requirements for the PrimeSim XA, FineSim, and PrimeSim simulation engines and VCS tool, as well as the correct versions for compile utilities, are provided in the compatibility table below:

Table 1 Tools Version Compatibility Table

PrimeSim™ XA	VCS®	Verdi®	PrimeSim™ Pro and SPICE
V-2023.12-SP2	V-2023.12-SP2	V-2023.12-SP2	V-2023.12-SP2
V-2023.12-SP1	V-2023.12-SP1	V-2023.12-SP1	V-2023.12-SP1
V-2023.12	V-2023.12	V-2023.12	V-2023.12
U-2023.03-SP2	U-2023.03-SP2	U-2023.03-SP2	U-2023.03-SP2
U-2023.03-SP1	U-2023.03-SP1	U-2023.03-SP1	U-2023.03-SP1
U-2023.03	U-2023.03	U-2023.03	U-2023.03
T-2022.06-SP2	T-2022.06-SP2	T-2022.06-SP2	T-2022.06-SP2
T-2022.06-SP1	T-2022.06-SP1	T-2022.06-SP1	T-2022.06-SP1
T-2022.06	T-2022.06	T-2022.06	T-2022.06
S-2021.09-SP2	S-2021.09-SP2	S-2021.09-SP2	S-2021.09-SP2
S-2021.09-SP1	S-2021.09-SP1	S-2021.09-SP1	S-2021.09-SP1
S-2021.09	S-2021.09	S-2021.09	S-2021.09
R-2020.12-SP2	R-2020.12-SP2	R-2020.12-SP2	R-2020.12-SP2
R-2020.12-SP1	R-2020.12-SP1	R-2020.12-SP1	R-2020.12-SP1
CustomSim™	VCS®	Verdi®	FineSim®
R-2020.12	R-2020.12	R-2020.12	R-2020.12
Q-2020.03-SP4/-SP5	Q-2020.03-SP2	Q-2020.03-SP2	Q-2020.03-SP2

Table 1 *Tools Version Compatibility Table (Continued)*

Q-2020.03-SP2/-SP3	Q-2020.03-SP1	Q-2020.03-SP1	Q-2020.03-SP1
Q-2020.03/Q-2020.03-SP1	Q-2020.03	Q-2020.03	Q-2020.03
P-2019.06-SP4/-SP5	P-2019.06-SP2	P-2019.06-SP2	P-2019.06-SP2
P-2019.06-SP2/-SP3	P-2019.06-SP1	P-2019.06-SP1	P-2019.06-SP1
CustomSim™	VCS®	Verdi®	FineSim®
P-2019.06/P-2019.06-SP1	P-2019.06	P-2019.06	P-2019.06
O-2018.09-SP4	O-2018.09-SP2	O-2018.09-SP2	O-2018.09-SP2
O-2018.09-SP2/-SP3	O-2018.09-SP1	O-2018.09-SP1	O-2018.09-SP1
O-2018.09/-SP1	O-2018.09	O-2018.09	O-2018.09
N-2017.12-SP4/-SP5	N-2017.12-SP2	N-2017.12-SP2	N-2017.12-SP2
N-2017.12-SP2/-SP3	N-2017.12-SP1	N-2017.12-SP1	N-2017.12-SP1
N-2017.12/N-2017.12-SP1	N-2017.12	N-2017.12	N-2017.12
M-2017.03-SP4/SP5	M-2017.03-SP2	M-2017.03-SP2	M-2017.03-SP2
M-2017.03-SP2/-SP3	M-2017.03-SP1	M-2017.03-SP1	M-2017.03-SP1
M-2017.03/M-2017.03-SP1	M-2017.03	M-2017.03	M-2017.03
L-2016.06-SP4/SP5	L-2016.06-SP2	L-2016.06-SP2	L-2016.06-SP2
L-2016.06-SP2/-SP3	L-2016.06-SP1	L-2016.06-SP1	L-2016.06-SP1
L-2016.06/L-2016.06-SP1	L-2016.06	L-2016.06	L-2016.06
L-2016.03	K-2015.09-SP2	K-2015.09-SP2	L-2016.03

Note:

The Unix Platform supported must be compliant with a specific Synopsys platform foundation (for example, QSC U-Foundation and T-Foundation). You can find details on supported versions in the following link: <https://>

www.synopsys.com/support/licensing-installation-computeplatforms/computeplatforms/release-specific-support.html

Note:

The compatibility [Table 1](#) only lists major releases (example, 2020.12) and Service Pack releases (example, 2020.12-SP1 and 2020.12-SP2). The table does not list all the tool patches that stem from major or Service Pack releases. Those patches follow the same tool compatibility as the root major release or Service Pack that they branched from. For example the PrimeSim XA or PrimeSim Pro patch 2020.12-SP1-1xxx follows the same tool compatibility as their root release 2020.12-SP1.

Note:

PrimeSim XA: By default, the PrimeSim XA tool supports FSDB version 6.1, but the versions 6.0 and 5.9 are also supported through the `set_waveform_option` command. The merged FSDB version is set by Verdi. Refer to Verdi's documentation for details.

Note:

PrimeSim™ Pro and SPICE: By default, the PrimeSim Pro and SPICE tool supports FSDB version 6.1, but the versions 6.0 and 5.9 are also supported through the `primesim_fsdb_version` command. The merged FSDB version is set by Verdi. Refer to Verdi's documentation for details.

Required UNIX Paths and Variable Settings

To set the paths for the PrimeSim XA, FineSim, and PrimeSim simulation engines and VCS, do the following:

- For the PrimeSim XA simulation engine:

```
source XA_installation_directory/CSHRC_xa
```

- For the FineSim simulation engine:

```
setenv install_directory FineSim_install_directory  
source $install_directory/finesim.cshrc
```

- For the PrimeSim simulation engine:

```
setenv install_directory PrimeSim_install_directory  
source $install_directory/primesim.cshrc
```

- For VCS:

```
setenv VCS_HOME VCS_installation_directory  
set path = ($VCS_HOME/bin $path)
```

Using the 64-bit Binaries

Currently, mixed-signal supports Suse and Linux (AMD) 64-bit platforms.

Note:

Starting with the J-2014.09 release, the PrimeSim XA and FineSim simulation engines are only available in 64-bit mode. Running a mixed-signal simulation in 32-bit mode causes an error, so use the VCS switch `-full64` at all times when running a mixed-signal simulation:

```
% vcs -ad -full64 ...
```

This directs the VCS tool to use 64-bit binaries and also instructs the analog engine (the PrimeSim XA, FineSim, and PrimeSim simulation engines) to use 64-bit binaries as well (assuming that the 64-bit binaries for the analog engines are installed and the environment variables for the analog engines are set as described above).

Reassigning Warning/Error Messages

For some types of message codes, you can upgrade warning messages to error messages, or downgrade error messages to warning messages. For more information about reassigning messages, see [downgrade_to_warn](#) and [upgrade_to_error](#).

Full Hierarchical Path

The term “full hierarchical path” is used frequently in this document and refers to the complete hierarchy of the element starting from the top level, as shown in the following examples.

Example 1

The top-level instance name is a digital block, `top`, with a block, `i0`, underneath it.

The full hierarchical path of the input port is: `top.i0.in`

Example 2

The top-level instance name is a SPICE view with no user-defined name (that is, the `spice_top;` command is used in a mixed-signal control file) with a block, `i0`, underneath it.

In this case, there is no need to specify the top-level name because the tool already knows about the top-level SPICE hierarchy, so the full hierarchical path of the input port is: `i0.in`

Example 3

The top-level instance name is a SPICE view with a user-defined name (that is, the `spice_top -name my_top;` command is used in a mixed-signal control file to set the top-level SPICE as `my_top`) with a block, `i0`, underneath it.

The full hierarchical path of the input port is: `my_top.i0.in`

Known Problems

The following problems currently exist:

- When a programming language interface (PLI) is used along with the g++ compiler on the Linux platform, an error messages for `multiple symbol definition...` might be generated. To suppress this error, set the `NO_STDPP` UNIX environment variable as follows: `setenv NO_STDPP 1`
- The following cases have limited or no support and can lead to incorrect results:
 - Registered nets that are connected to mixed-nets are not supported.
 - Bidirectional resistive pass switches (`rtran`, `rtranif1`, and `rtranif0`) that are connected to mixed-nets are not supported.
 - Bidirectional pass switches (`tran`, `tranif1`, and `tranif0`) that are connected to mixed-nets have limited support, and in certain cases the results might be incorrect. If the pass switch is connected to analog on one side, and to digital on the other side, and only one of the two sides drives while the other side is in HiZ state, the results are correct.

But when both sides of the pass switch drive a net, the results might be incorrect. Also, when both sides of the pass switch are connected to analog circuits, the results are incorrect as the two SPICE nets are not shorted.
- Jumper ports that are connected to the mixed nets are *not* supported and produce incorrect results.

The following example shows a sample jumper port.

```
module jumper (a, a);  
  inout a;  
  ...  
endmodule
```

- When you replace existing SPICE subcircuits with Verilog modules, the analog DC initialization in mixed-signal simulation may not be identical to the standalone analog simulation. For DC initialization condition-sensitive circuits, it may be necessary to apply specific DC initialization condition values at certain nodes to get expected simulation behaviors.

2

Mixed-Signal Simulation Feature Highlights

This chapter highlights the mixed-signal simulation features.

The following features are described in this chapter:

- [Interface A/D and D/A Signal Conversions](#)
- [Signal Conversion From Verilog-to-SPICE and SPICE-to-Verilog](#)
- [Verilog-Top/SPICE-Top Flows](#)
- [Donut Configurations](#)
- [Specifying a Voltage Reference for Interface Elements](#)
- [Using Library-Based Interface Element Options](#)
- [Synchronizing Analog and Digital Events](#)
- [Verilog-A Model Instantiation](#)

Interface A/D and D/A Signal Conversions

Mixed nets are the signals that enable the connection between analog and digital blocks in a mixed-signal simulation. These signals are the *interface* signals located at the analog-digital boundary.

Depending on whether the direction of the signal transfer is from digital-to-analog (D/A) or from analog-to-digital (A/D), mixed-signal simulation must perform D/A or A/D conversions to convert logic values to analog voltages or vice versa.

- In the Verilog-AMS-SPICE flow, the A/D and D/A conversions are done via *Connect Modules* (see [Converting Signals With Interface A/D and D/A Connect Modules](#)).
- In the Verilog-SPICE and VHDL/Verilog-SPICE flows, A/D and D/A interface elements are inserted automatically by the tool.

Note:

Verilog-SPICE allows SystemVerilog nets of type “real” to connect to SPICE ports and SPICE nets to connect to SystemVerilog “real” ports. In these cases, e2r (electrical to real) or r2e (real to electrical) interface elements are inserted to convert analog voltages to digital real values or vice versa. To allow such connectivity, use the `-realport` option with the VCS tool.

Interface elements are inserted automatically for mixed-nets based on the following principles:

- Mixed nets can be unidirectional, in which case they are either a2d or d2a.
- Mixed nets can be bidirectional, or inout, which implies that at different simulation times they can be either of an a2d or d2a nature.
- The direction of the mixed nets is determined from Verilog port directions:
 - In a Verilog child under a SPICE parent configuration, the port direction of the Verilog child determines the direction of the mixed nets.
 - In a SPICE child under a Verilog parent configuration, and if the SPICE child has a Verilog view as well, the port directions in that Verilog view determine the direction of the mixed nets.
 - If the SPICE child does not also have a Verilog view, the direction of all mixed nets are inout.
- The default high and low voltage values in a d2a conversion are the local analog VDD and 0V, respectively. You can use options with the [d2a](#) command to override these default values.
- The default high and low voltage threshold for A/D conversion is 50% of the local analog VDD. You can use options with the [a2d](#) command to override these default values.
- In the d2a conversion, the digital driver is modeled as an ideal source in series with a resistor on the analog side. The value of the resistor depends on the Verilog drive strength and is determined by the resistance map file (see [Creating a Resistance Map File](#)).
- The behavior of a2d conversion depends on the direction of the mixed-net interface:
 - If the interface is a unidirectional a2d mixed-net, the analog driver is modeled as a digital driver with the Verilog default drive strength of 6 or Strong.
 - If the interface is a bidirectional mixed-net, the a2d events are modeled as a digital driver whose drive strength is determined by the effective output resistance of the analog driver. The analog engine calculates that output resistance and uses it as an index *resistance map file* to get a corresponding drive strength for Verilog.

Smaller output resistances lead to stronger Verilog drive strengths and larger output resistances lead to weaker Verilog drive strength (see [Creating a Resistance Map File](#)).

Use the `-hiz on` and `-hiz off` options with the `a2d` mixed-signal command to change these default behaviors.

- **Note:**

Calculating a2d drive strength and estimating the output resistance for the a2d conversion is time consuming, which could noticeably slow down the simulation if there are too many ports requiring it. Therefore, you should limit the use of *inout* mixed-nets to interface nets that are functionally bidirectional. For interface nets that have become bidirectional only because they were ports of a single-view SPICE cell (with no Verilog view to determine port directions), you should use the `a2d` command with the `-hiz off` option to disable a2d drive strength calculation.

- Only resistive elements—equivalent channel resistance for MOS transistors and/or ideal resistors—that can be traced to the local power supply or to ground from a mixed net are used in the calculation of net output resistance, when such calculation is done.
- The Verilog drive strength calculation in the analog-to-digital conversion for a mixed net ignores the effects of BJTs, diodes, or coupling capacitors connected to the ports inside the subcircuits.

Cases Where A/D and D/A Interface Elements are Not Inserted

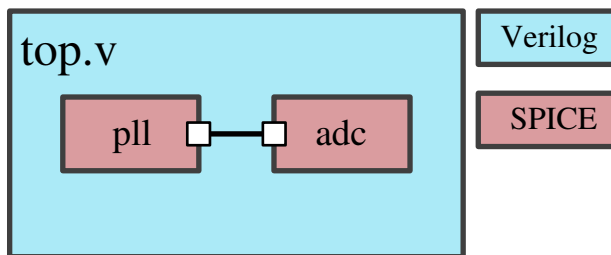
By default, the tool analyzes the design topology and inserts interface elements on analog signals that connect to the digital domain and vice versa. In some cases, analog signals may route through the digital domain, but not connect to a digital element. Likewise, digital signals may route through the analog domain, but not connect to an analog element. These nets are known as *through-nets* and interface elements inserted on through-nets are removed during optimization.

There are two cases for mixed nets where a2d or d2a interface elements are *not* inserted and are removed by through-net optimization:

Case #1

In case #1, a Verilog design contains instances represented by SPICE views. One or more SPICE pins are connected to a wire or nettype in the parent Verilog cell, however, there is no load or driver on that net in the digital domain. An example of this configuration is shown in [Figure 5](#), where blocks `pll` and `adc` are instantiated in a Verilog design, are represented by SPICE views, and are connected by a net. The tool determines that there is no connection from the analog domain to the digital domain for this net, performs through-net optimization on this net, and avoids inserting an interface element on the net.

Figure 5 SPICE is the Source and Destination With No Connection to Digital Domain

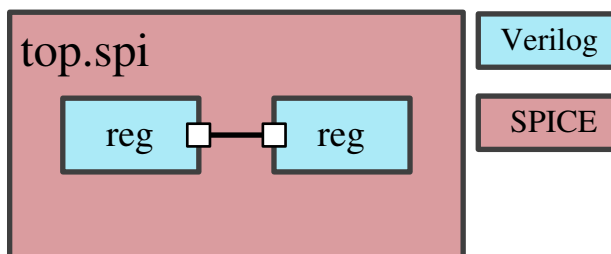


In this example, the through-net that connects the pll and adc analog blocks is referred to as an *a2a through net*. By default, a2a through nets are reported in the `through_net.rpt` file.

Case #2

In case #2, a SPICE design contains instances represented by Verilog views. One or more Verilog pins are connected to a wire or nettype in the parent SPICE block, however, there is no load or driver on that net in the analog domain. An example of this configuration is shown in Figure 6, where two register blocks are instantiated in a SPICE design, are represented by Verilog views, and are connected by a net. The tool determines that there is no connection from the digital domain to the analog domain for this net, performs through-net optimization on this net, and does not insert an interface element on the net.

Figure 6 Verilog is the Source and Destination With No Connection to SPICE Domain



In this example, the through-net that connects the two register blocks is referred to as a *d2d through-net*.

In either case, if more than one port is connected through the top-level net, those ports are connected through tunneling. The only exception to that rule is when a SPICE port is connected to top-level SystemVerilog net of type "reg", "logic", or a VHDL net of type "signal".

If the connection between the two SPICE ports was made with a nettype, the results are the same as the results for wire. If the connection was made by using SystemVerilog "logic", or "reg" or by using a VHDL "signal", then two interface elements are inserted, one from each SPICE port to the digital net. In other words, through-net optimization does

not occur for SPICE port or ports connected to a SystemVerilog "reg", "logic", or VHDL "signal".

If the ports of the two digital blocks blocka and blockb in [Figure 6](#) were defined as nettypes, the results are the same.

Controlling Through-net Optimization

An a2a through net is optimized as an analog node and, by default, is written only in the analog output file. You can disable a2a through-net optimization by using the `-ad_runopt=noa2dopt` option with the `simv` binary simulation executable as follows.

```
% simv -ad_runopt=noa2dopt ...
```

When you specify this option, an a2d, e2r, or e2n interface element is inserted and a digital image of the signal is generated and output in the digital output file. The type of interface element inserted by the tool depends on the type of the mixed-net. In addition, the net is no longer reported in the `through_net.rpt` file

Note:

When you specify the `-ad_runopt=noa2dopt` option, no interface element is inserted in the path *between* the two SPICE blocks. Instead, the new a2d interface element is added from the analog net between pll and adc ports and the digital domain.

The same principle applies to d2d through nets. By default, a d2d through-net is optimized as a digital node and is written only to the digital output file. You can disable d2d through-net optimization by using the `-ad_runopt=nod2aopt` option with the `simv` binary simulation executable as follows:

```
% simv -ad_runopt=nod2aopt ...
```

When you specify this option, a d2a, or r2e, or n2e interface element is inserted and an analog image of the signal is written to the analog output file. In addition, the net is no longer reported in the `through_net.rpt` file.

Note:

When you specify the `-ad_runopt=nod2aopt` option, no interface element is inserted *between* the two Verilog blocks. Instead, The new d2a interface element is added from the digital net between the Verilog blocks and the analog domain.

Caution:

These additional interface elements can create excess overhead for the mixed-signal simulation and degrade performance. Use these interface elements with caution.

To disable both a2a and d2d through-net optimization, you can combine runtime options as shown in the following example:

```
% simv -ad_runopt=noa2dopt+nod2aopt
```

Signal Conversion From Verilog-to-SPIICE and SPIICE-to-Verilog

Note that the term *SPIICE* refers to all transistor-level netlist formats supported by the analog engine. See the following conversion sections:

- [Converting Signal Values](#)
- [Modeling Dynamic Supply in Mixed-Signal Simulation](#)
- [Converting Signal Strength](#)
- [Creating a Resistance Map File](#)

Converting Signal Values

In the analog-to-digital and digital-to-analog conversions performed by the a2d and d2a interface elements, mixed-signal simulation must translate both the signal *value* and the signal *strength* from one domain to the other. The signal strength translation uses a resistance map file, which is described in [Creating a Resistance Map File](#).

The rules applied to digital-to-analog signal conversion are summarized in [Table 2](#).

Table 2 Signal Value Conversion Rules for Digital-To-Analog Conversion

Verilog value	Transistor-level value
0	• 0V (gnd) or, in case dynamic supply is enabled for d2a, a percentage of the voltage on the referenced VDD net. • Can be modified with the d2a mixed-signal control command.
1	• Local supply voltage value or, in case dynamic supply is enabled for d2a, a percentage of the voltage on the referenced VDD net. • Can be modified by the d2a mixed-signal control command.
Z	• The analog node is not be driven by Verilog. Treat Z" as an open circuit because 1'bz typically maps to a voltage source with half the supply voltage and a high internal resistance (example, 1e8 ohms). This high internal resistance eliminates current flow and is like an open circuit. The voltage of the node depends entirely on the analog circuitry.
X	• 0V • Can be modified by the d2a mixed-signal command.

Signal value conversion in the a2d direction from the transistor level to Verilog is based on the analog voltage crossing high and low thresholds. By default, 50% of the local voltage supply is used for both high and low a2d thresholds. To change the threshold values for digital event generation, the mixed-signal control command `a2d` must be used. If the dynamic supply feature is enabled in the `a2d` command, the high and low thresholds can be set as a percentage of the referenced supply net.

The analog-to-digital signal value conversion rules are displayed in [Table 3](#).

Table 3 Signal Value Conversion Rules for Analog-To-Digital Conversion

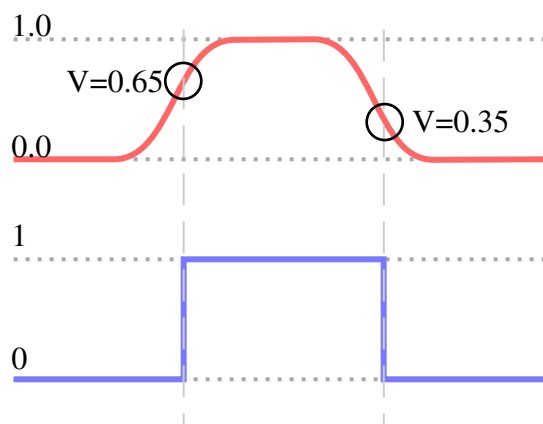
Transistor-level value	Verilog value
- Less than (<) or equal to (=) the low threshold voltage (default = 50% of local voltage supply) - Can be modified by the <code>a2d</code> command	0
- Greater than (>) or equal to (=) the high threshold voltage (default = 50% of local voltage supply) - Can be modified by the <code>a2d</code> command	1
In case of bidir interface nets or if the drive strength calculation is enabled for a2d interface nets and the analog node is HiZ	Z

No value is converted from transistor-level to X in Verilog.

The following example sets 0.35V as the low threshold value and 0.65V as the high threshold value for analog-to-digital conversion at the interface net `top.dout`. If the voltage on the `top.dout` interface node decreases to 0.35V, a digital event is generated and the logic value changes to 0. If the voltage on the `top.dout` interface node increases to 0.65V, a digital event is generated and the logic value changes from 0 to 1:

```
a2d -loth 0.35 -hith 0.65 -node top.dout;
```

Figure 7 Analog-to-Digital Conversion With a2d



Modeling Dynamic Supply in Mixed-Signal Simulation

By default, the a2d and d2a conversions assume that the analog supply voltage remains constant. However, in certain low-power designs and in other situations, the chip occasionally powers down and powers back up. Designers model this behavior by varying the supply voltage during simulation.

To model this behavior during mixed-signal simulation, specify the threshold voltages as a percentage of the power supply value and assign the power supply with the `vdd` option to the a2d and d2a commands. This allows the a2d and d2a voltage levels to dynamically track the changes in supply to correctly model the behavior of the circuit. The following example enables dynamic supply tracking for the a2d and d2a interface elements:

```
a2d -loth 20% -hith 80% -node top.i1.clk -vdd top.i2.vdd;  
d2a -hiv 100% -lov 0% -node top.i1.rst -vdd top.i2.vdd;
```

Figure 8 demonstrates how, with the dynamic supply enabled, the a2d input threshold voltage varies with changes to the analog supply voltage. In this example, both high and low a2d threshold voltages are set to 50% of VDD.

Figure 8 Thresholds Vary as the Analog Supply Varies

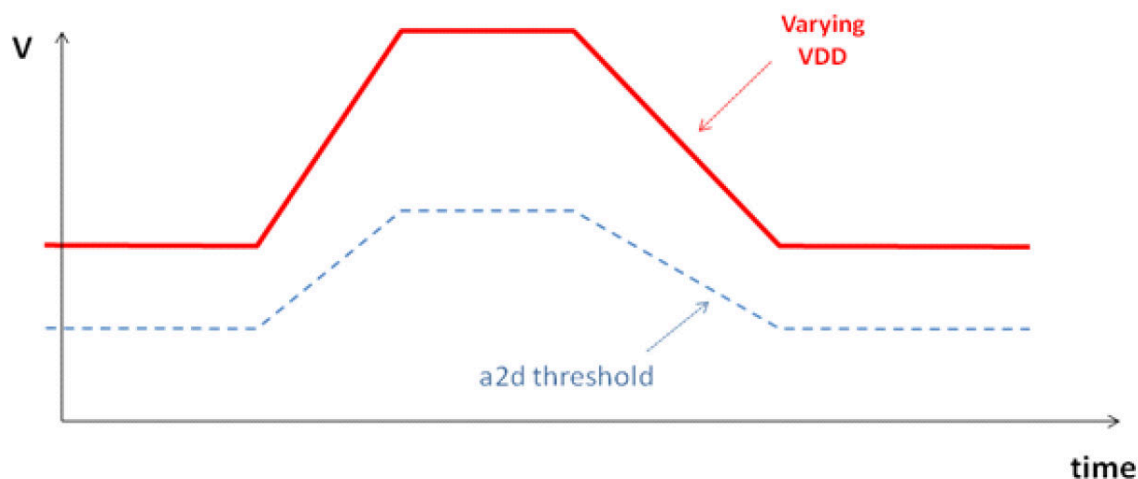
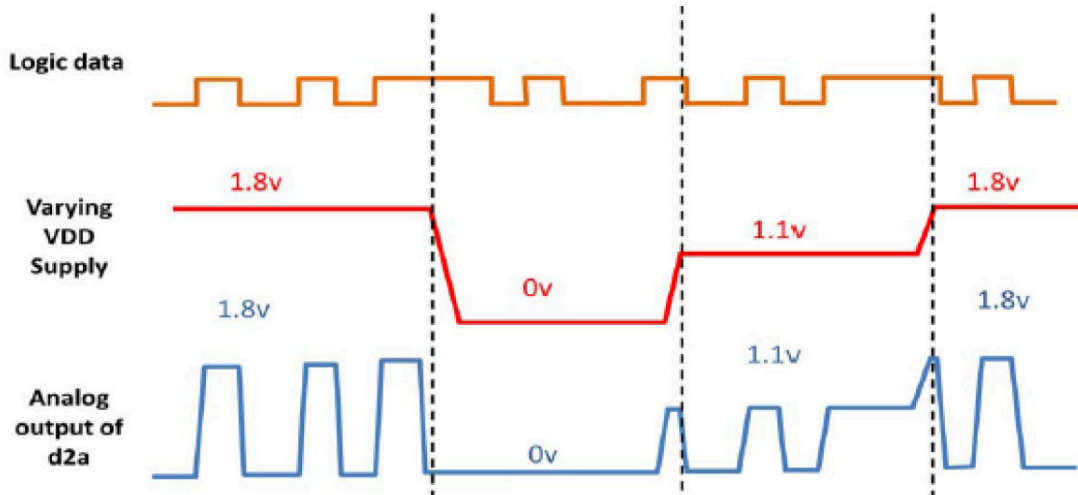


Figure 9 demonstrates how the d2a conversion is affected by a varying analog supply voltage if dynamic supply is enabled. Here, the *Logic data* is the input to the d2a interface element and the *Analog output* is the output. The analog output tracks the changes in VDD supply voltage. In this figure, the default high and low d2a levels are set to 100% and 0% of VDD.

Figure 9 Conversion Affected by a Varying Analog Supply Voltage



For more information, see the [a2d](#) and [d2a](#) command descriptions for details on about how to enable the dynamic supply feature.

Converting Signal Strength

The Verilog language defines eight different drive strengths, with 0 as the weakest drive strength and 7 as the strongest drive strength. By default, 6 is the drive strength applied by the tool if not otherwise specified. The following list shows the eight Verilog drive strengths and the keywords used to represent the strengths for logic 0 and 1 for each level:

Table 4 Verilog Signal Drive Strengths

Verilog strength level	Purpose	Logic 0 keyword	Logic 1 keyword
7	Supply drive	supply0	supply1
6	Strong drive	strong0	strong1
5	Pull drive	pull0	pull1
4	Large capacitive	large	large
3	Weak drive	weak0	weak1
2	Medium capacitive	medium	medium
1	Small capacitive	small	small
0	High impedance	highz0	highz1

Drive strengths are used to resolve the logic value on a net when multiple drivers are present. When a conflict occurs between multiple drivers on the same net, the driver with strongest driving value takes precedence and determines the net value.

For the analog-to-digital conversion, if the mixed net has a direction of inout, the tool calculates the effective analog output resistance of the mixed net to estimate an equivalent drive strength on the digital side.

By default, all SPIICE ports are assumed to be inout, unless the SPIICE cell has a Verilog view where the direction of ports are explicitly declared. If the direction of a SPIICE port connected to a mixed net is output, no drive strength mapping from analog to digital takes place during analog-to-digital conversion and the mixed net always assumes the default drive level of 6.

If the mixed net connects to a SPIICE port of type inout (either because there is no Verilog view for the SPIICE cell or the direction of the port in the Verilog view is defined as inout) then the tool calculates the effective output resistance for the analog output and maps it to a corresponding drive strength in Verilog. This can be a time-consuming task during simulation and it is recommended to avoid bidirectional mixed nets as much as possible.

For the digital-to-analog conversion, the Verilog drive strength is mapped to a resistance value. The resistor is placed in series with the voltage supply that models the Verilog driver in SPIICE. The following sections describe the rules governing these conversions.

Resistance Calculation for Digital-to-Analog Strength Conversion

To model the Verilog drive strength during digital-to-analog conversion, an equivalent resistance value is computed by interpolating values contained in a resistance map file. By default, the tool reads the `rmapAD.init` map file from the PrimeSim XA, FineSim, or PrimeSim installation directory, looks up the equivalent resistance map range for the digital strength value, and computes the average value for the range.

For example, consider a driver in Verilog that drives with a strength of 6 and uses the following `rmapAD.init` resistance map file:

Example 12 Example `rmapAD.init` Resistance Map File

```
resistance_map 90000.2-1e32      0 ;
resistance_map 70000.2-90000.2  1 ;
resistance_map 50000.2-70000.2  2 ;
resistance_map 7000.2-50000.2    3 ;
resistance_map 6000.2-7000.2     4 ;
resistance_map 1000.2-6000.2     5 ;
resistance_map 1.2-1000.2        6 ;
resistance_map 0-1.2             7 ;
```

The equivalent resistance value is calculated by referring to the resistance map value for strength 6 and taking the average value as follows:

Figure 10 Drive Strength Resistor Calculation

$$R_{avg} = \frac{R_{min(6)} + R_{max(6)}}{2} = \frac{1.2 + 1000.2}{2} = 500.7$$

The only exception is when Verilog drives with a `highz0` or `highz1` drive strength. In this case, no driver (no ideal supply or series resistor) is applied on the analog side, the digital driver acts as an open circuit, and the voltage of the analog node depends entirely on the drivers in the analog circuit.

When a digital block drives an analog block by using the default drive strength (6), the analog input might suffer a voltage drop if the analog block is drawing significant current. This can occur if there are multiple drivers on the net. To increase the drive strength, use the `resistance_map` command and specify a lower resistance value. For example, the following `resistance_map` command specifies a resistance range from 1.2 to 10 ohms.

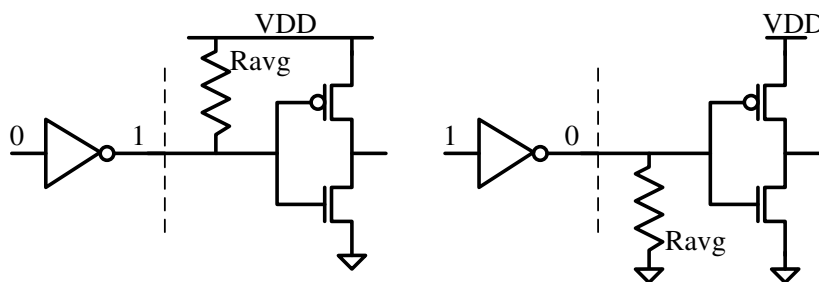
```
// rmapADcustom.init
...
resistance_map 1.2-10.0      6 ;
```

To use this custom resistance map file, specify the file name with the `rmap_file` command in the `vcsAD.init` file as follows:

```
rmap_file ./rmapADcustom.init;
```

When the mixed-net is in the high state, the series resistor is added between the net and VDD. When the mixed-net is in the low state, the series resistor is added between the net and ground as shown in [Figure 11](#).

Figure 11 Series Resistor to Model Drive Strength



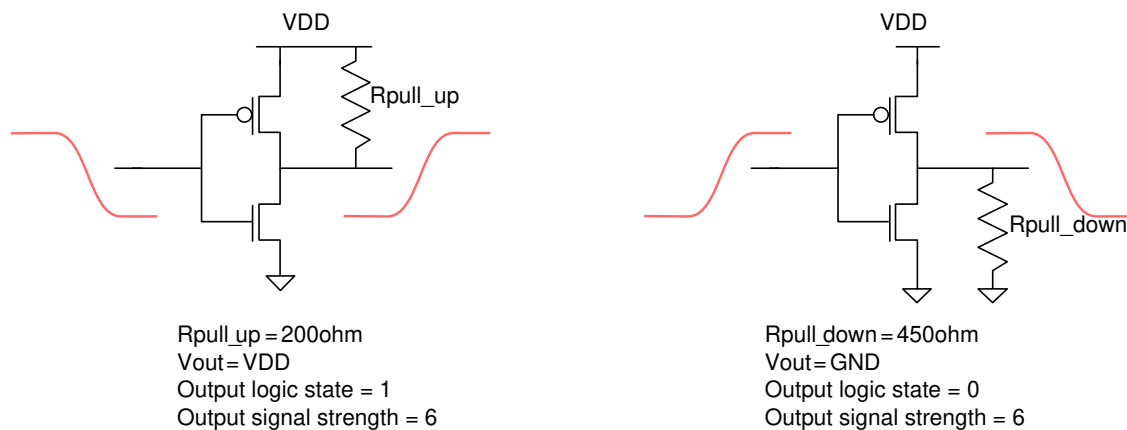
Strength Calculation for Analog-to-Digital Conversion

The driver strength of the analog node driving a digital signal is calculated based on the load resistance of the analog driver. The equivalent `Rpull_up` and `Rpull_down` values are used to compute the drive strength from the resistance map file.

For example, an equivalent resistance value 200 ohms for logic 1 and a resistance value of 450 ohms for logic 0 are converted to strength 6 based on the following line in the resistance map file (both 200 and 450 are in the range specified for drive strength 6).

```
resistance_map 1.2-1000.2 6 ;
```

Figure 12 Transistor Level to Verilog Strength Conversion



Creating a Resistance Map File

The default `rmapAD.init` file specifies eight ranges of values, one range for each Verilog driver strength value. The same range values are used for both analog-to-digital and digital-to-analog resistance mapping. The default resistance map file, shown in [Example 12](#), is available in the `include` directory of the simulator installation directory.

```
install_directory/include/rmapAD.init
```

You can create your own resistance map file with unique range values for both analog-to-digital and digital-to-analog conversion. To load your custom resistance map file, use the `rmap_file` command in the `vcsAD.init` mixed-signal simulation control file. If you do not use the `rmap_file` command, the default `rmapAD.init` file is used.

Mapping Resistance Values Separately for A2D and D2A Conversion

The default resistance map file provided with the tool specifies the same range values for both analog-to-digital and digital-to-analog conversion. To separate the conversions and provide unique ranges for analog-to-digital and digital-to-analog conversion, use the `-from` and `-to` options with the `resistance_map` command.

Use the following syntax to specify a user-defined resistance value range for converting signal values from analog to digital.

```
resistance_map -from analog resistance_value_range -to verilog strength;
```

Use the following syntax to specify a user-defined resistance value range for converting signal values from digital to analog.

```
resistance_map -to analog resistance_value_range -from verilog strength;
```

The following example shows a complete resistance map file that contains resistance map ranges for both analog-to-digital and digital-to-analog conversions for all eight Verilog strength values.

Example 13 Resistance Map File With Distinct Ranges for A2D and D2A Conversion

```
resistance_map -from analog 90000.2-1e32      -to verilog 0;
resistance_map -from analog 70000.2-90000.1    -to verilog 1;
resistance_map -from analog 50000.2-70000.1    -to verilog 2;
resistance_map -from analog 5000.2-50000.1     -to verilog 3;
resistance_map -from analog 4000.2-5000.1      -to verilog 4;
resistance_map -from analog 3000.2-4000.1      -to verilog 5;
resistance_map -from analog 1.2-3000.1         -to verilog 6;
resistance_map -from analog 0-1.1              -to verilog 7;

resistance_map -to analog 2002.2-1e32          -from verilog 0;
resistance_map -to analog 1500.2-2002.1        -from verilog 1;
resistance_map -to analog 1000.2-1500.1        -from verilog 2;
resistance_map -to analog 500.2-1000.1         -from verilog 3;
resistance_map -to analog 400.2-500.1          -from verilog 4;
resistance_map -to analog 300.2-400.1          -from verilog 5;
resistance_map -to analog 1.2-300.1            -from verilog 6;
resistance_map -to analog 0-1.1                -from verilog 7;
```

Verilog-Top/SPICE-Top Flows

Mixed-signal simulation supports both Verilog-top/VHDL-top and SPICE-top configurations. In a SPICE-top configuration, you must use the `spice_top` command in the `vcsAD.init` mixed-signal simulation control file. For Verilog-top or VHDL-top configuration, no specific command is required.

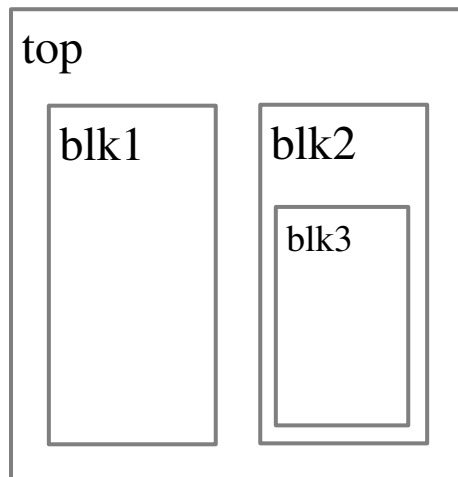
Donut Configurations

One of the factors that affects the setup of a mixed-signal simulation is the netlist formats used in different layers of the design hierarchy. For example, if the top level of a design is in SPICE format, the design is called *SPICE-top*. A design could also be *Verilog-top*, *VHDL-top* or *Verilog-AMS-top*.

Also, a design in which Verilog is on top of SPICE in the hierarchy is called a Verilog-SPICE donut configuration. There are many possible donut configurations for each of the three flows. There are also restrictions on certain types of donut configurations that are described in detail in following sections.

[Figure 13](#) shows a simple design and its hierarchy. The block named `top` instantiates two child blocks: `blk1` and `blk2`. Child block `blk2` contains `blk3`.

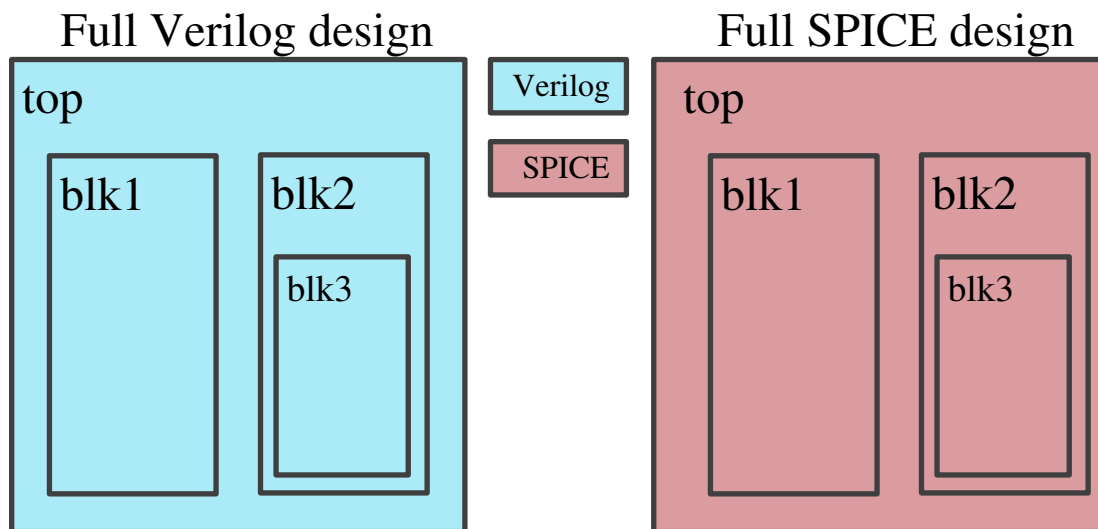
Figure 13 Sample Design With Hierarchy



Cells `blk1` and `blk3` are referred to as *leaf cells*, because they are located at the bottom of a hierarchy branch.

In a non mixed-signal design, the top block and all child blocks are either Verilog or SPICE as shown in [Figure 14](#).

Figure 14 Full Verilog and Full Spice Configurations



A mixed-signal design supports different simulation domains for digital and analog blocks within the same simulation. Figure 15 shows a Verilog-SPICE-Verilog donut configuration that is simulated by using the following mixed-signal simulation control file:

Example 14 Verilog top Mixed-signal Simulation Control File

```
use_spice -cell blk1;
use_spice -cell blk2;
use_verilog -module blk3;
choose ...
```

Figure 15 Verilog-top Configuration

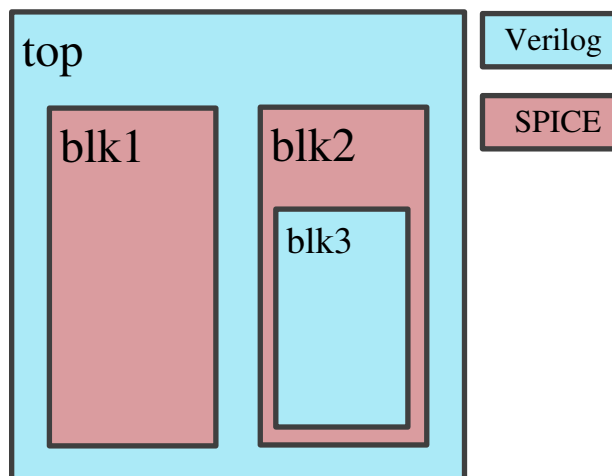
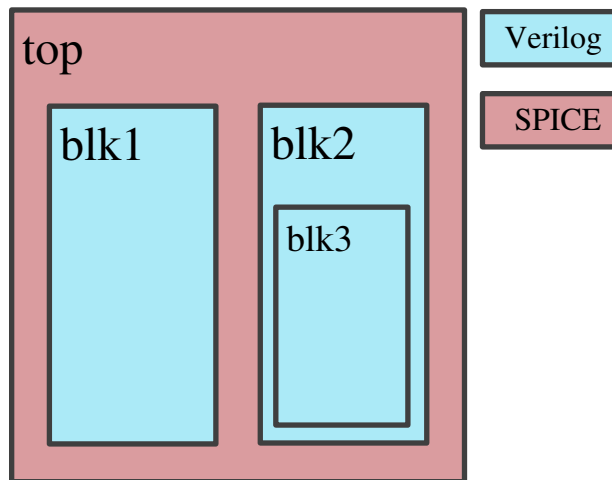


Figure 16 shows another possible donut configuration where the top-level design is represented in SPICE (SPICE-top design) and has a SPICE-Verilog donut configuration. The design is simulated with the following mixed-signal simulation control file:

Example 15 *SPICE top Mixed-Signal Simulation Control File*

```
spice_top;  
use_verilog -module blk1;  
use_verilog -module blk2;  
choose ...
```

Figure 16 *SPICE-top Configuration*



A block in the hierarchy might have both a Verilog description and a SPICE description available for simulation. This type of cell is called a *multi-view* cell. By default, the tool selects the view for a given cell based on the parent view. In the previous examples, the specific view was selected by specifying the `use_spice`, `use_verilog`, and `use_vhdl` commands in the `vcsAD.init` mixed-signal simulation control file.

In the preceding example, the Verilog views for `blk1` and `blk2` are used because of the `use_verilog` commands, while the view selected for the multi-view cell `blk3` is Verilog because `blk3` by default inherits the Verilog view of its parent `blk2`.

VHDL, Verilog or SPICE blocks can each instantiate blocks of the other two formats without restriction. The only requirement is for SPICE instantiations under VHDL; those SPICE cells must also have a Verilog view available in order for the instantiation under VHDL to take place. The PrimeSim XA, FineSim, and PrimeSim simulation engines support VHDL-top, Verilog-top, and SPICE-top design configurations.

Multiple Cell Views in the Nettype Flow

The a2a interface element through-net optimization in the nettype flow is not complete when there are multiple cell views. Typically, an extra e2n nettype is placed on the a2a connection, which can lead to erroneous results on the "n" side of the inserted e2n, especially if current-based nettypes are being used. You can use specific e2n options to resolve the erroneous results.

Specifying a Voltage Reference for Interface Elements

An Interface Element (IE) with levels or thresholds that have not been set by an IE command ([a2d](#) or [d2a](#)) can find an associated supply reference by tracing through the channel of connected transistors, or through resistors to an ideal voltage source. However, not all scenarios can be traced and the tool might select an incorrect voltage level. You can override these levels using the [a2d](#) or [d2a](#) commands, but this is impractical if there is a large number of interface elements.

The two kinds of IEs discussed in this section can be described by using the following terms:

- *Supply Interface Element (Supply IE)*: Any HDL to SPICE interface element whose SPICE side provides power to a SPICE block.
- *Signal Interface Element (Signal IE)*: An interface element that transfers a state or value but is not a power supply.

You can use the [ie_reference_voltage](#) command to define a reference node to guide the tool when tracing supplies. The reference voltage used can be static (where you specify the value) or dynamic (following the value on the reference node as it changes).

- Static reference voltage

A static reference voltage attaches a voltage value to a node. If supply tracing finds the node name, then any Signal IE that has traced its supply to that node uses the associated value to set thresholds and levels:

```
ie_reference_voltage -node node_name -voltage value;
```

For Signal IE's that can trace to the reference `node_name`:

- D2A levels are calculated as `hiv=value, lov=0`.
 - A2D levels are calculated as `hith=value/2, loth=value/2`
- Dynamic reference voltage

A dynamic reference voltage requires only the node. If supply searching finds the specified node, a dynamic supply reference to that node is established:

```
ie_reference_voltage -node node_name;
```

For Signal IE's that can trace to the reference *node_name*, this is equivalent to manually setting a dynamic supply for each:

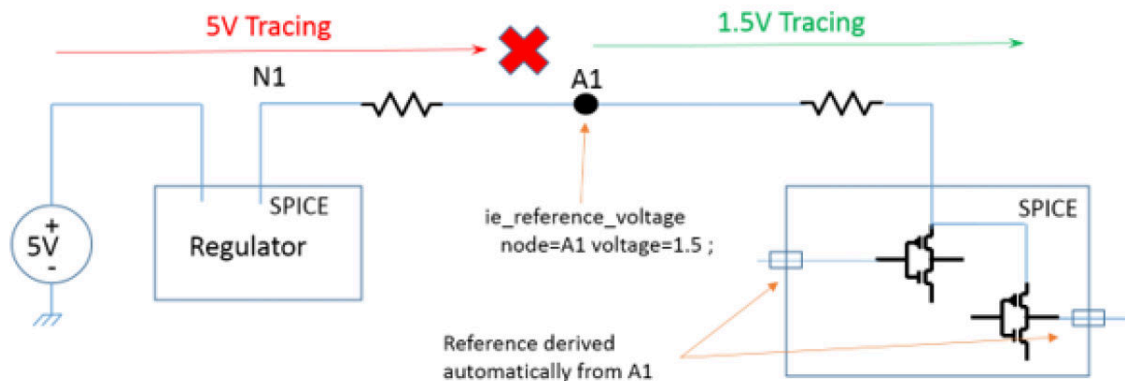
- D2A levels are calculated as *hiv*=100%, *lov*=0% referring to *VDD=node_name*
- A2D levels are calculated as *hith*=50%, *loth*=50% referring to *VDD=node_name*

A SPICE block often uses multiple supplies. You can use a separate [ie_reference_voltage](#) command for each of the supplies. If you use multiple [ie_reference_voltage](#) commands on the same net, the last one specified is used. Note that specifying multiple commands on the same net might consume extra time to resolve them, so it is not recommended.

Using [ie_reference_voltage](#) With Internal Supplies

A Signal IE that can trace its supply through circuitry to an internal supply still does not know what voltage the regulator is providing at its output. You can use the [ie_reference_voltage](#) command to define the node at the output of the regulator as the node that should be used to set interface element levels and thresholds.

In the following example, a voltage regulator has a 5V ideal supply and a 1.5V output. The 1.5V is distributed through the circuitry and traces through SPICE to several A2D and D2A interface elements.



You use the [ie_reference_voltage](#) command to identify the output of the regulator and either specify a value or declare it as a dynamic supply:

```
ie_reference_voltage -node A1 -voltage 1.5; //static
```

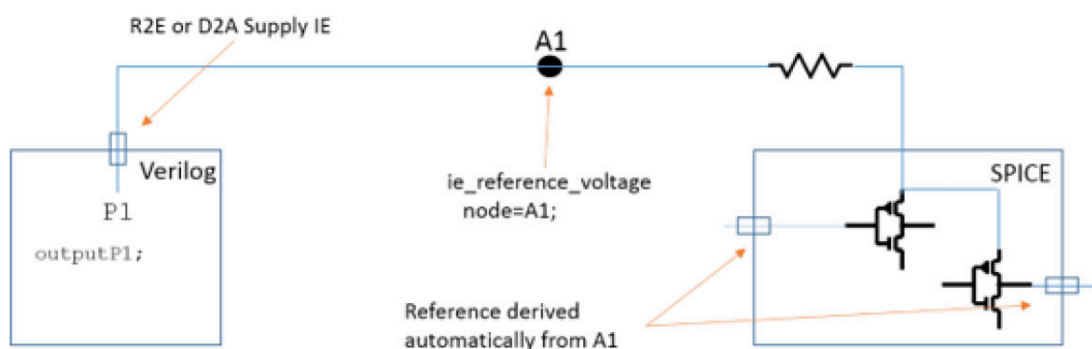
or

```
ie_reference_voltage -node A1; //dynamic
```

The Signal IEs that can trace supply to node A1 use the value specified by the `ie_reference_voltage` command, or dynamically follow the voltage on A1 and use it as their reference.

Supply IEs Connected to `ie_reference_voltage` nodes

You can also use the `ie_reference_voltage` command for a SPICE power supply driven from Verilog through a Supply IE. That supply IE could be a D2A power net, R2E, or N2E. For example:



If you specify:

```
ie_reference_voltage -node A1;
```

The two signal IEs (which can trace supply to A1) become:

```
d2a -node top.il.abc -hiv 100% -lov 0% -VDD A1;  
a2d -node top.il.def -hith 50% -loth 50% -VDD A1;
```

The previous reference is dynamic, so the signal IE levels follow the voltage on A1.

If you prefer to set a fixed value for the reference, you can specify the value:

```
ie_reference_voltage -node A1 -voltage 1.8;
```

Interface Element Supply Precedence

There are several ways to specify the thresholds of A2D and D2A interface elements. If multiple specifications are found, the Signal IE applies them with the following precedence (1 overrides 2, 2 overrides 3, and so on):

1. User-specified A2D/D2A thresholds
2. `ie_reference_voltage` node

3. Ideal supply traced from the Signal IE:

- a. The autotracing algorithm first searches for the supply voltage through channel connected networks, followed by gate-to-source, gate-to-drain connectivity, and so on.
- b. If the preceding search is unsuccessful, it searches for any voltage source defined in the netlist:
 - A voltage source is considered even if there is no direct connectivity between the current IE.
 - If there are multiple voltage sources defined in the netlist, the algorithm picks up the highest voltage.

Note:

The autotracing algorithm cannot trace through supplies defined inside a Verilog-A block as it considers them a black box.

The autotracing algorithm may provide different results between the simulators. That is, it may find different supplies with the PrimeSim XA simulation engine than with the FineSim or PrimeSim simulation engines. Results can also vary between different simulator versions.

4. If all the preceding cases fail (that is, nothing is found), use default (3.3V).

Report Files

If you enable verbose mode for the `interface_element.rpt`, a note for each a2d/d2a/inout indicates from where the value was derived. If the reference was derived from an [ie_reference_voltage](#) command, it is noted if you used the following command:

```
report_option -interface_element verbose;
```

Note:

This feature is currently available only in the PrimeSim XA simulation engine and is not supported in the FineSim or PrimeSim simulation engines.

Using Library-Based Interface Element Options

VCS PrimeSim AMS users often compile HDL standard cell libraries into different locations using the VCS HDL library compilation methodologies. Library-based interface element

options provide a mechanism to set global threshold options specific to each HDL compiled library. These options apply to:

- Cells compiled into Verilog/SystemVerilog libraries with the applicable VCS `vlogan` commands.
- Cells compiled into VHDL libraries with the applicable VCS `vhdlan` commands.

For more information about the `vlogan` and `vhdlan` commands, see the *VCS® User Guide*.

You specify the HDL library names with the `-library` options for all the interface element commands ([a2d](#), [d2a](#), and so on).

In the following example, when the [d2a](#) commands and options are applied, any interface elements inserted at the interface to HDL cells in the `cellLib1` library use the `-hiv 1.2` and `-lov 0.0` threshold options. Any interface elements inserted at the interface to HDL cells in the `cellLib2` library use the `-hiv 1.8` and `-lov 0.0` threshold options.

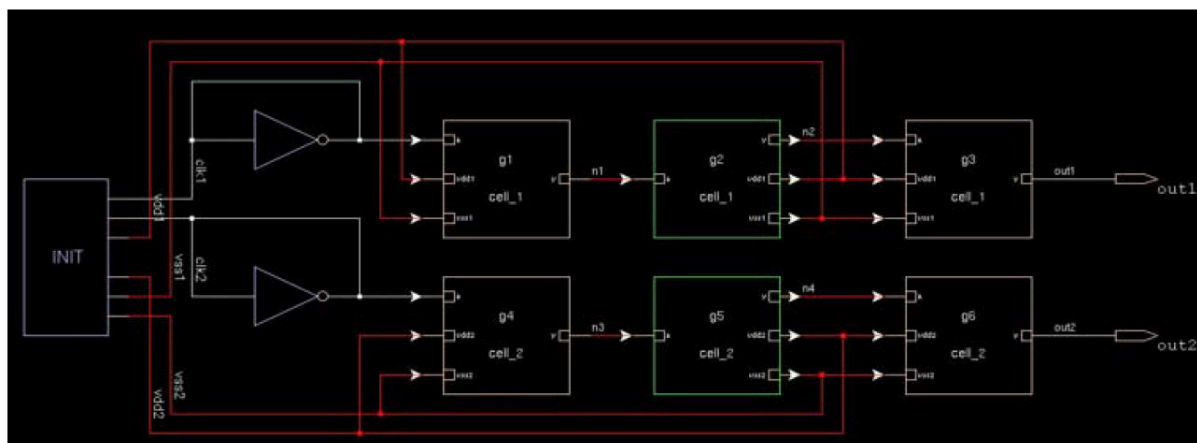
```
d2a -hiv 1.2 -lov 0.0 -library libCell1 -cell * -port *;
d2a -hiv 1.8 -lov 0.0 -library libCell2 -cell * -port *;
```

For more information about the interface element commands, see [Mixed-Signal Control Commands](#).

Library-Based Interface Element Example

This example is a Verilog top design with `d2a` and `a2d` interface elements. See [Figure 17](#).

Figure 17 Verilog Top Example

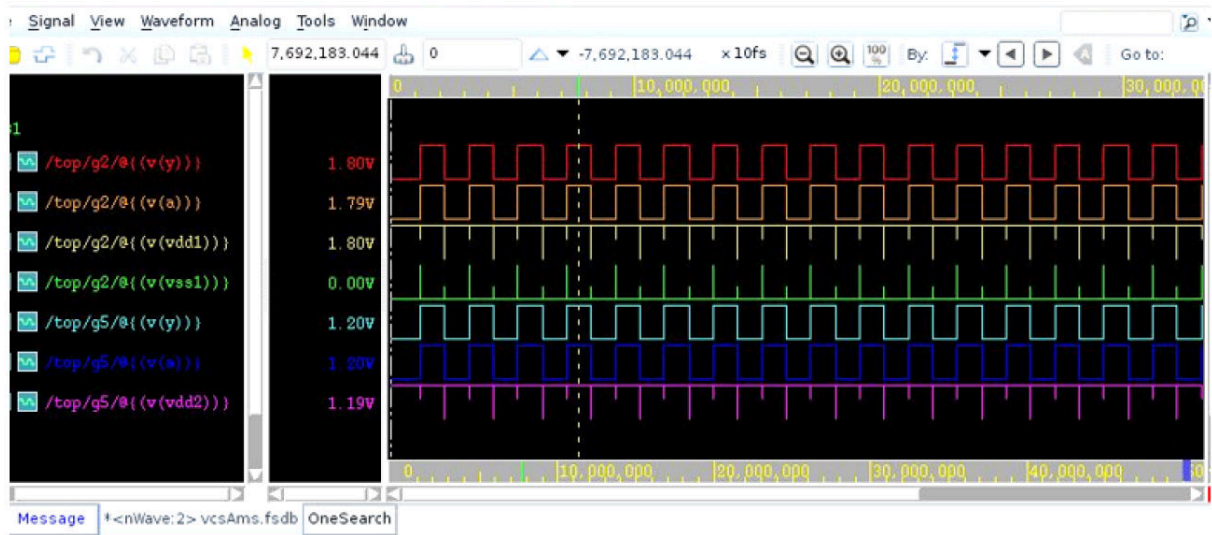


This design has the following configuration in the `vcsAD.init` file.

```
use_spice -cell cell_1 -inst top.g2;
use_spice -cell cell_2 -inst top.g5;
```

Figure 18 shows results that use manually-specified interface element command options to give the correct results.

Figure 18 Waveforms



In this case, the interface elements inserted for this design without the application of the new interface element library-based options in the `interface_element.rpt` file are:

Example 16 `interface_element.rpt` File

```
d2a -hiv 1.8v -lov 0v -node top.g2.a;
d2a -hiv 1.8v -lov 0v -node top.g2.vdd1;
d2a -hiv 1.8v -lov 0v -node top.g2.vss1;
a2d -loth 0.90v -hith 0.90v -node top.g2.y;
d2a -hiv 1.2v -lov 0v -node top.g5.a;
d2a -hiv 1.2v -lov 0v -node top.g5.vdd2;
d2a -hiv 1.2v -lov 0v -node top.g5.vss2;
a2d -loth 0.60v -hith 0.60v -node top.g5.y;
```

Eight specific interface element commands with thresholds must be provided to produce correct results for this design. With the new library-based interface element options, only two commands in the `vcsAD.init` file for each library and options are needed to give correct results:

Example 17 `vcsAD.init` File

```
d2a -hiv 1.8v -lov 0v -library cell1;
a2d -loth 0.9v -hith 0.9v -library cell1;
d2a -hiv 1.2v -lov 0v -library cell2;
a2d -loth 0.6v -hith 0.6v -library cell2;
```

Synchronizing Analog and Digital Events

To synchronize events between the digital and analog domains, ideally, both the analog and digital engines should have the same time resolution value, where time resolution is the smallest time step the simulator can use.

- The PrimeSim XA simulation engine does not have a predetermined time resolution. In the PrimeSim XA tool, the time resolution is constantly updated and adjusted for accuracy and performance during simulation.
- The FineSim and PrimeSim simulation engines have predetermined time resolutions, and you should align the analog and digital time resolutions so they match. This can be done by changing the digital time resolution to match the analog engine default resolution, or vice versa.

As a general rule, the analog time resolution cannot be greater than the digital time resolution, because a loss of synchronization could happen at runtime and cause a runtime error. At the same time, it is preferable that the analog time resolution is not smaller (or much smaller) than the digital time resolution unless necessary for accuracy. The smaller analog time resolutions usually cause slower performance for the analog engine and the mixed-signal simulation.

- In the VCS PrimeSim AMS tool using the PrimeSim XA simulation engine, a time resolution mismatch between the digital and analog engines does not occur because of the adjustable time resolution mechanism in the PrimeSim XA tool. In this case, you do not need to set and adjust the analog time resolution.
- In the VCS PrimeSim AMS tool using the FineSim simulation engine, to match the digital and analog resolutions, you can change either the FineSim or VCS time resolution. Because the analog engine time resolution is usually determined by the accuracy requirements of the analog circuit, you should set the FineSim time resolution based on the accuracy requirements for the circuit and adjust the VCS time resolution to match.

To override the VCS timescale, use the `-override_timescale` switch at compile time. In the following example, the VCS timescale/time resolution is overridden with a 1ps/100fs:

```
% vcs -ad -override_timescale=1ps/0.1ps
```

To change the default time resolution for the FineSim tool (not recommended, as explained above), set the `finesim_tunit` option:

```
.option finesim_tunit=0.1ps
```

- In the VCS PrimeSim AMS tool using the PrimeSim simulation engine, to match the digital and analog resolutions, you can change either the PrimeSim or VCS time resolution. Because the analog engine time resolution is usually determined by

the accuracy requirements of the analog circuit, you should set the PrimeSim time resolution based on the accuracy requirements for the circuit and adjust the VCS time resolution to match.

To override the VCS timescale, use the `-override_timescale` switch at compile time. In the following example, the VCS timescale/time resolution is overridden with a 1ps/100fs:

```
% vcs -ad -override_timescale=1ps/0.1ps
```

To change the default time resolution for the PrimeSim tool (not recommended, as explained above), set the `primesim_tunit` option:

```
.option primesim_tunit=0.1ps
```

Verilog-A Model Instantiation

In mixed-signal simulation, Verilog-A can be used as the analog view of a cell. Such a Verilog-A cell can be instantiated under SPICE or Verilog.

There are two ways to read in Verilog-A blocks in mixed-signal simulation depending on the mixed-signal flow used.

- For the Verilog-SPICE and VHDL/Verilog-SPICE flows, Verilog-A blocks are read in by:
 - Using the `.hdl` command in the SPICE netlist to read in Verilog-A files.
 - Using the appropriate command in the analog simulator to select the Verilog-A view, in case there are both SPICE and Verilog-A definitions for the same block. For the PrimeSim XA tool, the command is `set_va_view`. For details about the syntax of these commands, see the *PrimeSim XA Command Reference Manual*.
- For the Verilog-AMS-SPICE flow, all Verilog files, including Verilog-A, are passed to VCS at compile time. In this flow it is still possible, although not preferable, to read in Verilog-A files through SPICE `.hdl` as well.

Parameter Passing Rules

Parameters can be passed between HDL blocks (Verilog or VHDL) and SPICE.

- In the VCS PrimeSim AMS tool using the PrimeSim XA simulation engine, parameter passing between HVA blocks is supported by default.
- In Verilog-SPICE and VHDL/Verilog-SPICE flows, parameter passing between HDL and SPICE is not enabled by default. To enable parameter passing, the mixed-signal command `"param_pass enable;"` must be used (for details, see [param_pass](#)).

- Parameter passing between SPICE and Verilog-A is always enabled in all solutions if Verilog-A is read in via the SPICE `.hdl` command. For example:

```
.hdl "inv_verilog_a.va"  
x1 in out inv_verilog_a vhigh=3.3 vlow=0 td=10
```

- In the Verilog-AMS-SPICE flow, parameter passing between Verilog and SPICE and parameter passing between HVA blocks are enabled by default.

3

Using Advanced Features in Mixed-Signal Simulation

This chapter describes the advanced features in mixed-signal simulation.

- [Save and Restore Feature in VCS PrimeSim AMS](#)
- [Performing Sweeps and Alters for Efficient Mixed-Signal Simulation](#)
- [Reading and Driving Analog Signals from the Digital Domain](#)
- [Changing the Analog Configuration in the Middle of a Simulation](#)
- [Postlayout Simulation Through Back-Annotation](#)
- [Meta-Encrypted SPICE Netlists in Mixed-Signal Design](#)
- [Resolving Instance Name Conflicts for Testbench Reuse](#)
- [Running Concurrent Mixed-Signal Simulations From the Same Compiled Code](#)
- [Controlling the Number of Time Points at the Analog-Digital Boundary](#)
- [Running Monte Carlo Analysis in Mixed-Signal Simulation](#)
- [Running AC Analysis During Transient Analysis](#)
- [Running AC Analysis](#)
- [Adding Netlist Commands to the Control File](#)
- [Adding PrimeSim XA Commands at Runtime](#)
- [Skipping Analog Simulation for Specific Time Windows](#)
- [Running Transient Noise Analysis](#)

Save and Restore Feature in VCS PrimeSim AMS

You can use this feature to preserve simulation progress when interrupting long-running simulations (example, machine power down). Save the state of the simulation up to the

current time point and restore it later, preventing simulation data loss. For more details, refer to [Saving and Restoring Simulation State](#) chapter.

Performing Sweeps and Alters for Efficient Mixed-Signal Simulation

Sweeps and Alters are commonly used in SPICE and Fast SPICE to perform multiple simulations. This feature enables sweep methods commonly used in SPICE to be used in mixed-signal simulation as well. For more information, refer to [Sweep/Alter in Mixed-Signal Simulation](#) chapter.

Reading and Driving Analog Signals from the Digital Domain

Analog signals can be read or driven from the digital domain through these methods:

- Continuous assignment via Verilog language XMR: The XMR is used to access a SPICE signal anywhere in the design hierarchy for read or drive access.
- Continuous assignment via `$hdl_xmr` tasks:
 - These tasks allow a continuous transfer of events from a source to destination.
 - An analog signal can be the source or destination of the transfer.
- Event-based proprietary Synopsys functions (`$snps_...`): These can only be used in a sequential assignment (example, assignment in an initial or always block) to sample an analog voltage or assign a constant voltage.
- Unified Command Line Interface cross module reference (UCLI XMR): UCLI get, force, and release commands for cross-module reference.

For more detailed information, refer to [Cross Module Reference](#) chapter.

Changing the Analog Configuration in the Middle of a Simulation

After a simulation begins, you can change the analog configuration and simulate different conditions with the `ace reread` UCLI command. Use this feature to modify certain analog configurations on-the-fly, such as simulation accuracy, temperature, or limited aspects of the netlist. The PrimeSim XA, FineSim, or PrimeSim simulation engines support this feature in mixed-signal simulation, which relies on the OP-based save and restore feature.

ace reread

This command allows a new configuration file or netlist to be read in the middle of a simulation, with some limitations. The command saves the state of the analog simulator to files with a .ic and .ic.sup extension. Later, the saved state is restored while the analog engine reads in a new SPICE netlist file with a new set of options, temperature, or library corners or, in the case of the PrimeSim XA simulation engine, a new PrimeSim XA configuration file.

This operation allows you to make changes to the PrimeSim XA configuration or to the SPICE netlist (and embedded options) and pass the updated information to the PrimeSim XA, FineSim, or PrimeSim simulation engines. This command only applies to analog netlists and the PrimeSim XA configuration file. This command cannot apply changes to VCS command-line options or changes to the digital netlist.

You cannot make structural changes to the analog netlist, such as changing instances, instance names, port names, and the number of cell ports. The command supports only modifications that do not affect the structure of the analog netlist, such as changes to temperature, SPICE options, accuracy settings, and component values such as resistor, capacitor, or supply voltage and current values.

The syntax is:

```
ace reread [-c config_file] [-ad control_file] [-o output_file_name]
-op_optimize 0|1
```

Options	Description
-c config_file	Specifies a new PrimeSim XA configuration file or a file that contains additional commands for FineSim or PrimeSim.
-ad control_file	Specifies a new mixed-signal simulation control file. Only the hiz attributes of e2r and r2e interface elements can be overridden. All other commands in the file are ignored.
-o output_file_name	The restored waveform file and log file are prefixed with the specified output file name.
-op_optimize	The tool loads the operating point generated at the time point when ace reread is executed. 0 (default): flattens the hierarchical netlist and loads the IC file automatically generated by ace reread. 1: preserves netlist hierarchy while loading initial conditions (ICs) for performance and capacity on designs with many levels of hierarchy.

Changes Allowed in the Configuration File

This capability is limited to mixed-signal simulation with the PrimeSim XA simulation engine. The same limitations that apply in the case of OP-based save and restore in the PrimeSim XA simulation engine apply here as well. Those limitations mean that you can change the `set_sim_level`, `set_tolerance_level`, `set_ccap_level`, and `set_model_level` accuracy commands and use the `reread` command to apply the changes.

Changes in the SPICE Files Allowed With the `ace reread` Command

You can change simple resistance or capacitance only if it does not change the interface signals. You can also:

- Change the temperature
- Change the SPICE stimuli
- Add or remove probes

You cannot change the structure of the netlist such as changing the cell instantiations in any way; such as the instance name, the number of ports or names of ports, or their connectivity.

Postlayout Simulation Through Back-Annotation

Mixed-signal simulation supports back-annotation with DSPF and SPEF files for analog nodes and nodes at the analog/digital boundary. Mixed-signal simulation also supports back-annotation of digital blocks with SDF files, but mixed nets are back-annotated with some limitations. See [SDF Annotation at the Mixed-Signal Boundary](#) for more information.

Note that Verilog modules instantiated within a back-annotated analog block cannot be back-annotated by using DSPF and SPEF files; you must use the SDF files to back-annotate those Verilog modules separately. A SPICE subcircuit that is instantiated with a Verilog module is not affected by the SDF files. You must use the HSPF/HSPEF files to back-annotate those SPICE subcircuits.

To simulate donut-configured netlists with back-annotation, DSPF and SPEF or HSPF and HSPEF files must be used for the SPICE representations. The file type depends on whether the SPICE block is the top level or at lower levels. Use the SDF files to back-annotate the Verilog modules.

By default, SDF delays on D2D through-nets are annotated on SPICE parent netlists. Use the VCS elaboration option, `+msvsdf`, to disable this annotation as follows:

```
vcs options -ad +msvsdf ...
```

By default, SDF delays on A2D mixed-signal nets are enabled on SPICE parent netlists. Use either of the VCS elaboration options, `+msvsdf` or `+msvsdfext`, to disable this annotation as follows:

```
vcs options -ad +msvsdf ...
```

or:

```
vcs options -ad +msvsdfext ...
```

Using an SDF File

SDF back-annotation in a mixed-signal simulation is performed in the same way as in a pure-digital, VCS simulation by using the `$sdf_annotate` system task to specify SDF files to be applied to a Verilog module.

In the following example, the `$sdf_annotate` system task is called inside the `initial` block in the `my_back_annotation` module and loads the `dut.sdf` SDF file from the current directory. The `top` argument to the `$sdf_annotate` system task identifies the Verilog block on which to apply the SDF back-annotation.

```
module my_back_annotation();  
    initial $sdf_annotate("./dut.sdf",top);  
endmodule
```

In the following example, the `$sdf_annotate` system task is specified inside the `initial` block in the `top` module. Note that the `$sdf_annotate` system task specifies only one argument ("`./dut.sdf`"). In this case, the back-annotation data is applied to the current module (`top`).

```
module top ( );  
    initial $sdf_annotate ("./dut.sdf");  
endmodule
```

The *VCS® User Guide* contains more information on SDF annotation and the `$sdf_annotate` system function.

SDF Annotation at the Mixed-Signal Boundary

SDF annotation across the mixed-signal boundary is also supported with the following restrictions:

- SDF delay defined for nets that cross the mixed-signal boundary must be of type `INTERCONNECT` and must be defined from the output port of one module to the input port of another.
- If a net has defined SDF back-annotation delay and the net crosses the mixed-signal boundary from digital to analog (or vice versa), the SDF back-annotation delay is applied to the digital side.
- There must only be one driver. Annotating a bidirectional interface with SDF is not recommended.
- Multiple loads are allowed. The delay is annotated from the driver to each load.
- If a net that connects two SPICE blocks is instantiated within a digital block (A2A through-net), SDF is not annotated. Instead, the analog signals tunnel together and one SPICE node is formed.
- VHDL is not supported.

Meta-Encrypted SPICE Netlists in Mixed-Signal Design

SPICE subcircuits or models that have been encrypted by the PrimeSim HSPICE `metaencrypt` utility are supported in VCS PrimeSim AMS mixed-signal simulations. These encrypted SPICE entities can be used in simulations that use a HDL-top, SPICE-top, or donut configuration.

However, the encrypted SPICE subcircuits are treated as a black box. No block inside the encrypted SPICE can be mapped to Verilog or VHDL with the `use_verilog` or `use_vhdl` commands. If an encrypted SPICE block is instantiated at the analog/digital boundary, the tool inserts interface elements as if the block were not encrypted.

Resolving Instance Name Conflicts for Testbench Reuse

Naming restrictions in some SPICE simulators, such as PrimeSim HSPICE and Eldo, require that instance names for subcircuits begin with the letter `x`. In some cases, this restriction can cause problems when resolving instance names in a mixed-level simulation. Consider the following example which shows an equivalent Verilog view and SPICE view for module `dut`. Note that the instance names for `inv1`, `inv2`, and `inv3` are different between the two views.

Example 18 Verilog and SPICE Views for dut

```
// Verilog dut
module dut (out,clk);
    output out;
    input  clk;
    inv1 g1 (net1,clk);
    inv2 g2 (net2,net1);
    inv3 g3 (out,net2);
endmodule

* SPICE dut
.subckt dut out clk
xg1 net1 clk inv1
xg2 net2 net1 inv2
xg3 out net2 inv3
.ends
```

The conflict in instance names between the Verilog view and SPICE view creates problems when switching between the two views in a mixed-level simulation. In the following example, the testbench instantiates module `dut` and uses the `$hdl_xmr` system task to access node `top.g1.g2.Y`.

Example 19 Module Top Instantiates DUT

```
module top(out);
    output out;
    reg    clk, xmr;
    dut g1 (out,clk);
    initial begin
        $hdl_xmr("top.g1.g2.Y", "xmr", 1);
        ...
    end
```

As a result of the naming conflict, the mixed-signal simulation produces an error message when trying to resolve the `top.g1.g2.Y` node:

```
Warning-[HXSSI] Hdl_xmr:Source signal invalid
verilog/testbench.v, 6
Instance path: top
Source signal 'top.g1.g2.Y' could not be found in the design.
```

One solution to this problem is to modify the `$hdl_xmr` system task to use the SPICE instance name as follows:

```
$hdl_xmr("top.xg1.xg2.Y", "xmr", 1);
```

However, this requires keeping two or more copies of the testbench, one for digital simulation and one for mixed-signal simulation. To automatically resolve this naming conflict, use the `resolve_x_inst_prefix` enable mixed-signal command as described in this section.

Prerequisites

Note the following when preparing the design views:

- The testbench should run without error for the digital-only design hierarchy.
- Design hierarchies instantiated in an HDL testbench can be of different formats, such as mixed SPICE/HDL, but the hierarchies must be consistent using the same module/entity or SPICE instances and cell names.

Using `resolve_x_inst_prefix`

To automatically resolve instance name conflicts due to the `x` prefix in SPICE, use the `resolve_x_prefix` command in the `vcsAD.init` mixed-signal simulation control file as follows:

```
resolve_x_inst_prefix enable;
```

When you add this command to the file, the same testbench can be used without modification for digital-only and mixed-signal simulations. For the test case shown above, the same testbench and `$hdl_xmr` can be used for the design configured with either the Verilog or SPICE `dut` cell views.

```
initial begin $hdl_xmr("top.g1.g2.Y", "xmr", 1);
```

Note:

The `resolve_x_prefix enable;` command is not required for the 3rd-party SPICE format because 3rd-party instances do not require an `x` prefix.

Running Concurrent Mixed-Signal Simulations From the Same Compiled Code

When compiling a mixed-signal design, the tool creates a number of temporary files and directories. By default, the tool generates the `simv` simulation executable and a `simv.msv` mixed-signal report directory in the same location.

If you compile a design once and attempt to run multiple simulations in parallel from the same directory with the same configuration, the simulator tries to use the same output directory for the parallel simulations and fails. To avoid conflicts and perform multiple mixed-signal simulations in parallel, you must specify a unique output directory for each simulation result. This allows you to compile a mixed-signal design and run multiple simulations using different output directories. Alternatively, you can create separate directories, copy the simulation executable to the separate directories, and start the simulation in each directory to avoid conflicting output file paths.

The following sections describe the settings required to run concurrent mixed-signal simulations from the same directory and compiled code:

- [Enabling Concurrent Mixed-Signal Simulations](#)
- [Creating Separate Output Files During Concurrent Mixed-Signal Simulation](#)
- [Changing the Configuration File or SPICE Netlist File for Concurrent Mixed-Signal Simulation](#)

Enabling Concurrent Mixed-Signal Simulations

To enable concurrent mixed-signal simulations, specify the following `report_option` command in the `vcsAD.init` mixed-signal simulation control file:

```
report_option -msv_dir spice;
```

The `report_option` command directs the tool to generate local copies of the `simv.msv` directory. This prevents conflicts when multiple simulations point to the same compiled executable, allowing the simulation output to the `simv.msv` directory to avoid writing to the same file.

Creating Separate Output Files During Concurrent Mixed-Signal Simulation

In this step, you can run concurrent simulations at the same directory by pointing to the `simv` executable. All these simulations can run in parallel without conflicts. Each one creates a local copy of the equivalent `simv.msv` directory that contains all the mixed-signal report files, both the ones that were created at compilation time and those created at runtime.

In a mixed-signal simulation with the PrimeSim XA, FineSim, and PrimeSim simulation engines, you can use command-line options with the `simv` compiled simulation executable to run each simulation with specific command-line options and configuration files. The following example runs two mixed-signal simulations in parallel and specifies unique test settings, log files, UCLI control files, and SPICE configuration files for each simulation:

Example 20 Script to Run Two Mixed-Signal Simulations in Parallel

```
# Start first simulation in background
% ./simv \
  +TEST1 \
  -ucli \
  -do Vcs1.ucli \
  -l simv1.log \
  -ad_runopt=analogconfig:anlg_run_1.do &

# Start second simulation in foreground
```

```
% ./simv \  
+TEST2 \  
-ucli \  
-do Vcs2.ucli \  
-l simv2.log \  
-ad_runopt=analogconfig:anlg_run_2.do
```

Creating Configuration for PrimeSim XA

For a mixed-signal simulation with the PrimeSim XA simulation engine, the example uses the following two runtime files. Note that the `change_choose_arguments` command supports the same arguments as the `choose xa` command in the `vcsAD.init` mixed-signal simulation control file.

```
# anlg_run_1.do for PrimeSim XA  
change_choose_arguments  
-o ./xaRun1/anlg_dir_1 -c xa1.cfg  
  
# anlg_run_2.do for PrimeSim XA  
change_choose_arguments  
-o ./xaRun2/anlg_dir_2 -c xa2.cfg
```

Creating Configuration for FineSim

For a mixed-signal simulation with the FineSim simulation engine, the example uses the following two runtime files. Note that the `change_choose_arguments` command supports the same arguments as the `choose finesim` command in the `vcsAD.init` mixed-signal simulation control file.

```
# anlg_run_1.do for FineSim  
change_choose_arguments  
-o ./fsRun1/anlg_dir_1 -afile run1.spi  
  
# anlg_run_2.do for FineSim  
change_choose_arguments  
-o ./fsRun2/anlg_dir_2 -afile run2.spi
```

Creating Configuration for PrimeSim

For a mixed-signal simulation with the PrimeSim simulation engine, the example uses the following two runtime files. Note that the `change_choose_arguments` command supports the same arguments as the `choose primesim` command in the `vcsAD.init` mixed-signal simulation control file.

```
# anlg_run_1.do for PrimeSim  
change_choose_arguments  
-o ./psRun1/anlg_dir_1 -afile run1.spi  
  
# anlg_run_2.do for PrimeSim  
change_choose_arguments  
-o ./psRun2/anlg_dir_2 -afile run2.spi
```

Changing the Configuration File or SPICE Netlist File for Concurrent Mixed-Signal Simulation

In general, you can change PrimeSim XA commands for a concurrent simulation if they do not change the circuit structure. This means you can change commands such as `set_sim_level`, `set_model_level`, and `set_tolerance_level`. You cannot change commands such as `load_ba_file` because they may change the circuit structure.

Similarly, you cannot change the SPICE netlist for a concurrent simulation if the changes impact the circuit structure, such as: instance names, number of ports, and connectivity. All other changes are allowed, including:

- Change in temperature
- Change in SPICE stimuli
- Adding or removing probes
- Simulation time specified by `.tran` value

Controlling the Number of Time Points at the Analog-Digital Boundary

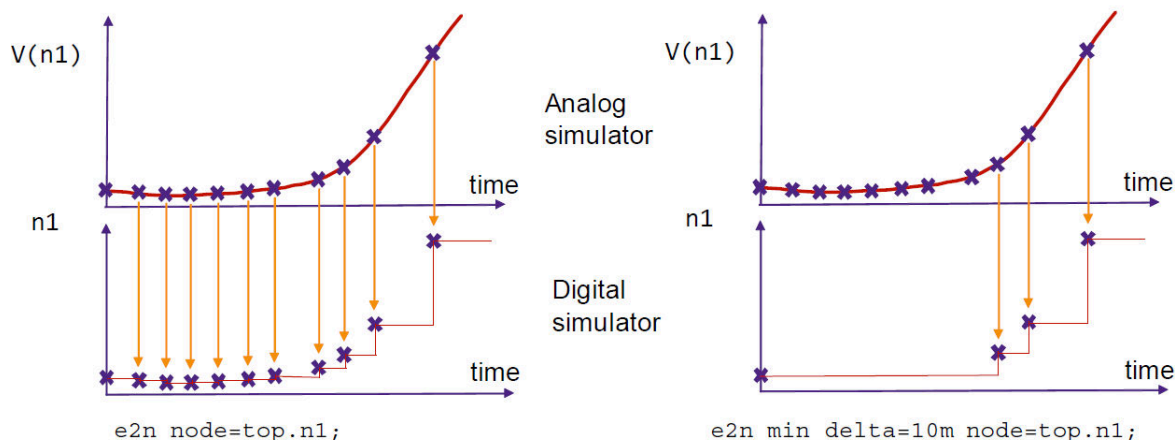
Mixed-signal simulation inserts interface elements (IEs) between nets with nettypes (on the digital side) and SPICE ports (on the analog side). A nettype-to-electrical IE (`n2e`) is inserted between a nettype output and an analog input. An electrical-to-nettype IE (`e2n`) is inserted between a nettype input and an analog output.

To tradeoff performance and accuracy, you can use the `-min_delta` and `-max_delta` arguments of the IE commands to control the number of simulation events passed between the digital and analog simulation domains. For example, the analog simulator might produce only a small change in voltage for a given node over time. To improve simulation performance, you might want to filter these small changes and send events to the digital domain only after the analog value changes by a large amount. You can use the `-min_delta` argument of the `e2n` command to set this filter value.

In the following example, the minimum delta is set to 10mV. Signal changes less than 10mV do not generate an event in the digital simulator.

```
e2n -min_delta 10m -node top.n1;
```

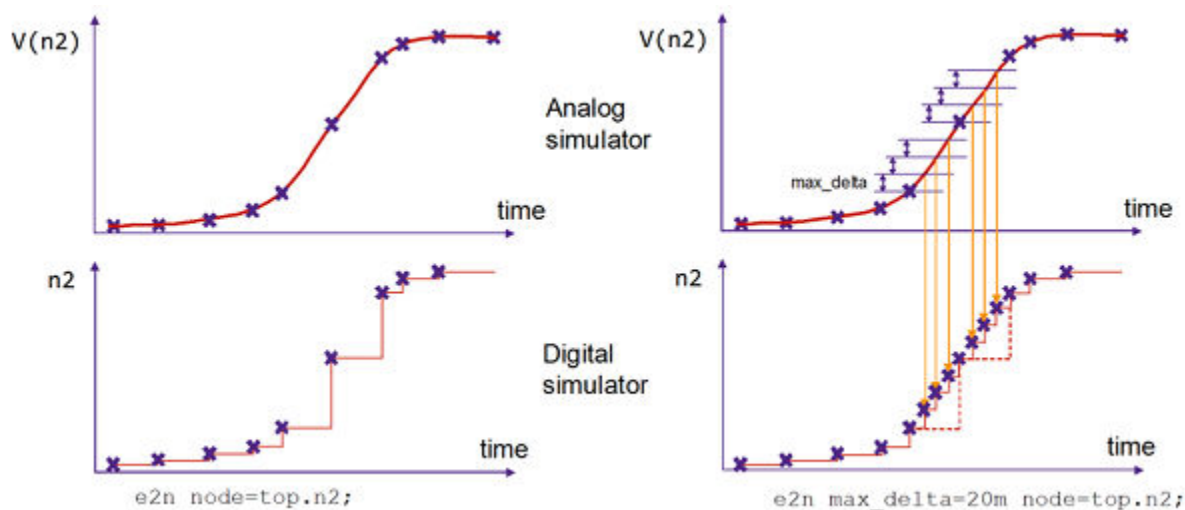
Chapter 3: Using Advanced Features in Mixed-Signal Simulation Controlling the Number of Time Points at the Analog-Digital Boundary



It is also possible that the analog engine might need to generate more events for the digital simulator to accurately record large changes in voltage. You can use the `-max_delta` argument of the `e2n` command to set the maximum change in voltage that triggers a simulation event. The analog simulator does not take an iteration during these extra events, so the analog simulator still runs as efficiently as possible. It calculates the additional events and sends the data to the digital simulator.

In the following example, the maximum delta is set to 20mV, meaning additional events every 20mV are generated and sent to the digital engine.

```
e2n -max_delta 20m -node top.n2;
```

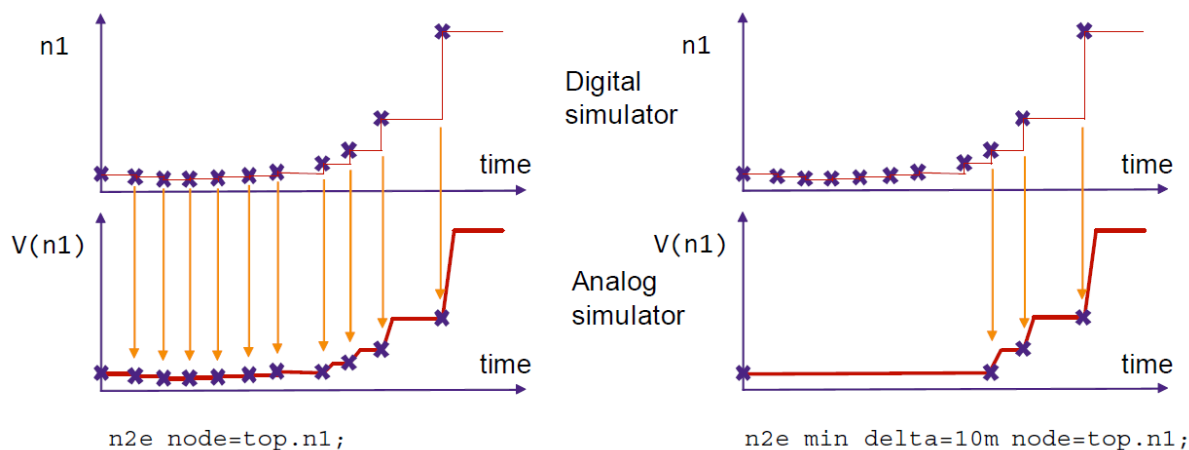


The digital simulator might take a lot of events, but when these events are sent to the analog simulator, the value at the boundary is changing very little. For performance reasons, you might want to filter these small value change events and have an event sent

to the digital simulator only after the analog values change by a large enough amount. You can use the `-min_delta` argument of the `n2e` command to set this filter value.

In the following example, the minimum delta is set to 10mV and signal changes less than 10mV do not generate an event in the digital simulator.

```
n2e -min_delta 10m -node top.n1;
```



Running Monte Carlo Analysis in Mixed-Signal Simulation

Monte Carlo analysis is a common method used to understand the effect of random process variations on circuit performance. During a Monte Carlo analysis, the simulation is run many times with small random variations in design and process variables. The simulation results are combined and correlated to determine key factors affecting performance.

Note:

The number of PrimeSim Continuum license tokens used during a mixed-signal simulation depends on the number of processes and follows the same calculation method as the PrimeSim XA, FineSim, and PrimeSim standalone tools. The VCS tool requires one license per Monte Carlo simulation, regardless of the VCS PrimeSim AMS configuration.

The following sections describe how to use the Monte Carlo feature in a mixed-signal simulation:

- [Running Sequential Monte Carlo Analysis](#)
- [Running Distributed Monte Carlo Analysis](#)

VCS PrimeSim AMS Monte Carlo analysis is supported as described in the user documentation for the PrimeSim XA, FineSim, and PrimeSim standalone tools.

Running Sequential Monte Carlo Analysis

Sequential Monte Carlo analysis runs on a single machine. In the SPICE netlist, a `.TRAN` statement defines the parameters for the Monte Carlo simulation:

```
.TRAN ... SWEEP Monte=val | parameter [firstrun=firstnum]
.TRAN ... SWEEP Monte=list(list_specification)
```

To run Monte Carlo analysis in a mixed-signal simulation, add the `SWEEP MONTE` keywords to the `.TRAN` statement and run the simulation using the same command you use for a non Monte Carlo analysis. The `simv` compiled simulation executable automatically launches a simulation run for each Monte Carlo iteration.

Output Files and Directories for PrimeSim XA

For a sequential Monte Carlo run, the output file and directory structure follow the PrimeSim XA tool output pattern with an added directory level. For example, if you specify `-o output/xa` in the `vcsAD.init` mixed-signal simulation control file, the simulator saves each Monte Carlo run to `output/mci/xa`, where *i* is the Monte Carlo iteration number.

For example, assuming that the simulation starts in the `rundir` directory, do the following:

1. Specify the following mixed-signal simulation control file:

```
// vcsAD.init
choose xa -o results/xa ...
```

2. Use the following command line to start the simulation:

```
% vcs ... -o simv -l compile.log
```

The tool creates the following files and directory structure:

```
rundir/
    simv, simv.daidir/, simv.msv/, csrc/, etc (VCS and MSV
    compile and run time directories and files)
    vcs.fsdb, vcs.vpd (VCS waveform files)

rundir/results/
    xa.pvadir/, xa.fsdb.grp, xa.log, xa.mc, xa.mc.csv,
    xa.mc_params, xa.meas, xa.valog (XA files)

rundir/results/mc1/
    xa.pvadir/, xa.m1.fsdb, xa.m1.meas, xa.ma, xa.valog
    (Iteration-specific XA files)

rundir/results/mc2/
    xa.pvadir/, xa.m2.fsdb, xa.m2.meas, xa.ma, xa.valog
    (Iteration-specific XA files)

...
rundir/results/mcn/
```


All VCS tool-generated compile and runtime files are placed in the run directory and are overwritten with each compile or run.

If waveforms are output from UCLI (FSDB or VPD), then VCS waveform files are added with a suffix of `m$i` and are generated inside the iteration-specific directories.

After simulation, the file and directory structure is:

```
rundir/  
    simv, simv.daidir/, simv.msv/, csrc/, etc (VCS and MSV  
    compile and run time directories and files)  
  
rundir/results/  
    xa.pvadir/, xa.fsdb.grp, xa.log, xa.mc, xa.mc.csv,  
    xa.mc_params, xa.meas, xa.valog (XA files)  
  
rundir/results/mc1/  
    xa.pvadir/, vcs.m1.fsdb, xa.m1.fsdb, xa.m1.meas, xa.ma,  
    xa.valog (Iteration-specific XA files & VCS waveform  
    files)  
  
rundir/results/mc2/  
    xa.pvadir/, vcs.m2.fsdb, xa.m2.fsdb, xa.m2.meas, xa.ma,  
    a.valog (Iteration-specific XA files & VCS waveform files)  
  
...  
rundir/results/mcn/
```

The following summary files are created by combining the results from all Monte Carlo iterations:

- Append one-by-one
 - `xa.log`
 - `xa.meas/mt` (if it exists)
 - `xa.mc_param` (if it exists)
- Statistical summary
 - `xa.mc` (if it exists)

Output Files and Directories for FineSim

For a sequential Monte Carlo run for the FineSim tool, the output file and directory structure follows the FineSim tool output pattern.

The FineSim tool does not create Monte Carlo iteration-specific directories. Instead, the waveform file name is appended with a `_si` suffix, where *i* is the Monte Carlo iteration number. For example:

```
fs.fsdb -> fs_s1.fsdb, fs_s2.fsdb
```

For example, assuming that the simulation starts in the `rundir` directory, do the following:

1. Specify the following mixed-signal simulation control file:

```
// vcsAD.init
choose finesim -o results/fs ...
```

2. Use the following command line to start the simulation:

```
% vcs ... -o simv -l compile.log
```

The simulation creates the following files and directory structure:

```
rundir/
  simv, simv.daidir/, simv.msv/, csrc/, etc (VCS and MSV
  compile and run time directories and files) vcs.fsdb,
  vcs.vpd (VCS waveform files)

rundir/results/
  fs.pvadir/, fs.ic, fs.im, fs.log, fs.mt0, fs.valog,
  fs_s1.fsdb , fs_s2.fsdb,..., fs_sn.fsdb, etc (FineSim files)
```

All VCS tool-generated compile and runtime files are placed in the run directory and are overwritten with each compile or run.

If waveforms are dumped from UCLI (FSDB or VPD), then VCS waveform files are added with a suffix of `m$i` to make the result unique.

After simulation, the file and directory structure is:

```
rundir/
  simv, simv.daidir/, simv.msv/, csrc/, etc (VCS and MSV
  compile and run time directories and files)

rundir/results/
  fs.pvadir/, fs.ic, fs.im, fs.log, fs.mt0, fs.valog,
  fs_s1.fsdb , vcs.m1.fsdb, fs_s2.fsdb, vcs.m2.fsdb, ...,
  fs_sn.fsdb, vcs.mn.fsdb, etc (FS files & VCS waveform
  files)
```

The following summary files are created by combining the results from all Monte Carlo iterations:

- Append one by one
 - `fs.log`
 - `fs.mcpt0` (if it exists)
- Statistical summary
 - `fs.mt0` (if it exists)

Output Files and Directories for PrimeSim

For a sequential Monte Carlo run for the PrimeSim tool, the output file and directory structure follows the PrimeSim tool output pattern.

The PrimeSim tool does not create Monte Carlo iteration-specific directories. Instead, the waveform file name is appended with a `_si` suffix, where `i` is the Monte Carlo iteration number. For example:

```
ps.fsdb -> ps_s1.fsdb, ps_s2.fsdb
```

For example, assuming that the simulation starts in the `rundir` directory, do the following:

1. Specify the following mixed-signal simulation control file:

```
// vcsAD.init
choose primesim -o results/ps ...
```

2. Use the following command line to start the simulation:

```
% vcs ... -o simv -l compile.log
```

The simulation creates the following files and directory structure:

```
rundir/
    simv, simv.daidir/, simv.msv/, csrc/, etc (VCS and MSV
    compile and run time directories and files) vcs.fsdb,
    vcs.vpd (VCS waveform files)

rundir/results/
    ps.pvadir/, ps.ic, ps.im, ps.log, ps.mt0, ps.valog,
    ps_s1.fsdb , ps_s2.fsdb,..., ps_sn.fsdb, etc (PrimeSim files)
```

All VCS tool-generated compile and runtime files are placed in the run directory and are overwritten with each compile or run.

If waveforms are dumped from UCLI (FSDB or VPD), then VCS waveform files are added with a suffix of `m$i` to make the result unique.

After simulation, the file and directory structure is:

```
rundir/
    simv, simv.daidir/, simv.msv/, csrc/, etc (VCS and MSV
    compile and run time directories and files)

rundir/results/
    ps.pvadir/, ps.ic, ps.im, ps.log, ps.mt0, ps.valog,
    ps_s1.fsdb , vcs.m1.fsdb, ps_s2.fsdb, vcs.m2.fsdb, ...,
    ps_sn.fsdb, vcs.mn.fsdb, etc (PS files & VCS waveform
    files)
```

The following summary files are created by combining the results from all Monte Carlo iterations:

- Append one by one
 - `ps.log`
 - `ps.mcpt0` (if it exists)
- Statistical summary
 - `ps.mt0` (if it exists)

UCLI Support

UCLI scripts are supported in Monte Carlo analysis as:

```
% simv -ucli -do run.ucli
```

Note:

If the UCLI script does not exit the simulation, the simulation generates an error message and stops.

If you use Ctrl+C to interrupt the simulation, the tool exits. Be aware that this is different from the UCLI normal behavior.

The `simv -ucli` command (interactive mode) is available for debugging only:

- After running a suite of Monte Carlo simulations, you can identify an outlier by the seed number.
- You can then duplicate that run using:

```
.tran ... SWEEP Monte=1 firstrun=seed
```

For example. If `mc27` is an outlier, use:

```
.tran ... SWEEP Monte=1 firstrun=27  
simv -ucli ...
```

In the previous example, the simulator begins a single simulation replicating 27 and stopping at time 0.

You can then use interactive commands to investigate the simulation.

Limitations

- The save/restore capability is not supported.
- The `ucli2proc` UCLI option is not supported.
- VCS PrimeSim AMS Monte Carlo analysis using the FineSim or PrimeSim simulation engines does not generate any waveform group file (*.grp).

Running Distributed Monte Carlo Analysis

Monte Carlo analysis simulations can be distributed across a network of machines. As in sequential analysis, the `.TRAN` statement in the SPICE netlist defines the parameters for the Monte Carlo simulation:

```
.TRAN ... SWEEP Monte=val | parameter [firstrun=firstnum]
.TRAN ... SWEEP Monte=list(list_specification)
```

To distribute these simulations across a network, you must provide additional information to the job distributor through the `-ad_runopt` option to `simv`. For example:

```
simv -ad_runopt=analogconfig:dp:worker#:[dp_cfg_file]:[dp_location]
```

The parameter descriptions are shown in [Table 5](#).

Table 5 *Distributed Monte Carlo Parameters*

Parameter	Description
<code>dp</code>	Keyword to enable distributed Monte Carlo analysis.
<code>worker#</code>	Required parameter that specifies the number of worker CPUs available to process Monte Carlo iterations. Must be a number greater than 0.
<code>dp_cfg_file</code>	Optional user-defined file. If not specified, uses the local host.
<code>dp_location</code>	Optional, can be NFS or TMP.

CDPL distributed processing for the LSF and GRID compute farms is supported in the VCS PrimeSim AMS tool. CDPL supports the following protocols for launching workers on remote nodes/machines:

- RSH (Remote Shell)
- SSH (Secure Shell)
- SGE (Oracle Grid Engine, originally Sun Grid Engine)
- LSF (Load Sharing Facility, from Platform Computing)

- SH (shell process on the local host)
- PBS (Portable Batch System)
- RTDA (Runtime Design Automation NetworkComputer™)

Caution:

Make sure you run Monte Carlo jobs on compatible OS farm machines. It is risky to build the `simv` simulation executable on a machine with a new OS version, then run the Monte Carlo simulation jobs on machines with an older OS.

Running AC Analysis During Transient Analysis

The FineSim simulator supports the `finesim_ac_at_diff_t` command. This command can also be used in mixed-signal simulation.

The following example runs two AC analyses: one at 50ns during the transient analysis and another at 100ns during the transient analysis. Each AC analysis sweeps from 1kHz to 500kHz with 100 steps.

```
.option finesim_ac_at_diff_t=1
.tran 1p 200n
.op 50n 100n
.ac log 100 1k 500k
```

Note:

The portion of the circuit that is simulated in VCS does not linearize, so the AC analysis results of the digital portion are meaningless.

Running AC Analysis

The FineSim tool in SPICE mode (`spicead`, `spicehd`, `spicemd`, and `spicexd`), the PrimeSim Pro tool in SPICE mode (`spicehd`, `spicemd`, and `spicexd`), and the PrimeSim SPICE tool all support AC analysis. This analysis can also be used in mixed-signal simulation. Add the SPICE command `.OP 0` to create an operating point at time=0. There is no need to run a transient analysis.

Note:

The portion of the circuit that is simulated in VCS does not linearize, so the AC analysis results of the digital portion are meaningless.

Adding Netlist Commands to the Control File

In a mixed-signal simulation with the PrimeSim XA, FineSim, or PrimeSim simulation engines, you can enter simulation commands in the `vcsAD.init` mixed-signal simulation control file. These commands are passed to the simulation engines and can be used to provide additional control over the mixed-signal simulation.

For the FineSim simulation engine, use the `netlist_commands_begin` and `netlist_commands_end` keywords within the `vcsAD.init` mixed-signal simulation control file to create a block of commands to run. In the following example, netlist commands are used to specify the temperature.

Example 21 Netlist Commands for FineSim

```
spice_top;
bus_format [%d];
use_verilog -module addr4;
choose finesim top.spi -o results/fsim/fsim;
netlist_commands_begin;
.temp 45
netlist_commands_end;
```

For the PrimeSim simulation engine, use the `netlist_commands_begin` and `netlist_commands_end` keywords within the `vcsAD.init` mixed-signal simulation control file to create a block of commands to run. In the following example, netlist commands are used to specify the temperature.

Example 22 Netlist Commands for PrimeSim

```
spice_top;
bus_format [%d];
use_verilog -module addr4;
choose primesim top.spi -o results/fsim/fsim;
netlist_commands_begin;
.temp 45
netlist_commands_end;
```

For the PrimeSim XA simulation engine, in addition to the `netlist_commands_begin` and `netlist_commands_end` keywords, you can also specify the `xa_commands_begin` and `xa_commands_end` keywords within the `vcsAD.init` mixed-signal simulation control file to create a block of commands to run.

Example 23 Netlist and Custom Commands for PrimeSim XA

```
choose xa -hspice test.spi -c xa.cfg;
use_spice -cell nand;
use_spice -cell xor;

netlist_commands_begin;
.temp 45
```

```
netlist_commands_end;  
xa_commands_begin;  
set_sim_level 5  
xa_commands_end;
```

See [netlist_commands_begin](#) and [netlist_commands_end](#) for more information.

Adding PrimeSim XA Commands at Runtime

In a mixed-signal simulation with the PrimeSim XA simulation engine, you can add PrimeSim XA commands at runtime by using the `include_xa_command` command before specifying PrimeSim XA commands in the runtime configuration file.

Example 24 PrimeSim XA Commands at Runtime

```
simv -ad_runopt=analogconfig:xa.runcfg -l simv.log
```

where `xa.runcfg` has the following lines:

```
include_xa_command  
set_sim_level 5  
set_model_level 6
```

Skipping Analog Simulation for Specific Time Windows

In a mixed-signal simulation using the PrimeSim XA, FineSim, or PrimeSim simulation engines, you can use the `skip_analog` command to reduce the simulation time by skipping or ignoring the analog simulation for certain time windows, provided you are sure that analog results are not needed during that time.

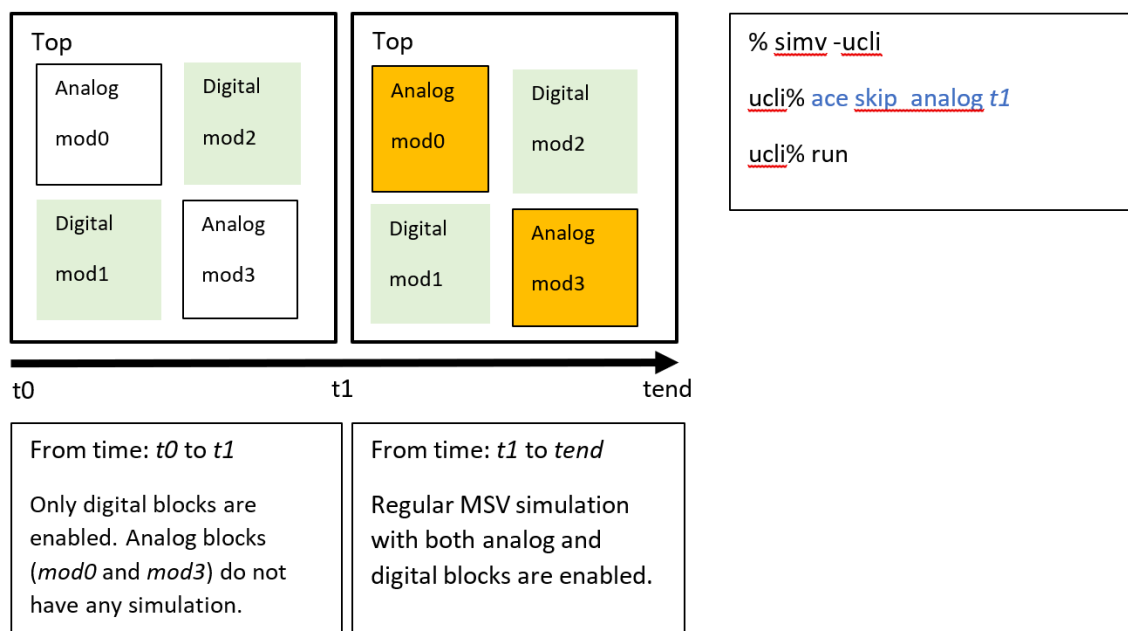
This feature is generally useful when you want to skip the long initialization time needed for the analog blocks in a mixed-signal design.

Note:

The analog simulation results (probing and so on) during the skip time window are impacted by this feature.

[Figure 19](#) illustrates an example of using the `skip_analog` command.

Figure 19 Skip Analog Example



In the example shown in Figure 19, all of the analog simulation and interface events (d2a, a2d, and so on) are skipped during the first window from t_0 to t_1 . However, the last digital-to-analog events (such as d2a) are recorded and applied right before $t=t_1$ as a DCOP (DC Operation) when the analog simulation starts. This ensures that the analog simulation gets the updated state from the digital simulation.

If you provide initial conditions for these analog nodes, the states and d2a events should be consistent. If there is any inconsistency, then the d2a events prevail because the d2a events are treated as a DCOP at $t=t_1$, which can trigger additional DC state updates from the interface nets to the affected analog internal nodes, and which is automatically taken care of by the simulator as a DCOP right before $t=t_1$.

There should be no a2d events in the first window $[t_0, t_1]$. A warning message is issued if there are such events and the events are ignored.

Note:

When the `skip_analog` command is issued at time 0, it automatically performs a DCOP even without any run at time 0. If `skip_analog` is issued at a non-zero time (for example, 10ns), it does not perform any DCOP at that time instant. When the analog simulator wakes up after the `skip_analog` window, a DCOP is performed automatically to initialize the analog nodes.

Specify the `skip_analog` command in UCLI mode to use this feature.

Syntax

```
ucli% ace skip_analog time_value
```

where, the default *time_value* unit is seconds.

The `skip_analog` command enables the analog engine to skip or ignore the analog simulation for the next *time_value*. If this command is specified before any run commands, it skips the first time window for *time_value*.

The `skip_analog` command takes effect from the current simulation time. There is only one active `skip_analog` session at any moment in the simulation. If the duration of two `skip_analog` commands overlap, the second `skip_analog` command overrides the first one that is, the most recent `skip_analog` command takes effect.

Example 1

The following example shows how analog simulation can be skipped during the initialization phase of the simulation.

```
% simv -ucli
ucli% ace skip_analog 200ns
ucli% run 300ns
```

In this example, the first UCLI command sets the skip analog period as $t=0$ to $t=200\text{ns}$. This sets the skip window only; it does not trigger the run.

The second command launches the run for 300ns, of which the first 200ns of analog simulation is skipped (as set in the previous command), then the regular mixed-signal simulation runs from $t=200\text{ns}$ to $t=300\text{ns}$.

Example 2

The following example shows how analog simulation can be skipped for multiple time windows.

```
% simv -ucli
ucli% ace skip_analog 200ns
ucli% run 300ns
ucli% ace skip_analog 100ns
ucli% run
```

In this example, the first UCLI command sets the skip analog period as $t=0$ to $t=200\text{ns}$. This sets the skip window only; it does not trigger the run.

The second command launches the run for 300ns, of which the first 200ns of analog simulation is skipped (as set in the previous command), then the regular mixed-signal simulation runs from $t=200\text{ns}$ to $t=300\text{ns}$.

The third UCLI command sets the skip analog period for the next 100ns, which sets the skip window from $t=300\text{ns}$ to $t=400\text{ns}$.

The last command runs the simulation from t=300ns to the end, of which the first 100ns of analog simulation is skipped for the window from t=300ns to t=400ns (as set in the previous command), and then regular mixed-signal simulation runs from 400ns to the end.

Example 3

The following example shows how analog simulation can be skipped during the initialization phase of the simulation and how you can choose to provide an initial condition after the skipped time.

```
% simv -ucli
ucli% ace skip_analog 200ns
ucli% run 200ns
ucli% ace reread -c rereadFile.cfg
ucli% run
```

In this example, the first UCLI command sets the skip analog period as t=0 to t=200ns. This sets the skip window only; it does not trigger the run.

The second command launches the run for 200ns, in which the analog simulation is skipped for t=0ns to t=200ns (as set in the previous command).

The third command reads in a new configuration file which has a PrimeSim XA, FineSim, or PrimeSim simulation engine-specific command to load the initial conditions at t=200ns.

PrimeSim XA:

```
load_operating_point -file "xa_200ns.ic" -type ic
```

FineSim:

```
.load file="fs_200ns.ic"
```

PrimeSim:

```
.load file="ps_200ns.ic"
```

The `xa_200ns.ic`, `fs_200ns.ic`, and `ps_200ns.ic` files contain initial conditions for the analog nodes at 200ns. Because there is no analog simulation during the initial phase, you can also choose to skip the DCOP at t=0ns by using the following.

PrimeSim XA configuration command specified in the `xa.cfg` file:

```
set_dc_option -skip_dc 1
```

FineSim option specified in the mixed-signal control file:

```
choose finesim -skipdc addr4.sp;
```

PrimeSim option specified in the mixed-signal control file:

```
choose primesim -skipdc addr4.sp;
```

The last command runs the simulation as usual from $t=200\text{ns}$ to the end.

Running Transient Noise Analysis

The PrimeSim and FineSim simulators, in SPICE mode, support large signal noise analysis in the time domain, see the *PrimeSim User Guide* for more details. Additional arguments to the `.tran` Spice command, such as `fmax` and `fmin` enable transient noise analysis. This analysis is also supported in mixed-signal simulation.

Note:

- Only the analog portion of the circuit is effected by the transient noise parameters, the digital portion of the circuit is not effected by transient noise.
- This analysis requires the use of the `.tran` Spice command. The actual end time of the simulation can be set in the HDL testbench or with a UCLI command.

4

Saving and Restoring Simulation State

In a VCS PrimeSim AMS mixed-signal simulation, you can save the state of the simulation at any given time point and then restore it at a later time to continue the simulation. When the simulation is restored, the simulation image is read from disk and copied into memory. This feature can be used in different situations:

- A simulation is running for a long time and must be interrupted. Instead of losing the simulation data, you can save the state of the simulation up to the current time point and restore it later.
- A simulation has a long initialization sequence. Instead of running the initialization sequence each time, you can complete the initialization sequence during the first simulation, save the state, and use the saved state for subsequent simulations.

Another application of this feature is when multiple tests are run on the same design (each with a different set of input stimulus for example) and all those tests share the same power-on or initialization phase (for example, they all wait for PLLs to lock). You can use save and restore to run one of the tests up to the point where the initialization phase is complete and save the state. For each test, you can restore the simulation, set unique parameters and input vectors, and continue the simulation. Note that the testbench must be designed to allow the selection of different tests after restore, for example, by forcing the content of an RTL variable that selects between tests to a different value after each restore.

This chapter describes:

- [Basic Save and Restore Usage](#)
- [Running Multiple Simulations With Save and Restore](#)

Basic Save and Restore Usage

The save and restore feature in the VCS PrimeSim AMS tool is based on the VCS save and restore mechanism. In VCS, these operations can be performed by using the `save` and `restore` commands in the Unified Command-Line Interface (UCLI) as follows:

```
ucli% save checkpoint_name
ucli% restore checkpoint_name
```

You can use these commands to capture and save the state of the simulation at a given timepoint and quit the simulation.

Note:

If you save a checkpoint with the `save` command and restore the checkpoint with the `restore` command at a later time on a different machine, you should try to match the CPU type and OS patch level between the machines. Otherwise you might encounter compatibility issues.

The following example starts the compiled simulation in the UCLI interactive mode by using the `-ucli` command-line option, runs the simulation for 100ns, saves an image of the simulation to `sim_state`, and exits the simulation.

```
% simv -ucli
ucli% run 100ns
ucli% save sim_state_100ns
ucli% quit
```

You can restore the saved state of the simulation at a later time. The following example starts the compiled simulator in interactive mode, loads the saved simulation state, and continues the simulation:

```
% simv -ucli
ucli% restore sim_state_100ns
ucli% run
```

Running Multiple Simulations With Save and Restore

You can use the save and restore feature in the VCS PrimeSim AMS tool to quickly advance the simulation to a given timepoint and continue the simulation. This is useful for a simulation that includes multiple tests that share a long initialization phase, such as for phase-locked loop synchronization or a long power-on reset sequence. By using the save feature, you can run a simulation to complete the initialization phase and save the simulation image at that timepoint for later use. Subsequent simulations can restore the saved state and continue the simulation. This methodology eliminates the simulation time required for initialization for all simulations except the first simulation.

To run the testbench with different tests after the restore operation, the testbench must be written to run the tests based on a control signal or variable. Using the UCLI, you can force the value of the signal or variable to select the test before resuming the simulation.

Here is an example Verilog testbench that allows multiple tests to be run. In this example, the value the `tst` variable is used to select different tests:

```
module tb;
  int tst;
  ...
```

Chapter 4: Saving and Restoring Simulation State

Running Multiple Simulations With Save and Restore

```

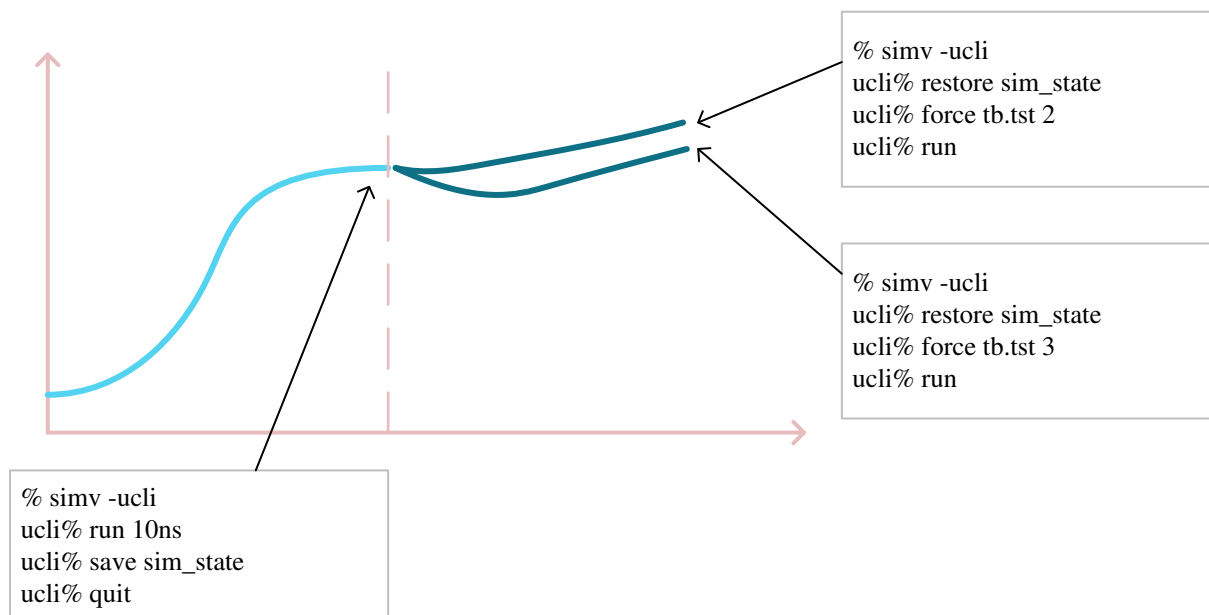
case (tst)
  1: test1( ); // Run test No 1
  2: test2( ); // Run test No 2
  3: test3( ); // Run test No 3
  4: test4( ); // Run test No 4
  5: test5( ); // Run test No 5
  default: tst0 ( );
endcase
...
end

```

The following example demonstrates this technique for running different tests based on a variable setting. The `simv -ucli` command starts the simulation in UCLI interactive mode. The simulation is run for 10ns and the simulation image is saved to `sim_state`. The saved image is used in two subsequent simulations, where each simulation runs a different part of the testbench. The second simulation restores the saved image and forces the `tb.tst` variable to 2 in order to select test number 2. The third simulation forces the variable to 3 to select test number 3.

As a result, each simulation runs a different test as shown in [Figure 20](#).

Figure 20 Save and Restore Example



The save and restore feature supports the following flows:

- Compile, save, and restore the simulation in the same directory
- Compile and save the simulation in the same directory and restore from different directories
- Compile in one directory, save the simulation in a different directory, and restore in some different directories

Note:

Parallel restore in the same directory is not supported.

5

Cross Module Reference

This chapter describes the different methods for reading and driving analog signals from the digital domain. These methods work in the PrimeSim XA, FineSim, and PrimeSim mixed-signal solutions.

- [Accessing, Forcing, and Releasing Analog Signals From the Digital Domain](#)
- [Accessing SPICE Nodes Through the Verilog Language XMR](#)
- [Accessing SPICE Nodes Through hdl_xmr, hdl_xmr_force, and hdl_xmr_release](#)
- [Accessing SPICE Nodes Through Proprietary System Tasks and Functions](#)
- [Accessing SPICE Nodes Through UCLI XMRs](#)
- [Using Bus Notation With an XMR Task](#)
- [Using form_spice_bus for SPICE Ports Appearing in the Form of a Bus](#)

Accessing, Forcing, and Releasing Analog Signals From the Digital Domain

- *Verilog XMR*: One method is through the Verilog language cross module referencing (XMR). In this method, you can use the Verilog language XMR to access a SPICE signal anywhere in the design hierarchy for read or drive access. [Accessing SPICE Nodes Through the Verilog Language XMR](#) describes this method in further detail.
- *hdl_xmr_force and hdl_xmr_release*: Because the VHDL language does not support the native XMR capability that is supported by the Verilog language, the VCS tool provides an alternative method through the use of system tasks `$hdl_xmr_force` and `$hdl_xmr_release` in Verilog, and `hdl_xmr_force` and `hdl_xmr_release` procedures in VHDL. These Verilog system tasks and VHDL procedures allow the Verilog and VHDL simulators to access signals across block and language boundaries. These system tasks and procedures also allow access to signals in the SPICE domain.

The capabilities in `hdl_xmr` can be viewed as equivalent to the capabilities offered by XMR in the Verilog simulation. This technique is recommended for cross-module references where the design contains VHDL blocks that can be replaced by SPICE.

[Accessing SPICE Nodes Through hdl_xmr, hdl_xmr_force, and hdl_xmr_release](#) and [\\$hdl_xmr System Task and hdl_xmr\(\) Procedure](#) describe this method in detail.

- *snps_get_* commands*: A third way accesses SPICE nodes for read or force operations through a family of Verilog system tasks and functions. These system tasks and functions allow Verilog to sample SPICE node voltages and port currents as real values, force a specific voltage value on a SPICE node, or sense if a node voltage has crossed a threshold value. There are a few differences between this method of accessing SPICE nodes and using XMR and `hdl_xmr` in Verilog to access the nodes.

One of the functions (`$snps_get_port_current`) allows sampling of a current, which the other two methods do not provide. In addition, the `$snps_get_volt` and `$snps_get_port_current` system tasks read a SPICE voltage or SPICE port current values at the time the system tasks are called. But in the other two methods perform sampling on an event-basis and provide up-to-date information each time the value of the source changes. [Accessing SPICE Nodes Through Proprietary System Tasks and Functions](#) provides more detail about this method.

- *UCLI force and release*: The fourth method of accessing SPICE nodes is by using UCLI `force` and `release` commands when running the simulation in UCLI mode. The capabilities in `force` and `release` can be viewed as the equivalent to the capabilities offered by the `$hdl_xmr_force` and `$hdl_xmr_release` system tasks in Verilog and the `hdl_xmr_force` and `hdl_xmr_release` procedures in VHDL. [Accessing SPICE Nodes Through UCLI XMRs](#) describes this method in detail.

Accessing SPICE Nodes Through the Verilog Language XMR

In this method, the Verilog code treats an internal SPICE node as a logic value for read or write operations. The same XMR principles used to access a digital net must be used to access an analog net by providing a [Full Hierarchical Path](#) to the net.

Mixed-signal simulation inserts a2d or d2a interface elements automatically, depending on whether Verilog is reading from or writing to the internal analog node. The interface elements are subject to the same rules used for conventional interface nets, including resistance map lookup. a2d and d2a interface elements are reported in the `simv.msv/interface_element.rpt` file along with all other interface elements. To change the default settings for the interface elements inserted for Verilog language XMR, use the [a2d](#) and [d2a](#) mixed-signal commands in the `vcsAD.init` mixed-signal simulation control file.

The following example performs a Verilog language XMR read on an analog node with the hierarchical path `top.i1.i2.x1.clk` into a Verilog wire. It assigns the logic value corresponding to the voltage of a SPICE node.

```
assign verilog_wire = top.i1.i2.x1.clk;
```

The following example performs a Verilog language XMR read on a SPICE node with the hierarchical path `top.i1.i2.x1.strb` into a Verilog register. It assigns the logic value corresponding to the voltage of a SPICE node.

```
initial begin
    ...
    verilog_reg = top.i1.i2.x1.strb;
    ...
end
```

The following example performs a Verilog language XMR write on a SPICE node. The d2a interface element inserted by mixed-signal simulation translates the logic values to voltage values and applies them to the SPICE node.

```
reg rst_reg;
assign top.i1.i2.x1.rst = rst_reg;

initial begin
    ...
    rst_reg = 1'b0;
    #5 rst_reg = 1'b1;
    ...
end
```

Because SPICE requires all buses to be split into individual signals, the bus signals in a SPICE netlist must be surrounded in braces (`{ }`) if the target of a Verilog language XMR is a SPICE bus. The following example shows a SPICE subcircuit with bus ports `a_2`, `a_1`, and `a_0` and how those SPICE bus members are accessed in a Verilog language XMR.

```
.subckt spice_blk a_2 a_1 a_0
```

The `a_2`, `a_1`, and `a_0` SPICE bus ports are accessed in an assign statement that uses Verilog language XMR as follows:

```
wire [2:0] verilog_wire;
assign verilog_wire = {top.x1.a_2, top.x1.a_1, top.x1.a_0};
```

In this example, `verilog_wire` is a Verilog bus. Note that braces (`{ }`) are used along with the full hierarchical paths to the SPICE bus ports to assign a vector composed of SPICE bus signals `a_2`, `a_1`, and `a_0`.

Accessing SPICE Nodes Through `hdl_xmr`, `hdl_xmr_force`, and `hdl_xmr_release`

This section describes force and release functionality for mixed HDL/SPICE cross-module references (XMRs). The VCS tool allows forcing or releasing any VHDL signal or Verilog wire or register in a mixed HDL design or a mixed-signal design. You can use the

`hdl_xmr_force()` and `hdl_xmr_release()` VHDL procedures or the `$hdl_xmr_force()` and `$hdl_xmr_release()` Verilog system tasks to perform this function.

To achieve this goal in a mixed-signal simulation that includes VHDL and SPICE (and possibly Verilog) the `hdl_xmr_force()` procedure or the `$hdl_xmr_force` system task is the recommended method for forcing a value on any existing VHDL, Verilog, or SPICE node at any level of the design hierarchy.

The syntax for the `$hdl_xmr` system task, the `hdl_xmr()` procedure, the `$hdl_xmr_force` system task, the `hdl_xmr_force` procedure, the `$hdl_xmr_release` system task, and the `hdl_xmr_release` procedure is discussed in the next section.

\$hdl_xmr System Task and hdl_xmr() Procedure

Description

Use the `$hdl_xmr()` Verilog system task or `hdl_xmr()` procedure to connect two signals specified in the argument list. With each value change, the value of the *source_object* is transferred to the *destination_object*.

Syntax

```
$hdl_xmr(source_object, destination_object, [verbose]);
hdl_xmr (source_object, destination_object, [verbose]);
```

Arguments

Argument	Description
<i>source_object</i>	Specifies the hierarchical path to the source object as a string. <i>source_object</i> can be a Full Hierarchical Path or relative path to an existing VHDL signal, Verilog register/net, or SPICE node. Relative paths are specified with respect to the calling block. The path can contain any hierarchy, including a SPICE hierarchy.
<i>destination_object</i>	Specifies the hierarchical path to the destination object as a string. <i>destination_object</i> can include the SPICE hierarchy, but cannot be a SPICE node as the PrimeSim XA simulation engine does not support deposit. You can specify a Full Hierarchical Path or relative path to an existing VHDL signal, Verilog register, SystemVerilog (SV) register, or SystemVerilog <i>scalar</i> nettype including the Synopsys predefined nettypes <i>voltage_r</i> and <i>current_r</i> . Relative paths are specified with respect to the calling block.
<i>verbose</i>	Specifies an optional integer value for setting the message level. A value of 0 indicates no verbosity, and value 1 enables verbosity. When the value is 1, a message is displayed every time a value of the source object is copied onto the destination object.

\$hdl_xmr_force System Task and hdl_xmr_force() Procedure

Description

The `$hdl_xmr_force` system task and `hdl_xmr_force()` procedures force a value on any existing VHDL or Verilog node at any level of the design hierarchy.

Syntax

```
$hdl_xmr_force("destination_object", "value", ["rel_time"],
  ["force_type"], ["cancel_time"], [verbose]);
hdl_xmr_force("destination_object", "value", ["rel_time"],
  ["force_type"], ["cancel_time"], [verbose]);
```

Arguments

Argument	Description
<i>destination_object</i>	Specifies the absolute path or a relative path to the destination object. The destination object can be a VHDL signal, Verilog register, or SPICE net. The VCS tool issues a warning message if the <i>destination_object</i> does not exist or is not specified. Note: Verilog nets are read-only by default and by default cannot be used as destination objects. To specify Verilog nets as destination objects, turn on the write capability on these nets using a tab file.
<i>value</i>	Specifies the value to force the <i>destination_object</i>. The specified value must be appropriate for the type and can be specified in any valid syntax for any radix as supported by the language.
<i>rel_time</i>	Specifies a time relative to the current simulation time. The VCS tool forces the specified value on the specified <i>destination_object</i> at the specified time, plus the current time. Specify the relative time in the following format: <pre>rel_time [fs ps ns us ms]</pre> For example: <pre>\$hdl_xmr_force("top.dut.reset", "0", "4 ns", "freeze", "5 ns");</pre> If the specified time is less than simulation precision, then the force happens at the end of the current time step.

Argument	Description
<i>force_type</i>	Specifies how the value is applied to the net. The <i>force_type</i> must be one of the following: <i>freeze</i> (default): Forces the specified value on the specified <i>destination_object</i> at the specified time. Any drivers later in time have no impact on the <i>destination_object</i>. <i>deposit</i> : Forces the specified value on the specified <i>destination_object</i> at the specified time. Any driver later in time can override the forced value. Note: The <i>deposit</i> setting is not supported on SPICE nets.
<i>cancel_time</i>	Specifies a time relative to the current simulation time. The VCS tool cancels the forced value on the specified <i>destination_object</i> at the specified time, plus the current time. Specify the relative time in the following format: <pre>cancel_time [fs ps ns us ms]</pre> For example: <pre>\$hdl_xmr_force("top.dut.reset", "0", "4 ns","freeze", "5 ns");</pre> If the specified time is less than simulation precision, the forced value is canceled at the end of the current time step. The default behavior is to never cancel the forced value, and is specified by leaving this field blank. Note: <i>cancel_time</i> is not used when <i>force_type</i> is <i>deposit</i>. If specified, <i>cancel_time</i> is ignored.
<i>verbose</i>	Specifies the output message level to use. By default, the message level is 0. A value of 0 indicates no verbosity, and a value of 1 enables verbosity.

\$hdl_xmr_release System Task and hdl_xmr_release() Procedure

Use the `hdl_xmr_release()` procedure or `$hdl_xmr_release()` system task to release the force applied to an existing VHDL, Verilog, or SPICE destination.

Syntax

```
$hdl_xmr_release (destination_object, [verbose]);
hdl_xmr_release (destination_object, [verbose]);
```

Arguments

Argument	Description
<code>destination_object</code>	Specifies the absolute path or a relative path to the destination object. The destination object can be a VHDL signal, Verilog register, or SPICE net. The VCS tool issues a warning message if the <code>destination_object</code> does not exist or is not specified.
<code>verbose</code>	Specifies the output message level to use. By default, the message level is 0. A value of 0 indicates no verbosity, and a value of 1 enables verbosity.

Description

You can use the `hdl_xmr_release()` procedure or `$hdl_xmr_release()` system task to release any objects forced with `hdl_xmr_force()`, `$hdl_xmr_force()`, or the force commands in the design. `hdl_xmr_release` works the same as the Verilog release command.

These signal release functions can be called concurrently or sequentially from a VHDL architecture (for example, a testbench), a process, or from any Verilog scope where a system call can be made. After the signal is released using `hdl_xmr_release`, the expected behavior of the released node is:

- If the destination node is of type VHDL signal, and if there is no driver on the destination node, then the node is forced to 'U'.
- If the destination node is of type Verilog wire, and if there is no driver on the destination node, then the node is forced to 'Z'.
- If the destination node is of type Verilog reg, and if there is no driver on the destination node, then the node retains the current value until future evaluations.

Runtime interface elements, `rt_a2d`, `rt_d2a`, `rt_e2n`, `rt_e2r`, `rt_n2e` and `rt_r2e` are inserted automatically for conversion to and from SPICE with the `hdl_xmr`, `hdl_xmr_force` procedures or `$hdl_xmr`, `$hdl_xmr_force` system tasks. Note that runtime interface elements are not saved to the `interface_element.rpt` file, they are saved to the `runtime_interface_element.rpt` file. You can view the console log to verify that runtime elements were inserted during simulation.

The following example shows a `$hdl_xmr_force` of 0.33 on an analog target when the analog target has an equivalent HDL view in which the target node is defined as nettype.

The `vcsAD.init` contains the following `use_spice` command:

```
use_spice -cell inv;
```

Example 25 Verilog Code

```
module inv (y, in);
    input in;
    output y;
    voltage_r y, in; // y and in are defined as voltage_r type
    ...
endmodule

module top();
    ...
    inv m1(y, in);
    initial begin
        #50 $hdl_xmr_force ("top.g1.y", "0.33", "50ns", , , 1);
        #10 $hdl_xmr_release ("top.g1.y", 1 );
    end
endmodule
```

In the previous example, the `top.g1.y` destination analog node is forced to a value of 0.33 at 50 ns by the `$hdl_xmr_force` system task. The tool inserts a `rt_n2e` element because the node is defined as a nettype of `voltage_r`. The `$hdl_xmr_release` system task releases the forced node after 10 time units.

The `runtime_interface_element.rpt` file reports the `rt_n2e` interface element as follows:

```
rt_n2e -node top.g1.y // -nettype voltage_r
```

Accessing SPICE Nodes Through Proprietary System Tasks and Functions

In this method, internal SPICE nodes are accessible to Verilog as real values. The access is made through calls to Verilog system tasks or system functions as described in [Table 6](#).

The following Verilog system tasks and system function provide real XMR access to SPICE nodes from Verilog:

Table 6 Accessing SPICE Nodes With System Tasks or System Functions

System Tasks	<code>\$snps_force_volt(SPICE_node_name, voltage)</code> <code>\$snps_release_volt(SPICE_node_name)</code> <code>\$snps_inject_current(SPICE_node_name)</code>
--------------	--

Table 6 Accessing SPICE Nodes With System Tasks or System Functions (Continued)

System Functions	<code>\$snps_get_aint(int_variable)</code> <code>\$snps_get_areal(real_variable)</code> <code>\$snps_get_dint(int_variable)</code> <code>\$snps_get_dreal(real_variable)</code> <code>\$snps_get_port_current (SPICE_port_name spice_primitive_pin)</code> <code>\$snps_get_volt(SPICE_node_name)</code> <code>snps_above(expression)</code> <code>snps_absdelta(expression, delta, time_tol, expr_tol, enable)</code> <code>snps_cross(expression, dir)</code>
------------------	---

Caution:

These real XMR system tasks and functions are not yet supported in the VHDL/Verilog-SPICE and Verilog-AMS-SPICE flows.

`$snps_force_volt()`

This system task allows Verilog to force a voltage on any SPICE node, even one connected to an ideal voltage source. In such a case, this system task overrides the ideal voltage source.

Syntax

```
$snps_force_volt(SPICE_node_name, verilog_real_value  
| verilog_real_variable);
```

Argument	Description
<code>SPICE_node_name</code>	Specifies the full hierarchical node name for the internal SPICE node. This can also be a mixed net (an A/D interface net).
<code>verilog_real_value</code> <code>verilog_real_variable</code>	Specifies a real value or a Verilog real variable. The value is applied to the SPICE node as an ideal voltage source.

Examples

```
$snps_force_volt (top.i1.spcell.n1, 3.3);
```

or

```
$snps_force_volt (top.i1.spcell.n1, real_var);
```

The voltage applied to the SPICE node stays at the specified value until the next `$snps_force_volt` or `$snps_release_volt` system task call is applied.

\$snps_inject_current()

Allows Verilog to inject additional current on a SPICE node.

Syntax

```
$snps_inject_current(SPICE_node_name, verilog_real_value  
| verilog_real_variable);
```

Argument	Description
<i>SPICE_node_name</i>	Specifies the full hierarchical node name for the internal SPICE node. This can also be a mixed net (an A/D interface net).
<i>verilog_real_value</i> <i>verilog_real_variable</i>	Specifies an explicit real value or a Verilog real variable. The value is applied to the SPICE node as an ideal voltage source.

Examples

```
$snps_inject_current (top.il.spcell.n1, 20.0e-6);
```

An additional 20uA of current is added to the SPICE node. This additional current is applied until the next `$snps_inject_current` system task call. To stop injecting current, set the value to 0.0.

Note:

This feature is currently available only in the PrimeSim XA simulation engine and is not supported in the FineSim or PrimeSim simulation engines.

\$snps_release_volt()

This system task removes the voltage source applied by a previous `$snps_force_volt` task, and from that point on, allows the SPICE node to assume voltages determined by the SPICE circuit. If the node has not been forced with `$snps_force_volt` or is already released, this system task has no effect.

Syntax

```
$snps_release_volt(SPICE_node_name);
```

Argument	Description
<i>SPICE_node_name</i>	Specifies the full hierarchical node name for the internal SPICE node. This can also be a mixed net (an A/D interface net).

Examples

```
$snps_release_volt (top.i1.spcell.n1);
```

\$snps_get_volt()

This is a Verilog function that allows sampling of voltage values for internal SPICE nodes.

This function can be used to assign a value to a Verilog real variable or it could be used as a real value in an expression.

Syntax

```
$snps_get_volt (SPICE_node_name);
```

Argument	Description
<i>SPICE_node_name</i>	Specifies the full hierarchical node name for the internal SPICE node. This can also be a mixed net (an A/D interface net).

Examples

```
always @(posedge clk) begin  real_var =
    $snps_get_volt(top.i1.spcell.n2);end

if ($snps_get_volt(top.i1.i2.sp1_node) > 2.5)
...
else
...
end
```

\$snps_get_port_current()

This is a Verilog function that allows sampling of current through SPICE subcircuit ports or SPICE primitive pins.

You can use this function to assign a value to a Verilog real variable or as a real value in an expression.

Syntax

```
$snps_get_port_current(SPICE_port_name | spice_primitive_pin);
```

Argument	Description
<i>SPICE_port_name</i>	The Full Hierarchical Path to the SPICE subcircuit port or the pin of a SPICE primitive such as a resistor, capacitor, inductor, or transistor.
<i>spice_primitive_pin</i>	The names of SPICE primitives, which follow the PrimeSim HSPICE convention. For example, "p" and "n" represent positive and negative pins of two port primitives such as R's, C's, and L's, and "d", "g", and "s" represent drain, gate, and source of MOS transistors.

Example 26 \$snps_get_port_current Example

```
real_var = $snps_get_port_current(top.i1.amp.out);
res_curr = $snps_get_port_current(top.i1.x1.r2.p);

// current through the "p" pin of a resistor
mos_curr = $snps_get_port_current(top.i3.x1.m5.d);

// current through the drain of a MOS transistor
if ($snps_get_port_current(top.i1.i2.bias > 1e-3))
...
else
...
end
```

Caution:

These system tasks and functions operate on an on-demand basis and are not event-driven. This means that they sample and apply SPICE voltages only at times when they are called in the Verilog code.

\$snps_get_areal()

This is a Verilog function that allows sampling of a real variable defined in Verilog-A. The function returns a real value to the digital domain. The function can be used to assign a value to a Verilog real variable or used as a real value in an expression.

Syntax

```
$snps_get_areal(real_variable);
```

Argument	Description
<i>real_variable</i>	The hierarchical path to a real variable inside a Verilog-A module. The path can be a full path or a relative path.

Example 27 \$snps_get_areal Example

```
// SystemVerilog module
real r_sup;
always @(posedge clk) begin
    r_sup = $snps_get_areal(I2.v_sup);
end

// Verilog-A module
real v_sup;
analog begin
    v_sup = V(sup);
end
```

\$snps_get_aint()

This is a Verilog function that allows sampling of an integer variable defined in Verilog-A. It returns an integer value to the digital domain. This function can be used to assign a value to a Verilog int variable or used as an integer value in an expression.

Syntax

```
$snps_get_aint(int_variable);
```

Argument	Description
<i>int_variable</i>	The hierarchical path to a integer variable inside a Verilog-A module. The path can be a full path or a relative path.

Example 28 \$snps_get_aint Example

```
// SystemVerilog module
int i_sup;
always @(posedge clk) begin
    i_sup = $snps_get_aint(I2.v_sup);
end

// Verilog-A module
int v_sup;
analog begin
    if (V(sup) > 0.7) v_sup=1;
end
```

\$snps_get_dreal()

This is a Verilog-A function that allows sampling of a real variable defined in SystemVerilog. It returns a real value to the analog domain. This function is limited to accessing the SystemVerilog parent of a Verilog-A module.

Syntax

```
$snps_get_dreal(real_variable);
```

Argument	Description
<i>real_variable</i>	A real variable inside the SystemVerilog parent module.

Example 29 \$snps_get_dreal Example

```
// SystemVerilog module
real r_v1;
initial r_v1 = 1.11;

// Verilog-A module
real v_v1;
analog begin
    v_v1 = $snps_get_dreal(r_v1);
    V(out) <+ transition(v_v1,0,1n,1n);
end
```

\$snps_get_dint()

This is a Verilog-A function that allows sampling of an integer variable defined in SystemVerilog. It returns an integer value to the analog domain. It is limited to accessing the SystemVerilog parent of a Verilog-A module.

Syntax

```
$snps_get_dint(int_variable);
```

Argument	Description
<i>int_variable</i>	An integer variable inside the SystemVerilog parent module.

Example 30 \$snps_get_dint Example

```
// SystemVerilog module
int i_v1;
initial i_v1 = 2;

// Verilog-A module
real r_v1;
analog begin
    r_v1 = $snps_get_dint(i_v1);
    V(out) <+ transition(r_v1,0,1n,1n);
end
```

snps_above()

Generates a digital event that allows Verilog to sense when a given SPICE voltage has gone above or below a certain threshold. This event can be used in the sensitivity list of a Verilog `always` block, which would then allow a piece of Verilog code to be executed depending on the voltage transitions on the given SPICE node.

Syntax

```
snps_above(expression);
```

In the following example, `snps_above` is used with `$snps_get_volt()` in a Verilog `always` block to sense when the voltage of SPICE node `top.dut.x4.rst` rises above 1.85V.

```
always @(snps_above($snps_get_volt(top.dut.x4.rst) - 1.85))
begin
...
end
```

In the following example, `snps_above` triggers an event when the `test_top.i1.x3.strb` signal falls below 0.9V.

```
always @(snps_above(0.9 - $snps_get_volt(test_top.i1.x3.strb)))
begin
...
end
```

snps_absdelta()

Generates a digital event when the expression changes by more than the specified delta value. This event can be used in the sensitivity list of a Verilog `always` block to allow execution of Verilog code based on a changing SPICE node. This function is useful when real valued variables are used in Verilog modules and the analog signals must be sampled continuously.

Syntax

```
snps_absdelta(expression, delta, time_tol, expr_tol, enable);
```

Argument	Description
<i>expression</i>	Specifies a mathematical expression which would cause an event to be generated whenever the value changes by more than <i>delta</i> . Normally the expression contains a <code>\$snps_get_volt()</code> system task.
<i>delta</i>	Sets the delta value that causes the function to create an event. Must be a non-negative number. If <i>delta</i> is set to 0, all analog changes trigger events.

Argument	Description
<i>time_tol</i>	Sets the value of the time tolerance. Must be a nonnegative number.
<i>expr_tol</i>	Sets the value of the expression tolerance. Must be a nonnegative number.
<i>enable</i>	Expression that enables the function. This expression evaluates to an integer and when the integer is 0, no events are generated.

In the following example, `snps_absdelta` is used with `$snps_get_volt()` in a Verilog `always` block to sense when the voltage of SPICE node `i1b.xb.x02.y` changes by more than 0.15V.

```
always @ (snps_absdelta($snps_get_volt(i1b.xb.x02.y), 0.15))
begin
...
end
```

In the following example, `snps_absdelta` is used with `$snps_get_port_current()` in a Verilog `always` block to sense when the current through a SPICE port `i_sp.x1.a_in` changes by more than 2uA and variable `en_abs` evaluates to a nonzero value.

```
always @ (snps_absdelta($snps_get_port_current(i_sp.x1.a_in),
2.0e-6,,,en_abs))
begin
...
end
```

snps_cross()

Generates a digital event that allows Verilog to sense when a given SPICE voltage has passed a certain threshold voltage. This event can be used in the sensitivity list of a Verilog `always` block to allow execution of Verilog code based on a voltage transition at the given SPICE node.

Syntax

```
snps_cross(expression, dir);
```

Argument	Description
<i>expression</i>	Specifies a mathematical expression which would cause an event to be generated whenever the value of the expression crosses 0. Normally the expression contains a <code>\$snps_get_volt()</code> system task.

Argument	Description
<i>dir</i>	Determines when the event is generated. A value of "1" generates an event when the expression crosses 0 in the rising direction, when transitioning from a negative value to a positive value. A value of "-1" generates an event when the expression crosses 0 in the falling direction, when transitioning from a positive value to a negative value. A value of "0" generates an event in both directions.

In the following example, `snps_cross` is used with `$snps_get_volt()` in a Verilog `always` block to sense when the voltage of SPICE node `top.dut.x4.rst` rises above 1.85V.

```
always @(snps_cross($snps_get_volt(top.dut.x4.rst) - 1.85, 1))
begin
...
end
```

In the following example, `snps_cross` triggers an event when signal `test_top.i1.x3.strb` rises above or falls below 0.9V.

```
always @(snps_cross($snps_get_volt(test_top.i1.x3.strb) - 0.9, 0))
begin
...
end
```

Accessing SPICE Nodes Through UCLI XMRs

This section describes the `force` and `release` commands that can be supplied through UCLI mode to force or release a value on any existing VHDL or Verilog node or SPICE node.

UCLI force and release Commands

Use the `force` command to force a value onto an HDL object (signal or variable) or SPICE node. The syntax is:

```
force node value time {, value time}*
[-repeat time]
[-cancel time]
```

Where `node` is the hierarchical path name of the node to be forced. The effect of this command can be canceled with the following commands:

- A `release` command
- Another `force` command
- A `force` command with the `-cancel` option

Use the `release` command to release the value forced to a signal, variable, net, or reg previously forced by the `force` command. After this command is executed, the drivers of signal, variable, net or reg have their original drivers.

Note:

If the nettype is `reg`, then it retains its value until the original driver forces a new value.

The `release` syntax is:

```
release node
```

where `node` is the nested hierarchical identifier of the signal, variable, net or reg or SPICE node.

The following example applies a UCLI `force` command on an analog target:

```
ucli% run 20ns
ucli% force top.g1.g2.y 1
```

With the following screen output at 20ns:

```
Notice [MSV-RT-D2A] - runtime d2a conversion:
rt_d2a -hiv 1.8v -lov 0.0v -node top.g1.g2.y;
```

And the following content in the `runtime_interface_element.rpt` report file:

```
rt_d2a -hiv 1.8v -lov 0.0v -node top.g1.g2.y;
```

The following example applies a UCLI `force` command on an analog target when the analog target has an equivalent HDL view in which the target node is defined as nettype.

Example 31 Force Example

```
module inv (y, in);
  input in;
  output y;
  voltage_r y, in; // y and in are defined as voltage_r type
  ...;
endmodule

// "g2" is an instance of "inv"
```

```
ucli% run 20ns
ucli% force top.g1.g2.y 1.1
```

The tool writes out the following notice at 20ns:

```
Notice [MSV-RT-N2E] Runtime Interface Element: "n2e" inserted
rt_n2e -node top.g1.g2.y // -nettype voltage_r
```

The tool writes the following content to the `runtime_interface_element.rpt` file:

```
rt_n2e -node top.g1.g2.y // -nettype voltage_r
```

Support of Verilog Force and Release Assignments on wreal Nets

The force and release on wreal nets allows you to force and release wreal nets from the testbench code written in UCLI, MHPI, or VPI and allows you to use `$hdl_xmr_force` and `$hdl_xmr_release` system tasks. It is useful when there are large number of designs and instances.

It does not support deposit functionality. It generates an error message if you try to deposit values on wreal nets. This applies to all sources of deposit including:

- `$deposit`
- UCLI `force -deposit`
- `$hdl_xmr`
- `vpi_put_value` without `vpiForceFlag`

Usage Example

The following example supports usage of force and release assignments on wreal nets:

top.v

```
module top;
    mid m();
endmodule

module mid;
    wreal wr[1:2];
    bot b();
    always @(wr[1]) $display ("wr[1]=%g at %0t", wr[1], $time);
endmodule

module bot;
    real r;
    assign top.m.wr[1] = r;
    initial begin
        r = 0.1;
        repeat (5) #10 r = r * 2.0;
    end
endmodule
```

```
        $finish;  
    end  
endmodule
```

Command line:

```
% vcs top.v -sverilog -wreal res_sum -debug_access+all  
% ./simv -ucli
```

Tool session:

```
ucli% catch {force {top.m.wr[1]} 1.0 -deposit} err  
1  
ucli% puts $err  
Error-[UCLI-FORCE-ERR-WREAL] NYI force on WREAL  
Force command failed on object 'top.m.wr[1]' with type  
'vpiRealWire'.  
Command force deposit on wreal net is not yet supported.  
Please refer to UCLI user guide for more documentation on force  
commands.  
ucli% force {top.m.wr[1]} 8.0  
ucli% run 1  
ucli% get {top.m.wr[1]}  
8.000000  
ucli% release {top.m.wr[1]}  
ucli% run 3  
ucli% get {top.m.wr[1]}  
0.100000  
ucli% quit
```

Limitations

This feature has the following limitations:

- The `$deposit` system task on the wreal net is not supported.
- The force and release on wreal arrays that are present in modules instantiated in VHDL is not supported.
- The force and release on wreal arrays are not supported when wreal scalarization is disabled using the `-wreal noscalarize` option.

Using Bus Notation With an XMR Task

When using bus notation with square brackets (`[]`) in an XMR task, you must escape the identifier as described in the *VCS®/VCSi™ Unified Command-Line Interface User Guide*.

Using `form_spice_bus` for SPICE Ports Appearing in the Form of a Bus

Escaped identifiers begin with the backslash (\) character. The entire identifier is escaped by the backslash. Escaped identifier is terminated by white space. For example:

```
real_var1 = $snps_get_volt(snps_sptop.\x1_[2] .in);
```

Using `form_spice_bus` for SPICE Ports Appearing in the Form of a Bus

Use `form_spice_bus enable`; when the source object of `$hdl_xmr` or destination object of `$hdl_xmr_force` or UCLI force is in SPICE and the ports appear as a bus in the equivalent digital view. See the [form_spice_bus](#) command for details.

6

Sweep/Alter in Mixed-Signal Simulation

Sweeps and Alters are commonly used in SPICE and Fast SPICE to perform multiple simulations. This feature enables sweep methods commonly used in SPICE to be used in mixed-signal simulation.

Sweeps and Alters are described in the following sections:

- [Types of Sweeps](#)
- [Combining Sweeps](#)
- [Running Distributed Sweeps and Alters](#)
- [Output Files and Directories](#)
- [Placing Waveform Files in One Directory](#)
- [Merged FSDB Files](#)
- [Group Files](#)
- [UCLI Support](#)
- [Sweep and Alter With Save/Restore or ACE-Reread](#)
- [Known Limitations](#)

Types of Sweeps

The following types of sweeps and alters are supported in mixed-signal simulation:

- [Parameter Sweeps](#)
- [Data Sweeps](#)
- [Temperature Sweeps](#)
- [Alter Blocks](#)
- [DC Sweeps](#)

The sweep mechanism is specified in the SPICE netlist the same way as a SPICE-only simulation. You can use the `SWEEP` keyword with the `.TRAN` statement to enable sweep

analysis. Refer to the respective user guides for the PrimeSim XA, FineSim, or PrimeSim simulation engines for more information regarding the `.TRAN` command.

Parameter Sweeps

You can specify parameter sweeps as:

```
.TRAN tstep tstop [UIC] [SWEEP var type np pstart pstop]
```

or

```
.TRAN tstep tstop [UIC] [SWEEP var start stop incr]
```

Argument	Description
<i>var</i>	Can be an independent source, parameter, or temperature.
<i>type</i>	Can be <code>lin</code> , <code>poi</code> , <code>dec</code> , or <code>oct</code> .
<i>np</i>	Specifies the number of points for <code>lin</code> or <code>poi</code> or the number of points per decade or octave for <code>dec</code> and <code>oc</code> sweeps, respectively.
<i>pstart</i>	Specifies the first point of a range of parameter values.
<i>pstop</i>	Specifies the final point of a range of parameter values.

For example, each of the following `.tran` statements sweeps the simulation using six points:

```
.tran 1ns 100ns sweep p1 10 110 20
.tran 1ns 100ns sweep p2 lin 6 10 110
.tran 1ns 100ns sweep temp poi 6 10 30 50 70 90 110
```

Data Sweeps

You can define sweep parameters in a `.DATA` block:

```
.TRAN tstep tstop [UIC] [SWEEP DATA=dataname]
.DATA datanm pnam1 [pnam2 pnam3 ... pnamx]
+ pval1 [pval2 pval3 ... pvalx]
+ pval1' [pval2' pval3' ... pvalx']
.ENDDATA
```

For example:

```
.TRAN 1ns 100ns sweep DATA=dat1
.DATA dat1 param1 param2
10 20
```

```
30 40
.ENDDDATA data1
```

Temperature Sweeps

There are four ways to sweep temperature:

- The `.TEMP` command.
- The `TEMP` parameter in the `.TRAN` statement.
- The `TEMP/TEMPER` parameter in the first column of the `.DATA` statement.
- As a parameter in a `.ALTER` block.

Note:

The `TEMP` parameter in a `.TRAN` statement or in `.DATA` statement is treated as a parameter sweep. Samples have an extension of `.s#` in file names. `TEMP` as a parameter in a `.ALTER` block is treated as an alter. Samples have a `.a#` extension to their file names.

For example:

```
.TEMP 10 20 30
.TRAN 10ns 1us UIC SWEEP TEMP -55 75 10
.TRAN 1ns 100ns sweep DATA=data1
.DATA data1 temp p2
0 20
25 30
125 40
.ENDDDATA data1
```

Alter Blocks

The `.ALTER` statement resimulates a netlist using different parameter values, model libraries, components, temperatures, and so on for each run.

The following example changes a parameter:

```
.param rvalue = 10
Vsource source 0 PULSE(0 5 1n 1n 1n 20n 40n)
Rsource source 0 'rvalue'
.alter rvalue=10
.alter rvalue=100
.end
```


The following example changes libraries:

Example 32 Changing Libraries

```
mn out in Vss Vss nch l=.1u w=.4u
mp out in Vdd Vdd pch l=.1u w=.4u

.lib 'mymodel.l' TT
.alter
.del lib 'mymodel.l' TT
.lib 'mymodel.l' SS
.alter
.del lib 'mymodel.l' SS
.lib 'mymodel.l' FF
.END
```

The following example sets the temperature and changes a resistor value:

Example 33 Changing Temperature and Resistance

```
* Netlist
.param rval=1K
V1 1 0 pulse 0 1 0 1n 1n 2n 4n
R1 1 2 rval tc1=0.01
R2 2 0 rval tc1=0.005
.temp 60
.tran 1n 20n
.alter
rval=2k
.temp -40
.alter
rval=3k
.temp 110
.end
```

DC Sweeps

The FineSim tool in SPICE mode (spicead, spicehd, spicemd, and spicexd), the PrimeSim Pro tool in SPICE mode (spicehd, spicemd, and spicexd), and the PrimeSim SPICE tool all support DC sweep analysis in mixed-signal simulation. Add the SPICE command `.DC` to enable DC Sweep.

The following example runs a DC sweep of a mixed-signal circuit. The parameter `vd1` is linearly swept from 1.0 to 1.4 in five steps.

```
.dc vd1 lin 5 1.0 1.4
```

Combining Sweeps

The sweeps and alters may be combined and nested. The following example combines temperature, parameter, and alter sweeps for a total of 30 simulations:

Example 34 Combined Sweeps

```
.temp 0 25 55 $ 3pts
.tran .1p 2n sweep vset 1.3 1.7 0.1 $ 5pts
.lib 'c1n90g_1k.1' TT
.measure tran vol find v(aout) when v(in)=0.5 fall=2
.alter $2pts
.lib 'c1n90g_1k.1' FF
.end
```

Running Distributed Sweeps and Alters

To distribute sweep and alter simulations across a network of machines, add the `dp` runtime option to `simv`:

```
simv -ad_runopt=analogconfig:dp:worker#:[dp_cfg_file]:[dp_location]
```

Argument	Description
dp	Keyword to enable distributed analysis.
worker#	Required parameter that specifies the number of workers (CPUs) available to process sweep/alter iterations. Must be a number greater than 0.
dp_cfg_file	Optional user-defined file. If not specified, uses the local host.
dp_location	Optional, can be NFS or TMP.

For more information about distributed processing, see [Running Distributed Monte Carlo Analysis](#).

Output Files and Directories

This section describes the PrimeSim XA, FineSim, PrimeSim, and VCS-generated output files and directories.

PrimeSim XA-Generated Files

Output file names and locations follow the PrimeSim XA tool convention. You specify the directory and file name using `-o` on the PrimeSim XA command line. The tool makes file names unique by adding `s` (sweep), `t` (temperature) or `a` (alter).

By default, an additional directory hierarchy is added and the output files (`.ic`, `.fsdb`, `.meas`, and so on) are named as follows (# is the sample number):

Sweep

```
-o output/xa -> output/s#/xa.s#.*
```

Temperature:

```
-o output/xa -> output/t#/xa.t#.*
```

Alter:

```
-o output/xa -> output/a#/xa.a#.*
```

When a combination of parameter sweeps, temperature sweeps, and alters are performed, the results of each simulation are recorded using file names with concatenated suffixes:

```
-o results/xa -> results/[a#.t#.s#]/xa.a#.t#.s#.*
```

FineSim-Generated Files

Output file names and locations follow the FineSim tool convention. You specify the directory and file name with the `-o` option on the FineSim command line as:

```
-o results/fs
```

The FineSim tool does not create sample-specific directories. Instead, it makes the file names (`.fsdb`, `.vpd`) unique by adding `s` (sweep), `t` (temperature) or `a` (alter) with the sample number and places them inside the output results directory. For example:

```
results/
  fs_s0.fsdb, fs_s1.fsdb ... (Sweep)
  fs_t0.fsdb, fs_t1.fsdb ... (Temperature)
  fs_a0.fsdb, fs_a1.fsdb ... (Alter)
```

PrimeSim-Generated Files

Output file names and locations follow the PrimeSim tool convention. You specify the directory and file name with the `-o` option on the PrimeSim command line as:

```
-o results/ps
```

The PrimeSim tool does not create sample-specific directories. Instead, it makes the file names (.fsdb, .vpd) unique by adding *s* (sweep), *t* (temperature) or *a* (alter) with the sample number and places them inside the output results directory. For example:

```
results/  
  ps_s0.fsdb, ps_s1.fsdb ... (Sweep)  
  ps_t0.fsdb, ps_t1.fsdb ... (Temperature)  
  ps_a0.fsdb, ps_a1.fsdb ... (Alter)
```

VCS-Generated Output Files

The output files generated by VCS command line or by Verilog system-tasks are placed in the run directory and are overwritten with each sweep simulation.

FSDB or VPD files output from UCLI are created and then moved to the respective PrimeSim XA/FineSim/PrimeSim output directories:

```
results/a#.t#.s#/ (SPICE results)  
results/a#.t#.s#/vcs.a#.t#.s#.fsdb (VCS waveform files)
```

Examples of a Parameter Sweep of Two Values

PrimeSim XA Tool:

```
xa ... -o rslt_dir/f_name
```

VCS (UCLI) Tool:

```
VCS (UCLI): fsdbDumpfile vcs_rslt.fsdb
```

The resulting names and locations of the output files are:

```
rslt_dir/s0/  
  f_name.s0.0.ic  
  f_name.s0.fsdb  
  f_name.s0.meas  
  vcs_rslt.s0.fsdb  
simv.daidir/  
simv.msv/  
  
rslt_dir/s1  
  f_name.s1.0.ic  
  f_name.s1.fsdb  
  f_name.s1.meas  
  vcs_rslt.s1.fsdb  
simv.daidir/  
simv.msv/
```

FineSim Tool

```
fs ... -o rslt_dir/f_name
```

VCS (UCLI) Tool:

```
fsdbDumpfile vcs_rslt.fsdb
```

The resulting names and locations of the output files are:

```
simv.daidir/  
simv.msv/  
vcs_rslt.s0.fsdb  
vcs_rslt.s1.fsdb  
f_name_s0.fsdb  
f_name_s1.fsdb  
f_name.mt0
```

PrimeSim Tool

```
ps ... -o rslt_dir/f_name
```

VCS (UCLI) Tool:

```
fsdbDumpfile vcs_rslt.fsdb
```

The resulting names and locations of the output files are:

```
simv.daidir/  
simv.msv/  
vcs_rslt.s0.fsdb  
vcs_rslt.s1.fsdb  
f_name_s0.fsdb  
f_name_s1.fsdb  
f_name.mt0
```

Placing Waveform Files in One Directory

The waveform and measurement files can be relocated and consolidated into one directory by using the PrimeSim XA `-merge` command-line option.

Example

PrimeSim XA Tool:

```
xa ... -o rslt_dir/f_name -merge
```

VCS (UCLI) Tool:

```
fsdbDumpfile vcs_rslt.fsdb
```

The resulting names and locations of the output files are:

```
rslt_dir/  
  f_name.s0.0.ic  
  f_name.s0.fsdb
```

```
f_name.s0.meas
vcs_rslt.s0.fsdb
f_name.s1.0.ic
f_name.s1.fsdb
f_name.s1.meas
vcs_rslt.s1.fsdb
s0/
    simv.daidir/
    simv.msv/
s1/
    simv.daidir/
    simv.msv/
...
```

Merged FSDB Files

In the previous example, analog and digital waveforms are saved to separate FSDB files. As described in [Output Files and Directories for PrimeSim XA](#), the analog and digital waveforms can be merged into a single file. In that case, each sweep or alter produces a single FSDB file rather than two. In the previous examples, if merged FSDB is used, the analog and digital signals are written to the `vcs_rslt.s0.fsdb` and `vcs_rslt.s1.fsdb` files.

Group Files

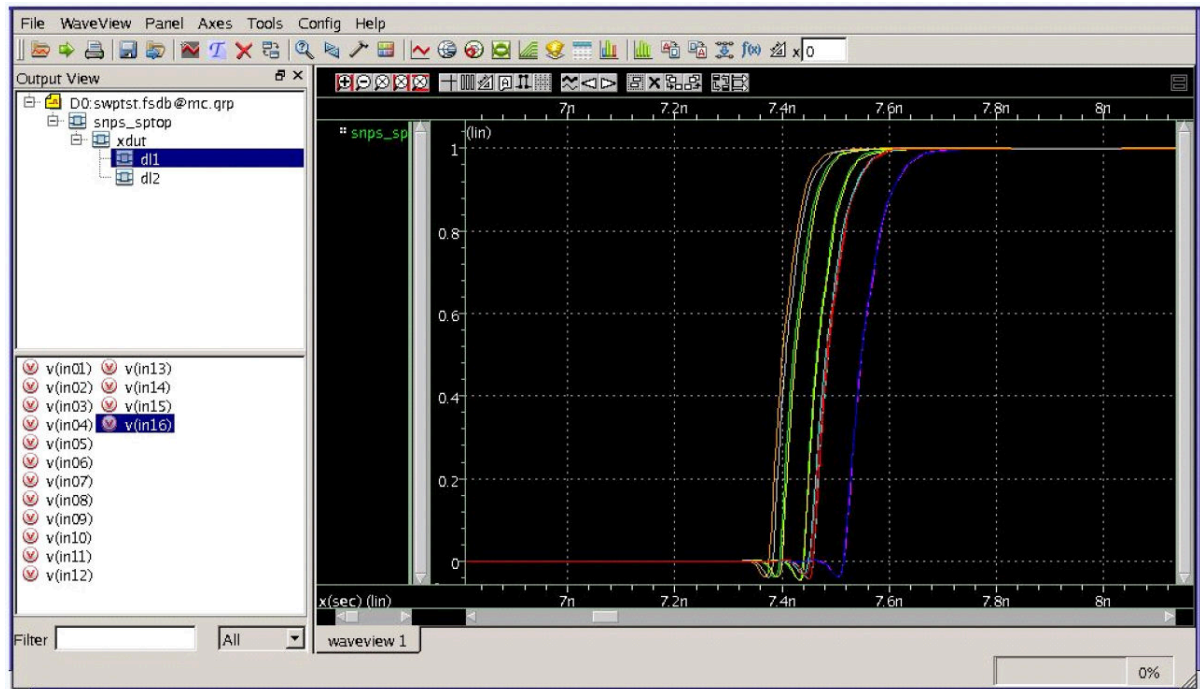
A WaveView group (`.grp`) file is generated to make it easy to load all the resulting waveform files into WaveView at one time and overlay the results.

For example, suppose you ran a series of 12 sweeps and used the following command:

```
xa -o rslt_dir/f_name
```

In WaveView, choose **File > Open Waveform** and open the following `.grp` file to load all of the waveforms at one time:

```
f_name@000.fsdb@sw.grp
```



UCLI Support

The following UCLI capabilities are supported:

- `simv -ucli -do run.ucli`
- Note that if the UCLI script does not contain a means to exit, the simulation automatically adds one.
- A digital or analog force can be applied at any time from the UCLI command file.
- You can use `ace` commands in the `ucli.cmd` file.
- The UCLI `ace reread` command is supported.
- You can apply breakpoints.

The following features are not supported:

- Interactive mode is not supported. Ctrl+C causes the simulation to exit.
- The UCLI save/restore commands are not supported. These commands are ignored and a warning is given.
- `-ucli2proc`

Sweep and Alter With Save/Restore or ACE-Reread

OP-based save and restore are supported for sweeps and alters with one limitation: you cannot add or change alter or sweep control by new `ace reread` statements. All such later changes are ignored with warnings.

Image-based save and restore is not supported.

Known Limitations

- Only transient analysis is supported.
- If swept analog parameters are passed to digital, only the initial value is passed. This is because the parameter passing between analog and digital is determined at compile time.
- Alter blocks cannot change the interface element. You cannot change views using an alter block.
- You cannot combine data sweep and parameter driven sweeps. This is consistent with the PrimeSim XA standalone tool. If you specify both, data sweep is used and parameter sweep is ignored.
- Sweep statements cannot be inside `.alter` statements:

```
.param rvalue=1
.tran 1ns 100ns
.lib 'Cmos90nm.lib' tt
.alter 1
.tran 1ns 100ns sweep rvalue 1 2 1 $ This line will be ignored
.lib 'Cmos90nm.lib' ss
```
- You cannot mix sweeps or alters with Monte Carlo analysis. If encountered, the tool produces an error message and exits.
- UCLI interactive mode is not supported. Ctrl+C causes the simulation to exit.

7

Mixed-Signal Simulation in the Verilog-SPICE Flow

This chapter provides information about the Verilog-SPICE mixed-signal simulation flow.

This chapter describes:

- [Required Input Files for the Verilog-SPICE Flow](#)
- [Compiling the Design for a Verilog-SPICE Simulation](#)
- [Running a Verilog-SPICE Simulation](#)

Required Input Files for the Verilog-SPICE Flow

To run a mixed-signal simulation, you need to create a mixed-signal simulation control file. This file is passed to VCS during compile time, and contains the call to the analog engine (PrimeSim XA, FineSim, or PrimeSim tools) and optional mixed-signal commands.

Creating a Mixed-Signal Simulation Control File

By default, when you specify the `-ad` switch with the `vcs` command, the tool opens and reads the `vcsAD.init` mixed-signal simulation control file. You can specify a different file name with the `-ad=control_file_name` compile-time option. The mixed-signal simulation control file contains all the commands to configure mixed-signal simulation.

At a minimum, the mixed-signal simulation control file must contain the `choose` command, which specifies the analog simulator. The mixed-signal simulation control file can also include:

- Block comments
- Include files
- Search paths

To comment out lines one-by-one, use double slashes (//) as shown in the following example:

```
// TEST A
// use_spice -cell blocka;
// a2d -loth 1.65v -hith 1.65v -node top.s[0];
```

You can also use Verilog-style block comments (/* */) to comment out multiple lines as shown in the following example:

```
//TEST A
/*
use_spice -cell blocka ;
a2d -loth 1.65v -hith 1.65v -node top.s[0] ;
a2d -loth 1.65v -hith 1.65v -node top.s[1] ;
*/
```

You can include the content of other files with the ``include` command. In the following example, the contents of File1 and File2 are inserted into the `vcsAD.init` file:

```
`include "/path/mydir/File1" ;
`include "/path/mydir/File2" ;
// `include "/path/mydir/File3"
```

The ``include` command calculates relative paths to the included files based on the simulation directory (the directory where the `simv` simulation executable is created and executed). For example, suppose the simulation directory is `/user/unit/case/test` and the `vcsAD.init` file contains the following ``include` commands:

```
`include /a/b/c/d/sub1.init
`include dir1/dir2/sub2.init
```

In the previous example, the `/a/b/c/d/sub1.init` file and the `/user/unit/case/test/dir1/dir2/sub2.init` file are read by the tool. Note that include files can reference other include files. For example, if the `sub2.init` file contained the following command:

```
`include ../../e/sub3.init
```

The tool also reads the `/user/unit/e/sub3.init` file.

You can use the include file syntax to expand environment variables. For example, if you set the following environment variables:

```
setenv MYFILE1 /path/to/directory/file1.inc
setenv MYFILE2 file2.init
```

You can reference the `$MYFILE1` and `$MYFILE2` environment variables in your `vcsAD.init` file as follows:

```
choose xa -hspice addr4.spi -c cfg ;
`include "$MYFILE1" ;
`include "$MYFILE2" ;
```

Using a Mixed-Signal Initialization File

Mixed-signal simulation supports an additional mixed-signal simulation control file, `snps_vcsAD.ini`, that is loaded *before* the `vcsAD.init` file. The `snps_vcsAD.ini` file can contain the same commands as the mixed-signal simulation control file, but the `snps_vcsAD.ini` file is read first. The tool searches for the initialization file in the following paths and in the following order:

1. Current simulation directory
2. Your home directory (`$HOME`) /
3. `PrimeSim_XA_installation_directory/etc/`

The contents of the first initialization file found from the search list are read before the contents of the mixed-signal simulation control file. If a conflict exists, the latest command read takes precedence.

For example, if the `snps_vcsAD.ini` file contains the following commands:

```
// snps_vcsAD.ini
resolve_x_inst_prefix enable;
a2d -loth 1.65v -hith 1.65v -node top.s[0];
```

And the `vcsAD.init` file contains the following commands:

```
// vcsAD.init
choose xa -hspice netlist.sp -c xa.cfg -o xa/xa;
a2d -loth 1.65v -hith 1.65v -node top.s[1];
a2d -loth 1.35v -hith 1.35v -node top.s[0];
```

The following commands are passed to the mixed-signal simulator:

```
resolve_x_inst_prefix enable ;
choose xa -hspice netlist.sp -c xa.cfg -o xa/xa;
a2d -loth 1.65v -hith 1.65v -node top.s[1];
a2d -loth 1.35v -hith 1.35v -node top.s[0];
```

In the previous example, note that the threshold values for `top.s[0]` set by the `snps_vcsAD.ini` file are overridden by the `vcsAD.init` file.

The path for each control file is recorded to the compile log in the order in which it was read.

```
% vcs testbench.v -debug_access+all -ad=vcsAD.init -l ./vcs.log
...
[MSV-MC-FRO] - Notice: Mixed-signal commands were read in this order from
the following files:
  snps_vcsAD.ini
  vcsAD.init
```

For a complete list of the mixed-signal control commands, see [Mixed-Signal Control Commands](#).

Before using the Verilog-SPICE flow, the following files must also be in place:

- Verilog netlist files (for example, `testbench.v`)
- Verilog-A module files, if used
- SPICE netlist files (including device model libraries)
- Mixed-signal simulation control file (for example, `vcsAD.init`)
- Command file, depending on which analog engine is being used (for example, `cfg`)

Compiling the Design for a Verilog-SPICE Simulation

To prepare the design, use the Unified Use Model (UUM) or three-step flow with the commands `vhdlan` and `vlogan` for analysis, `vcs` for elaboration, and `simv` for simulation. Use the VCS option `-ad` to enable mixed-signal simulation.

```
vcs top_design -ad[=mixed-signal_control_file] [vcs_options]
```

Use the VCS option `-full64` to run in 64-bit mode. The following example uses VCS to elaborate the design named `top` with the `-ad` compile-time option. The command reads the default `vcsAD.init` file as the mixed-signal simulation control file and creates the `simv` simulation executable.

```
% vcs top -ad -full64
```

The following example elaborates the design named `my_top` and loads `my_setup.init` as specified by the `-ad` compile time option. The command creates the `my_simv` simulation executable.

```
% vcs -full64 my_top -ad=my_setup.init -o my_simv
```

Note:

The `-y` command-line switch in VCS is not recommended in the mixed-signal flow and may interfere with or prevent the instantiation of SPICE blocks in the design.

Recompiling the Design

In the current standalone version of the VCS tool, the `-Mupdate` option is on by default. As mixed-signal requires both the compile-time phase and runtime phase to work together, do not compile incrementally.

When you modify any design files (Verilog, Verilog-A, or SPICE) or the mixed-signal simulation control file:

- Remove all the intermediate files/directories generated by the previous mixed-signal simulation by removing the `simv.daidir` and `csrc` directories.
- Restart the compilation.

Also, if you use the analog configuration commands to change the case sensitivity in SPICE netlists, then you must recompile the design before those changes can take effect.

Automatic Verilog Dummy Module Generation

Mixed-signal simulation requires an internal [Verilog view](#) for every cell in the design—even SPICE cells.

The tool automatically generates dummy Verilog modules for all subcircuits available in the SPICE netlist that *do not* have a corresponding Verilog module. A Verilog dummy module, also called a *dummy wrapper* or *shadow file*, is defined as a `spicemodule` instead of `module`, and contains the port name, port direction, net name, and instance module definitions (if available). These dummy Verilog modules are generated at compile time and are kept in the `simv.daidir` directory.

Running a Verilog-SPICE Simulation

After the `simv` compiled simulation executable is generated, the simulation can be run using the following syntax:

```
simv [vcs runtime_options]
```

You can use the `-R` option to `vcs` to compile and run the design at the same time as shown in the following example:

```
% vcs -full64 -R -ad=vcsAD.init testbench.v adder.v -l comp.log \  
  -debug_pp -o simv -l sim.log
```

The following example specifies the `simv.log` file as the output log file for the simulation.

```
% simv -l simv.log
```

Caution:

All time step values for the VCS tool must be set to a value that is an even power of 10; for example 1ns, 10ps, 100ps.

8

Mixed-Signal Simulation in the VHDL/Verilog-SPICE Flow

This chapter describes how to prepare the input data to run a mixed-signal simulation with the VCS PrimeSim AMS tool.

The VHDL/Verilog-SPICE flow provides a mixed-signal, mixed-HDL language verification solution. VHDL/Verilog-SPICE enables simulating a design described in SPICE (or other transistor-level description language that the analog engine supports), Verilog-HDL (“Verilog”), and [VHDL](#).

You must be familiar with the SPICE, Verilog, and VHDL languages, as well as the PrimeSim XA, FineSim, or PrimeSim tools (depending on which analog engine is being used) as well as VCS.

In VHDL/Verilog-SPICE simulation, different parts of the design can be simulated in SPICE, Verilog, or VHDL models.

VHDL/Verilog-SPICE supports both Verilog-top and VHDL-top flows. SPICE-top is also supported.

This chapter describes the following topics:

- [Required Input Files for the VHDL/Verilog-SPICE Flow](#)
- [Using a VHDL Setup File](#)
- [Using a Verilog Wrapper](#)
- [Using the VHDL/Verilog-SPICE Autowrapper Utility](#)
- [Running a VHDL/Verilog-SPICE Simulation](#)
- [Known Limitations](#)

Required Input Files for the VHDL/Verilog-SPIKE Flow

Before using the VHDL/Verilog-SPIKE flow, the following files must be in place:

- Verilog netlist files (for example, testbench.v)
- Verilog-A module files, if used
- SPICE netlist files (including device model libraries)
- VHDL and/or Verilog descriptions of the digital blocks
- Dummy Verilog wrappers for SPICE blocks instantiated under VHDL
- VHDL setup file
- Mixed-signal simulation control file (for example, vcsAD.init)
- Command file, depending on which analog engine is being used (for example, xa.cfg)

Creating a Mixed-signal Simulation Control File for VHDL/Verilog-SPIKE

In VHDL/Verilog-SPIKE, as in Verilog-SPIKE, you must create a `vcsAD.init` mixed-signal simulation control file (see [Creating a Mixed-Signal Simulation Control File](#)) and load the file with the `-ad` option to the `vcs` command. You can also use the `-ad=control_file` option to the `vcs` command to specify the file name. The file contains mixed-signal control commands which specifies the HDL or SPICE views for modules and instances in the designs, specifies A/D and D/A conversions, and so on.

The mixed-signal simulation commands supported by Verilog-SPIKE are also supported in the VHDL/Verilog-SPIKE flow—with the following exceptions:

- Use the [use_vhdl](#) command to specify VHDL views in a VHDL/Verilog-SPIKE simulation.
- Use the [use_verilog](#) command to select the Verilog view of a multi-view cell instantiated under SPICE.
- Use the [use_spice](#) command to select the SPICE view of a multi-view cell instantiated under Verilog. Note that you must also use the conventional VHDL methods such as configuration files or explicit `use lib.cell_name` must be used to select the Verilog wrapper for the SPICE block in addition to using the [use_spice](#) command.

- Use the `use_spice` command to perform instanced-based selection of SPICE cells in a Verilog module
- Use the `use lib_name.cell_name` or other configuration in the VHDL design files to perform instance-based view selection for cells instantiated under VHDL
- The following is an example mixed-signal simulation control file for a design that contains Verilog, VHDL, and SPICE views:

```
use_spice -cell chargepump;
use_verilog -module counter;
use_vhdl -cell d_flop;
use_vhdl -cell rtl_lib.mux_1;

choose xa -hspice spice_cells.spi -c cfg;
bus_format <%d>;
```

When using the FineSim or PrimeSim simulation engines, replace `choose xa` with `choose finesim` or `choose primesim`.

Selecting Multiple Views

By default, VHDL/Verilog-SPICE selects the view for the multi-view cell that is identical to the parent block view. If that view is not available, VHDL/Verilog-SPICE selects the next available view. A particular view of a multi-view cell can also be selected explicitly. The method of explicit view selection for a child cell depends on the view of the parent cell.

View Selection for Cells Under a VHDL Parent

For a VHDL parent, a Verilog or VHDL view for a child cell can be selected by pointing to the design library that contains that view. This can be done in a number of ways:

- By placing the VHDL `use lib_name.cell_name` construct inside a configuration in the VHDL code that explicitly points to a particular cell in a particular library.
- By creating a VHDL configuration that defines the architecture and library to use for a specified instance or instances.
- By specifying the library name during the elaboration of the design. The elaboration stage of design compilation is described later in more detail.

For more details on VHDL and Verilog view selections, see the *VCS User Guide*.

To select the SPICE view for a child cell,

1. Use one of the three methods described above to select the Verilog wrapper within the VHDL code, since SPICE cells instantiated under VHDL require a Verilog view of the SPICE cell.
2. Use the [use_spice](#) mixed-signal command to indicate that the SPICE view of the cell replaces the Verilog wrapper.

Using a VHDL Setup File

The VCS tool uses the `synopsys_sim.setup` VHDL setup file to configure the environment for VHDL and mixed-HDL simulation. The `synopsys_sim.setup` file can exist in the VCS installation directory, in your home directory, or in your run directory. The VCS tool searches these locations—in that order—and the last file found overrides any previously found file.

This setup file maps VHDL logical library names to physical directories, sets search paths, sets the VHDL simulation time base and time resolution, and assigns values to some simulation control variables.

One of the most frequent uses of the VHDL setup file is to map logical VHDL libraries to physical host directories.

In VHDL, design blocks can be analyzed and stored in one or more logical libraries. This capability offers great flexibility in maintaining multiple versions of a VHDL block by analyzing each version in a different library.

The VHDL setup file supports the mapping of multiple logical library names to their physical location.

By default, all VHDL blocks are analyzed and stored in the `WORK` logical library. The default physical directory for this library is `./WORK`.

For more information on the VCS setup file, refer to the *VCS User Guide*.

The following example shows a sample `synopsys_sim.setup` file. Here, `rtl_lib` and `gate_lib` (two logical libraries) and their physical locations are defined. Also, the `WORK` default logical library is mapped to the `rtl_lib` library.

```
WORK > rtl_lib
rtl_lib: ./rtl_library
gate_lib: ./gate_library

TIMEBASE=ns
TIME_RESOLUTION=10ps
```

Using a Verilog Wrapper

If there is a SPICE block instantiated under VHDL, a dummy Verilog wrapper for the SPICE block must be generated.

This dummy module must have the same name, number of ports and port names as the SPICE cell. The body of the dummy module can be empty, as VHDL/Verilog-SPICE does not read the body of that module if it is used as a dummy wrapper for SPICE.

A dummy Verilog wrapper is not required for any type of Verilog or SPICE donuts, nor for SPICE instantiating VHDL.

If a Verilog description already exists for the SPICE block, use the existing Verilog module as the Verilog wrapper. If not, the wrapper can be created either manually or by using the `autowrapper` utility—see [Using the Autowrapper Utility](#) for the `autowrapper` usage.

If real ports are used in the VHDL description, ensure that a `wreal` declaration is made for the same ports in the Verilog wrapper module (also include the `-realport` switch when calling `vlogan` and `vcs`, which is discussed later in this document). When using a `wreal` declaration, only `input` and `output` ports are allowed. Note that `inout` ports are not supported.

The following example shows a `wreal` declaration file sample.

Example 35 Verilog Module With `wreal` Declaration

```
// Verilog wrapper for using real ports
//
// In addition to module name, port name, and directions,
// wreal declaration is also required

module test (a, b);
  input a;
  wreal a;
  output b;
  wreal b;
endmodule
```

Note:

Using real ports can result in a simulation performance penalty. Specifically, real values passed from the analog engine to VHDL when combined with smaller time resolution values can result in decreased simulation performance.

The following example shows a SPICE subcircuit, its instance in the VHDL description, and the required corresponding Verilog wrapper. Note that only the subcircuit ports are required.

```
.subckt chargepump_com
+ com_inv<3> com_inv<2> com_inv<1> com_inv<0>
```

```
+ com_rsh<3> com_rsh<2> com_rsh<1> com_rsh <0>
+ clk
.ends
```

The following example instantiates the `chargepump_com` SPICE block from the previous example into a VHDL description. The `sig_inv`, `sig_rsh` and `sig_clk` signals are defined in the VHDL block and connect to the `chargepump` ports.

```
I3 : chargepump_com
  port map (
    com_inv => sig_inv(3 downto 0),
    com_rsh => sig_rsh(3 downto 0),
    clk     => sig_clk
  );
```

The following example shows a Verilog wrapper that must be generated to allow the instantiation of the SPICE cell under VHDL.

```
'timescale 1ns/10ps
module chargepump_com(com_inv, com_rsh, clk);
input [3:0] com_inv, com_rsh;
input clk;
// The body of the Verilog wrapper can be empty.
// Any content here will be ignored.
endmodule
```

VHDL and Verilog Descriptions

Here are the issues to consider regarding the VHDL and Verilog netlists:

- A VHDL block can contain a *real* data type port, as long as that port is meant for data exchange between two VHDL blocks, or a VHDL and SPICE block.
- In the case of VHDL instantiating SPICE, where a Verilog wrapper is required for SPICE, the Verilog port that corresponds with the VHDL real port must be defined as *wreal*.
- The VHDL *real* ports and the Verilog wrapper *wreal* ports cannot have the direction *inout*. These ports must have an *input* or *output* direction.
- Cross-module reference (XMR) across digital/analog boundary is only supported between Verilog and SPICE as explained in the Verilog-SPICE section of this document. Cross referencing an analog node from inside VHDL is not supported.
- A mixed-net cannot connect to Verilog ports that are connected to bidirectional pass switches (`tran`, `rtran`, `tranif0`, `rtranif0`, `tranif1`, and `rtranif1`)
- A mixed-net cannot connect to Verilog jumper ports (see the following example).

```
module jumper (a, a);
  inout a;
```

```
...  
endmodule
```

Transistor-Level Descriptions

For SPICE netlists, note the following:

- PrimeSim HSPICE and VHDL netlist formats are case-insensitive, but Verilog is case-sensitive.

So the best practice is to create all subcircuit, port, and signal names consistently, in either lowercase or uppercase, across all SPICE, Verilog and VHDL files.

- Unused ports must be removed from the subcircuits that are going to be instantiated under VHDL, and a Verilog wrapper is generated for them.

Using the VHDL/Verilog-SPICE Autowrapper Utility

A SPICE subcircuit cannot be directly instantiated in a VHDL or a Verilog block. Instead, you must create a Verilog wrapper with port definitions that corresponds to the subcircuit. The wrapper can be manually created or automatically created using the `autowrapper` utility.

After creating the wrapper file, you must edit the wrapper.v file and change the port directions to match the design intent for the circuit. By default, the port direction in a SPICE subcircuit is `inout` and the `autowrapper` utility writes out all ports with an `inout` direction. Using the default `inout` port direction setting could slow the runtime because `inout` mixed-nets can lead to many successive back-and-forth D/A and A/D conversions.

The wrapper.v file contains all Verilog wrapper modules that correspond to subcircuits defined in the SPICE file. For instance, if there are four subcircuits specified in the SPICE file, the utility creates four Verilog wrapper modules in the wrapper.v file. The syntax for the `autowrapper` utility is:

```
autowrapper -n[fmt] netlist_files  
[-bus_fm bus_format] [-cell subckt_name(s)]  
[-xcell subckt_name(s)] [-file netlist_file_name]  
[-o output_file_name] [-case s|S|l|L|u|U]
```

For example, if a SPICE file contains the following subcircuit:

```
* SPICE inv subcircuit  
.subckt inv a zn  
m1 zn a vdd vdd p 1 0.35  
m2 zn a gnd gnd n 2 0.35  
.ends
```

The autowrapper utility creates the following Verilog wrapper:

```
//generated verilog wrapper example
module inv (a, zn);
    inout a;
    inout zn;
endmodule
```

Note that all ports are defined as `inout` in module `inv`. For the best simulation performance, you must change port directions as shown in the following example:

```
// modified verilog wrapper example
module inv (a, zn);
    input a;
    output zn;
endmodule
```

For a description of the autowrapper utility options, see [Table 7](#).

Table 7 autowrapper Utility Options

Utility Option	Description
<code>-n[fmt] netlist_file_names</code>	Specifies the file names to be read.
<code>-bus_fm bus_format</code>	Specifies the format of bus signals in the input netlist file. The default bus format is [%d]. See Table 8 for examples of different bus formats.
<code>-cell subckt_name(s)</code>	Specifies the subcircuits, in the netlist file, that must be converted into Verilog wrapper modules. All other subcircuits are ignored. The module name uses the same case-sensitivity as the <code>subckt_name</code> .
<code>-xcellsubckt_name(s)</code>	Specifies the subcircuits that must not be converted into Verilog wrapper modules. All other subcircuits are converted into Verilog wrapper modules. This option <i>excludes</i> specified subcircuits, and is case-insensitive.
<code>-file netlist_file</code>	Specifies a file that contains a list of subcircuits, one subcircuit name per line, to convert to Verilog wrapper modules. Use the semicolon (;) character as a comment character as shown in the following example: File:name.txt ;inv ;xor ;f_add adder
<code>-o output_file_name</code>	Specifies the output file name where the Verilog wrappers are written and sets the .log file prefix. If the same file exists in the output directory, it is overwritten without a WARNING message.

Table 7 *autowrapper Utility Options (Continued)*

Utility Option	Description
<code>-case s S l L u U</code>	<code>-case s</code> or <code>-case S</code> maintains case-sensitivity for the port names <code>-case l</code> or <code>-case L</code> generates lowercase port names (default) <code>-case u</code> or <code>-case U</code> generates uppercase port names

Table 8 *-bus_fm bus_format options*

SPICE Bus Signal Name	Autowrapper -bus_fm Bus Format Option
<code>A[0]</code>	<code>[%d]</code>
<code>B_1</code>	<code>_%d</code>
<code>C<2></code>	<code><%d></code>
<code>D_3_</code>	<code>_%d_</code>
<code>E{4}</code>	<code>{%d}</code>
<code>F5</code>	<code>%d</code>
<code>G6_</code>	<code>%d_</code>

For example, the `_%d` bus format directs the `autowrapper` utility to recognize bus signals defined as `A_1`, `A_2` ..., `A_n`.

Using the Autowrapper Utility

The following guidelines apply to using the `autowrapper` utility:

- If an `inout` port is connected to a net of type `register`, the VCS tool generates an error message and stops compilation. In Verilog, a `register` net should connect to an `input` port, not an `inout` port. The `autowrapper` utility only generates `inout` ports since direction does not exist in SPICE netlists. Therefore, before compiling, edit the `wrapper.v` file and specify the correct port direction.
- The `autowrapper` utility generates one Verilog wrapper module per subcircuit. Therefore, it can generate unnecessary Verilog wrappers. Some of these modules might be using the same module name as other Verilog modules in the original Verilog code. If this occurs, the VCS tool generates an error message and stops compilation. Therefore, before compiling, check the module names in the Verilog wrapper file. If the name is used elsewhere in the Verilog description, and the generated Verilog wrapper is not needed, remove the module from the Verilog wrapper file. The following

example displays the SPICE definition for subcircuits `adder4` and `inv`, and how a Verilog wrapper can be generated.

```
.subckt adder4 a[3] a[2] a[1] a[0] b[3] b[2] b[1] b[0]
+ clk cin cout
xinv1 clk clkcn inv
...
.ends

.subckt inv a zn
m1 zn a vdd vdd p 1 0.35
m2 zn a gnd gnd n 2 0.35
.ends
```

When you run the following command:

```
% autowrapper -nspi netlist -o net.v
```

The tool generates the `net.v` file as shown in the following example:

```
module adder4 (a, b, clk, cin, cout);
    inout [3:0] a;
    inout [3:0] b;
    inout clk;
    inout cin;
    inout cout;
endmodule

module inv (a, zn);
    inout a;
    inout zn;
endmodule
```

The `inv` module is unnecessary here and can create confusion if another `inv` module exists in the original Verilog code. You must remove the `inv` module description from the `net.v` file.

- Verilog is case-sensitive. SPICE is not case-sensitive. The `autowrapper` utility maintains case-sensitivity for module names; but, you must use the `-case` option if you want to maintain case-sensitivity for port names.
- The `autowrapper` utility does not generate timescale information. Therefore, the wrapper file must be placed at the end of the Verilog file's compilation input:

```
vcs -ad a.v c.v d.v wrapper.v
```

- If the signal in the SPICE netlist is bus- or array-type, it must be expanded.

The `autowrapper` utility automatically generates bus- or array-type signals in the Verilog wrapper file (original SPICE subcircuit), as shown in the following example. Refer to [Table 7](#) for details.

```
.subckt mem DATA[3], DATA[2], DATA[1], DATA[0],
+ WL[0], WL[1], WL[2], WL[3],
+ WL[4], WL[5], WL[6], WL[7],
+ R_WB, RAM_ENB
.ends
```

The `autowrapper` utility automatically forms bus- or array-type signals in the Verilog wrapper file, as shown in the following example.

```
module mem (data, wl, r_wb, ram_enb);
  inout [3:0] data;
  inout [0:7] wl;
  inout r_wb;
  inout ram_enb;
endmodule
```

- If special characters and Verilog-specific keywords are used for the signal or subcircuit name, the name is assigned a backslash (\) leading character.

In addition, a space is inserted at the end of the name in the Verilog wrapper file, as shown in the following example.

```
module \inverter.test ( \if , \1 , \2 );
  inout \if ;
  inout \1 ;
  inout \2 ;
endmodule
module vsources (\0 , \B#1 , B, \CE# , CLK,
  DECOU, \Q^ , XDPD, \b#2 );
  inout \0 ;
  inout \B#1 ;
  inout [5:5] B;
  inout \CE# ;
  inout CLK;
  inout [7:7] DECOU;
  inout \Q^ ;
  inout [7:7] XDPD;
  inout \b#2 ;
endmodule
```

- If subcircuit ports in a bus are randomly ordered in the transistor netlist, the `autowrapper` utility cannot function properly. See the following example for a sample (unsupported) file.

```
.subckt p7ibptacddcwr clk wlv0[0] wlv0[10] wlv0[11] wlv0[12]
+ wlv0[13] wlv0[14] wlv0[15] wlv0[1] wlv0[2]
+ wlv0[3] wlv0[4] wlv0[5] wlv0[6] wlv0[7]
+ wlv0[8] wlv0[9] wlv1[0] wlv1[10] wlv1[11]
+ wlv1[12] wlv1[13] wlv1[14] wlv1[15] wlv1[1]
+ wlv1[2] wlv1[3] wlv1[4] wlv1[5] wlv1[6]
+ wlv1[7] wlv1[8] wlv1[9] writeadd[0] writeadd[1]
```



```
+writeadd[2]   writeadd[3]   writeen[0]   writeen[1]
.ends
```

- Nested subcircuits in the SPICE netlist are supported. A module is generated for the nested subcircuit; the module name is made of the nested subcircuit name preceded by the parent subcircuit name, itself preceded by a backslash character (\).

A nested subcircuit sample is shown in the following example.

```
.subckt s in out
.subckt inv13 in out
.ends inv13
.ends s
```

The Verilog wrapper generated by the `autowrapper` utility is shown in the following example.

```
module s (in, out)
  inout out;
  inout in;
endmodule

module \s.inv13 (in, out);
  inout out;
  inout in;
endmodule
```

Running a VHDL/Verilog-SPICE Simulation

To run the mixed-signal simulation, the executable generated during the compilation must be run. By default, `simv` is the executable, unless the default name is overridden by the VCS `-o` option. The syntax for running `simv` follows:

```
simv [runtime options]
```

Argument	Description
<code>-include [VCS command file_name]</code>	Specifies a file that contains VCS commands to be run during run-time.
<code>-l [log_file_name]</code>	Generates a run-time log file.
<code>-ucli</code>	Starts the simulation in the UCLI interactive mode. The UCLI feature must have been enabled at compile time by passing the <code>-ucli</code> option to VCS.
<code>-version</code>	Prints the version of the VCS used in compilation.

In the following example, `simv` generates a `simv.log` runtime log file.

```
% simv -l simv.log
```

In the following example, the `-ucli` option starts the simulation in the UCLI interactive mode. Note that you must also specify the `-debug` or `-debug_all` option VCS compilation.

```
% simv -ucli
```

In the following example, a command file is passed to `simv` at runtime.

```
% simv -include command.txt
```

An example `command.txt` file is as follows:

```
run 1000
echo "Simulation run for 1000 time-units"
quit
```

Known Limitations

The known VHDL/Verilog-SPICE limitations are:

- Verilog wrappers are required for instantiations of SPICE under VHDL.

However, donut configurations of Verilog and SPICE blocks, or SPICE instantiating VHDL, do not require a Verilog wrapper.

- Through-net optimization is not supported for SPICE ports connecting through a top-level Verilog/VHDL net. But Verilog/VHDL ports connecting through a top-level SPICE net are subject to through-net optimization.

Note:

As an alternative, `real` type connections in VHDL can be used to generate unidirectional connections of SPICE ports via VHDL.

9

Mixed-Signal Simulation in the Verilog-AMS-SPICE Flow

This chapter provides an overview of the Verilog-AMS-SPICE flow.

The Verilog-AMS-SPICE flow is supported by the VCS PrimeSim AMS tool. This flow supports Verilog-AMS, as described in the Verilog-AMS LRM (with some exceptions and limitations described in this section).

The Verilog-AMS-SPICE flow supports many of the same features as those in Verilog-SPICE, described in [Mixed-Signal Simulation in the Verilog-SPICE Flow](#).

Some of the concepts in the Verilog-AMS language that must be considered when using the Verilog-AMS-SPICE flow are:

- Analog and digital blocks in Verilog-AMS
- Connect rules and connect modules
- Continuous and discrete domains
- Nets and disciplines

This chapter describes:

- [Required Input Files for the Verilog-AMS-SPICE Flow](#)
- [Nets and Disciplines](#)
- [Converting Signals With Interface A/D and D/A Connect Modules](#)
- [Compiling the Design for a Verilog-AMS-SPICE Simulation](#)
- [Running a Verilog-AMS-SPICE Simulation](#)
- [Resolving Keyword Conflicts between SystemVerilog and Verilog-AMS](#)
- [Support for Wreal Nets in Verilog-AMS Flow](#)
- [Unsupported Features in Verilog-AMS-SPICE](#)
- [Support for SystemC Designs in Verilog-AMS](#)

Required Input Files for the Verilog-AMS-SPICE Flow

Before using the Verilog-AMS-SPICE flow, the following files must be in place:

- Verilog netlist files (for example, `testbench.v`)
- Verilog-A module files, if used
- SPICE netlist files (including device model libraries)
- Mixed-signal simulation control file (for example, `vcsAD.init`)
- Command file, depending on which analog engine is being used (for example, `cfg`)
- Verilog files for Verilog-AMS files (for example, `mux2to1.vams`)

Preparing a Mixed-Signal Simulation in Verilog-AMS-SPICE

Before compiling netlists for a Verilog-AMS-SPICE simulation, you must check the netlist and other simulation files as follows to ensure identical subcircuit and module names for multi-view cells, port name matching between SPICE and Verilog views, and so on.

1. Define proper connect modules and connect rules as needed.
2. Use or modify the default connect rule and connect module files that come with the PrimeSim XA, FineSim, and PrimeSim installations:
 - The PrimeSim XA files are located in the `XA_install_dir/include/snps_connect_lib` directory.
 - The FineSim files are located in the `FineSim_install_dir/include/snps_connect_lib` directory.
 - The PrimeSim files are located in the `PrimeSim_install_dir/include/snps_connect_lib` directory.
3. Include the following two lines at the top of the first Verilog file passed to the simulator:

```
`include "constants.vams"  
`include "disciplines.vams"
```

Mixed-Signal Simulation Control File

A Verilog-AMS-SPICE simulation requires a mixed-signal simulation control file (see [Creating a Mixed-Signal Simulation Control File](#)). The format and content of this file is called at compile time using the `-ad` switch. If all analog blocks are described in Verilog-AMS or Verilog-A code, no SPICE netlist is required.

In this case, the call to the PrimeSim XA tool in the mixed-signal simulation control file requires the following command:

```
choose xa -c xa.cfg;
```

The call to the FineSim tool requires the following command:

```
choose finesim;
```

The call to the PrimeSim tool requires the following command:

```
choose primesim;
```

Selecting Multiple Views

In the Verilog-AMS-SPICE flow, cells can have multiple views with both a SPICE subcircuit and a Verilog module definition. The required view can be selected by using [use_spice](#) or [use_verilog](#) (mixed-signal control commands). Both of these commands—just as in Verilog-SPICE—support instance-based, as well as cell-based, view selection.

Note:

In the Verilog-AMS-SPICE flow, there is no distinction between Verilog-A, digital Verilog-HDL or Verilog-AMS code. Verilog-A and digital Verilog-HDL are both subsets of Verilog-AMS. Legacy digital Verilog-HDL code or legacy Verilog-A code are both considered Verilog-AMS code in the Verilog-AMS-SPICE flow.

In the Verilog-AMS-SPICE flow, a cell can only have one module definition. Regardless of the type of Verilog language used, the definition is Verilog-AMS and can be selected by using the [use_verilog](#) command.

The following example shows a mixed-signal simulation control file where both instance-based and cell-based view selection commands are used.

```
choose xa -nspice spice_files.spi -c cfg;

// cell-based use_verilog
use_verilog -module mux;

// cell-based use_spice
use_spice -cell inverter;

// instance-based use_verilog
use_verilog -module mux -inst top.i1.i2;

// instance-based use_spice
use_spice -cell inverter -inst top.i1.i3;
```

Note:

In the Verilog-AMS-SPICE flow, all Verilog files are passed to VCS at compile time. In the Verilog-SPICE flow, Verilog-A files are passed to FastSPICE, using

the ``hdl` command. Although this feature is also supported in the Verilog-AMS-SPICE flow, it is recommended that, in the Verilog-AMS-SPICE flow, all Verilog files, including legacy Verilog-A, be passed to VCS at compile time.

Files Containing Connect Rule and Connect Module Definitions

In Verilog-AMS-SPICE, definitions for connect rules and connect modules are passed to VCS just as any other Verilog file. They can be used as is or as templates for customized connect modules and connect rules.

These default files can be used, or new files can be created by creating a copy of the default files and modifying them to suit the specific design characteristics (for example, changing the `vsup` supply voltage parameter for the connect module).

The following example shows how connect rule and connect module definitions can be passed to the simulator.

```
% vcs -ad -ams snps_cm_a2d.vams snps_cm_d2a.vams snps_crules.vams ...
```

In this example, it is assumed that the files are in the local directory. If not, a full path to the location of these files is required (just as in any other Verilog file).

Understanding Analog and Digital Blocks in Verilog-AMS

Verilog-AMS supports both analog and digital block definitions within the same module. Digital blocks can access nets in the analog block and vice-versa.

The following example shows Verilog-AMS code with both digital and analog blocks. The port `in` is defined as a net of discipline *logic*, while port `out` is declared as a net of discipline *electrical*.

```
`include "constants.vams"
`include "disciplines.vams"

module mydesign (in, out);
  input  in;
  output out;
  logic  in;
  electrical out;

  // Digital block
  reg clk;
  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end

  // Analog block
  analog begin
    if (in == clk)
```

```
V(out) <+ 1.8;
else
  V(out) <+ 0.0;
end
endmodule
```

A Verilog-AMS module can contain many digital blocks, but it can only contain one analog block as defined in the *Verilog-AMS Language Reference Manual*.

An analog block is identified by the `analog` keyword. All other blocks inside a module definition, such as `initial` and `always` blocks, are considered to be digital blocks (for example, `initial` and `always` blocks).

Because Verilog-AMS is a superset of the Verilog-A and digital Verilog-HDL languages, a module that only contains digital blocks (conventional digital Verilog HDL) or a module that only contains an analog block (conventional Verilog-A) is also viewed and treated as Verilog-AMS code by the simulator.

Nets and Disciplines

In Verilog-AMS, the term *net* refers to nodes inside a module that provide connections between two or more submodules. Every port of an instantiated module provides a connection between two nets.

A net can belong to a digital or analog domain. The way to declare the domain for a net is by associating it with a predefined discipline in Verilog-AMS. Two of the most common disciplines in Verilog-AMS are *electrical*—for analog nets, and *logic*—for digital nets.

The discipline declaration for each net can be made explicitly in the code, or the simulator can resolve it, based on the discipline resolution algorithm. However, the discipline of all nets in a Verilog-AMS code must be determined before the simulation can start.

In the following example, a module contains internal nets `n1`, `n2`, and `n3` that connect different submodules. No explicit disciplines are declared for these nets, and the simulator can use the discipline resolution method to assign the proper discipline.

```
`include "constants.vams"
`include "disciplines.vams"

module test;

  // no disciplines are declared for nets "n1", "n2" and "n3"

  blk_a i1 (.a( n1), .b(n2), .z(n3) );
  blk_b i2 (.out(n1) );
  blk_c i3 (.a(n3), .out(n2) );

endmodule
```

In the digital Verilog-HDL language, no explicit discipline declaration is made for `wire`, `reg`, or `port`, so the compilation of the Verilog-HDL files can fail when they are used in the Verilog-AMS-SPICE flow. To resolve this problem, declare a default discipline by placing the ``default_discipline` directive inside the Verilog code or by using the `-ams_discipline` VCS compile switch.

The following example uses the ``default_discipline` directive in Verilog code to define the discipline `logic` as the default discipline. In this example, ports `a` and `b` of module `mydesign` have no explicit discipline declaration. However, because of the ``default_discipline logic` statement, the `logic` discipline is assigned.

```
`include "constants.vams"
`include "disciplines.vams"

`default_discipline logic
module mydesign ( a , b);
input a;
output b;
...
endmodule
```

In the following example, the `-ams_discipline` VCS switch defines a default discipline.

```
vcs testbench.v -R -ad -ams_discipline logic
```

Unlike nets, disciplines are not explicitly declared for variables. The domain of a variable is determined by the context in which the assignment is made. If the variable is assigned a value in an analog context (that is, in an analog block), the variable is considered *analog*. However, if the assignment is made in a digital context (that is, in a digital block), the variable is considered to be *digital*. A variable can only be assigned a value in one domain, but it can be accessed for reading from any domain.

The following example shows a digital domain variable. The real variable `v` is assigned a value in the digital domain (inside the `initial` block), and as a result is considered a digital variable.

```
`include "constants.vams"
`include "disciplines.vams"

module mydesign;
  real v;

  initial begin
    v=1.5;
  end
  ...
endmodule
```


The following example shows a variable definition in Verilog-AMS code. The variable is assigned a value in the analog domain, and is treated as an analog variable.

```
`include "constants.vams"
`include "disciplines.vams"

module mydesign;
  real v;

  analog begin
    ...
    v = 1.5;
    ...
  end
  ...
endmodule
```

Converting Signals With Interface A/D and D/A Connect Modules

In Verilog-AMS, the conversion of signals between the analog and digital domains is done by interface blocks called *connect modules*. Connect modules are inserted automatically by the simulator at the interface between analog and digital nets. Connect modules can also be inserted manually.

A connect module is a predefined Verilog-AMS module with two ports—one analog and one digital. The following example shows a sample connect module named `snps_cm_a2d` that takes an input of type `electrical` and produces an output of type `logic`.

```
connectmodule snps_cm_a2d (ain, dout);
output dout;
input ain;
logic dout;
electrical ain;

// body of the connect module goes here

endmodule
```

Identifying the Correct Connect Module

The simulator performs the following steps to identify the appropriate connect module for each interface:

1. The simulator ensures that every net in a module has a defined discipline. If a net does not have an explicit discipline definition, the discipline resolution algorithm is called to resolve and assign a discipline based on the discipline of other nets connected to it.
2. After the disciplines for all nets are determined, the simulator identifies connections between nets of digital and analog disciplines.
3. Connect modules with matching disciplines are then inserted between the analog and digital nets. Direction of the ports are also accounted for.

Note:

These steps occur at compilation time. If any steps are unsuccessful, the compile process exits with an error message.

Verilog-AMS associates a particular connect module, based on the disciplines and port directions of the two nets connected, by defining a connect rule that is passed to the simulator.

Connect modules of this type are commonly called `a2d` connect modules. Connect modules that take a digital input and deliver an analog output are usually called `d2a` connect modules. The connect modules with bidirectional analog and digital ports are called `bidi` connect modules.

Understanding Connect Rules

The following example shows a connect rule where two connect module associations are made. The first rule defines the connect module to be used in case an `a2d` interface must be inserted between two nets. The second rule defines the connect module to be used as a `d2a` interface between two nets.

```
connectrules snps_crules;  
  connect snps_cm_a2d input electrical, output logic ;  
  connect snps_cm_d2a input logic, output electrical ;  
endconnectrules
```

A connect rule may contain many more associations; for example, to define interfaces between other types of digital and analog disciplines, or to define *bidi* connect modules.

Note:

Connect rules and connect modules are only deployed by the simulator *if* there is a connection between a digital and an analog net. If no direct connection exists throughout the netlist, connect rules and connect modules are not used.

In Verilog-AMS-SPICE, all conversions of signals between digital and analog are performed by connect modules.

The following example does not require a connect module:

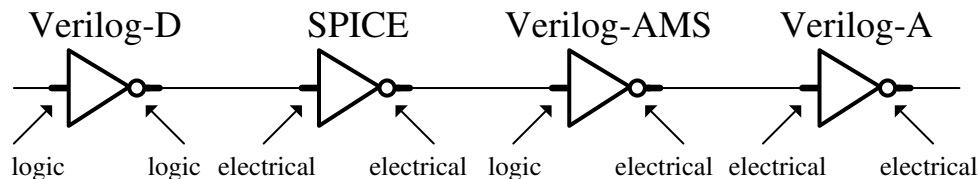
```
module mydesign (dig_in, dig_out, an_in, an_out);
    input dig_in, an_in;
    output dig_out, an_out;
    reg dig_out;
    logic dig_in, dig_out;
    electrical an_in, an_out;

    // Analog signal accessed in digital domain
    always @ (above(V(an_in), +1))
        dig_out = 1b'1;

    analog begin
        // Digital signal accessed in analog domain
        if (dig_in == 1b'1)
            V(an_out) <+ 1.8;
    end
    ...
endmodule
```

Figure 21 shows an example of a simple inverter chain circuit where connect modules are used to model the direct connection of analog and digital nets.

Figure 21 Cells of an Inverter Chain Modeled in SPICE, Verilog-D, Verilog-AMS and Verilog-A



The circuit in this example contains four inverters:

- The first inverter is modeled in digital Verilog (Verilog-D)
 The inverter is defined by a module that contains only digital blocks
- The second inverter is modeled in SPICE
 All ports of a SPICE cell assume `electrical` discipline in Verilog-AMS-SPICE
- The third inverter is modeled in Verilog-AMS with `logic` input and `electrical` output
 The inverter is defined by a module that contains both an analog block and digital blocks

- The fourth inverter is modeled in Verilog-A

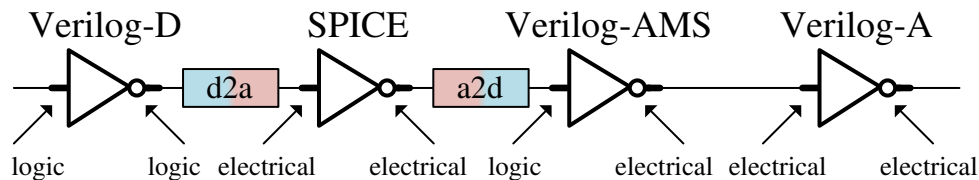
The inverter is defined by a module that contains only an analog block

Note:

In the Verilog-AMS-SPICE flow, all "Verilog" modules, regardless of whether they contain an analog or digital block, or both, are considered Verilog-AMS code. The examples of Verilog-A and Verilog-D referenced in [Figure 21](#) are only used for clarification.

[Figure 22](#) shows where the simulator inserts connect modules. There are two inserted connect modules: d2a and a2d.

Figure 22 *Mixed-Signal Design With Connect Modules*



Note:

Connect rules and connect modules are not used if there is no direct connection between analog and digital nets. This occurs if the interactions between analog and digital nets are implemented by analog/digital cross-boundary sampling within a Verilog-AMS module, not through port connections.

However, if there are direct connections between analog and digital nets (for example, nets of electrical and logic disciplines), the definitions for connect rules and connect modules must be passed to the simulator at compile time.

Files Containing Connect Rule and Connect Module Definitions

The files that contain default definitions for connect rules and connect modules are:

- `snps_cm_a2d_1.vams` and `snps_cm_d2a_1.vams` (connect module)

These files contain the default `a2d` and `d2a` connect module definitions. The connect modules contain many parameters that define their behavior, such as parameters to define the analog supply voltage, high and low thresholds for `a2d` or `d2a` conversions, delay times, and so on.

These parameters have default values that can be overridden when the modules are referenced in the connect rule files.

- `snps_crules_1_xx.vams` (connect rule)

Specific `xx` strings signify specific connect rule definitions for specific analog supply voltages. For example, the default connect rule file for a 1.8V supply is:

```
snps_crules_1_18.vams
```

If the design has specific characteristics (for example, an analog supply voltage of 1.6V) that do not match any of the default connect rule files, define a connect rule file by copying one of the default files to a new file and change the parameters for the connect module referenced inside of it.

Compiling the Design for a Verilog-AMS-SPICE Simulation

The following compile options are specific to the Verilog-AMS-SPICE flow:

- `-ams` (mandatory)
- `-ams_discipline logic` (optional, but highly recommended)
- `-ams_dresolution`
- `-ams_iereport` (optional)

-ams

Enables the Verilog-AMS-SPICE feature. Specify this option on the VCS command line as follows:

```
% vcs -ad -ams ...
```

-ams_discipline logic

Treats all nets without an explicit discipline definition as discipline type `logic`. This compile option is used when importing legacy Verilog-D code in which no disciplines are defined for module ports. By using this option, discipline `logic` is assigned to all such ports.

While `logic` is the digital discipline commonly used, any other discipline name can be used in Verilog-AMS as the default discrete discipline. Especially if Verilog-AMS is used along with SystemVerilog, a name different from `logic` must be used for default discrete discipline because `logic` is a reserved SystemVerilog type. The following example specifies a discrete discipline called `logical` as the default discipline.

```
% vcs -ams_discipline logical ...
```

-ams_dresolution

Enables detail discipline resolution. The Verilog-AMS LRM describes basic and detail discipline resolution. By default, basic discipline resolution is used, but this compile time option switches to detail discipline resolution. Typically, detail discipline resolution causes more connect modules to be inserted.

```
% vcs -ams_dresolution ...
```

-ams_iereport

Records a list of all instances of connects modules in the design. The list is displayed at the console and saved in the log file with the following information:

- Instance name under which the connect module was inserted
- Instance name of the connect module
- Module name of the connect module
- Discipline resolution mode used (that is, merged or split)
- Net and port that connect to each end of the connect module

A full report, which also lists the final parameter values for each connect module instance, is output to `simv.msv/connectmodule.rpt`.

Verilog Netlist Files

In Verilog-AMS-SPICE, all Verilog files are specified on the command line at compile time. Verilog files include Verilog-D, Verilog-A, or full Verilog-AMS files as shown in the following example:

```
% vcs -ams -ad testbench.v block1.va block2.vams ...
```

In this example, `testbench.v` contains Verilog-D code, `block1.va` contains Verilog-A code, and `block2.vams` contains Verilog-AMS code.

Note:

It is possible to pass Verilog-A files to the simulator using the SPICE `.hdl` command, but it is highly recommended to pass the files at compile time.

Recompiling the Design

In the current standalone version of the VCS tool, the `-Mupdate` option is on by default. As mixed-signal requires both the compile-time phase and runtime phase to work together, do not compile incrementally.

Do the following when you modify any design files (Verilog, Verilog-A, or SPICE) or the mixed-signal simulation control file:

- Remove the `simv.daidir` and `csrc` directories generated by the previous mixed-signal simulation.
- Restart the compilation.

Also, if you use the analog configuration commands to change the case sensitivity in SPICE netlists, then you must recompile the design before those changes can take effect.

Running a Verilog-AMS-SPICE Simulation

To run the Verilog-AMS-SPICE simulation, add the `-R` switch to the compile script so the simulation starts automatically after compiling, or specify the following to run the mixed-signal simulation:

```
% simv runtime_options
```

Note:

`simv` is the default name for the binary simulation executable generated after compilation. You can change this name by using the `-o exec_file_name` compile time switch.

Resolving Keyword Conflicts between SystemVerilog and Verilog-AMS

There are certain function names and keywords in SystemVerilog that cannot be used in Verilog-AMS, such as `analog`, `max`, `sin` and so on. Likewise, there are certain keywords in Verilog-AMS that cannot be used in SystemVerilog, such as `interface` and `logic`.

To avoid such conflicts, SystemVerilog and Verilog-AMS code should be parsed separately by using their own language parsers. One way to achieve that is to specify distinct file name extensions for SystemVerilog and Verilog-AMS files. For example:

- `*.v`, `*.sv`, or `*.svh` for SystemVerilog files
- `*.vams` for Verilog-AMS files

If you use this naming convention, you can use the `+verilogamsext` and `+systemverilogext` switches to identify the file extensions used for SystemVerilog and Verilog-AMS as shown in the following example:

```
% vcs -ams -ad +verilogamsext+vams +systemverilogext+sv+v+svh ...
```

This technique separates the language context for SystemVerilog and Verilog-AMS at compilation time and avoids the potential conflict between keywords and function names.

Support for Wreal Nets in Verilog-AMS Flow

Connect modules are inserted based on the specified connect rules. Connect rules are created to control which connect modules must be used and where they must be inserted. The connect modules are also searched as per the connect rules defined and first matched connect module is selected.

To select the connect module, the discipline of the wreal net or port should also match along with the `wreal` type.

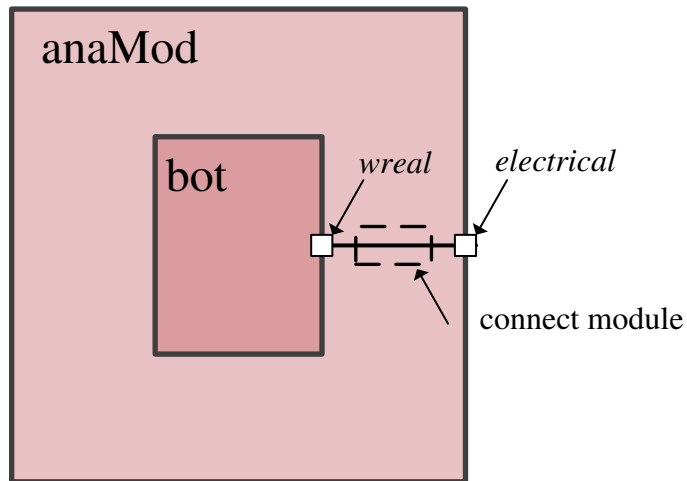
Usage Example

The following example defines connect modules that matches the discipline of the `wreal` net or port along with the `wreal` type:

```
module bot(output wreal p1);
    myLogical p1;
    assign p1 = 1.0;
    anaMod ANAMOD (p1);
endmodule

// Analog module
module anaMod (e1);
    output e1;
    electrical e1;
endmodule
```


Figure 23 Connect Module



The following connect modules are used in this example.

```
// Following connect module has output port with myLogical discipline
// with wreal port type.
connectmodule elect_to_wreal (ain, dout);
input ain;
electrical ain;
output dout;
wreal dout;
myLogical dout;
...
endmodule

// The following connect module has output port with myLogical discipline
// with wire type.
connectmodule elect_to_logic (ain, dout);
input ain;
electrical ain;
output dout;
myLogical dout;
...
endmodule

connectrules mixedsignal;
  connect elect_to_logic input electrical, output myLogical;
  connect elect_to_wreal input electrical, output myLogical;
...
endconnectrules
```

In the preceding example, a connect module is required inside the `bot` module because `p1 wreal` signal is connected to the `e1 electrical` port of the `anaMod` module. To select the connect module, there are two connect rules that refer to two different connect modules, that is, `elect_to_logic` and `elect_to_wreal` respectively.

The `elect_to_wreal` connect module has digital output port of type `wreal` with `myLogical` discipline. Therefore, the `elect_to_wreal` connect module is selected as per the connect module selection algorithm.

Unsupported Features in Verilog-AMS-SPICE

The following Verilog-AMS 2.4 features are not currently supported in the Verilog-AMS-SPICE flow:

- Analog voltage or current contribution from Verilog using cross-Module Referencing (XMR) across the analog/digital boundary

Note:

You can use XMR from inside Verilog only to probe (read) an analog voltage or current.

- `abstol` parameter in `ddt()` (LRM chapter 4.4.4)
- `aliasparam` (LRM chapter 3.2)
- `ddx()` (LRM chapter 4.4.7)
- driver access functions, `$driver_XXX` (LRM chapter 8.10)
- hierarchical system parameters (LRM chapter 7.2.6)
- `last_crossing` (LRM chapter 4.4.11)
- `localparam` (LRM chapter 3.2)
- parameterized-width analog buses
- parameter arrays (LRM chapter 3.2.4)
- `paramsets` (LRM chapter 7.3)
- `$param_given()` (LRM chapter 10.11)
- `$port_connected()` (LRM chapter 10.11)
- predefined macros (LRM chapter 11.7)
- `$rdist_XXX` functions (LRM chapter 10.3)
- `$simparam()` (LRM chapter 10.1)
- VPI routines (LRM chapter 13)

Support for SystemC Designs in Verilog-AMS

SystemVerilog-SPICE flow is supported with SystemC in the design topologies using the `-sysc=ams` option during VCS elaboration. Verilog-AMS is supported with SystemC in the design topologies using the same option `-sysc=ams`.

Note:

The `-ams` option must be provided along with the `-sysc=ams` option.

Use Model

This section provides the use model to support Verilog-AMS and SPICE with SystemC designs.

The other options of VCS elaboration for mixed-signal simulations remain the same.

If the `-sysc=ams` option is not used for Verilog-AMS elaboration, then an error message is generated that mentions Verilog-AMS is not supported.

Usage Example

The following example illustrates the usage of a SystemC design in a simulation with Verilog-AMS and SPICE:

Example 36 `test_sc.v`

```
// test_sc.v
`include "connectmodules.vams"
`include "bottom2.vams"
`timescale 1ns/1ps

interface myport (
    output wire  iw1,
    output wire  iw2,
    output wire  iw3,
    output bit   ib1,
    output bit   ib2,
    output bit   ib3,
    output logic il1,
    output logic il2,
    output logic il3);
endinterface

// ===== VERILOG TOP =====
module top;
    wire  w1, w2, w3;
    bit   b1, b2, b3;
    logic l1, l2, l3;
```

```

myport MYPORT (w1,w2,w3, b1,b2,b3, l1,l2,l3);
wire x;
wire y;

middle1 VM1 (.p1(x), .p2(MYPORT), .p3(y));

initial begin
    $fsdbDumpfile("vcs.fsdb");
    $fsdbDumpvars("+all");
end
endmodule

// ===== VERILOG MIDDLE =====
module middle1 (output wire p1, myport p2, output wire p3);
    // SPICE
    myspice MY_SPICE_1 (p2.iw1, p2.iw2, p2.iw3, p2.ib1, p2.ib2, p2.ib3,
        p2.il1, p2.il2, p2.il3);
endmodule

// ===== SPICE MIDDLE: MULTIPLE VIEW MODULE =====
module spiceInv (output wire out, input wire in);
    assign out = ~in;
endmodule

module myspice (output wire w1, w2, w3, w4, w5, w6, w7, w8, w9);
    wire z1, z2, z3, z4, z5, z6, z7, z8, z9;

    // VERILOG
    bottom1 VLOG_BOTTOM_1 (z1, z2, z3, z4, z5, z6, z7, z8, z9);

    spiceInv SP1 (w1, z1);
    spiceInv SP2 (w2, z2);
    spiceInv SP3 (w3, z3);
    spiceInv SP4 (w4, z4);
    spiceInv SP5 (w5, z5);
    spiceInv SP6 (w6, z6);
    spiceInv SP7 (w7, z7);
    spiceInv SP8 (w8, z8);
    spiceInv SP9 (w9, z9);
endmodule

// ===== VERILOG BOTTOM =====
module bottom1 (output wire w1, w2, w3, w4, w5, w6, w7, w8, w9);
    sc_bottom1 SC_BOTTOM_1 (w1, w2, w3, w4, w5, w6, w7, w8, w9);
endmodule

```

Example 37 test_sc.spi

```

.inc "spiceinv.spi"

subckt myspice w1 w2 w3 w4 w5 w6 w7 w8 w9
// VERILOG

```

```
VLOG_BOTTOM_1 z1 z2 z3 z4 z5 z6 z7 z8 z9 bottom1
// SPICE
SP1 w1 z1 spiceInv
SP2 w2 z2 spiceInv
SP3 w3 z3 spiceInv
SP4 w4 z4 spiceInv
SP5 w5 z5 spiceInv
SP6 w6 z6 spiceInv
SP7 w7 z7 spiceInv
SP8 w8 z8 spiceInv
SP9 w9 z9 spiceInv
ends myspice
```

Example 38 *spiceinv.spi*

```
simulator lang=spice
.inc 'models.spi'
*epic tech="voltage 3.3"

.global vss vdd
vss vss 0 0
vdd vdd 0 3.3

.subckt spiceInv out in
mn1 out in vss vss n w=5.00u l=0.35u
mp1 out in vdd vdd p w=10.00u l=0.35u
.ends
```

Example 39 *disciplines.vams*

```
// disciplines.vams
`ifdef DISCIPLINES_VAMS
`else
`define DISCIPLINES_VAMS 1
//
// Natures and Disciplines
//
discipline logical
    domain discrete;
enddiscipline

// Electrical
// Current in amperes
nature Current
    units = "A";
    access = I;
    idt_nature = Charge;
`ifdef CURRENT_ABSTOL
abstol = `CURRENT_ABSTOL;
`else
abstol = 1e-12;
`endif
endnature
```

Chapter 9: Mixed-Signal Simulation in the Verilog-AMS-SPICE Flow Support for SystemC Designs in Verilog-AMS

```

// Charge in coulombs
nature Charge
  units = "coul";
  access = Q;
  ddt_nature = Current;
`ifdef CHARGE_ABSTOL
abstol = `CHARGE_ABSTOL;
`else
abstol = 1e-14;
`endif
endnature
// Potential in volts
nature Voltage
  units = "V";
  access = V;
  idt_nature = Flux;
`ifdef VOLTAGE_ABSTOL
abstol = `VOLTAGE_ABSTOL;
`else
abstol = 1e-6;
`endif
endnature
// Flux in Webers
nature Flux
  units = "Wb";
  access = Phi;
  ddt_nature = Voltage;
`ifdef FLUX_ABSTOL
abstol = `FLUX_ABSTOL;
`else
abstol = 1e-9;
`endif
endnature
// Conservative discipline
discipline electrical
  potential Voltage;
  flow Current;
enddiscipline
// Signal flow disciplines
discipline voltage
  potential Voltage;
enddiscipline
discipline current
  potential Current;
enddiscipline
`endif

```

Example 40 *connect_modules.vams*

```

// connect_modules.vams
`include "disciplines.vams"
`timescale 1ns/1ps

```

Chapter 9: Mixed-Signal Simulation in the Verilog-AMS-SPICE Flow Support for SystemC Designs in Verilog-AMS

```

connectmodule elect_to_logic(e1, cm);
input e1; electrical e1;
output cm; logical cm;
reg cm_1;
real vel;
assign cm = cm_1;

always @(above(V(e1) - 1.65)) begin
    cm_1 = 1;
end
always @(above(1.65 - V(e1))) begin
    cm_1 = 0;
end
endmodule

connectmodule logic_to_elect(l2e_cm,l2e_el);
input l2e_cm; logical l2e_cm;
output l2e_el; electrical l2e_el;

assign l2e_cm = l2e_cm;
analog begin
    V(l2e_el) <+ transition((l2e_cm == 1) ? 3.3 : 0.0, 0.1n, 0.1n ,0.1n);
end
endmodule

connectrules mixedsignal;
connect elect_to_logic input electrical, output logical;
connect logic_to_elect input logical, output electrical;
endconnectrules

```

Example 41 *sc_bottom1.cpp*

```
#include "sc_bottom1.h"
```

Example 42 *sc_bottom1.h*

```

#ifndef sc_bottom1_h
#define sc_bottom1_h

#include "systemc.h"
#include "bottom2.h"

SC_MODULE(sc_bottom1)
{
    sc_out<bool> w1;
    sc_out<bool> w2;
    sc_out<bool> w3;
    sc_out<bool> w4;
    sc_out<bool> w5;
    sc_out<bool> w6;
    sc_out<bool> w7;
    sc_out<bool> w8;
    sc_out<bool> w9;
}

```

```

bottom2 VLOG_BOTTOM_2;

SC_CTOR(sc_bottom1):
w1("w1")
  ,w2("w2")
  ,w3("w3")
  ,w4("w4")
  ,w5("w5")
  ,w6("w6")
  ,w7("w7")
  ,w8("w8")
  ,w9("w9")
  ,VLOG_BOTTOM_2("VLOG_BOTTOM_2")
{
  VLOG_BOTTOM_2.11(w1);
  VLOG_BOTTOM_2.12(w2);
  VLOG_BOTTOM_2.13(w3);
  VLOG_BOTTOM_2.14(w4);
  VLOG_BOTTOM_2.15(w5);
  VLOG_BOTTOM_2.16(w6);
  VLOG_BOTTOM_2.17(w7);
  VLOG_BOTTOM_2.18(w8);
  VLOG_BOTTOM_2.19(w9);
}
};
#endif

```

Example 43 vcsAD.init

```

choose xa -nspice test_sc.spi -c xa.cfg;
use_spice -cell spiceInv;
use_spice -cell myspice;

```

To run the example, use the following commands:

```

% vlogan -kdb -full64 -sverilog -sc_model bottom2 test_sc.v -ams
% syscan -full64 sc_bottom1.cpp:sc_bottom1
% vcs -kdb -lca -debug_access+all -full64 \
  -cc /path/to/gcc -ams_discipline logical -sysc top \
  -sysc=ams -ad -ams
% simv

```


10

Output Files

This chapter describes how to generate digital and analog waveform output files and a unified dump output file.

- [Capturing Analog and Digital Signals in Output Files](#)
- [Generating Merged FSDB Files](#)
- [Unified Dump of Voltage and Current Signals](#)
- [Writing Out Both Sides of an Interface Element](#)

Capturing Analog and Digital Signals in Output Files

To capture digital and analog signals in output files, use two sets of commands: one set for analog signals and another set for digital signals.

Generating an Analog Output File

Any method of printing analog signals that is used in standalone analog simulations for the PrimeSim XA tool can also be used in the Verilog-SPICE flow, such as:

- The PrimeSim XA `probe_waveform_voltage * -port 1 -limit 99` and `probe_waveform_current * -port 1 -limit 99` commands to output all analog nodes throughout the hierarchy in the output file.
- The PrimeSim XA `probe_waveform_va *.* -limit 99` configuration command to capture Verilog-A variables in the output.
- The `$display`, `$strobe` and `$write` functions within the Verilog-A code to output Verilog-A variables or voltage and current values on the screen.
- SPICE-specific `print` commands, such as the PrimeSim HSPICE-compatible `.print` and `.probe` commands.

The format of the analog output file can be any one of the formats supported by the analog engine, including `.vpd`, `.fsdb` and ASCII `.out`.

The following configuration commands must be used to set the output format for analog signals:

- For the PrimeSim XA tool:

Use the `set_waveform_option -format fsdb|wdf|out|vpd` command inside the PrimeSim XA command file. See the *PrimeSim XA User Guide* for more details.

- For the FineSim tool:

Use the SPICE option:

```
.option finesim_output=fsdb|wdf|psf|tr0|psfascii|none|out
```

For more information about this command, see the *FineSim User Guide: Pro and SPICE Reference*.

- For the PrimeSim tool:

Use the SPICE option:

```
.option primesim_output=fsdb|wdf|psf|tr0|psfascii|none|out
```

For more information about this command, see the *PrimeSim User Guide: Pro and SPICE*.

By default, the hierarchical nets that appear in multiple levels of the SPICE hierarchy with different names (aliases) appear only once under their top-level alias. This causes the size of the analog output file to be more compact but could make tracing signals through the hierarchy difficult. The following configuration commands enable printing of all analog hierarchical aliases:

- For the PrimeSim XA tool, use the `-port 1` option of the `probe_waveform_voltage` and `probe_waveform_current` commands. For example:

```
probe_waveform_voltage * -port 1
```

- For the FineSim tool, use the following SPICE option:

```
.OPTION POST PROBE  
.probe v(*) i(*)
```

- For the PrimeSim tool, use the following SPICE option:

```
.OPTION POST PROBE  
.probe v(*) i(*)
```

Generating a Digital Output File

The preferred output format for digital signals is FSDB, since FSDB supports merging of analog and digital outputs into a single merged FSDB file as described in the following section. Here are the steps to generate an FSDB output file for digital signals:

1. Use the `-debug_access+all` compile time options with VCS to enable VPD generation by VCS, as follows:

```
% vcs -ad -debug_access+all
```

2. Make sure that the Verdi tool is installed and is included in the path.

3. Use the simv runtime options `-ucli -i ucli_command_file` to pass UCLI commands to output an FSDB file. The command file must contain a `dump` command to create an FSDB file. The command file can also specify the name of the output FSDB file. The following example shows how the UCLI command file is read at runtime:

```
% simv -ucli -i ucli_command_file
```

Where the content of `ucli_command_file` is:

```
dump -file vcs.fsdb -type fsdb
dump -add top.dut -iall
run 1000
quit
```

These commands specify the type of the output file (FSDB), the name of the output file, and the signals and levels of hierarchy to write to the FSDB output. For a complete list of all options for the UCLI `dump` command, refer to the *VCS Unified Command-Line Interface User Guide*.

When you run parallel simulations (Monte Carlo Analysis, `.alter` and `.sweep`), you must output the waveforms by using UCLI commands and specify the `-fid` option. Other waveform output commands, such as system tasks or command-line options, are not supported.

The following example generates an FSDB format file:

```
ucli% dump -file vcs.fsdb -type fsdb
ucli% dump -add / -depth 10 -fid FSDB0
```

The following example generates a VPD-format file:

```
ucli% dump -file vcs.vpd -type vpd
ucli% dump -add / -depth 10 -fid VPD0
```

This limitation also applies when merged analog and digital waveforms are used, or when unified dump is used.

Generating Merged FSDB Files

This section describes how to generate merged FSDB files in a mixed-signal simulation. Merged FSDB is supported in the VCS PrimeSim AMS tool.

Generating Merged FSDB

You can use the following method to generate a unified FSDB output file that contains both analog and digital waveforms.

Note:

The PrimeSim XA, FineSim, or PrimeSim simulation engines support a merged FSDB file only in FSDB version 5.4 or older.

To generate a merged FSDB file, do the following:

1. Follow the steps to generate an FSDB output from VCS mentioned in [Generating a Digital Output File](#).
2. Follow the steps to generate an FSDB output from the PrimeSim XA, FineSim, or PrimeSim simulation engines described in [Generating an Analog Output File](#).
3. The PrimeSim XA, FineSim, or PrimeSim simulation engines must be instructed to merge their FSDB output with the digital FSDB file. Use the following commands to merge a FSDB file containing both analog and digital signals:

- For the PrimeSim XA simulation engine:

Use the `set_waveform_option -format fsdb -file merge` configuration command.

- For the FineSim simulation engine:

Use the `.option finesim_output=fsdb` and `.option finesim_merge_fsdb=1` commands.

- For the PrimeSim simulation engine:

Use the `.option primesim_output=fsdb` and `.option primesim_merge_fsdb=1` commands.

4. Add the UCLI `dump -file file_name -type fsdb` command to the UCLI command file as shown in the following example.

```
# ucli.cmd
dump -file OUTPUT/out.fsdb -type fsdb
dump -add / -depth 10 -fid fsdb0
run 10 us
exit
```

5. Run the VCS tool with the `-debug_access+all` command line switch:

```
% vcs -ad -debug_access+all
```

6. Run the simulation executable and specify the UCLI command file. Make sure the name of the fsdb file (OUTPUT/out.fsdb in this example) matches the fsdb file name specified in the UCLI command file in the previous step.

```
% simv -ucli -i ucli.cmd +fsdbfile+OUTPUT/out.fsdb
```

For more information on FSDB file generation in the VCS tool, see the *VCS/VCSi User Guide*.

Unified Dump of Voltage and Current Signals

The unified dump command is the preferred mechanism for probing and dumping analog and digital signals by using one command, regardless of target type. If the mixed-signal mode of the `dump` command is enabled, the command outputs the target node whether it is analog or digital. This capability ensures that the signal is written out, regardless of the view used during simulation.

Note:

The unified dump capability in the VCS PrimeSim AMS tool only supports the FSDB file format.

To enable the unified dump of voltage and current signals, do the following steps.

1. Compile your design with the `-debug_access` option. For example:

```
% vcs ... -debug_access+all
```

For more information about the `-debug_access` option, see the *VCS® User Guide* and *Unified Command-Line Interface User Guide*.

2. Create a unified dump output data file with the existing VCS UCLI `dump` command. For example:

```
ucli% dump -file vcsAms.fsdb -type fsdb
```

3. By default, the `dump -add` command outputs voltages. You need to specify additional `dump -add` commands with options to output mixed-signal terminal voltages and currents. By default, the `dump -add` command outputs voltages.

For example:

```
ucli% dump -add top -depth 0 -v -vN -vall -i -iN -iall -isub  
ucli% dump -add top.dut -iall -fid FSDB0
```

You can add these UCLI commands to a text file and use the `-ucli -i filename` option to the `simv` simulation executable to run the commands as shown in the following example:

```
# Create the command file
% cat ucli.cmd
dump -file vcsAms.fsdb -type fsdb
dump -add top.dut -iall -fid FSDB0
run 1000

# Run the simulation
% simv -ucli -i ucli.cmd
```

For more information about the `dump` command, see the *VCS® User Guide*.

Setting the Scope of Dumping Current Signals

By default, the `dump -add scope` commands dump both analog and digital signals. If you need to dump only digital signals, use the `dump -msv off` command to disable analog dumping. Note that:

- Scope protection is required when dumping current values. To do this:
 - Specify the `iprobe_waveform_current` command in a PrimeSim XA configuration file.
 - Specify the `.option finesim_iprobe_scope="current_probe_pattern"` command for the FineSim tool.
 - Specify the `.option primesim_iprobe_scope="current_probe_pattern"` command for the PrimeSim tool.
- Dumping Verilog-A port voltages, currents, and variables is supported for the VCS PrimeSim AMS flow using the PrimeSim XA, FineSim and PrimeSim simulation engines by using the following options: `-vaV`, `-vaI`, and `-va`.

For example:

```
dump -file vcsAms.fsdb -type fsdb
dump -msv off
dump -add top.dut -fid FSDB0
run 1000
```

Table 9 *Dumping Voltages*

UCLI Dump Voltages	PrimeSim XA Probe Equivalent
<code>dump -add top.dut -v</code>	<code>iprobe_waveform_voltage -port 1 top.dut.*</code>

Table 9 *Dumping Voltages (Continued)*

UCLI Dump Voltages	PrimeSim XA Probe Equivalent
<code>dump -add top.dut -vN</code>	<code>iprobe_waveform_voltage -vN top.dut.*</code>
<code>dump -add top.dut -vall</code>	<code>iprobe_waveform_voltage -vall top.dut.*</code>

Table 10 *Dumping Currents*

UCLI Dump Currents	PrimeSim XA Probe Equivalent	FineSim / PrimeSim Probe Equivalent
<code>dump -add top.dut -i</code>	<code>iprobe_waveform_current top.dut.*</code>	<code>.option finesim_iprobe_scope = "i(top.dut.*)"</code> <code>.option primesim_iprobe_scope = "i(top.dut.*)"</code>
<code>dump -add top.dut -iN</code>	<code>iprobe_waveform_current -iN top.dut.*</code>	<code>.option finesim_iprobe_scope = "iN(top.dut.*)"</code> <code>.option primesim_iprobe_scope = "iN(top.dut.*)"</code>
<code>dump -add top.dut -iall</code>	<code>iprobe_waveform_current -iall top.dut.*</code>	<code>.option finesim_iprobe_scope = "iall(top.dut.*)"</code> <code>.option primesim_iprobe_scope = "iall(top.dut.*)"</code>
<code>dump -add top -isub</code>	<code>iprobe_waveform_current -isub top.*</code>	<code>.option finesim_iprobe_scope = "x(top.*)"</code> <code>.option primesim_iprobe_scope = "x(top.*)"</code>

Writing Out Both Sides of an Interface Element

You must use the `+IE_Only` option in the `-fsdb_opt` argument of the UCLI `dump -add` command or `$fsdbDumpvars` system task to dump both the analog and digital sides of the interface element. This option can help you debug issues with interface elements by capturing the activities of the signals at the mixed-signal boundary. The UCLI command syntax is:

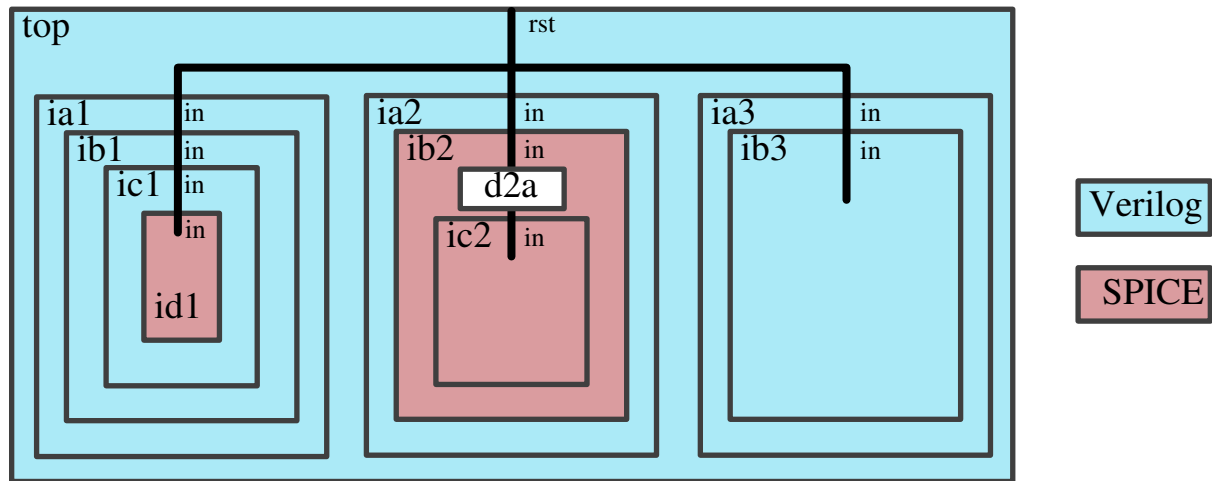
```
dump -add <scope> [-fsdb_opt +IE_Only]
```

or

```
$fsdbDumpvars (" +IE_Only" );
```

The following example writes out the listed signals in the FSDB waveform file for the design shown in [Figure 24](#):

Figure 24 Dumping Both Sides of Interface Elements



```
ucli% dump -add top -fsdb_opt +IE_Only
```

Digital signals: top.ia2.in, top.ia1.ib1.ic1.in

Analog signals: top.ia2.ib2.v(in), top.ia1.ib1.ic1.id1.v(in)

For more information, see the *Linking Novas Files with Simulators and Enabling FSDB Dumping* manual.

11

Report Files

This chapter provides a detailed description of the report files generated in the Verilog-SPICE, VHDL/Verilog-SPICE, and Verilog-AMS-SPICE mixed-signal simulation flows.

In all flows of a mixed-signal simulation, the tool creates a `simv.msv` directory to store mixed-signal report files. The name is based on the simulation executable, which is named “simv” by default. You can change the name by using the `-o` option with the `vcs` command.

The following sections describe the report files that are stored in the `simv.msv` directory and explains their contents.

- [connectmodule.rpt](#)
- [hierarchy.rpt](#)
- [interface_activity.rpt](#)
- [interface_connectivity.rpt](#)
- [interface_element.rpt](#)
- [interface_element_temporal.rpt](#)
- [interface_tracing.rpt](#)
- [mview.rpt](#)
- [port.rpt](#)
- [runtime_interface_element.rpt](#)
- [through_net.rpt](#)
- [use_cell_view.rpt](#)

Note:

There are two ways to generate the `names_map.rpt` file, depending on the version of the tool you are using:

- For older releases (O-2018.09 and earlier): By default, the `names_map.rpt` file is not automatically generated in the mixed-signal simulation flow. To force its generation, set the environment variable `COSIM_USE_MSV_COMMON` to 0 as follows.

```
% setenv COSIM_USE_MSV_COMMON 0
```

- For latest releases: You can enable generation of the `names_map.rpt` file by adding the following option in your `vcsAD.init` file.

```
report_option -names_map enable;
```

connectmodule.rpt

This file lists the connect module instances in the design at compile time by using the VCS switch `-ams_iereport`.

If you use this switch, the tool displays an abbreviated report at the console and in the `vcs.log` file, as shown in the following example:

```
# =====LIST OF CONNECT MODULE INSTANCES=====
# Instance: snps_sptop
#   (1) \net1[0]__snps_cm_a2d_1__ddiscrete (ConnectModule: snps_cm_a2d_1)
#       Mode: merged
#       Net:  snps_sptop.net1[0] (electrical)
#       Ports: snps_sptop.xg1.clk[0] (ddiscrete)
#   (2) \net1[1]__snps_cm_a2d_1__ddiscrete (ConnectModule: snps_cm_a2d_1)
#       Mode: merged
#       Net:  snps_sptop.net1[1] (electrical)
#       Ports: snps_sptop.xg1.clk[1] (ddiscrete)
```

The full report, which also lists the final parameter values for each connect module instance, is output to `simv.msv/connectionmodule.rpt`, as shown in the following example:

```
# =====LIST OF CONNECT MODULE INSTANCES=====
# Instance: snps_sptop
#   (1) \net1[0]__snps_cm_a2d_1__ddiscrete (ConnectModule: snps_cm_a2d_1)
#       Mode: merged
#       Net:  snps_sptop.net1[0] (electrical)
#       Ports: snps_sptop.xg1.clk[0] (ddiscrete)
#       Parameters:
#         vsup = 1.7999999523162842
#         hir  = 0.5999999999999998
#         lor  = 0.40000000000000002
#         tdx  = 4.0000000000000001e-08
#         rin  = 10000000000.00000
#         cin  = 0.0000000000000000
#         vxh  = 1.0799999713897706
#         vxl  = 0.71999998092651374
```

```
#          hysteresis = 4.0000000000000000
#          hyst_int = 0.089999997615814203
#          vxhl = 0.98999997377395632
#          vxll = 0.80999997854232797
# (2) \net1[1]__snps_cm_a2d_1__ddiscrete (ConnectModule: snps_cm_a2d_1)
#       Mode: merged
#       Net: snps_sptop.net1[1] (electrical)
#       Ports: snps_sptop.xgl.clk[1] (ddiscrete)
#       Parameters:
#           vsup = 1.7999999523162842
#           hir = 0.5999999999999999
#           lor = 0.4000000000000000
#           tdx = 4.0000000000000001e-08
#           rin = 10000000000.00000
#           cin = 0.0000000000000000
#           vxh = 1.0799999713897706
#           vxl = 0.71999998092651374
#           hysteresis = 4.0000000000000000
#           hyst_int = 0.089999997615814203
#           vxhl = 0.98999997377395632
#           vxll = 0.80999997854232797
```

hierarchy.rpt

This file lists the hierarchical paths to all cells in the design, along with their cell names. Each hierarchical instance name is followed by the cell name of the instance wrapped in:

- Parentheses, "`( )`", if the cell is in Verilog/VHDL
- Angle brackets, "`< >`", if it is in SPICE or Verilog-A (which is read in through SPICE ``hdl`)
- Curly braces, "`{ }`", if it is Verilog-AMS

An example of the file content is shown as follows:

```
top(top).dut<addr4>.x4<addr>.x9<nor2>
top(top).dut<addr4>.x1<addr>.x2<xor2>.x2<inv>
top(top).dut<addr4>.x1<addr>.x2<xor2>.x3<inv>
top(top).dut<addr4>.x1<addr>.x2<xor2>.x4<xfer>
```

The first entry above, `top(top).dut<addr4>.x4<addr>.x9<nor2>`, shows that the [Full Hierarchical Path](#) to instance `x9` of the SPICE cell `nor2` is: `top.dut.x4.x9`

It also shows that the cells `addr4`, `addr` and `nor2` are using the SPICE view (cell names are wrapped in angle brackets, "`< >`"), while the cell `top` is using the Verilog or VHDL view (cell name is wrapped in parentheses "`( )`").

interface_activity.rpt

This file contains either the interface element (IE) activity statistics for the Verilog-SPICE and VHDL/Verilog-SPICE flows or the connect module (CM) activity statistics for the

Verilog-AMS flow. It reports the number of data transfers between digital and analog domains during the simulation.

Note:

This file is generated only if the interface activity report is enabled with the `ie_activity_rpt` command.

The following example report shows the number of events on each interface element:

```
// interface_activity.rpt
top.dut.net2 // voltage_r nettype
    n2e      162
top.dut.net1 // voltage_r nettype
    e2n      161
top.clk
    d2a      11
top.voltage_r_vdd
    r2e      1
top.dut.net2 // voltage_r nettype
    rt_e2n    352

Summary:
Number of d2a events (including HighZ) : 11
Number of HighZ d2a events             : 0
Number of n2e events (including HighZ) : 162
Number of HighZ n2e events             : 0
Number of e2n events (including HighZ) : 161
Number of HighZ e2n events             : 0
Number of r2e events (including HighZ) : 1
Number of HighZ r2e events             : 0
Number of rt_e2n events (including HighZ): 352
Number of HighZ rt_e2n events          : 0
Total number of events : 687 for 5 interface signals

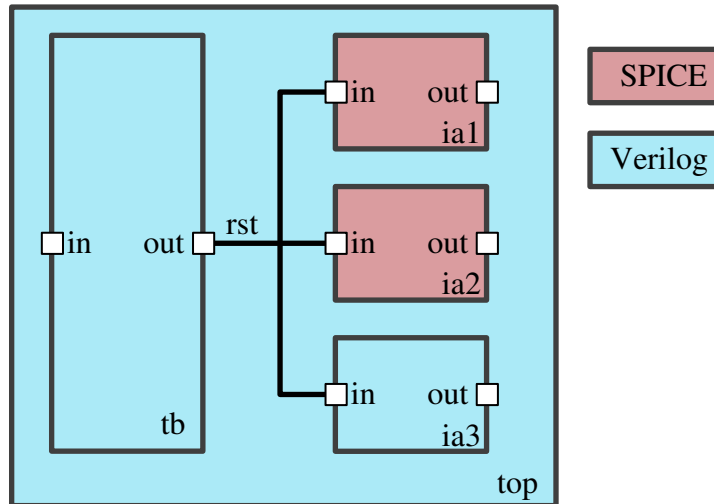
x1.clk // nettype voltage_r
    n2e      10
x1.vss // nettype current_r
    n2e      1
x1.vdd // nettype current_r
    n2e      1
```

interface_connectivity.rpt

This file lists additional connectivity information for the interface elements in the simulation, including the digital drivers and the connected SPICE nets and ports for each interface element. To generate this report, specify `ie_connect_rpt` enable in the `vcsAD.init` mixed-signal simulation control file.

In the following example, the top-level net `rst` connects digital module `tb` to SPICE blocks `ia1`, `ia2`, and digital module `ia3`.

Figure 25 Mixed-Signal Design



The `interface_connectivity.rpt` file contains the following information for the d2a interface element on net `rst`, including the Verilog statements that drive the net.

```
# Interface element:
top.ia1.in //d2a
# Digital driver(s):
top.tb.rst top.tb top.v 17 : always #25 rst =~ rst;
top.tb.rst top.tb top.v 20 : rst = 1'b1;
# Analog port(s):
top.ia1.xu0.in
top.ia1.in
top.ia2.in
```

interface_element.rpt

This file lists the interface elements inserted by the tool and specified in the `vcsAD.init` mixed-signal simulation control file. This file is only generated in Verilog-SPICE and VHDL/Verilog-SPICE flows. The report contains all information related to interface nets in the design in the following format:

- A header explaining the meanings of interface elements used in the file (a2d, e2r, and so on)
- Total number of all resistors added to the netlist because of interface elements

- A list of resistance map files used
- A list of all interface nodes

The following comment lines appear at the top of the report file and provide a guide to the possible interface elements contained in the file:

```
# interface net information
# a2d:    Analog to Digital interface node
# d2a:    Digital to Analog interface node
# inout:  bidirectional interface node
# e2r:    Real interface node with an Analog to Digital direction
# r2e:    Real interface node with a Digital to Analog direction
# e2n:    Nettype interface node with an Analog to Digital direction
# n2e:    Nettype interface node with a Digital to Analog direction
```

The header is followed by a section that indicates the total number of resistors and capacitors added by interface elements.

```
Total number of resistors added by interface elements = 9
Total number of capacitors added by interface elements = 9
```

The next section in the report file is a list of resistance map files used in the design. If no explicit resistance map file is used, the default resistance map is listed as follows:

```
rmap_file 1 tool_install_dir/resistance.map
```

If resistance map files were specified in the `vcsAD.init` file, they are listed in the report file as follows:

```
rmap_file 1 ./global_res.map
rmap_file 2 ./cust_res_a.map
rmap_file 3 ./cust_res_b.map
rmap_file 4 ./cust_res_lv.map
rmap_file 5 ./cust_res_hv.map
```

The bottom of the report file contains the list of interface elements as shown in the following example:

```
a2d -loth 0.6v -hith 1.2v -node top.id<0>;
d2a -hiv 1.8v -lov 0v -rf_time 3e-10 -x2v 3 -node top.mux_en;
e2n -node top.vref;      // -nettype voltage_r
n2e -node top.i1.BUS[0]; // -nettype current_r
e2r -node vcc_i1;
r2e -node top.vin;
inout -loth 0.6v -hith 1.2v -hiv 1.8v -lov 0v -rf_time 3e-10 -x2v 3 -node
io;
```

Note:

Recent versions of the `interface_element.rpt` file use the `inout` term instead of `bidir`.

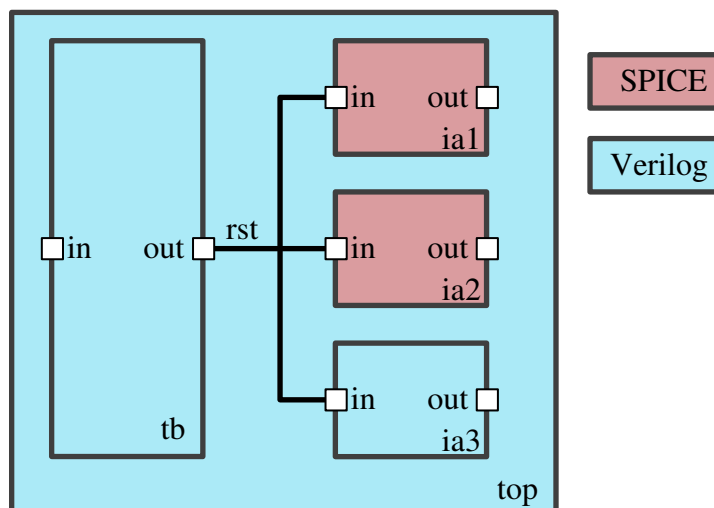
The `loth` and `hith` values for `a2d` and `inout` nodes are reported as absolute values, not as ratios (percentages). Note that, for the `a2d` and `d2a` interface elements, the reports are generated with the correct command syntax. You can copy lines from the `interface_element.rpt` file, paste them into your `vcsAD.init` mixed-signal simulation control file, edit the `loth` and other settings as needed, and rerun the compilation and simulation.

The `node` argument specifies the location of the interface element. If you specify multiple interface elements for the same net, a net alias condition exists and the tool:

- Combines the parameters specified for the interface elements on the same net
- Uses the last parameter specified in the `vcsAD.init` file if conflicts exist

In the following example, the `rst` net connects the output of module `tb` to the inputs of SPICE blocks `ia1` and `ia2`.

Figure 26 Design With Net Aliases



The following interface elements are specified in the following `vcsAD.init` mixed-signal simulation control file; note that the node references all refer to the same net:

```
// vcsAD.init
d2a -hiv 3.1v      -node top.ia1.in;
d2a -lov 0.1v     -node top.rst;
d2a -rf_time 1.5n -node top.ia2.in;
```

The tool consolidates the multiple `d2a` interface elements to a single interface element by combining the parameters. The `interface_element.rpt` file reports the interface element and aliases as follows:

```
// interface_element.rpt
d2a -hiv 3.1v -lov 0.1v -rf_time 1.5e-09 -node top.ia1.in;
```

```
// User Specified Aliases
// top.rst
// top.ia2.in
```

Verbose Mode

To produce additional debugging information, specify the `report_option -interface_element` command with the `verbose` option in the `vcsAD.init` mixed-signal simulation control file as follows:

```
report_option -interface_element verbose;
```

When the verbose mode is enabled, the tool adds additional information to the `interface_element.rpt` file. For d2a and a2d interface elements, the tool reports the top net (the highest hierarchical alias of a mixed-net name with the smallest number of hierarchical separators ".") and all the hierarchical boundary nets in the SPICE domain that touch a digital net or port.

In the previous example shown in [Figure 26](#), the top-level signal `rst` connects digital module `tb` with SPICE blocks `ia1`, `ia2`, and digital block `ia3`. For this design, the following listing shows the default report for a d2a interface element on the left. The verbose report, generated when you specify the `report_option -interface_element verbose` command, is shown on the right.

Example 44 Default and Verbose Interface Element Reports

// Default Report	// Verbose Report
d2a -hiv 3.1v -lov 0.1v	# d2a interface elements:
-rf_time 1.5e-09 -delay 1e-11	# 1 total
-node top.ia1.in;	d2a -hiv 3.1v -lov 0.1v
// User Specified Aliases	-rf_time 1.5e-09 -delay 1e-11
// top.ia2.in	-node top.ia1.in;
// top.rst	// Top-Net
// top.ia3.in	// top.rst
	// All Boundary Nets
	// top.ia1.in
	// top.ia2.in
	// User Specified Aliases
	// top.ia2.in
	// top.rst
	// top.ia3.in

The `-ad_iereport` option writes out the interface element report in the Verilog SPICE flow to the console and the PrimeSim XA log file.

The equivalent report in the Verilog-AMS-SPICE flow is the Connect Module report (see [connectmodule.rpt](#)), which can be generated with the `-ams_iereport` option on the VCS command line.

interface_element_temporal.rpt

This file records changes to interface elements by using the UCLI `ace` command when running the simulation. This file is only generated in the Verilog-SPICE and VHDL/Verilog-SPICE flows and contains the details of the interface element options and values that were changed during the simulation.

In the following example based on [Figure 25](#), the simulation is run with the `-ucli` option and UCLI `ace` commands are used to modify the parameters of the same d2a interface element by using different node aliases for the same net.

```
% simv -ucli
ucli% ace d2a -hiv 3.3v -lov 0.0v -node top.ia1.in;
ucli% run 10
100 ps
ucli% ace d2a -hiv 2.5v -lov 0.0v -node top.ia2.in
ucli% run 10
200 ps
ucli% ace d2a -hiv 3.3v -lov 0.0v -node top.rst
ucli% run 10
300 ps
ucli% ace d2a -hiv 2.5v -lov 0.0v -node top.ia3.in
ucli% run 10
400 ps
```

Here is an example of the `interface_element_temporal.rpt` file after running these commands:

```
# Temporal view of Interface Elements
# Interface_element_type options=values //time=time_instant

d2a -hiv 3.3v -lov 0.0v -node top.ia1.in    // time = 0 ns
d2a -hiv 2.5v -lov 0.0v -node top.ia1.in
// User Specified Alias: top.ia2.in // time = 0.1 ns
d2a -hiv 3.3v -lov 0.0v -node top.ia1.in
// User Specified Alias: top.rst    // time = 0.2 ns
d2a -hiv 2.5v -lov 0.0v -node top.ia1.in
// User Specified Alias: top.ia3.in // time = 0.3 ns
```

interface_tracing.rpt

This file helps you check whether your user-defined `vdd` nodes are correct. If a user-defined `vdd` differs from the supply reached by the tracing algorithm, the report includes a warning message.

Note:

This file is generated only if the interface activity report is enabled with the `ie_tracing_rpt` command.

The following is an example report:

```
#           ie_tracing_rpt -enable | -disable [-refLov <val>]
#
#   The interface_tracing.rpt file helps to trace the value
# of hiv/lov, hith/loth and Vdd/Vss.'ie_tracing_rpt' command
# provides 'refLov' option to help customer to estimate the
# value of hiv and lov for a2d node.
#
#   When the user sets constant hith and loth, the tool will
# assume that the thresholds set by the user are always
# symmetrical and that the estimated lov is given from the refLov
# (default is 0.0 ) option. So, the tool will first estimate
# the hiv and lov as described below:
#   Estimated hiv = hith+loth-refLov

d2a -hiv 2v -lov 0v -node top.I1.in1
  Traced Vdd: node = myvdd2 hiv = 2
d2a -hiv 2v -lov 0v -node top.I1.in2
  Traced Vdd: node = myvdd2 hiv = 2
a2d -loth 0.5v -hith 2v -node top.I1.out
  Traced Vdd: node = myvdd2 hiv = 2 <- mismatch!
  Estimated user defined hiv = 2.5
```

mview.rpt

This file lists all cells in the design that have more than one view (for example, SPICE, Verilog, Verilog-A).

Here is an example of the file content:

```
; Has unresolved modules?
0
; For Spice: case 1 by default
; Lists of modules: Verilog Spice Verilog-A Adfmi
addr4 addr4 * *
; Verilog Top Module(s)
1 top
;done
```

In this example, cell “addr4” has Verilog and SPICE views.

port.rpt

This file contains information about how ports are mapped when one SPICE or HDL view is replaced by the other. If multiple views are present, the port order, names, or direction are often not consistent between them. The tool tries to reconcile the differences according to the rules and commands described in this manual.

The results in the `port.rpt` file contain syntactically correct commands that can be cut, edited, and pasted back into the mixed-signal simulation control file.

Note:

The `port.rpt` file does not display connectivity from the high to low (from parent to child). It displays the mapping between child views

The `port.rpt` file is useful when ports were mapped by the tool successfully and you want to compare the results to the intended mapping, or when some ports were not mapped successfully and you want to find out why.

The report contains:

- A header that provides a reminder of the syntax used within the report
- Reports on cells with multiple views, organized by cell name
- Reports on cells with single view; contains port direction only

Entries are grouped by cell name.

The following example shows cell names, modules and subcircuit references for cells with multiple views, including:

- Each cell name
- Where to find the cell definitions within the overall design
- Port list for the Verilog and SPICE views
- Total port count for both views

For example:

```
-----Cell: addr4 -----
*** Port Warnings Encountered For this Cell ***

subckt addr4: file "addr4.spi" line "49" ports=15verilog addr4: file
"adder.v" line "1" ports=14

verilog module addr4([3:0]a, [3:0]b, cin, [3:0]s, cout);
    input    a, b, cin;
    output   s, cout;
subckt addr4 a_4, a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0, cin, s_3,
            s_2, s_1, s_0, cout
```

Each `port_map` entry contains the following set of resolved and unresolved ports for each multi-view cell:

- Ports specifically mapped by the `port_map` command
- Unresolved ports
- Ports mapped by default as `snps_by_name`

Buses are reported, where possible, in the `busname[m:n]` format. They are resolved and printed, as much as possible, as a unit. A mismatch in one bus should not generally affect the reporting of another.

Unresolved ports are reported with double question marks (`??`). They are collapsed where possible, but if you define the port map, it is expressly shown. For example:

```
use_spice -cell addr4 port_map(  
    a[3:0]=>a[4:0]??, //bus width mismatch  
    *=>snps_by_name);
```

When extra SPICE ports are detected, they are shown mapped to `??`. For example:

```
use_spice -cell addr4d port_map(  
    <??=>vddl, <??=>vssl,  
    *=>snps_by_name);
```

After the port mapping information, the resolved port directions of cells with a SPICE view are reported in the format of the `port_dir` command. For example:

```
port_dir -cell addr4b(  
    input    a, b, cin;  
    output   s, cout  
)          //derived from verilog
```

runtime_interface_element.rpt

This file lists the runtime interface elements (`rt_a2d`, `rt_d2a`, `rt_e2r`, `rt_r2e`) inserted by the tool. In an HDL testbench and UCLI run scripts, the `hdl_xmrs`, `hdl_xmr_force` and UCLI `force` commands are used to force a value on a node. When a cell view is switched from a digital HDL view to an analog view, the same `hdl_xmr`, `hdl_xmr_force` or UCLI `force` commands must be applicable for the analog target.

When a node is forced to 0, 1, X, or Z by using the `hdl_xmr`, `hdl_xmr_force` or UCLI `force` commands, the tool must correctly apply the correct value even after the digital target has changed to an analog target. The tool uses an automatic VDD detection mechanism, similar to that used for traditional interface element thresholds and voltage swings, to select voltage levels and thresholds for digital-to-analog and analog-to-digital conversions.

You can manually change the voltage thresholds and swings with the following interface elements:

- [rt_a2d](#)
- [rt_d2a](#)
- [rt_e2r](#)
- [rt_r2e](#)

The `runtime_interface_element.rpt` file is generated and updated each time an `hdl_xmr`, `hdl_xmr_force`, or `hdl_xmr_release` system task is executed, or a `force` or `release` command is entered through the UCLI. The tool automatically generates voltage level and threshold settings for the runtime interface element based on the design context and includes the settings in the report file.

You can cut and paste the runtime interface element commands from the `runtime_interface_element.rpt` file and use them in the `vcsAD.init` mixed-signal simulation control file. When you use an `hdl_xmr`, `hdl_xmr_force`, or `hdl_xmr_release` system task, or a `force` or `release` UCLI command on an analog target, the tool issues a message to the console with information about the options that have been used for any analog-to-digital or digital-to-analog conversion, electrical-to-real, or real-to-electrical conversion.

An example `runtime_interface_element.rpt` file is shown as follows:

```
rt_d2a -hiv 3.3v -lov 0v -node top.u1.u2_1.a;  
rt_a2d -hith 1.65v -loth 1.65v -node top.u1.u2_1.a;  
rt_d2a -hiv 3.3v -lov 0v -node top.u1.u2_1.b;  
rt_a2d -hith 1.65v -loth 1.65v -node top.u1.u2_1.b;  
rt_a2d -hith 1.65v -loth 1.65v -node top.u1.u2_2.a;  
rt_a2d -hith 1.65v -loth 1.65v -node top.u1.u2_2.b;
```

through_net.rpt

This file contains a list of nets that connect two digital ports within a SPICE subcircuit, or two SPICE ports within a digital block. This file is only generated in Verilog-SPICE and VHDL/Verilog-SPICE flows and is generated only if there is at least one a2a or d2d net in the design. If both a2a and d2d through-nets exist in the design, the a2a nets are listed first, followed by d2d nets. Any n2n through-nets are listed with a comment that indicates the specific type. The through-nets that share the same connectivity appear together while the ones that do not share the same connectivity are reported after a blank line as shown in the following example:

```
// This report contains a list of a2a, d2d and n2n through-nets in the design.  
//  
// VCS runtime options can be used to disable, globally, through-net optimization.  
//  
// Use the VCS runtime option -ad_runopt=noa2aopt to disable a2a through net
```

```
// optimization: example simv -ad_runopt=noa2aopt
//
// Use the VCS runtime option -ad_runopt=nod2dopt to disable d2d through net
// optimization: example simv -ad_runopt=nod2dopt
//
// The VCS runtime options can be combined to disable, globally, all through net
// optimization: example simv -ad_runopt=noa2aopt+nod2dopt

xtop.xia1.temp1 A2A
xtop.xia1.temp2 A2A

xtop.xia1.temp4 D2D

xtop.xia1.n1 D2D
xtop.xia1.n2 D2D
xtop.xia1.n3 D2D

xtop.xia1.temp5 D2D
xtop.xia1.temp3 D2D

xtop.xia7.t1 D2D

xtop.g1.in2 n2n // voltage_r nettype
```

use_cell_view.rpt

This file lists all the cell names and instance names that match each of the `use_spice`, `use_verilog` and `use_vhdl` commands in the `vcsAD.init` file. The list is partitioned into sections, and each section corresponds to each use statement. Each section contains a [Full Hierarchical Path](#) to the matching element.

In the following example, the SPICE view is used for the `buffer` cell and the Verilog view is used for the `inv` cell.

```
#=====
# Command used on line 5 of mixed-signal control file vcsAD.init
# followed by instances partitioned by that command
#=====
use_spice -cell buffer;

#top.spi.sm.G10.G20.buffer
#top.spi.sm.G10.G21.buffer
#top.spi.sm.G11.G20.buffer
#top.spi.sm.G11.G21.buffer

#=====
# Command used on line 7 of mixed-signal control file vcsAD.init
# followed by instances partitioned by that command
#=====
use_verilog -module inv;

#top.spi.sm.G10.G20.buffer.inv1
#top.spi.sm.G10.G21.buffer.inv1
#top.spi.sm.G11.G20.buffer.inv1
```

```
#top.spi.sm.G11.G21.buffer.inv1  
#top.inv
```

12

Interactive Debug

Interactive debug is a useful feature that is widely used to debug the issues in design and verification stages. This feature allows you to interrupt the simulation anytime, interact with the tool and circuit to access or change the settings during the runtime.

- [Invoking the Interactive Mode With the UCLI Debugging Feature With Verilog-SPICE or VHDL/Verilog-SPICE](#)
- [UCLI Ace Analog Interactive Commands](#)
- [Pausing and Resuming Simulations](#)
- [Debugging Current Consumption](#)
- [Viewing and Modifying Interface Element Options at Runtime](#)

Invoking the Interactive Mode With the UCLI Debugging Feature With Verilog-SPICE or VHDL/Verilog-SPICE

To invoke the UCLI interactive mode, first use the appropriate VCS debug compile options:

```
vcs -debug_access+<options> or vcs -debug/-debug_pp/-debug_all ...
```

For example:

```
vcs -debug_access+all -ad=vcsAD.init ...
```

For more information see the *VCS User Guide* and *Unified Command-Line Interface User Guide*..

Next, invoke the `simv` binary file by passing the `-ucli` option to it:

```
% simv -ucli
```

This switch stops the mixed-signal simulation at simulation time 0, and generates the UCLI prompt:

```
ucli%
```


Alternatively, you can stop the simulation any time during the simulation by pressing Ctrl +C, provided that the UCLI was enabled through `-debug` or `-debug_all` at compile time and `simv` was run with the `-ucli` switch.

All UCLI commands can be used at this prompt, such as

```
ucli% run 10
```

which runs the simulation for 10 (VCS) units of time.

The VCS time unit is usually defined by the ``timescale` directive in the Verilog code.

The following command prints all the digital sources that drive the given `node_name` with the file and line numbers locating the contributing Verilog code

```
ucli% drivers -full hier_node_name
```

UCLI Ace Analog Interactive Commands

You can use analog interactive commands at the UCLI prompt if they are preceded by the `ace` command. The following example shows how the PrimeSim XA interactive command `iprint_time` can be used at the UCLI interactive prompt.

```
ucli% ace iprint_time
```

For a complete list of the UCLI commands, see the *VCS Unified Command-Line Interface User Guide*.

Note:

The analog simulation time cannot be advanced using the UCLI command.
The analog simulation time can only be advanced by advancing the digital simulation time in UCLI. See the *VCS User Guide* for more information.

To use the FineSim interactive command `exi` to report devices with excessive current:

```
ucli% ace exi
```

To use the PrimeSim interactive command `exi` to report devices with excessive current:

```
ucli% ace exi
```

Pausing and Resuming Simulations

The UCLI interactive mode, you can pause and resume simulations at any point. For example,

Example 1

```
simv <sim_options> -i ucli.cmd
```

where `ucli.cmd` contain following commands:

```
cmd-1
cmd-2
run 100us
cmd-3
```

You can interrupt the simulation anytime pressing **Ctrl+C** and execute any different set of commands other than `ucli.cmd`.

Note:

Use `-keep` option with `run` command to prevent commands in `ucli.do` from being overwritten by commands entered after interrupting with **Ctrl+C**.

Example 2

```
simv <simv_options> -i ucli.cmd
```

where `ucli.cmd` contain following commands:

```
cmd-1
cmd-2
run -keep 100us
cmd-3
cmd-4
```

In this example, `ucli.cmd` commands will still be executed even if there is an interrupt using **Ctrl+C** and different sets of commands are supplied.

Debugging Current Consumption

There are two ways to debug current consumption in a design:

1. Using Verdi “Current Debug Focus Connection”:
 - Traces current through the Verdi nSchema (schematics)
 - Easy to follow high current consumption path through schematic
 - Ability to analyze currents backward in time
 - Current values are very accurate
 - Requires currents to be probed
2. XA interactive mode (using `iprint_connectivity` command):
 - Generates current report in csv format using the XA interactive `iprint_connectivity` command.
 - Does not require currents to be probed.

- Limited to interactive mode without post processing capabilities.
- Cannot analyze currents backward in time.
- Provides less accurate current values.

Syntax of XA `iprint_connectivity` command:

```
ace iprint_connectivity -node node_name -on current_value \
  [-file_format text|csv] [-except_inst inst_name {inst_name}] \
  [-except_subckt subckt_name {subckt_name}] -xprobe 0|1 \
  [-short_resistor resistor_value] [-file file_name]
```

Viewing and Modifying Interface Element Options at Runtime

After a mixed-signal simulation is set up, it is often necessary to view or modify interface element options after the interface element locations and associated interface element reports are generated. The feature described in this section uses runtime-specific UCLI `ace` commands to view or modify interface element options without having to restart and rerun the simulation.

Viewing Interface Element Details at Runtime

You can use the `ace show_ie` command to view the options and values of the interface elements that are valid at the current simulation time on the screen or in a specified file.

Editable options that have a dependent option (such as `-minv` or `-minv_logic` for dynamic supply) are output only when the dependent option (`-vdd`, for example) is specified. If the non-editable options are already applicable, they are output, or else they are not output.

`ace show_ie`

In interactive simulation mode, this `ace` UCLI command displays the interface elements on the screen or in a specified file. You can use this command to output all the actual option values used by the tool for each interface element, including default option values and those you set.

Syntax

```
ace show_ie node=[hierarchical_node_name]
  type=[a2d|d2a|n2e|e2n|r2e|e2r]
  input=[0|1|X|Z]
  output_range={min_value,max_value}
  output=[0|1|X|Z]
  input_range={min_value,max_value}
  cond=["expr"]
  mode=[concise|verbose]
```

```
property_rpt=[report_list|all]  
csv=filename;
```

Arguments

```
node=hierarchical_node_name
```

Specifies the hierarchical path to the interface element and can be any SPICE net name (which includes primary-net name (PNN) reported by the tool in the `interface_element.rpt` file, or any hierarchical alias in the SPICE hierarchy) and only the digital net name that is used in the `vcsAD.init` mixed-signal simulation control file.

You can specify wildcard characters (*) in the path, but it is limited only to point to a PNN, that is, a hierarchical name for an interface element reported in the `interface_element.rpt` file. The wildcard cannot be used to target a hierarchical alias.

```
type=a2d|d2a|n2e|e2n|r2e|e2r|u2e|e2u
```

Shows only the type specified.

```
input=0|1|X|Z
```

Shows only the **d2a** interface elements where the digital signal is the specified value.

```
output_range={min_value,max_value}
```

Shows only the non-**a2d** interface elements where the analog voltage is the between the `min_value` and the `max_value`.

```
output=0|1|X|Z
```

Shows only the **a2d** interface elements where the digital signal is the specified value.

```
input_range={min_value,max_value}
```

Shows only the non-**d2a** interface elements where the analog voltage is the between the `min_value` and the `max_value`.

```
cond="expr"
```

A Boolean expression in which `input`, `output`, `input_range`, and `output_range` can be used. When `cond` is specified, `input`, `output`, `input_range`, and `output_range` cannot be used as separate arguments. Expressions can include `&&` and `||` for logical AND and OR functions, respectively.

```
mode=[concise|verbose]
```

Generates a `concise` or `verbose` output:

- In `concise` mode, you see the most commonly displayed options (like the options that you see in `interface_element.rpt`) for the interface elements. The options reported in concise mode are *sticky* in nature, which means that if you use any specific option (such as the `x2v` option for the `d2a` command in the mixed-signal simulation control file (`vcsAD.init`)), it is reported. In typical cases, `interface_element.rpt` does not show the `x2v` option unless it is explicitly specified in `vcsAD.init`.
- The `verbose` mode displays all the editable options and values applicable for the interface elements. Options can be of two types: editable and non-editable. Editable options are the ones whose values can be modified during simulation using ace runtime interface element commands, whereas non-editable ones are just the opposite. See [Modifying Interface Element Options at Runtime](#) for more details.

```
property_rpt={report_list|all}
```

When `all` is specified, all the relevant properties are listed. When a space-separated list is specified, those properties are listed.

```
csv=filename
```

Creates command separated `.csv` files of the form, `filename_ietype.csv`.

Examples

The following examples show the command input and tool output for various `ace show_ie` commands:

- Show interface element details for a specific node:

```
ucli% ace show_ie node=top.dut.cin
d2a -hiv 3.3v -lov 0v -node top.dut.cin
```

If the node does not exist, the tool issues an error message as follows:

```
ucli% ace show_ie node=top.dut.not_a_node
Warning: [MSV-CMD-008] Can't find interface net
top.dut.not_a_node in command "ace show_ie".
The command is ignored.
```

By default, the command reports by using concise mode. You can explicitly specify concise mode by including the `mode=concise` option.

- Show additional debugging information for a specific interface element by using `mode=verbose`:

```
ucli% ace show_ie node=top.dut.cin mode=verbose
d2a -hiv 3.3v -lov 0v -rf_time 1e-11 -rise_time 1e-11
    -fall_time 1e-11 -delay 0 -x_state 0 -x2v 0
    -vdd_filter 0 -node top.dut.cin
```

Note that some options that are set to their default settings are not reported, such as `-minv` and `-minv_analog` in this example.

- Show all interface elements that match a wildcard expression by using the wildcard character (*):

```
ucli% ace show_ie node=*
d2a -hiv 3.3v -lov 0v -node top.dut.cin
d2a -hiv 3.3v -lov 0v -node top.dut.a_3
d2a -hiv 3.3v -lov 0v -node top.dut.a_2
...

ucli% ace show_ie node=top.dut.a*
d2a -hiv 3.3v -lov 0v -node top.dut.a_3
d2a -hiv 3.3v -lov 0v -node top.dut.a_2
d2a -hiv 3.3v -lov 0v -node top.dut.a_1
d2a -hiv 3.3v -lov 0v -node top.dut.a_0
```

- Show all interface elements of type a2d (note that you must also specify the `node=*` option):

```
ucli% ace show_ie type=a2d node=*
a2d -loth 1.65v -hith 1.65v -node top.dut.s_3
a2d -loth 1.65v -hith 1.65v -node top.dut.s_2
...
```

- Show how to display only d2a interface elements whose output range is between 0.8V and 1.2V:

```
ucli% ace show_ie node=* type=d2a output_range={0.8,1.2}
```

- Show how the `cond` argument is used to build a more complex filter (in this case, only d2a interface elements are displayed whose input is either 1 or 0 and output voltage is between 0.8V and 1.2V):

```
ucli% ace show_ie node=* type=d2a cond="(input=1 || input=0) &&
    output_range={0.8,1.2}"
```

- Show how the `.csv` files are generated (in this case, a file called `design_ie_d2a.csv` with a list of all the d2a interface elements and a file called `design_ie_a2d.csv` with a

list of all the a2d interface elements are created; similar files for r2e, e2r, n2e, and e2n are created, as necessary):

```
ucli% ace show_ie node=* csv=design_ie
```

- Show how the `property_rpt` argument is used to display the specified arguments (in this case, all d2a interface elements and their corresponding `hiv` and `lov` values are displayed):

```
ucli% ace show_ie node=* type=d2a property_rpt={hiv lov}
```

Modifying Interface Element Options at Runtime

You can use the `ace element_type` UCLI command to change interface element settings at runtime. Supported interface element types are a2d, d2a, e2n, n2e, r2e, and e2r. Note that the following properties cannot be changed for the listed interface element:

- **all:** `-ceff`, `-cell/-port/-inst`, `-except_port`, `-library`, `-powernet`, `-vdd`, and `-vss`
- **e2n:** `-type`
- **e2r:** `-res`, `-type i`
- **n2e:** `-res`, `-type i`
- **r2e:** `-type i`

The following example displays all interface elements in the design, modifies an interface element by setting new parameters, and displays the updated interface element.

```
% simv -ucli
ucli% ace show_ie node=*
d2a -hiv 3.3v -lov 0v -node top.dut.a_3
d2a -hiv 3.3v -lov 0v -node top.dut.a_2
d2a -hiv 3.3v -lov 0v -node top.dut.a_1
d2a -hiv 3.3v -lov 0v -node top.dut.a_0
...

ucli% ace d2a -hiv 3.7 -lov 0.22 -rise_time 3e-09 -fall_time 4e-09
      -delay 1e-09 -node top.dut.a_3

ucli% ace show_ie node=top.dut.a_3
d2a -hiv 3.7v -lov 0.22v -rise_time 3e-09 -fall_time 4e-09
      -delay 1e-09 -node top.dut.a_3
```

All UCLI commands are written to the VCS `ucli.key` file, which you can replay for subsequent VCS PrimeSim AMS simulations. The runtime interface element modifications are also recorded in the `interface_element_temporal.rpt` file; use this file to track any changes to interface elements that occur during runtime.

Note:

The same set of rules that is applicable to the `node=node_name` for the `ace show_ie` command is also applicable for these `ace` runtime interface element commands.

A

Mixed-Signal Control Commands

This appendix lists the syntax descriptions for the mixed-signal control commands.

The `vcsAD.init` mixed-signal simulation control file contains commands to specify the views to use for different cells and instances, interface element types and locations, reporting modes, and so on. You can specify a different file name with the `-ad <control_file_name>` compile-time option.

To use these commands, the following rules apply:

- All commands must be completed with a semicolon (;)
- Comments are created by inserting a double forward-slash character (//)
- Commands can span more than one line with no line continuation character needed. The Return character at the end of each line serves as the line continuation character.
- If a mixed-signal command has a `-node` option, it refers to the name of the interface element it is trying to configure. For this option, either the hierarchical name of the interface element, as reported in the `interface_element.rpt` file, must be used, or a hierarchical alias of the interface element. A hierarchical alias is the name of a net at a different level of hierarchy, which through hierarchy connects to the interface element. See [Using Hierarchical Aliases for Interface Elements in Mixed-Signal Commands](#) for more information.

The mixed-signal control commands are:

- [a2d](#)
- [ams_cdef_inst](#)
- [ams_cdef_net](#)
- [ams_cm](#)
- [ams_set_discipline](#)
- [ams_supply](#)
- [bus_format](#)

- [choose](#)
- [d2a](#)
- [disable_ie](#)
- [downgrade_to_warn](#)
- [duplicate_net_inst_name](#)
- [dynamic_supply_filter](#)
- [e2n](#) for [current_r](#) Nettype
- [e2n](#) for [i_wire](#) Nettype
- [e2n](#) for [th_wire](#) Nettype
- [e2n](#) for [voltage_r](#) Nettype
- [e2n](#) for [v_wire_avg](#), [v_wire_sum](#), and [v_wire_one](#) Nettypes
- [e2r](#)
- [e2u](#)
- [form_spice_bus](#)
- [gen_spice_wrapper](#)
- [ie_activity_rpt](#)
- [ie_connect_rpt](#)
- [ie_reference_voltage](#)
- [ie_tracing_rpt](#)
- [insert_cell](#)
- [map_by_node](#)
- [n2e](#) for [current_r](#) Nettype
- [n2e](#) for [i_wire](#) Nettype
- [n2e](#) for [th_wire](#) Nettype
- [n2e](#) for [voltage_r](#) Nettype
- [n2e](#) for [v_wire_avg](#), [v_wire_sum](#), and [v_wire_one](#) Nettypes
- [netlist_commands_begin](#)

- [netlist_commands_end](#)
- [optimize_shadowfile](#)
- [param_pass](#)
- [port_connect](#)
- [port_dir](#)
- [print_ie_res](#)
- [r2e](#)
- [remove_d2a](#)
- [report_option](#)
- [resolve_x_inst_prefix](#)
- [rmap_file](#)
- [rt_a2d](#)
- [rt_d2a](#)
- [rt_e2n](#)
- [rt_e2r](#)
- [rt_n2e](#)
- [rt_r2e](#)
- [shadow_file](#)
- [shadow_file_dir](#)
- [skip_xmr_name_check](#)
- [spice_top](#)
- [transient_analysis](#)
- [u2e](#)
- [udn_bidir](#)
- [udn_e2n](#)
- [udn_n2e](#)
- [upf_port_connect](#)

- [upgrade_to_error](#)
- [use_spice](#)
- [use_verilog](#)
- [use_veriloga](#)
- [use_vhdl](#)

a2d

Controls all aspects of the analog-to-digital (a2d) interfaces in the VCS PrimeSim AMS tool. If you do not specify this command, all a2d events are triggered at 50% of the local VDD by default.

Syntax

```
a2d
[-loth lo_thrsh [V | %]]
[-hith hi_thrsh [V | %]]
[-xband xband]
[-hiz on | off]
[-library library_name]
[-cell cell_name -port port_name] |
[-inst inst_name -port port_name] | [-node hier_name]
[-vdd hier_name]
[-vss hier_name]
[-vdd_port port_name]
[-vss_port port_name]
[-minv min_vdd_voltage]
[-minv_logic 0 | 1 | X | Z]
[-ceff value]
[-midv_time value]
[-midv_logic 0 | 1 | X | Z]
[-except_port port_name]
[-strength supply | strong | pull | large | weak |
    medium | small | hiz | rmap]
```

Arguments

`-loth lo_thrsh [V | %]`, `-hith hi_thrsh [V | %]`

Specifies the low and high a2d threshold values. Threshold values are specified as absolute values (1.1V, 2.2) or as a percentage of the supply (10%, 90%). Voltages can be specified with or without the `v` character.

For a dynamic supply specification using the `-vdd` option, values can only be specified as a percentage of the supply voltage. This forces the a2d thresholds to change as the supply voltage changes.

`-xband xband`

Sets a hysteresis level for X-state detection. Parameters `-hith` and `-loth` set the thresholds for high, low, and X-state generation, as follows:

- When the voltage is above `-hith`, a 1 is generated.
- When the voltage is below `-loth`, a 0 is generated.
- When the voltage is between `-hith` and `-loth` for a time greater than `-midv_time`, an X-state is generated.

Parameter `-xband` creates additional threshold values, based on the equations:

```
hith_hys = hith - (hith - loth)/xband
loth_hys = loth + (hith - loth)/xband
```

- When a falling voltage is between `hith_hys` and `-loth` for a time greater than `-midv_time`, an X-state is generated.
- When a rising voltage is between `loth_hys` and `-hith` for a time greater than `-midv_time`, an X-state is generated.

There is no default value for `-xband`. If not set, the value of `loth_hys` is the same as `-loth` and the value of `hith_hys` is the same as `-hith`.

`-hiz off`

Turns off the a2d drive strength calculation. All a2d events are passed to digital with the Verilog default drive strength of 6 (strong). Use this option when specifying an a2d interface element for inout (bidirectional) nets. When this option is enabled, the a2d interface element always outputs digital signals with a **strong** drive strength (`strong0` or `strong1`) and does not output values with a `highz0` or `highz1` strength. For a list of Verilog drive strengths, see [Converting Signal Strength](#).

You can use this option to remove (mask) the HiZ glitches on bidirectional interface nets. The HiZ glitches are caused during the periods when neither the analog nor the digital circuits are driving the interface net. With this option enabled, the interface element always drives the digital side with a strength of `strong0` or `strong1` based on the voltage on the analog side and the a2d thresholds.

Note:

HiZ glitches usually reflect the correct behavior of the circuit. Removing them by using the `-hiz off` option can potentially mask a real phenomenon or problem in the circuit and is discouraged.

`-hiz on`

Turns on the a2d drive strength calculation. For each a2d event, the analog engine calculates the analog output resistance and uses that value as an index to the resistance map file to get the corresponding Verilog output drive strength. With this option enabled, HiZ states are passed to digital if the analog engine identifies them. This feature is enabled by default for bidirectional interface nets.

`-library library_name`

Specifies a *library_name* to apply global a2d threshold options. Use this option to specify unique threshold options for all cells in the specified library. For more information, see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies cell and port names for the interface element. The *cell_name* must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical instance path to the port for the interface element. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name for the interface element. You can specify the asterisk (*) wildcard character in a node name. You must also specify the `-debug_access` option with the `vcs` command when you specify the wildcard character.

`-vdd hier_name, -vss hier_name`

Specifies the reference supply to use when calculating high and low thresholds. If you use this option, the high and low thresholds change as the combination of VDD and VSS supply voltages identified by the *hier_name* changes during the simulation. By default, without this option the reference supply is VDD, if defined, or the interface element behaves as if the reference supply voltage was constant. With this option enabled, only the % format can be used to specify *loth* and *hith* arguments. The supply net identified by the *hier_name* can be either an internal analog node, a top-level analog net, a regular interface net (d2a, inout), or a *real* interface net.

`-vdd_port port_name, -vss_port port_name`

Specifies the vdd or vss port to use as the reference supply. Use the `-vdd_port` and `-vss_port` arguments to assign the supply sensitivity to the a2d interface element when you only know the vdd and vss ports of the block. The `-vdd_port` and `-vss_port` arguments allow the tool to trace the hierarchy to identify the actual vdd and gnd parent nets. The `-vdd_port` and `-vss_port` arguments

must be used together with the `-port` argument. The `port_name` can also refer to a node one level higher in the hierarchy, such as `-vdd_port ../VDD1`.

Note that the `interface_element.rpt` contains the `vdd` and `vss` that were used, not the `-vdd_port` and `-vss_port` statements from the `vcsAD.init` file.

`-minv min_vdd_voltage`

Specifies the minimum voltage for VDD (or VDD-VSS) that turns off the a2d conversion. Use this option to disable a2d conversions when the analog supply (or the difference of supplies) is very low. In this case, it does not make sense to continue converting analog signals to logic values and generate simulation events.

When specified, if the VDD supply (or VDD-VSS supplies) associated with the interface element goes below this value, the a2d conversion is turned off and a fixed logic value is output on the digital side as specified by the `-minv_logic` argument. By default, `minv` is 100mV.

Note:

The `-minv` argument is required when both `-vdd` and `-vss` of the dynamic supply pair are used.

`-minv_logic 0 | 1 | X | Z`

Specifies the output logic value when the a2d conversion is disabled with the `-minv` trigger. By default, a `Z` value is output.

`-ceff value`

Specifies a capacitance value to model the loading effect of the digital blocks driven by the analog interface net. By default, no extra capacitance is inserted at the a2d interface net. Use this option to specify a loading capacitance to insert at the a2d interface net or nets. The unit of the value is Farad.

`-midv_time value, -midv_logic 0 | 1 | X | Z`

The `-midv_time` option sets the time (seconds) when the `a2d` command generates an X or Z-state. These options work with the `-hith` and `-loth` options. There is no default value.

The `-midv_logic` option specifies the output logic value. The default value is `X`.

`-except_port port_name`

Prevents dynamic supply sensitivity on the specified ports when you used wildcards in a previous `a2d` command with `-port`. You can specify the asterisk

(*) wildcard character partially in a port name to filter out certain ports. For example,

```
use_verilog -cell F32;
a2d -loth 0.4 -hith 0.8 -cell F32 -port VDDV12DIG;
a2d -loth 20% -hith 80% -cell F32 -port * -minv 0.6 \
    -minv_logic 0 -vdd tb.VDD -except_port VDDV12DIG;
a2d -loth 20% -hith 80% -cell F32 -port * -minv 0.6 \
    -minv_logic 0 -vdd tb.VDD -except_port VDD*;
```

```
-strength [supply | strong | pull | large | weak | medium | small | hiz
| rmap]
```

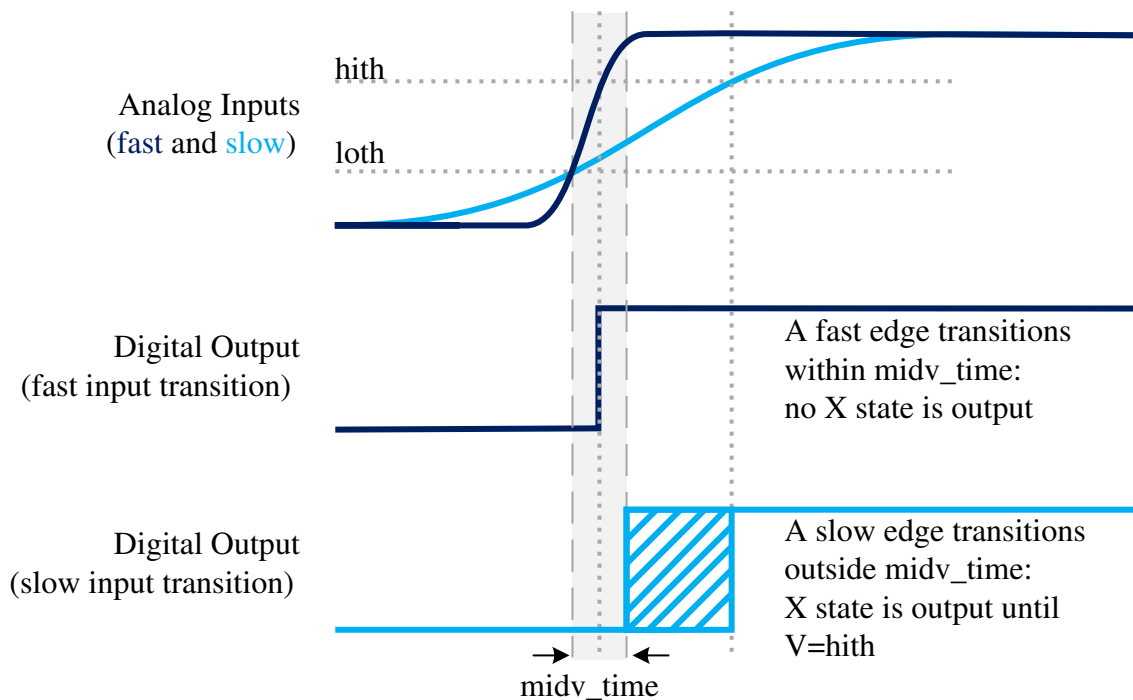
Disables the output strength calculation based on values in the `rmapAD.init` file and always outputs a fixed drive strength. By default, the value is `rmap` and the strength is calculated based on the `rmapAD.init` file.

Description

The `-midv_time` argument is used to model the behavior of slow-to-transition signals. When the analog input voltage rises above the `-hith` value before the `-midv_time` expires, the digital signal transitions between 0 and 1 without generating an X-state, as shown in middle waveform of [Figure 27](#). Fast transitions times should not generate an X-state at the boundary as these X-states corrupt the digital simulation results.

Slow transitions generate an X-state after the time specified by `-midv_time` expires. By default, an X-state is generated on the digital output. You can change this default to a Z-state with the `-midv_logic Z` option. After the voltage goes above `-hith`, a 1 is output as shown in the bottom waveform of [Figure 27](#).

Figure 27 *midv_time with Fast and Slow Input Edge Transitions*



Examples

The following examples illustrate the use of different options with the a2d interface element:

- Set the a2d low and high thresholds for node `top.i1.ct1` to 0.2V and 1.7V respectively. Use either command to perform this task. Note that the units character, *v*, is optional.

```
a2d -loth 0.2V -hith 1.7V -node top.i1.ct1;
a2d -loth 0.2 -hith 1.7 -node top.i1.ct1;
```

- Set the a2d low and high thresholds for node `top.i1.ct1` to 20% and 80% of the local supply voltage.

```
a2d -loth 20% -hith 80% -node top.i1.ct1;
```

- Enable the dynamic supply feature and set the a2d thresholds for node `top.i1.ct1` to 50% of the referenced VDD node. As the voltage of the VDD node changes, so do the a2d thresholds.

```
a2d -loth 50% -hith 50% -node top.i1.ct1 -vdd top.vdd;
```

- Enable the dynamic supply feature and set the a2d thresholds for node `top.i1.ct1` to 40% and 60% of the referenced VDD node. The `-minv` value is set to 500mV. As a

result, if the VDD voltage drops below that value, the a2d conversion is shut off and HiZ is output on the digital side instead.

```
a2d -loth 40% -hith 60% -node top.i1.ct1 -vdd top.vdd -minv 500mV;
```

- Enable the dynamic supply feature and set the a2d thresholds for node top.i1.ct1 to 40% and 60% of the referenced VDD node. Set the -minv value to 500mV and specify that logic value x is output on the digital side if the VDD voltage drops below the -minv value.

```
a2d -loth 40% -hith 60% -node top.i1.ct1 -vdd top.vdd -minv 500mV
    -minv_logic x;
```

Note:

If multiple a2d commands in the mixed-signal simulation control file target the same node, the last command overrides all the others.

- Specify top.n1 to be an a2d interface net and add 1pF capacitance at the top.n1 node.

```
a2d -node top.n1 -ceff 1p;
```

- Specify the ports in1 of cell blk1 to be a2d interface nets and add 2pF capacitance at this port.

```
a2d -cell blk1 -port in1 -ceff 2p;
```

- Output a Z-state on the digital output when the analog voltage is between 0.7V and 0.2V for more than 10ns.

```
a2d -hith 0.7 -loth 0.2 -midv_time 10n -midv_logic Z -node top.w1;
```

- The following example uses the -vdd_port and -vss_port arguments to correctly specify vdd and vss for the circuit. The circuit contains three inverter blocks: inva, invb, and invc. inva contains an additional vdd supply; invb and invc contain one vdd connection and one vss connection. The vcsAD.init mixed-signal simulation control file contains two a2d commands to correctly specify the vdd and vss connections for the simulation.

cells.v

```
module inva(a1);
module invb(b1);
module invc(c1);
```

* SPICE subcircuits

```
.subckt inva a1 vdda vssa vddh
.subckt invb b1 vddb vssb
.subckt invc c1 vddc vssc
...
X1 n1 vdd1 vss1 vddh1 inva
```

```
X2 n2 vdd2 vss2 invb
X3 n3 vdd3 vss3 invc
```

The `vcsAD.init` file contains the following `a2d` specification. Note that the second `a2d` command overrides the first `a2d` command for the `inva` cell.

```
// vcsAD.init
a2d -cell * -port * -vdd_port vdd* -vss_port vss*
a2d -cell inva -port * -vdd_port vdda -vss_port vssa
```

ams_cdef_inst

Description

Overrides connectivity to power and ground nets given from `ams_cdef_net` commands. This command is used when specific instances in the designs have different power and ground connections.

Note:

This command is used with the `ams_supply` and `ams_cdef_net` commands to allow power-aware connect modules in the design.

Syntax

```
ams_cdef_inst -cell cell_name [-lib library_name]
               -inst {inst_name [, inst_name] ...}
               -supply {attribute_name => net_name, [attribute_name => net_name];
```

Arguments

`-cell cell_name`

Specifies the cell name.

`-lib library_name`

Specifies the library name. This is the VCS compiled library.

`-inst {inst_name [, inst_name] ...}`

Specifies a list of instances that are overridden.

`-supply {attribute_name => net_name, [attribute_name => net_name}`

Specifies the name of the net associated with the list in the `-net` argument of the `ams_cdef_net` command.

Examples

The following example shows one use of the `ams_cdef_net` and `ams_cdef_inst` commands:

- This example electrically connects the net `vdd2` in the cell `blk02` to the net `top.vdd` for all instances of `blk02` in the design with the exception of instance `top.i1`. For this instance, net `vdd2` is connected to `top.vdd1`.

```
ams_cdef_net -cell blk02 -net {vdd2} -default_supply {vdd2_tag =>
top.vdd};
ams_cdef_inst -cell blk02 -inst {top.i1} -supply {vdd2_tag =>
top.vdd1};
```

ams_cdef_net

Description

Connects an electrical net to an analog power or ground electrical net. When the netlist for the design does not contain the connectivity for the power and ground network, this command is used to create that connection.

Note:

This command is used with the `ams_supply` and `ams_cdef_inst` commands to allow power-aware connect modules in the design.

Syntax

```
ams_cdef_net -cell cell_name [-lib library_name]
-net {net_name [, net_name] ...}
-default_supply {attribute_name => net_name};
```

Arguments

`-cell cell_name`

Specifies the cell name.

`-lib library_name`

Specifies the library name. This is the VCS compiled library.

`-net {net_name [, net_name] ...}`

Specifies a list of net names that are connected to the net specified by the `-default_supply` argument. These nets are declared inside the specified cell.

`-default_supply {attribute_name => net_name}`

Specifies the name of the net associated with the list in the `-net` argument.

Examples

The following example shows one use of the `ams_cdef_net` command:

- This example electrically connects the net `vdd2` in the cell `blk02` to the net `top.vdd`.

```
ams_cdef_net -cell blk02 -net {vdd2} -default_supply {vdd2_tag =>
    top.vdd};
```

ams_cm

Description

Enables the override of connect rules and connect module parameters in the Verilog-AMS-SPICE flow. For more information, see [Converting Signals With Interface A/D and D/A Connect Modules](#).

Using this command, you can override the following:

- General parameters of one or more connect module instances
- Connect module insertion mode (split | merge)
- Connect rule for connect module insertion

Note:

This command does not change any disciplines in the design (for this, use [ams_set_discipline](#)).

This command is functional only when the Verilog-AMS-SPICE flow is enabled using the VCS `-ams` option. If the Verilog-AMS-SPICE flow is not enabled, this command is ignored and a warning message is output.

Syntax

```
ams_cm -connect_rule_module_parameter_name parameter_value
[-mode split | merge]
[-crules connect_module_library_name.connect_rule_name]
[-library library_name]
[-cell cell_name [-node node_name]]
[-inst inst_name [-node node_name]]
[-node hier_name]
[-vddnet global_supply_name | hierarchical_path_to_supply_node]
[-vssnet global_supply_name | hierarchical_path_to_supply_node]
```

Arguments

`-connect_rule_module_parameter_name parameter_value`

Specifies the connect rule or connect module parameter and the value to which it is set. The `parameter_value` can be either real or a string parameter. If you

specify override information for a nonexistent parameter of a connect rule or connect module, then that override is ignored.

`-mode split | merge`

Specifies the connect module insertion mode. The default is `merge`.

`-crules connect_module_library_name.connect_rule_name`

Specifies the connect rules. The default for the `.connect_rule_name` extension is `snps_crules`.

Note:

Multiple connect rules cannot be specified for the same `ams_cm` command, but multiple connect rules can be referenced with separate `ams_cm` commands.

`-library library_name`

Specifies the name of the logical library for which the connect module is inserted or connected globally.

`-cell cell_name [-node node_name]`

Specifies the cell or cell/node names for which the connect module is inserted or connected. The node name is optional; if it is not specified, this argument is treated as `-cell cell_name -node *`.

`-inst inst_name [-node node_name]`

Specifies the hierarchical instance path or instance/node names for which the connect module is inserted or connected. The node name is optional; if it is not specified, this argument is treated as `inst inst_name -node *`.

`-node hier_name`

Specifies the hierarchical node name for which the connect module is inserted or connected.

`-vddnet global_supply_name|hierarchical_path_to_supply_node`

Specifies the `-vddnet` parameter.

Note:

This parameter is not applicable with the default connect rule, so a different connect rule must be given if this argument is used. The connect rule containing the `-vddnet` argument references connect modules containing the Verilog-AMS `$node_alias` function and is used for dynamic supply sensitivity in the connect modules.

```
-vssnet global_supply_name|hierarchical_path_to_supply_node
```

Specifies the `-vssnet` parameter.

Note:

This parameter is not applicable with the default connect rule, so a different connect rule must be given if this argument is used. The connect rule containing the `-vssnet` argument references connect modules containing the Verilog-AMS `$node_alias` function and is used for dynamic supply sensitivity in the connect modules.

Examples

The following examples show different uses of the `ams_cm` command:

- This example globally overrides the `vsup` parameter value (3.3) for all connect modules connected or inserted throughout the design.

```
ams_cm -vsup 3.3;
```

- This example overrides the `vsup` parameter value (3.2) for any connect modules connected or inserted on node `ClockTop.I3.y`.

```
ams_cm -vsup 3.2 -node ClockTop.I3.y;
```

Note:

There could be multiple connect modules inserted on the node `ClockTop.I3.y` depending on whether the connect module insertion mode is set to `split` or `merge`.

- This example overrides the `vsup` parameter value (3.222) for all connect modules connected or inserted for any cell named `dflop` in the design.

```
ams_cm -vsup 3.222 -cell dflop;
```

- This example overrides the `vsup` parameter value (3.333) for all connect modules connected or inserted for the instance `ClockTop2.I3` in the design.

```
ams_cm -vsup 3.333 -inst ClockTop2.I3;
```

- This example:
 1. Globally overrides the `vsup` parameter value (3.3) for all connect modules connected or inserted throughout the design,
 2. Then overrides the `vsup` parameter value (3.222) for all connect modules connected or inserted for the instance `ClockTop2.I3.I5` in the design,
 3. Then overrides the `vsup` parameter value (3.444) for any connect modules connected or inserted on node `ClockTop2.I3.net18`.

```
ams_cm -vsup 3.3;  
ams_cm -vsup 3.222 -inst ClockTop2.I3.I5;  
ams_cm -vsup 3.444 -node ClockTop2.I3.net18;
```

- This example:
 1. Globally overrides the `vsup` parameter value (3.3) for all connect modules connected or inserted throughout the design,
 2. Then overrides the `vsup` parameter value (3.222) for all connect modules connected or inserted for the instance `ClockTop2.I3` with node `CLK` in the design.

```
ams_cm -vsup 3.3;  
ams_cm -vsup 3.222 -inst ClockTop2.I3 -node CLK;
```

- This example:
 1. Globally overrides the `vsup` parameter value (3.3) for all connect modules connected or inserted throughout the design,
 2. Then overrides the `-vddnet` parameter for dynamic supply on instance `ClockTop.I3` in the design using the connect rules in `snpsConnectLib.snps_ana_crules`.

```
ams_cm -vsup 3.3;  
ams_cm -crules snpsConnectLib.snps_ana_crules -vddnet top.vdd1 -inst  
ClockTop.I3;
```

- This example globally changes the connect mode to `split`.

```
ams_cm -mode split;
```

ams_set_discipline

Description

Enables the override of net disciplines (or setting the discipline of a net that does not have any declared discipline) in the Verilog-AMS-SPICE flow. For more information, see [Nets and Disciplines](#).

- Global `ams_set_discipline` commands override the discipline provided by the switch `-ams_discipline`
- Cell / cell-node / library / global level `ams_set_discipline` commands:
 - Can only have discrete discipline
 - Are applied after discipline resolution on nodes having no declared or resolved discipline
 - Cannot influence discipline resolution outcome
- Node / inst-node / inst level `ams_set_discipline` commands:
 - Can have either discrete or continuous discipline
 - Are applied before discipline resolution on nodes having no declared discipline
 - Can influence discipline resolution outcome

Syntax

```
ams_set_discipline -discipline discipline_name
  [-library library_name]
  [-cell cell_name [-node node_name]]
  [-inst inst_name [-node node_name]]
  [-node hier_name]
```

Arguments

`-discipline discipline_name`

Specifies the name of the discipline to declare.

`-library library_name`

Specifies the name of the logical library for which the discipline is declared.

`-cell cell_name [-node node_name]`

Specifies the cell or cell/node names for which the discipline is declared. The node name is optional; if it is not specified, this argument is treated as `-cell cell_name -node *`.

`-inst inst_name [-node node_name]`

Specifies the hierarchical instance path or instance/node names for which the discipline is declared. The node name is optional; if it is not specified, this argument is treated as `-inst inst_name -node *`.

`-node hier_name`

Specifies the hierarchical node name for which the discipline is declared.

ams_supply

Description

Associates the analog supply information with digital signal ports in the Verilog-AMS-SPICE flow. This command is used for Verilog-AMS modules that have digital signal ports and analog supply ports.

Note:

- This command works with connect modules that use the `port_power` and `port_ground` attributes as internal electrical supplies.
- This command is used with the `ams_cdef_net` and `ams_cdef_inst` commands to allow power-aware connect modules in the design.

Syntax

```
ams_supply -cell cell_name [--lib library_name]  
-digital_port {digital_port_name [-, digital_port_name] ...}  
-supply {port_power => port_power,  
        port_ground => ground_port_name};
```

Arguments

`-cell cell_name`

Specifies the cell name.

`-lib library_name`

Specifies the library name. This is the VCS compiled library.

`-digital_port {digital_port_name, digital_port_name, ...}`

Specifies a list of port names that are associated with the supply set.

`-supply {port_power => power_port_name,
 port_ground => ground_port_name}`

Specifies the names of the power and ground associated with the list of port names in the `-digital_port` argument.

Examples

The following examples show different uses of the `ams_supply` command:

- This example associates the logic ports `y` and `a` to the power port `vdd` and ground port `gnd` for cell `inv01`.

```
ams_supply -cell inv01 -digital_port {y, a} -supply {port_power => vdd,  
        port_ground => gnd};
```

This Verilog-AMS module has four ports: two are declared as `ddiscrete` (`y` and `a`) and two are declared as `electrical` (`vdd` and `gnd`).

- This example associates the logic port `y` to the power port `vdd1`, and ground port `gnd` and the logic port `a` to the power port `vdd2` and ground port `gnd` for cell `inv01`.

```
ams_supply -cell inv01 -digital_port {y} -supply {port_power => vdd1,
port_ground => gnd};
ams_supply -cell inv01 -digital_port {a} -supply {port_power => vdd2,
port_ground => gnd};
```

This Verilog-AMS module has five ports: two are declared as `ddiscrete` (`y` and `a`) and three are declared as `electrical` (`vdd1`, `vdd2`, and `gnd`).

bus_format

Identifies the bus notation used in the SPICE netlist.

Syntax

```
bus_format -format [open_char] %d [close_char] [[open_char] %d
[close_char]...];
```

Description

This command causes the tool to treat multiple SPICE ports as members of a bus and group them together when making connections between SPICE and Verilog/VHDL buses at the analog/digital boundary. It allows more than one bus format to be identified as the SPICE bus notation. The `[%d]` notation is the default bus format for SPICE. The ports in a SPICE subcircuit that follow the `[%d]` notation, such as `a[2]` `a[1]` `a[0]`, are considered members of a bus when connecting to Verilog by default.

Examples

The following examples illustrate the use of different options with the `bus_format` command:

- Specify that angle brackets (`<` `>`) are used as the bus notation used in the SPICE netlist:

```
bus_format -format <%d>;

*SPICE subckt
.subckt addr4 a<3> a<2> a<1> a<0>
+b<3> b<2> b<1> b<0> cin
+s<3> s<2> s<1> s<0> cout
.ends
```

- Define multiple bus formats for SPICE. Both ports `inp` and `out` are considered as buses when connecting to Verilog.

```
bus_format -format <%d> _%d ;

*SPICE subckt definition
.subckt data_blk inp<2> inp<1> inp<0> out 2 out_1 out_0
...
.ends
```

- Specify that the underscore character (`_`) is used as `open_char` in SPICE bus notation, and no character is used as `close_char`.

```
bus_format -format _%d;

*SPICE subckt
.subckt addr4 a_3 a_2 a_1 a_0
+b_3 b_2 b_1 b_0 cin
+s_3 s_2 s_1 s_0 cout
.ends
```

Note:

Mixed-signal simulation *only* supports the following bus format characters for SPICE: `<` `>` `[` `]` `_`

choose

Description

Use the `choose` command to select the analog simulator (the PrimeSim XA, FineSim, and PrimeSim simulation engines) and command options for mixed-signal simulation.

Mixed-signal simulation supports all the PrimeSim XA command-line options.

Syntax

```
choose xa xa_command_line_options;
choose finesim finesim_command_line_options;
choose primesim primesim_command_line_options;
```

Examples

The following examples illustrate different usages of the `choose` command:

- Specify that the PrimeSim XA tool is used during mixed-signal simulation and the PrimeSim XA tool reads a netlist named `net.spi`. The `-c` option passes PrimeSim XA analog configuration commands stored in the `xa.cmd` file. Comments are specified with double slashes (`//`).

```
//This is a comment
choose xa -hspice net.spi -c xa.cmd;
```

- Specify that the FineSim Pro tool is used during mixed-signal simulation. The FineSim Pro tool reads a netlist named `net.spi`. The `-o` option specifies the prefix for output files created by FineSim.

```
//This is a comment
choose finesim net.spi -o out;
```

- Specify that FineSim in SPICE-mode is used during mixed-signal simulation. The FineSim tool reads a netlist named `net.spi`. The `-o` option specifies the prefix for output files created by FineSim.

```
// This is comment
choose finesim -spice net.spi -o out;
```

- Specify that the PrimeSim tool is used during mixed-signal simulation. The PrimeSim tool reads a netlist named `net.spi`. The `-o` option specifies the prefix for output files created by PrimeSim.

```
//This is a comment
choose primesim net.spi -o out;
```

- Specify that the PrimeSim tool in SPICE-mode is used during mixed-signal simulation. The PrimeSim tool reads a netlist named `net.spi`. The `-o` option specifies the prefix for output files created by PrimeSim.

```
// This is comment
choose primesim -spice net.spi -o out;
```

d2a

Description

Controls all aspects of the digital-to-analog (d2a) interfaces in the VCS PrimeSim AMS tool.

Syntax

```
d2a [-powernet 0 | 1] [-rf_time slope_time] | [-rise_time rise_time]
    [-fall_time fall_time]
    [-delay delay_time] | [-rise_delay rise_delay]
    [-fall_delay fall_delay]
    [-x2v 0 | 1 | 2 | 3 | 4]
    [-hiv high_voltage | %]
    [-lov low_voltage | %]
    [-library library_name]
    [-cell cell_name -port port_name] |
    [-inst inst_name -port port_name] | [-node hier_name]
```

```
[-vdd vdd_hier_name]
[-vss vss_hier_name]
[-vdd_port port_name]
[-vss_port port_name]
[-minv value]
[-minv_analog value]
[-vdd_filter filter_thresh]
```

Arguments

`-powernet 0 | 1`

Identifies a d2a node as an ideal voltage source on the analog side without the resistance map resistor. You must use this option when Verilog drives analog power nets in order to remove the series resistance map resistors and to allow efficient partitioning of the analog circuit.

- `0`: to not treat as an ideal voltage source (default).
- `1`: to treat as an ideal voltage source.

Note that If the Verilog wires driving the SPICE supply are defined as `supply1` or `supply0` types in the Verilog code, the tool treats the d2a interfaces connected to those wire types as if the d2a command with the `-powernet` option was used. This means that no resistance map resistors are inserted at the d2a interface and the supply nets are treated as ideal sources in analog.

`-rf_time slope_time`

Specifies the analog rise and fall times. The default time unit is in seconds, so specify the subunit with the value, for example, `-rf_time 1.5n`. With this option, both rise and fall times are set to the same value.

`-rise_time rise_time`

Specifies the analog rise time. The default time unit is in seconds, so specify the subunit with the value, for example, `-rise_time 1n`.

`-fall_time fall_time`

Specifies the analog fall time. The default time unit is in seconds, so specify the subunit with the value, for example, `-fall_time 2n`.

`-delay delay_time`

Specifies the inertial delay before the analog transition starts. The default time unit is in seconds, so specify the subunit with the value, for example, `-delay 1.5n`. With this option, the inertial delays for both rising and falling signals are set to the same value.

`-rise_delay rise_delay`

Specifies the inertial delay before the analog transition starts for rising digital signals. The default time unit is in seconds, so specify the subunit with the value, for example, `-rise_delay 1n`.

`-fall_delay fall_delay`

Specifies the inertial delay before the analog transition starts for falling digital signals. The default time unit is in seconds, so specify the subunit with the value, for example, `-fall_delay 2n`.

`-x2v 0 | 1 | 2 | 3 | 4`

Sets the rule for how a logic X must be translated to a voltage level on the analog side: Use this option to manage the translation of the X to a voltage level.

- 0: always set to the logic 0 voltage (default)
- 1: always set to the logic 1 voltage
- 2: set to the (logic 0 voltage + logic 1 voltage)/2
- 3: set to previous voltage
- 4: Set the output based on the following conditions
 - If the input is logic 0, set to the logic 1 voltage
 - If the input is logic 1, set to the logic 0 voltage
 - Otherwise, set to the previous voltage

`-hiv high_voltage`

Specifies the default voltage for logic 1 as an absolute voltage value (for example, 1.2V or 1.2) or as a percentage of the supply voltage (for example, 90%). By default, the logic 1 voltage value is the voltage of the local supply. If the tool cannot trace the d2a net to an ideal supply, it assumes 3.3V.

If the dynamic supply feature is enabled by using the `-vdd` option, the values for `-hiv` can only be specified as a percentage of the VDD net. You must use this option together with the `-lov` option.

`-lov low_voltage`

Specifies the default voltage for logic 0 as an absolute voltage value (for example, 0.1V or 0.1) or as a percentage of the supply voltage (for example, 10%). By default, the logic 0 voltage value is assumed to be 0V.

If the dynamic supply feature is enabled by using the `-vdd` option, values for `lov` can only be specified as a percentage of the VDD net. You must use this option together with the `-hiv` option.

`-library library_name`

Specifies a library name to apply global d2a threshold options. You can apply threshold options on a library-by-library basis as described in [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies the cell and port names for the interface element. The name must be the same as the `cell_name` used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port for the interface element. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name for the interface element. You can specify the asterisk (*) wildcard character in a node name. You must also specify the `-debug_access` option with the `vcs` command when you use the wildcard character.

`-vdd vdd_hier_name, -vss vss_hier_name`

Specifies the nodes for high and low reference supplies. Distinguishes the reference supply associated logic high and low values. If you use this option, the d2a logic high and low values change as the combination of vdd and vss supply voltages identified by the `hier_name` changes during the simulation. By default, the d2a command either refers to vdd, if defined, or behaves as if the reference supply voltage was constant. With this option enabled, values to the `-lov` and `-hiv` arguments must be specified as percentages using the % format. The supply net identified by the `hier_name` can be either an internal analog node, a top-level analog net, a regular interface net (d2a, inout). or a `real` interface net.

`-vdd_port port_name, -vss_port port_name`

Specifies the vdd or vss port to use as the reference supply. Use the `-vdd_port` and `-vss_port` arguments to assign the supply sensitivity to the a2d interface element when you only know the vdd and vss ports of the block. The `-vdd_port` and `-vss_port` arguments allow the tool to trace the hierarchy to identify the actual vdd and gnd parent nets. The `-vdd_port` and `-vss_port` arguments must be used together with the `-port` argument. The `port_name` can also refer to a node one level higher in the hierarchy, such as `-vdd_port ../VDD1`.

Note that the `interface_element.rpt` contains the vdd and vss that were used, not the `-vdd_port` and `-vss_port` statements from the `vcsAD.init` file.

`-minv value`

Specifies the minimum voltage for VDD (or VDD-VSS) that turns off the d2a conversion. The purpose of the option is to disable d2a conversions when the analog supply (or the difference of supplies) is very low and it does not make sense to continue converting analog signals to logic values, giving the impression to the digital domain that the analog circuit is still functioning.

When specified, if the VDD supply (or VDD-VSS supplies) associated with the interface element goes below this value, the d2a conversion is turned off and the d2a output is floating, keeping the current value (or DC value at `time=0`). If `-minv_analog` is specified, the d2a output is equal to the `-minv_analog` value instead. By default, `-minv` is set to 100mV.

Note:

The `-minv` argument is required when both `-vdd` and `-vss` of the dynamic supply pair are used.

`-minv_analog value`

Specifies the analog value passed to the analog engine when the d2a conversion is disabled because `vdd-vss` is less than `-minv`. It is undefined by default, so the d2a output is left floating.

`-vdd_filter filter_thresh`

Triggers the evaluation of the d2a interface. The interface element is evaluated only if the variation of the power supply defined with `vdd_hier_name` is greater than `filter_thresh`.

This argument is only valid if the power supply is a dynamic supply. If you specify both the global command [dynamic_supply_filter](#) and `d2a -vdd_filter`, then the [dynamic_supply_filter](#) value should be smaller than the `d2a -vdd_filter` threshold value, and the `-vdd_filter` value overrides the global [dynamic_supply_filter](#) value. For more information, see the [dynamic_supply_filter](#) command.

Examples

The following examples illustrate different usages of the d2a command:

- Set the high and low d2a voltages to 1.8V and 0.1V.

```
d2a -hiv 1.8 -lov 0.1 -node top.i1.ct1;
```

- Set the high and low d2a voltages to 1.2V and 0V.

```
d2a -hiv 1.2V -lov 0V -node top.i1.ct1;
```

- Define interface nets `top.vdd` and `top.vss` as ideal supplies while setting their high and low voltage values to 1.2V and 0V respectively.

```
d2a -powernet 1 -hiv 1.2 -lov 0 -node top.vdd;
d2a -powernet 1 -hiv 1.2 -lov 0 -node top.vss;
```

- Enable dynamic supply and define the interface net `top.i1.rst` as a d2a interface for which high and low voltages track the changes on node `top.i2.vdd`. The high and low voltages are set to 90% and 10% of the voltage on the node `top.i2.vdd`.

```
d2a -hiv 90% -lov 10% -node top.i1.rst -vdd top.i2.vdd;
```

- Enable dynamic supply for all d2a interface elements below the hierarchical level `top.i1`. All those d2a interfaces track the changes on supply net `top.i1.vdd` dynamically. The `-hiv` and `-lov` values are set to 100% and 0% of the voltage on node `top.i1.vdd`.

```
d2a -hiv 100% -lov 0% -node top.i1.* -vdd top.i1.vdd;
```

- Enable multiple dynamic supplies with respect to `vdd1` and `vss1` as the power of reference to define the logic high and low to all ports of the instance `top.x2`. If the difference between the `vdd1` and `vss1` supplies is less than the `-minv` value of 0.1v, the d2a is in a high impedance state and the analog output is equal to the `-minv_analog` value of 0V.

```
d2a -hiv 100% -lov 0% -inst top.x2 -port * -vdd vdd1 -vss vss1
    -minv_analog 0 -minv 0.1;
```

Note:

If multiple d2a commands in the mixed-signal simulation control file target the same node, the last command overrides all the others.

disable_ie

Description

Disables an interface element from the digital-analog boundary. The `-node` argument is the path to a mixed-signal net. The `-node` argument cannot take the asterisk wildcard character (*), such as `-node *`. However, a partial node search using the asterisk character is legal; for example, `-node top.din*` matches all signals with names starting with the `top.din` string.

This command disables any type of interface element, including:

- Logic/electrical (a2d and d2a)
- Real/electrical (r2e and e2r)

- Nettype (n2e and e2n)
- User-defined nettype/electrical (udn_n2e and udn_e2n)
- UPF (u2e and e2u)

Note:

A warning message is displayed during VCS elaboration when this command has been used. The interface element that has been disabled is printed in the [interface_element.rpt](#) file. In the report, it is prefaced with a # sign and includes the string `disable_ie`.

Syntax

```
disable_ie -node node_name;
```

Arguments

```
node -node_name
```

Specifies the node from which to disable the interface element. This argument cannot take the asterisk wildcard character (*), for example, `-node *`. However, a partially-specified node search using the asterisk character is accepted, such as `-node N*`.

Examples

- Disable the interface element from node `top.i1.outa`.

```
disable_ie -node top.i1.outa;
```

The [interface_element.rpt](#) shows:

```
#e2r -min_delta 1e-05 disable_ie -node top.i1.outa;
```

downgrade_to_warn

Downgrades or lowers the priority of an error message to a warning message.

Syntax

```
downgrade_to_warn -id message_ID [message_ID ...];
```

Description

Use the `downgrade_to_warn` command to downgrade the `[MSV-IE-OPT-TNF]` and `[MSV-RTIE-OPT-TNF]` error messages. The `[MSV-IE-OPT-TNF]` message identifier applies to error and warning messages for interface element commands. The `[MSV-RTIE-OPT-TNF]` message identifier applies to error and warning messages for runtime interface element commands.

The [MSV-IE-OPT-TNF] message identifier applies to the following error/warning message:

```
[MSV-IE-OPT-TNF]: Interface Element command, Option Target Not Found
```

It applies to the following interface element command arguments.

- -node incorrect path/node name
- -cell incorrect cell name
- -port incorrect port name
- -inst incorrect path/instance name
- -vdd incorrect path/vdd name
- -vss incorrect path/vss name

Note:

The -vdd and -vss arguments do not apply to the runtime interface element commands.

The [MSV-RTIE-OPT-TNF] message identifier applies to the following error/warning message,

```
[MSV-RTIE-OPT-TNF]: Runtime Interface Element command, Option Target Not Found
```

It applies to the following runtime interface element command arguments.

- -node incorrect path/node name
- -cell incorrect cell name
- -port incorrect port name
- -inst incorrect <path/instance name>

Examples

The following example defines an a2d interface element in the vcsAD.init file:

```
d2a -hiv 1.8 -lov 0.0 -node top.fail;
```

If the preceding command generates the following error message:

```
Error: [MSV-IE-OPT-TNF]: Interface Element command, Option Target Not Found.
```

In the "vcsAD.init" file line no. 2, "d2a" command, option target "node=top.fail" was not found, the command will be ignored.

To resolve the problem please modify the command to contain the correct "node=<path to node>.<node name>" for the Interface Element.

You can downgrade this type of error message to a warning message with the following command:

```
downgrade_to_warn -id MSV-IE-OPT-TNF
```

duplicate_net_inst_name

Allows the same name to be used for both an instance and a net in SPICE.

Syntax

```
duplicate_net_inst_name -enable | -disable;
```

Description

The `duplicate_net_inst_name -enable` command internally renames the net names that are used for both an instance and a net in SPICE. Nets are renamed to `net_name_DUPLICATE_INST_n` in Verilog (not in SPICE, original duplicate names are preserved in SPICE) where `n` is a numerical index starting from 0 and going up if the duplicate name is used for more than one net.

Using the same name for a node and an instance is allowed in SPICE but it is illegal in Verilog and causes a compilation error. Because mixed-signal simulation builds a Verilog shadow module for each instance of SPICE subcircuit by default, if the same name is used both as an instance and a net name anywhere in the SPICE netlist, the Verilog shadow modules inherits the same net and instance names which violate the Verilog rule and lead to a mixed-signal compilation error.

One of the options (`-enable` or `-disable`) must be used with the command. The `-enable` option allows identical names to be used for both SPICE nets and instance in mixed-signal by renaming the net name to `net_name_DUPLICATE_INST_n` in the Verilog shadow module created for the SPICE instance, where `n` is a numerical index.

By default, this option is disabled and duplicate names used for nets and an instance in SPICE cause a compilation error. You can specify this behavior with the `duplicate_net_inst_name -disable` command.:

Examples

In this example, the subcircuit `and2x2` is instantiated with instance name `x_and1` while at the same time one of the nodes connected to its ports is also called `x_and1`. This is allowed in SPICE, but invalid in Verilog, and causes an error during mixed-signal compilation.

```
x_and1 a x_and1 y and2x2
```

To resolve it, use the following command inside the mixed-signal simulation control file:

```
duplicate_net_inst_name -enable;
```

By doing so, the net name is renamed to `x_and1_DUPLICATE_INST_0` in Verilog to avoid the conflict, but the names remain unchanged in SPICE.

dynamic_supply_filter

Triggers an evaluation at the digital/analog interface, based on a change in supply voltage, only if the variation of the power supply is greater than the specified voltage change. This feature can be used only when you specify an interface element with a dynamic power supply by using the `vdd` or `vss` argument.

Syntax

```
dynamic_supply_filter -deltav value;
```

Description

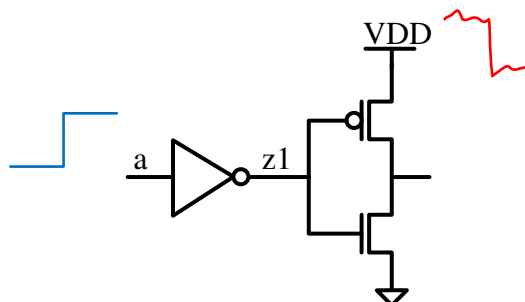
Use this command to adjust the number of simulation events produced when there is a continuous variation of the power supply over the simulation time, such as a power up mode. Do not overestimate the `dynamic_supply_filter` value. It is a tradeoff for performance and accuracy. If you specify too large a value, it can lead to unpredictable results.

The default value of `-deltav` is 10% of the largest `vdd` value in the design. You must set an absolute voltage value to see the option become effective. It represents the absolute voltage change threshold, forcing the engine to reevaluate the interface element. To assess the benefits of this option, it is recommended to apply it in an additional `ie_activity_rpt` in the `vcsAD.init` mixed-signal simulation control file to better estimate the outcome by looking at the total reported number of events.

Examples

In the following example there are two inverters, with one in a Verilog description and the other in SPICE.

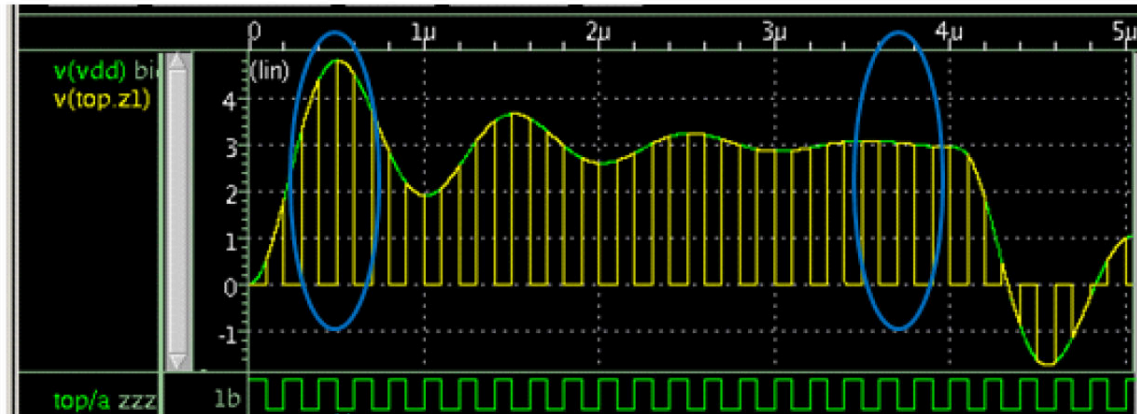
Figure 28 Digital Inverter Driving an Analog Inverter



```
//vcsAD.init  
use_spice -cell inv ;
```

```
choose xa 2inv_test.spi -c xa.cfg -o test;  
d2a -hiv 100% -lov 0% -node top.z1 -vdd vdd;  
ie_activity_rpt enable -flush 1u;  
dynamic_supply_filter -deltav 0.1;
```

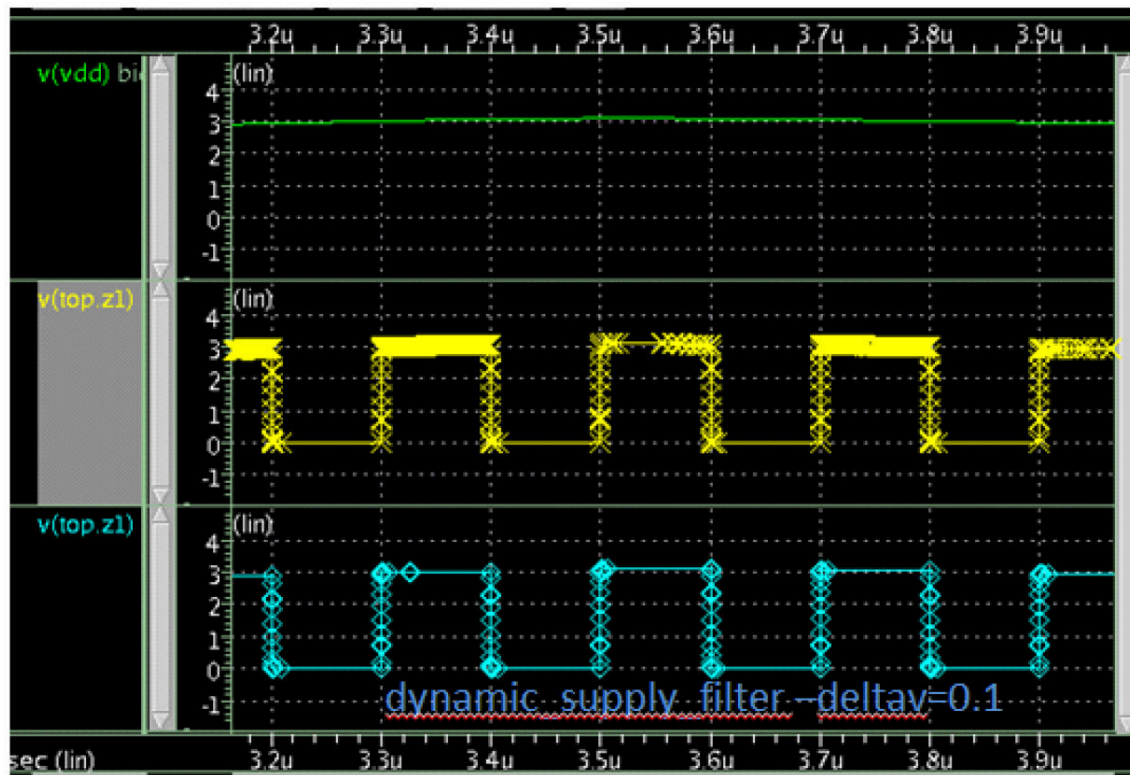
The expected result is:



The next two examples show that you can observe the different number of time points with and without the `dynamic_supply_filter` command.

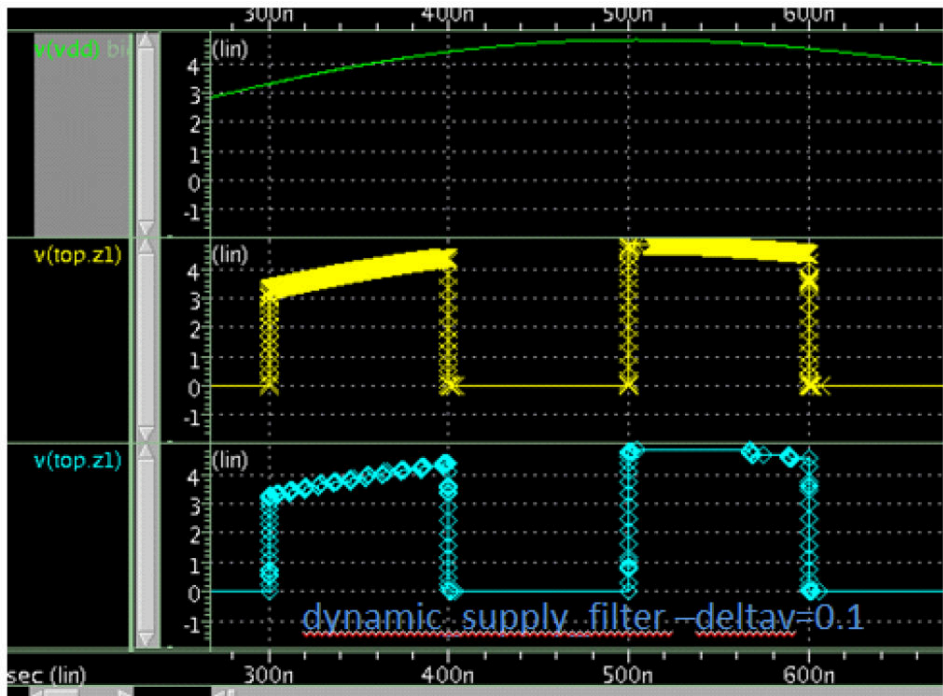
In [Figure 29](#), the power supply is stable and very few events are generated when the input is at a steady state.

Figure 29 Stable VDD Power Supply



In [Figure 30](#), the power supply is continuously varying and many events are generated.

Figure 30 Continuously Varying VDD Power Supply



The `simv.msv/interface_activity.rpt` file reports the event count for nodes that are connected to interface elements in the mixed-signal simulation. The following abbreviated example is a comparison of two interface activity reports. The report on the left shows the interface element activity with no supply filtering; the report on the right is generated with `dynamic_supply_filter -deltav 0.1` and shows fewer simulation events and potentially better simulation performance.

Example 45 Default Activity Report and Report With `dynamic_supply_filter`

# interface_activity.rpt	# interface_activity.rpt
# No dynamic_supply_filter	# dynamic_supply_filter -deltav 0.1;
#	#
top.x1.a	top.x1.a
d2a 132762	d2a 3624
top.x3.a	top.x3.a
d2a 132113	d2a 2967
top.x1.z	top.x1.z
a2d 351	a2d 343
Summary:	Summary:
d2a events (incl HighZ) : 264875	d2a events (incl HighZ) : 6591
HighZ d2a events : 0	HighZ d2a events : 0

a2d events (incl HighZ) :	351		a2d events (incl HighZ) :	343
HighZ a2d events	: 0		HighZ a2d events	: 0
Total events	: 265226		Total events	: 6934
for 5 interface signals			for 5 interface signals	

e2n for current_r Nettype

Description

Controls the behavior of an e2n interface element for the `current_r` nettype. For more information about nettypes, see [Support for Nettypes in Mixed-Signal Simulation](#).

Syntax

```
e2n [-node hierarchical_node_name]
    [-min_delta value]
    [-max_delta value]
    [-gain value]
    [-res value]
    [-res_node node_name]
    [-library library_name]
    [-cell cell_name -port port_name] |
    [-inst inst_name -port port_name] | [-node hier_name]
    [-ceff value]
    [-type nettype_name]
```

Arguments

`-node hierarchical_node_name`

Specifies the hierarchical path to the e2n interface. You can specify wildcard characters (*) in the path.

`-min_delta value`

Specifies the absolute current threshold value for e2n events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default value is 1% of the maximum value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-max_delta value`

Sets the current difference between two e2n events to be less than or equal to the specified value, which must be greater than 0. This option disables the `min_delta` option. Note that if you specify a very small `max_delta` value, it can slow down a simulation. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-gain value`

Multiplies the analog value by the specified *value*. The *value* can be a negative number. The default value is 1.

`-res value`

Specifies resistance. The default is 1000 Ohms.

`-res_node node_name`

Specify a path to a SPICE node name to connect the resistor. This argument is typically used to mimic a pull-up resistor. By default, the resistor is internally connected to ground.

`-library library_name`

Specifies a *library_name* to apply global e2n threshold options. Use this option to specify unique threshold options for all cells in the specified library. For more information, see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies the cell and port names for connecting the e2n interface element. The *cell_name* must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port for connecting the e2n interface element. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name for connecting the e2n interface element. You can specify the asterisk (*) wildcard character in a node name.

`-ceff value`

Specifies a capacitance value to model the loading effect of the digital blocks driven by the analog interface net. By default, there is no extra capacitance inserted at the e2n interface net. The unit of the *value* is Farad.

`-type nettype_name`

Applies e2n (or n2e) command arguments to a specified nettype. For example:

```
n2e -min_delta 1e-09 -type voltage_r -node *;  
e2n -min_delta 1e-09 -type voltage_r -node *;  
n2e -min_delta 1e-10 -node top.g1.g4.a;  
e2n -gain 100 -node top.g1.g5.y;
```

These commands set the `-min_delta` argument for `n2e` and `e2n` interface elements to 1e-09V for all nettypes of `voltage_r`. On line 3, the `n2e -min_delta` command specifies a value of 1e-10V for the `top.g1.g4.a` node and overrides the previous `-min_delta` value of 1e-09V.

Note:

The `-type nettype_name` argument limits the scope of the `e2n` (or `n2e`) command only; it does not change how the `e2n` (or `n2e`) interface element is modeled.

e2n for i_wire Nettype

Description

Controls the behavior of an `e2n` interface element for the `i_wire` nettype. For more information about nettypes, see [Support for Nettypes in Mixed-Signal Simulation](#).

Syntax

```
e2n [-node hierarchical_node_name]
    [-min_delta value]
    [-max_delta value]
    [-gain value]
    [-res value] [-res_node node_name]
    [-library library_name]
    [-cell cell_name -port port_name] |
    [-inst inst_name -port port_name] | [-node hier_name]
    [-ceff value]
    [-type nettype_name]
```

Arguments

`-node hierarchical_node_name`

Specifies the hierarchical path to the `e2n` interface. You can specify wildcard characters (*) in the path.

`-min_delta value`

Specifies the absolute voltage threshold value for `e2n` events. The `value` field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum value. For more information about this option, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-max_delta value`

Sets the voltage difference between two `e2n` events to be less than or equal to the specified value, which must be greater than 0. This option disables the

`-min_delta` option. Note that if you specify a very small `max_delta` value it can slow down a simulation. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-gain value`

Multiplies the analog value by the specified `value`. The `value` can be a negative number. The default value is 1.

`-res value`

Specifies resistance. The default is 1000 Ohms.

`-res_node node_name`

Specify a path to a SPICE node name to connect the resistor. This argument is typically used to mimic a pull-up resistor. By default, the resistor is internally connected to ground.

`-library library_name`

Specifies a `library_name` to apply global e2n threshold options. Use this option to specify unique threshold options for all cells in the specified library. For more information, see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies the cell and port names for connecting the e2n interface element. The `cell_name` must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port for connecting the e2n interface element. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name for connecting the e2n interface element. You can specify the asterisk (*) wildcard character in a node name.

`-ceff value`

Specifies a capacitance value to model the loading effect of the digital blocks driven by the analog interface net. By default, there is no extra capacitance inserted at the e2n interface net. You can use this option to specify a loading capacitance to be inserted at the e2n interface net or nets. The unit of the value is Farad.

`-type nettype_name`

Applies `e2n` (or `n2e`) command arguments to a specified nettype. For example:

```
n2e -min_delta 1e-09 -type voltage_r -node *;  
e2n -min_delta 1e-09 -type voltage_r -node *;  
n2e -min_delta 1e-10 -node top.g1.g4.a;  
e2n -gain 100 -node top.g1.g5.y;
```

These commands set the `-min_delta` argument for `n2e` and `e2n` interface elements to 1e-09V for all `n2e` and `e2n` nettypes of `voltage_r`. On line 3, the `n2e -min_delta` command specifies a value of 1e-10V for the `top.g1.g4.a` node and overrides the previous `-min_delta` value of 1e-09V.

Note:

The `-type nettype_name` argument limits the scope of the `e2n` (or `n2e`) command only; it does not change how the `e2n` (or `n2e`) interface element is modeled.

e2n for th_wire Nettype

Description

Controls the behavior of an `e2n` interface element for the `th_wire` nettype. For more information about nettypes, see [Support for Nettypes in Mixed-Signal Simulation](#).

Syntax

```
e2n [-node hierarchical_node_name]  
    [-min_delta value]  
    [-max_delta value]  
    [-library library_name]  
    [-cell cell_name -port port_name] |  
    [-inst inst_name -port port_name] | [-node hier_name]  
    [-ceff value]  
    [-type nettype_name]
```

Arguments

`-node hierarchical_node_name`

Specifies the hierarchical path to the `e2n` interface. You can specify wildcard characters (*) in the path.

`-min_delta value`

Specifies the absolute voltage threshold value for `e2n` events. The `value` field is specified in engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1%

of the maximum value. For more information about this option, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-max_delta value`

Sets the voltage difference between two e2n events to be less than or equal to the specified value, which must be greater than 0. This option disables the `-min_delta` option. Note that if you specify a very small `-max_delta` value, it can slow down a simulation. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-library library_name`

Specifies a `library_name` to apply global e2n threshold options. Use this option to specify unique threshold options for all cells in the specified library. For more information see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies the cell and port names for connecting the e2n interface element. The `cell_name` must be the same as the name used in the `use_spice`, `use_verilog`, or `use_vhdl` commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port for connecting the e2n interface element. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name for connecting the e2n interface element. You can specify the asterisk (*) wildcard character in a node name.

`-ceff value`

Specifies a capacitance value to model the loading effect of the digital blocks driven by the analog interface net. By default, there is no extra capacitance inserted at the e2n interface net. You can use this option to specify a loading capacitance to be inserted at the e2n interface net or nets. The unit of the value is Farad.

`-type nettype_name`

Applies e2n (or n2e) command arguments to a specified nettype. For example:

```
n2e -min_delta 1e-09 -type voltage_r -node *;
e2n -min_delta 1e-09 -type voltage_r -node *;
n2e -min_delta 1e-10 -node top.g1.g4.a;
e2n -gain 100 -node top.g1.g5.y;
```

These commands set the `-min_delta` argument for `n2e` and `e2n` interface elements to 1e-09V for all nettypes of `voltage_r`. On line 3, the `n2e -min_delta` command specifies a value of 1e-10V for the `top.g1.g4.a` node and overrides the previous `-min_delta` value of 1e-09V.

Note:

The `-type nettype_name` argument limits the scope of the `e2n` (or `n2e`) command only; it does not change how the `e2n` (or `n2e`) interface element is modeled.

e2n for voltage_r Nettype

Description

Controls the behavior of an `e2n` interface element for the `voltage_r` nettype. For more information about nettypes, see [Support for Nettypes in Mixed-Signal Simulation](#).

Note:

The `e2n` interface element for this nettype produces a hiZ value of `realZ` on the digital side if the simulator estimates that the output resistance seen on the analog side of the `e2n` is greater than 10G Ohms.

Syntax

```
e2n [-node hierarchical_node_name]
    [-min_delta value]
    [-max_delta value]
    [-drive_strength 0 | 1]
    [-library library_name]
    [-cell cell_name -port port_name] |
    [-inst inst_name -port port_name] | [-node hier_name]
    [-ceff value]
    [-type nettype_name]
    [-r_off_th value]
```

Arguments

`-node hierarchical_node_name`

Specifies the hierarchical path to the `e2n` interface. You can specify wildcard characters (*) in the path.

`-min_delta value`

Specifies the absolute voltage threshold value for `e2n` events. The value field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The

default is 1% of the maximum value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-max_delta value`

Sets the voltage difference between two e2n events to be less than or equal to the specified value, which must be greater than 0. This option disables the `-min_delta` option. Note that if you specify a very small `-max_delta` value it can slow down a simulation. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-drive_strength 0 | 1`

Enables the e2n drive strength calculation. For each e2n event, the analog engine calculates the analog output resistance. When the calculated resistance is above the value of the `-r_off_th` argument, the net is considered to be in a highZ state and the interface element passes highZ to the VCS tool. For bidirectional nets, the default is 1, which means the calculation is on. For unidirectional nets, the default is 0 and the calculation is off.

`-library library_name`

Specifies a *library_name* to apply global e2n threshold options. Use this option to specify unique threshold options for all cells in the specified library. For more information see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies the cell and port names for connecting the e2n interface element. The *cell_name* must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port for connecting the e2n interface element. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name for connecting the e2n interface element. You can specify the asterisk (*) wildcard character in a node name.

`-ceff value`

Specifies a capacitance value to model the loading effect of the digital blocks driven by the analog interface net. By default, there is no extra capacitance inserted at the e2n interface net. You can use this option to specify a loading capacitance to be inserted at the e2n interface net or nets. The unit of the value is Farad.

Appendix A: Mixed-Signal Control Commands

e2n for v_wire_avg, v_wire_sum, and v_wire_one Nettypes

`-type nettype_name`

Applies e2n (or n2e) command arguments to a specified nettype. For example:

```
n2e -min_delta 1e-09 -type voltage_r -node *;
e2n -min_delta 1e-09 -type voltage_r -node *;
n2e -min_delta 1e-10 -node top.g1.g4.a;
e2n -gain 100 -node top.g1.g5.y;
```

These commands apply the n2e and e2n -min_delta argument value of 1e-09 for all n2e and e2n nettypes of voltage_r. Note that a specific n2e -min_delta value of 1e-10 applies to the top.g1.g4.a node and overrides the previous -min_delta value of 1e-09 value for a single n2e command.

Note:

The -type nettype_name argument limits the scope of the e2n (or n2e) command only; it does not change how the e2n (or n2e) interface element is modeled.

`-r_off_th value`

Sets the threshold value for the impedance calculation. When the impedance of the analog net is above this value and drive strength calculation is enabled, the interface element passes hiZ to the VCS tool. The default is 100KOhms.

e2n for v_wire_avg, v_wire_sum, and v_wire_one Nettypes

Description

Controls the behavior of an e2n interface element for the v_wire_avg, v_wire_sum, and v_wire_one nettypes. For more information about nettypes, see [Support for Nettypes in Mixed-Signal Simulation](#).

Syntax

```
e2n [-node hierarchical_node_name]
    [-min_delta value]
    [-max_delta value]
    [-drive_strength 0 | 1]
    [-library library_name]
    [-cell cell_name -port port_name] |
    [-inst inst_name -port port_name] | [-node hier_name]
    [-type nettype_name]
    [-r_off_th value]
```

Arguments

`-node hierarchical_node_name`

Specifies the hierarchical path to the e2n interface. You can specify wildcard characters (*) in the path.

`-min_delta value`

Specifies the absolute voltage threshold value for e2n events. The value field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-max_delta value`

Sets the voltage difference between two e2n events to be less than or equal to the specified value, which must be greater than 0. This option disables the `min_delta` option. Note that if you specify a very small `max_delta` value it can slow down a simulation. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-drive_strength 0 | 1`

Enables the e2n drive strength calculation. For each e2n event, the analog engine calculates the analog output resistance. When the calculated resistance is above the value of the `-r_off_th` argument, then net is considered in a highZ state and the interface element passes highZ to the VCS tool. For bidirectional nets, the default is 1 and the calculation is on. For unidirectional nets, the default is 0 and the calculation is off.

`-library library_name`

Specifies a `library_name` to apply global e2n threshold options. Use this option to specify unique threshold options for all cells in the specified library. For more information see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies the cell and port names for connecting the e2n interface element. The `cell_name` must be the same as the name used in the `use_spice`, `use_verilog`, or `use_vhdl` commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port for connecting the e2n interface element. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name for connecting the e2n interface element. You can specify the asterisk (*) wildcard character in a node name.

`-type nettype_name`

Applies e2n (or n2e) command arguments to a specified nettype. For example:

```
n2e -min_delta 1e-09 -type voltage_r -node *;
e2n -min_delta 1e-09 -type voltage_r -node *;
n2e -min_delta 1e-10 -node top.g1.g4.a;
e2n -gain 100 -node top.g1.g5.y;
```

These commands set the n2e and e2n -min_delta argument value of 1e-09 for all n2e and e2n nettypes of voltage_r. Note that a specific n2e -min_delta value of 1e-10 applies to the top.g1.g4.a node and overrides the previous min_delta value of 1e-09 value for a single n2e command.

Note:

The `-type nettype_name` argument limits the scope of the e2n (or n2e) command only; it does not change how the e2n (or n2e) interface element is modeled.

`-r_off_th value`

Sets the threshold value for the impedance calculation. When the impedance of the analog net is above this value and drive strength calculation is enabled, the interface element passes hiZ to the VCS tool. The default is 100KOhms.

e2r

Description

Use the e2r command to control the behavior of an e2r interface element. This command sets the threshold at which e2r events occur and allows e2r to convert analog voltage values (default) or current values to digital.

Syntax

```
e2r [-type i]
    [-min_delta value]
    [-max_delta value]
    [-gain value]
    [-res value]
    [-res_node node_name]
    [-library library_name]
    [-cell cell_name -port port_name] |
    [-inst inst_name -port port_name] | [-node hier_name]
```

Arguments

`-type i`

Specifies whether the e2r interface element is converting voltage values to digital (default) or the current through a resistor (with `-type i`). When you specify `-type i`, the current direction follows the PrimeSim HSPICE rule, which is a positive value means the current is flowing into the subcircuit and a negative value means that the current is flowing out of the subcircuit. You can change the sign of the `gain` argument to change the current direction.

`-min_delta value`

Specifies the absolute current threshold value for e2r events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of possible maximum value of e2r, that stabilizes after several e2r conversions. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-max_delta value`

Sets the voltage difference between two e2r events to be less than or equal to the specified value, which must be greater than 0. The `-max_delta` option does not disable the `-min_delta` option. Therefore it is recommended to use both the arguments together for precise control over e2r filtering. For example, if you specify only `-max_delta`, the default `-min_delta` (1% of possible maximum value of e2r) is still applied. And if `-max_delta` is less than the default `-min_delta`, then the `-min_delta` takes precedence, and `-max_delta` is ignored. Note that if you specify a very small `-max_delta` value it can slow down a simulation. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-gain value`

Determines if the analog voltage/current needs to be multiplied by a multiplier when converted to digital. The multiplier could be a negative number as well. The default value is 1.

`-res value`

Specifies the value of the resistor that is placed between the interface net and ground to measure the current through it. This option is only applicable with `-type i`.

`-res_node node_name`

Specify a path to a SPICE node name to connect the resistor. This option provides a path to a SPICE node name, typically used to mimic a pull-up resistor. By default, the series resistor is internally connected to ground.

`-node hierarchical_node_name`

Specifies the hierarchical path to the e2r interface. You can specify wildcard characters (*) in the path.

`-library library_name`

You can specify a *library_name* to apply global e2r threshold options that interface to HDL cells that have been compiled into separate libraries and are used in the mixed-signal design configuration. For more information see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies the cell and port names for connecting the e2n interface element. The *cell_name* must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port for connecting the interface element. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name for connecting the interface element. You can specify the asterisk (*) wildcard character in a node name.

Examples

The following examples show different usages of the e2r interface element.

- The following example sets the e2r interface element to a current type at hierarchical path `top.i1.ictrl1`. The current through the 10 Ohm resistor, specified by the `-res 10` option, is converted to digital real values.

```
e2r -type i -node top.i1.ictrl1 -res 10;
```

- The following example sets the `-min_delta` for the e2r interface located at `test.pblk.clk` to 1mV.

```
e2r -min_delta 1e-3 -node top.pblk.clk;
```

e2u

Specifies the behavior of an e2u interface element. This interface element is used to connect a net from SPICE to a UPF supply net.

Syntax

```
e2u [-node hierarchical_node_name | -cell cell_name -port port_name]
    [-type power | pwr | ground | gnd]
    [-rth res_value]
    [-res_tth time_value]
    [-vth voltage_value1]
    [-vth_low voltage_value2]
    [-glitch_time time_value]
    [-min_delta value]
    [-max_delta value]
```

Arguments

`-node hierarchical_node_name`

Specifies the hierarchical node name. You can specify the asterisk (*) wildcard character in a node name.

`-cell cell_name -port port_name`

Specifies the cell and port names for the interface element. The `cell_name` must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-type power | pwr | ground | gnd`

Specifies whether the e2u interface element is driving a power supply net (`-type power | pwr`) or a ground net (`-type ground | gnd`). The default is power.

`-rth res_value`

Specifies the output resistance of the SPICE supply driver connected to the e2u input. If this resistance is greater than `res_value`, the interface element outputs an OFF value. Otherwise, FULL_ON is output.

`-res_tth time_value]`

Specifies how much time must pass after the output impedance crosses the output resistance threshold, `rth`, before switching the interface element output from FULL_ON to OFF or from OFF to FULL_ON. The default is 0.

`-vth voltage_value1`

If `-type power | pwr`, specifies the threshold voltage. If the input voltage to the e2u interface element is less than `voltage_value1`, the interface element outputs an OFF value. Otherwise, FULL_ON is output.

If `-type ground | gnd`, specifies the upper boundary of a range with `-vth_low voltage_value2`. When the input voltage is within the specified range, the interface element outputs FULL_ON. Otherwise, it outputs an OFF value.

`-vth_low voltage_value2`

If `-type power | pwr`, this argument is ignored.

If `-type ground | gnd`, specifies the lower boundary of a range with `-vth voltage_value1`. When the input voltage is within the specified range, the interface element outputs FULL_ON. Otherwise, it outputs an OFF value. The default is $-\infty$.

`-glitch_time time_value`

Specifies how much time must pass after the input voltage crosses the threshold voltage (`-vth`, if `-type power | pwr`) or crosses a boundary of the voltage range (`-vth` and `-vth_low`, if `-type ground | gnd`) before switching the interface element output from FULL_ON to OFF or from OFF to FULL_ON. The default is 0.

`-min_delta value`

Specifies the absolute voltage threshold value for e2u events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-max_delta value`

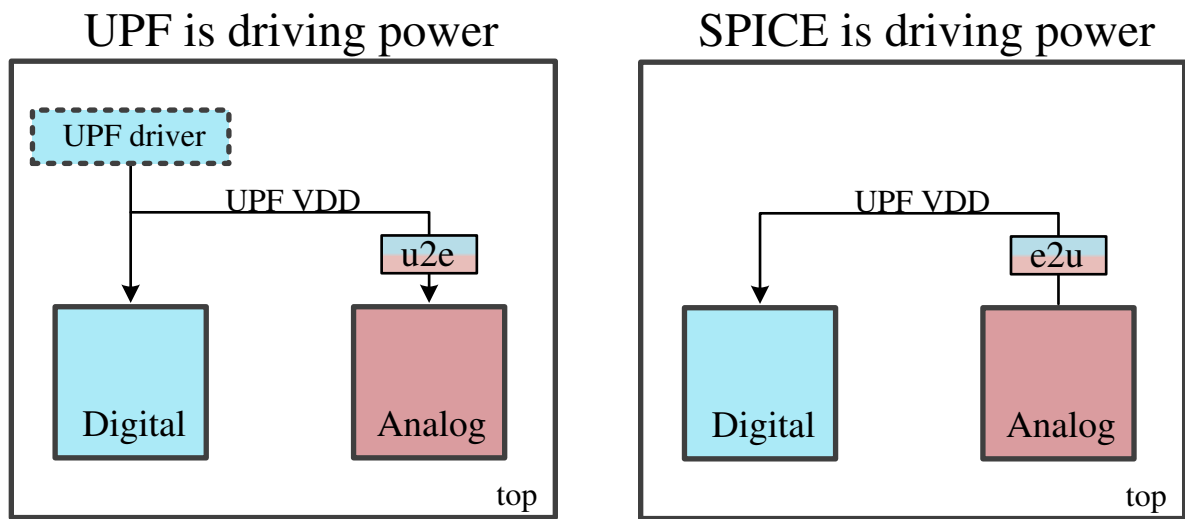
Sets the voltage difference between two e2u events to be less than or equal to the specified value, which must be greater than 0. This option disables the `-min_delta` option. Note that if you specify a very small `-max_delta` value, it can slow down a simulation. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

Description

The e2u interface element is used to convert a SPICE power output signal to a UPF supply net value. Use the e2u command to configure and change the default behavior for an e2u interface element placed at the SPICE-to-UPF interface. Use options to the command to define the threshold voltage or output resistance to determine whether a FULL_ON or OFF state is passed to UPF.

In the following figure, an e2u interface element is used to convert the VDD SPICE power output signal to a UPF supply net value for the digital block.

Figure 31 Mixed-Signal Design With UPF



Examples

The following examples use the `-rth` and `-vth` options to control the behavior of the `e2u` interface element:

- Specify a threshold voltage of 0.9V:

```
e2u -node Top.I0.I0.vdd_high -vth 0.9
```

In this example, if the voltage of node `Top.I0.I0.vdd_high` is less than 0.9, the interface element outputs an OFF value. Otherwise, FULL_ON is output.

- Specify a threshold voltage of 0.9V and an output resistance of 10:

```
e2u -node Top.I0.I0.vdd_high -rth 1k;
```

If the resistance at node `Top.I0.I0.vdd_high` is greater than 1k, the interface element outputs an OFF value. Otherwise, FULL_ON is output.

form_spice_bus

Description

Enables or disables the use of Verilog language XMR or UCLI `force` statements to be developed by using a vector format. This command is used for designs that contain Verilog language XMR, `$hdl_xmr`, `$hdl_xmr_force`, or UCLI `force` commands on SPICE ports that appear as a bus in the digital view of the target cell. By default, `form_spice_bus` is `-enable`.

If there are errors for designs that are not using XMR or UCLI `force` statements, you can disable this command. If disabled, XMR or UCLI `force` statements cannot be used in a vector format.

Syntax

```
form_spice_bus -enable | -disable;
```

Arguments

`-enable`

Enables (default) the `form_spice_bus` command.

`-disable`

Disables the `form_spice_bus` command.

Examples

The following examples show the usage of the `form_spice_bus` command:

- The following digital testbench uses the Verilog language XMR feature on a SPICE port that appears as a bus. The `form_spice_bus` command allows the SPICE vector `out` to be treated as a Verilog vector, which allows the parts of the Verilog code that make vector-to-vector assignments to remain unchanged in mixed-signal simulation.

dut.spi

```
.subckt my_design in<2> in<1> in<0> out<2> out<1> out<0>
...
.ends
```

testbench.v

```
module top;
  wire [-2:0] read1, read2, read3;
  ...
  my_design dut (.in(x[-2:0]), .out(z[-2:0]));
  assign read1 = top.dut.out;
  assign read2[-1:0] = top.dut.out[-1:0];
  assign read3 = {top.dut.out[-2], top.dut.out[-1],
top.dut.out[-0]};
endmodule
```

- In the following example, the digital testbench uses `$hdl_xmr_force` on SPICE ports (`top.dut.out<2>`, `top.dut.out<1>`, `top.dut.out<0>`), which appears as a bus.

testbench.v

```
module top;
  ...
  my_design dut (.in(x[-2:0]), .out(z[-2:0]));
  initial begin
    #20 $hdl_xmr_force ("top.dut.out", "101", "0ns", , , 0);
    #20 $hdl_xmr_release ("top.dut.out", 0);
  end
```

```
        end
    endmodule
    module my_design (in, out);
        input [-2:0] in;
        output [-2:0] out;
        ...
    endmodule

    dut.spi
    .subckt my_design in<2> in<1> in<0> out<2> out<1> out<0>
    ...
    .ends
```

In this case, you must use `form_spice_bus -enable;` in the mixed-signal simulation control file to enable the force on the SPICE bus. The command is disabled by default.

gen_spice_wrapper

Helps the tool to read in the list of ports in the Verilog or VHDL descriptions and generate the wrapper with the port definitions.

Syntax

```
gen_spice_wrapper -cell cell_list -add_port port_list;
```

Arguments

`-cell cell_list`

Specifies a list of the Verilog or VHDL cells that are instantiated in a structural Verilog netlist and require a wrapper. The cell names are separated by a space. These cells must also appear either in the `use_verilog` or `use_vhdl` commands.

`-add_port port_list`

Specifies a list of the extra ports you want to add in the SPICE wrapper.

Description

When a design uses structural Verilog for its SPICE blocks, structural Verilog is read in using the `load_verilog_file` PrimeSim XA command and is assumed to be a SPICE view in a VCS PrimeSim AMS simulation. However, when you have VHDL or Verilog cells inside the structural Verilog, the VCS PrimeSim AMS tool is not able to find the ports defined in the VHDL or Verilog descriptions.

You only need to use this command if your design meets all the following conditions:

- You are using a structural Verilog netlist as a SPICE view (using `load_verilog_file` in the PrimeSim XA configuration file).
- You are using a VHDL or Verilog view for one of the instances in the structural Verilog.
- Your design references ports by name, not by position, in the structural Verilog for the blocks specified with the `use_vhdl` or `use_verilog` commands.

Port reference by name is:

```
INV U0 ( .A(A[0]) , .Z(Z[0]) );
```

Port reference by position is:

```
INV U0 ( A[0] , Z[0] );
```

- There is no SPICE `.subckt` description for the Verilog or the VHDL blocks instantiated in the structural Verilog.

In order for the `gen_spice_wrapper` command to execute, there must be a VHDL or Verilog description for the blocks specified in the `use_verilog` or `use_vhdl` command. The `gen_spice_wrapper` command reads in the VHDL or the Verilog description of the block and generates a wrapper that gives the analog simulator the information it needs about the ports.

When there are buses, the buses are separated in single bit signals by using the Verilog or VHDL bus notation for each bit. When there is a mismatch between the port names used in the Verilog module (or VHDL model) and the structural Verilog instance port names, an error message is issued.

Examples

The following examples show different use cases for the `gen_spice_wrapper` command:

- Generate a SPICE wrapper for the Verilog INV cell and the VHDL ND2 cell. The tool reads the `inv.v` file to build the wrapper for INV and reads the `nd2.vhd` file to build the wrapper for ND2.

vcsAD.init

```
use_verilog INV;
use_vhdl ND2;
gen_spice_wrapper -cell INV ND2;
```

top.v

```
module top();
...
INV U0 ( .A(A[0]) , .Z(Z[0]) );
```

Appendix A: Mixed-Signal Control Commands

gen_spice_wrapper

```
ND2 U1 ( .A(A[1]) , .B(A[2]), .Z(Z[1]) );
endmodule
```

Generated SPICE wrapper

```
.subckt INV A Z
.ends
.subckt ND2 A B Z
.ends
```

- Generate a SPICE wrapper for the Verilog INV cell and the VHDL BUF cell. The tool reads the `inv.v` file to build the wrapper for INV and reads the `buf.vhd` file to build the wrapper for ND2.

vcsAD.init

```
use_verilog INV;
use_vhdl BUF;
gen_spice_wrapper -cell INV;
gen_spice_wrapper -cell BUF;
```

top.v

```
module top()
...
INV U0 ( .A(A[0]) , .Z(Z[0]) , .VSS (VSS), .VDD(VDD));
BUF U1 ( .A(A[1]) , .Z(Z[1]) );
endmodule
```

Generated SPICE wrapper

```
.subckt INV A Z VSS VDD
.ends
.subckt BUF A Z
.ends
```

- Generate a SPICE wrapper for the Verilog MUX cell. Note that the MUX cell instance contains VDD and VSS ports; these are generated in the SPICE wrapper by specifying `-add_port VDD VSS`.

vcsAD.init

```
use_verilog MUX;
gen_spice_wrapper -cell MUX -add_port VDD VSS;
```

top.v

```
module top ();
...
MUX U0 ( .A(IN) , .Z(OUT), .VDD(inh_sup), .VSS(inl_sup) );
...
endmodule
```

```

module MUX (A, Z);
input [5:0] A;
output Z;
...
endmodule

```

Generated SPICE wrapper

```

.subckt MUX A[5] A[4] A[3] A[2] A[1] A[0] Z VDD VSS
...
.ends

```

ie_activity_rpt

Enables outputting either the interface element (IE) activity statistics for the Verilog-SPICE and VHDL/Verilog-SPICE flows or the connect module (CM) activity statistics for the Verilog-AMS flow into the report file, `simv.msv/interface_activity.rpt`.

Syntax

```

ie_activity_rpt -enable | -disable
[-flush flush_percentage | flush_time];

```

Arguments

`-enable` | `-disable`

Enables or disables outputting of the `interface_activity.rpt` file to the `simv.msv` directory.

`-flush flush_percentage`

Specifies the simulation time interval at which the report file gets updated. The interval is given as a percentage of the end time specified in the `.tran` statement. By default, the interval is 10% of the `.tran` end time. If there is no `.tran` statement in SPICE, this option is ignored.

`-flush flush_time`

Specifies the absolute simulation time intervals at which the report file gets updated.

Description

By default, the tool updates the `interface_activity.rpt` file at time intervals equal to 10% of the simulation time specified by the `.tran` statement.

This file is generated only if the interface activity report is enabled with the `ie_activity_rpt` command. The report file has an entry for each interface element (a2d, d2a, e2r, or r2e) and displays how many events have occurred for each interface.

Examples

- The following example writes out the interface activity report to the `simv.msv/interface_activity.rpt` file.

```
ie_activity_rpt -enable;
```
- The following example writes out the interface activity report and sets the update interval for the file to 2ns.

```
ie_activity_rpt -enable -flush 2ns;
```
- The following example writes out the interface activity report and sets the update interval for the file to 5% of the `.tran` end time.

```
ie_activity_rpt -enable -flush 5%;
```

ie_connect_rpt

Description

Enables outputting the interface element connectivity information into the report file, `simv.msv/interface_connectivity.rpt`. This command is disabled by default.

Syntax

```
ie_connect_rpt -enable | -disable -msg_count value;
```

Arguments

`-enable | -disable`

Enables or disables output of the `interface_connectivity.rpt` file to the `simv.msv` directory. Because the `ie_connect_rpt` command can potentially impact performance and memory usage, `-disable` is the default.

`-msg_count value`

Controls the count of messages dumped in both digital drivers and analog port connections. By default, both analog connections and digital drivers are dumped into the report. Default is 20.

Examples

```
ie_connect_rpt -enable -msg_count 99
```

This command enables the count of messages, for both analog connections and digital drivers, dumped with a maximum message count of 99.

```
ie_connect_rpt -enable -msg_count 99 -type digital
```

With the `-type digital` the report only has digital drivers checked and dumped with a maximum message count of 99.

ie_reference_voltage

Description

Defines a reference node that can assist automatic supply tracing to set the appropriate thresholds or levels of a2d or d2a interface elements. See [Specifying a Voltage Reference for Interface Elements](#) for more information.

Syntax

```
ie_reference_voltage -node node_name [-voltage value];
```

or

```
ie_reference_voltage -skip_node skip_node_name;
```

Arguments

`-node node_name`

Specifies the reference to a hierarchical node name inside SPICE.

`-voltage value`

Specifies a voltage value used to calculate the thresholds and levels of interface elements whose SPICE side can trace supply to the specified reference node. If this argument is not set, the reference is dynamic.

`-skip_node skip_node_name`

Specifies the node that is skipped from supply tracing.

Note:

This argument cannot be used with the other arguments of this command.

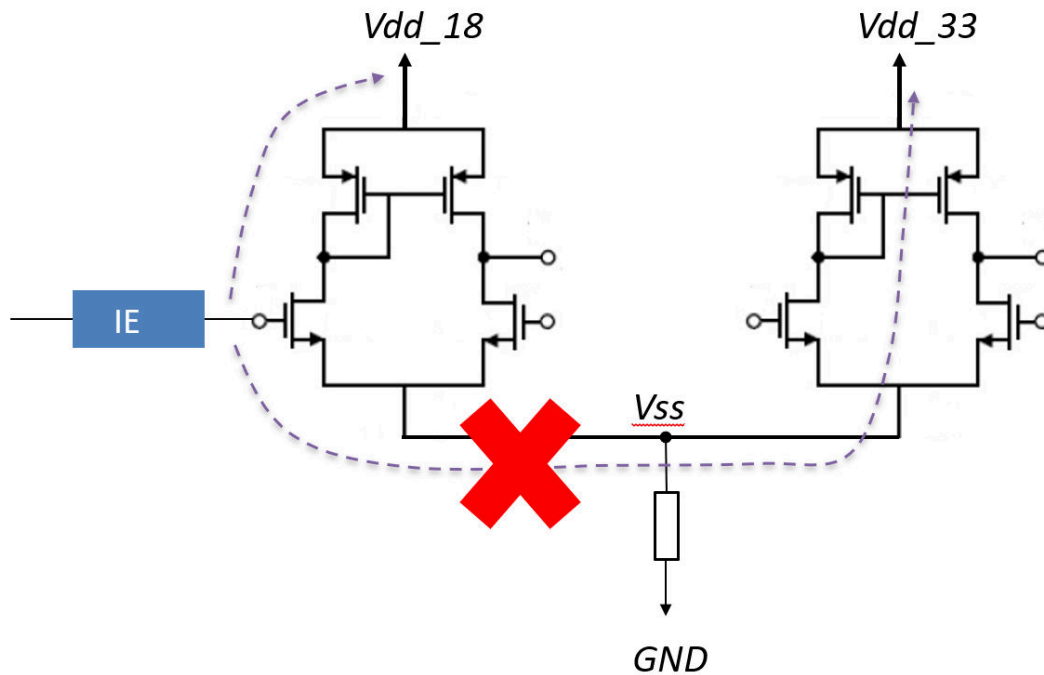
Examples

Automatic supply tracing may not work correctly if the circuit has a floating ground node, that is, a node that is not directly connected to true ground. In the example shown in [Figure 32](#), the floating ground node `vss` may create a false path for tracing `vdd_33` to the interface element (IE). To eliminate such a path, you can skip `vss` from supply tracing by setting the following:

```
ie_reference_voltage -skip_node vss;
```

This results in `vdd_18` being the only path for supply tracing to the interface element.

Figure 32 *ie_reference_voltage Example*



ie_tracing_rpt

Description

Enables outputting the report file, `simv.msv/interface_tracing.rpt`. This file helps you check whether your user-defined `vdd` nodes are correct. If a user-defined `vdd` differs from the supply reached by the tracing algorithm, the report includes a warning message.

Syntax

```
ie_tracing_rpt -enable | -disable -refLov val;
```

Arguments

`-enable | -disable`

Enables or disables output of the `interface_tracing.rpt` file to the `simv.msv` directory.

`-refLov val`

Used for estimating `-hiv` when you set constant `-hith` and `-loth` (Estimated `hiv=hith+loth-refLov`). The default for `-refLov` is 0.0.

Examples

The following examples demonstrate the use of the `ie_tracing_rpt` command:

Example 1

In this example, the user-defined `vdd` (`vdd20`) matches the traced `vdd` (`vdd20`).

Set the following commands in the `vcsAD.init` file:

```
ie_tracing_rpt -enable  
d2a -node net1 -vdd vdd20 -vss vss00 -hiv 90% -lov 10%
```

The resulting `interface_tracing.rpt` file includes the following:

```
d2a -vdd vdd20 -vss vss00 -hiv 90% -lov 10% -node net1  
Traced -vdd vdd20
```

Example 2

In this example, the user-defined `-vdd` (`vdd18`) does not match the traced `-vdd` (`vdd20`).

Set the following commands in the `vcsAD.init` file:

```
ie_tracing_rpt -enable  
d2a -node net1 -vdd vdd18 -vss vss00 -hiv 90% -lov 10%
```

The resulting `interface_tracing.rpt` file includes the following:

```
d2a -vdd vdd18 -vss vss00 -hiv 90% -lov 10% -node net1  
Traced -vdd vdd20 <- mismatch!
```

Example 3

If you set a static interface element (constant `-hiv` and `-hith`), the tool checks only the results of the tracing algorithm against `-hith` because the simulator does not run the tracing for those interface nodes that have constant `-hiv` (in fact, the user-defined constant `-hiv` has higher priority than the supply tracing).

In this example, the estimated `-hiv` (1.5) does not match the traced `-hiv` (2.0).

Set the following commands in the `vcsAD.init` file:

```
ie_tracing_rpt -enable -refLov 0.0  
a2d -node net1 -hith 1.0 -loth 0.5
```

The resulting `interface_tracing.rpt` file includes the following:

```
a2d -hith 1.0 -loth 0.5 -node net1  
Traced -vdd vdd20 -hiv 2 <- mismatch!  
Estimated -hiv 1.5
```

Example 4

In this example, the estimated `p-hiv (2.0)` matches the traced `-hiv (2.0)`.

Set the following commands in the `vcsAD.init` file:

```
ie_tracing_rpt -enable -refLow 0.0
a2d -node net1 -hith 1.5 -loth 0.5
```

The resulting `interface_tracing.rpt` file includes the following:

```
a2d -hith 1.5 -loth 0.5 -node net1
Traced -vdd vdd20 -hiv 2
Estimated -hiv 2.0
```

insert_cell

Description

Inserts a 2-port SPICE netlist on the analog side of an interface net. This can be used to convert a voltage into a current.

Syntax

```
insert_cell
  [-param parameter_list]
  [-subckt subcircuit_name -apin port_name -dpin port_name]
  [-cell cell_name -port port_name | -node hier_name]
  subckt subcircuit_name -apin port_name -dpin port_name
```

Arguments

`-subckt subcircuit_name -apin port_name -dpin port_name`

Identifies a two-port subcircuit to be inserted on the analog side of the interface and determines which port faces the analog side and which the digital side. The `-apin` and `-dpin` options are only needed if the two ports of the inserted cell are not called the default names `-apin` and `-dpin`.

`-cell cell_name, -port port_name, -node hier_name`

The command can be applied to cells and ports, or nodes.

- For cells, specify the cell name and port name.
- For nodes, specify the hierarchical path to the instance port.
- Cell and port names can contain the asterisk (*) wildcard character, for example,

```
-node top.i1.ad*,
-cell *foo, -port *.
```

Examples

- Insert a 2-port SPICE subcircuit named `multiplier` with pins `in` and `out` at the interface net `top.rst`:

```
// vcsAD.init
insert_cell -subckt multiplier -apin in -dpin out -node top.rst;

* multiplier subcircuit
.subckt multiplier in out ...
.ends
```

- Insert a 2-port SPICE subcircuit named `term` at the interface net `test.il.d_in`. Note that the default pin names, `-apin` and `p-dpin`, are used and the `-dpin` and `-apin` arguments are not required:

```
// vcsAD.init
insert_cell -subckt term -node top.il.d_in;

* term subcircuit
.subckt term apin dpin
...
.ends
```

map_by_node

Description

Specifies a resistance value for d2a conversion at an interface node, instead of the value calculated from the resistance map file. This command is only applied in the digital-to-analog direction.

Syntax

```
map_by_node -r resistance_value -node node_names;
```

Examples

In the following example, the 5 ohm resistance value is applied to the interface resistor connected to the `top.n1` node.

```
map_by_node -r 5 -node top.n1;
```

When multiple `map_by_node` commands are used for the same mixed-nets in the mixed-signal simulation control file, the last command takes precedence.

n2e for current_r Nettype

Description

Controls the behavior of an n2e interface element for the `current_r` nettype. For more information about nettypes, see [Support for Nettypes in Mixed-Signal Simulation](#).

Syntax

```
n2e [-node hierarchical_node_name]  
    [-min_delta value]  
    [-gain value]  
    [-rf_rate value]  
    [-res value]  
    [-x2i value]  
    [-library library_name]  
    [-cell cell_name -port port_name] |  
    [-inst inst_name -port port_name] | [-node hier_name]  
    [-type nettype_name]
```

Arguments

`-node hierarchical_node_name`

Specifies the hierarchical path to the n2e interface. You can specify wildcard characters (*) in the path.

`-min_delta value`

Specifies the absolute current threshold value for n2e events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-gain value`

Multiplies the output value by the specified *value*. The multiplier can be a negative number. The default value is 1.

`-rf_rate value`

Specifies how fast the voltage changes from the current value to the target value. Units are specified as second/ampere, with the default set to 10ps/A. You should avoid rapid changes in voltage values that might create unwanted current peaks.

`-res value`

Specifies parallel resistance. The default is 1100G Ohms.

`-x2i value`

Specifies the current source value when the interface element real value is set to `realX`.

`-library library_name`

Specifies a `library_name` to apply global `n2e` threshold options. Use this option to specify unique threshold options for all cells in the specified library. For more information see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies the cell and port names for the interface element. The `cell_name` must be the same as the name used in the `use_spice`, `use_verilog`, or `use_vhdl` commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name for the interface element. You can specify the asterisk (*) wildcard character in a node name.

`-type nettype_name`

Applies `n2e` (or `e2n`) command arguments to a specified nettype. For example:

```
n2e -min_delta 1e-09 -type voltage_r -node *;  
e2n -min_delta 1e-09 -type voltage_r -node *;  
n2e -min_delta 1e-10 -node top.g1.g4.a;  
e2n -gain 100 -node top.g1.g5.y;
```

These commands apply the `n2e` and `e2n -min_delta` argument value of `1e-09` for all `n2e` and `e2n` nettypes of `voltage_r`. Note that a specific `n2e -min_delta` value of `1e-10` applies to the `top.g1.g4.a` node and overrides the previous `-min_delta` value of `1e-09` value for a single `n2e` command.

Note:

The `-type nettype_name` argument limits the scope of the `n2e` (or `e2n`) command only; it does not change how the `n2e` (or `e2n`) interface element is modeled.

n2e for i_wire Nettype

Description

Controls the behavior of an n2e interface element for the `i_wire` nettype. For more information about nettypes, see [Support for Nettypes in Mixed-Signal Simulation](#).

Syntax

```
n2e [-node hierarchical_node_name]  
    [-min_delta value]  
    [-gain value]  
    [-rf_rate value]  
    [-res value]  
    [-x2i value]  
    [-library library_name]  
    [-cell cell_name -port port_name] |  
    [-inst inst_name -port port_name] | [-node hier_name]  
    [-type nettype_name]
```

Arguments

`-node hierarchical_node_name`

Specifies the hierarchical path to the n2e interface. You can specify wildcard characters (*) in the path.

`-min_delta value`

Specifies the absolute voltage threshold value for n2e events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-gain value`

Multiplies the analog value by the specified *value*. The value can be a negative number. The default value is 1.

`-rf_rate value`

Specifies how fast the voltage changes from the current value to the target value. Units are specified as second/volt, with the default set to 10ps/V. You should avoid rapid changes in voltage values that might create unwanted current peaks.

`-res value`

Specifies parallel resistance. The default is 1100G Ohms.

`-x2i value`

Specifies the current source value when the interface element real value is set to `realX`.

`-library library_name`

Specifies a `library_name` to apply global `n2e` threshold options. Use this option to specify unique threshold options for all cells in the specified library. For more information, see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies the cell and port names for connecting the interface element. The `cell_name` must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port to apply the voltage value to the ports referring to the target instance. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name. You can specify the asterisk (*) wildcard character in a node name.

`-type nettype_name`

Applies `n2e` (or `e2n`) command arguments to a specified nettype. For example:

```
n2e -min_delta 1e-09 -type voltage_r -node *;
e2n -min_delta 1e-09 -type voltage_r -node *;
n2e -min_delta 1e-10 -node top.g1.g4.a;
e2n -gain 100 -node top.g1.g5.y;
```

These commands apply the `n2e` and `e2n -min_delta` argument value of `1e-09` for all `n2e` and `e2n` nettypes of `voltage_r`. Note that a specific `n2e -min_delta` value of `1e-10` applies to the `top.g1.g4.a` node and overrides the previous `-min_delta` value of `1e-09` value for a single `n2e` command.

Note:

The `-type nettype_name` argument limits the scope of the `n2e` (or `e2n`) command only; it does not change how the `n2e` (or `e2n`) interface element is modeled.

n2e for th_wire Nettype

Description

Controls the behavior of an n2e interface element for the th_wire nettype. For more information about nettypes, see [Support for Nettypes in Mixed-Signal Simulation](#).

Syntax

```
n2e [-node hierarchical_node_name] [-min_delta value] [-rf_rate value]
    [-library library_name]
    [-cell cell_name -port port_name] |
    [-inst inst_name -port port_name] | [-node hier_name]
    [-type nettype_name]
```

Arguments

`-node hierarchical_node_name`

Specifies the hierarchical path to the n2e interface. You can specify wildcard characters (*) in the path.

`-min_delta value`

Specifies the absolute voltage threshold value for n2e events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-rf_rate value`

Specifies how fast the interface element internal voltage changes from the present value to the target value. Units are specified as second/volt, with the default set to 10ps/V. You should avoid rapid changes in voltage values that might create unwanted current peaks

`-library library_name`

Specifies a *library_name* to apply global n2e threshold options. For more information see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies the cell and port names for connecting the n2e interface element. The *cell_name* must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

```
-inst inst_name -port port_name
```

Specifies the hierarchical path to the instance port to apply the voltage value to the ports referring to the target instance. You can specify the asterisk (*) wildcard character in an instance name.

```
-node hier_name
```

Specifies the hierarchical node name. You can specify the asterisk (*) wildcard character in a node name.

```
-type nettype_name
```

Applies n2e (or e2n) command arguments to a specified nettype. For example:

```
n2e -min_delta 1e-09 -type voltage_r -node *;
e2n -min_delta 1e-09 -type voltage_r -node *;
n2e -min_delta 1e-10 -node top.g1.g4.a;
e2n -gain 100 -node top.g1.g5.y;
```

These commands apply the n2e and e2n -min_delta argument value of 1e-09 for all n2e and e2n nettypes of voltage_r. Note that a specific n2e -min_delta value of 1e-10 applies to the top.g1.g4.a node and overrides the previous -min_delta value of 1e-09 value for a single n2e command.

Note:

The -type nettype_name argument limits the scope of the n2e (or e2n) command only; it does not change how the n2e (or e2n) interface element is modeled.

n2e for voltage_r Nettype

Description

Controls the behavior of an n2e interface element for the voltage_r nettype. For more information about nettypes, see [Support for Nettypes in Mixed-Signal Simulation](#).

Syntax

```
n2e [-node hierarchical_node_name]
    [-min_delta value]
    [-gain value]
    [-rf_rate value]
    [-res value]
    [-r_off value]
    [-x2v value]
    [-powernet 0| 1]
    [-library library_name]
    [-cell cell_name -port port_name] |
```

```
[-inst inst_name -port port_name] | [-node hier_name]  
[-type nettype_name]
```

Arguments

`-node hierarchical_node_name`

Specifies the hierarchical path to the n2e interface. You can specify wildcard characters (*) in the path.

`-min_delta value`

Specifies the absolute voltage threshold value for n2e events. The *value* field is specified in engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-gain value`

Multiplies the analog value by the specified *value*. The multiplier can be a negative number. The default value is 1.

`-rf_rate value`

Specifies how fast the voltage changes from the present value to the target value. Units are specified as second/volt, with the default set to 10ps/V. You should avoid rapid changes in voltage values that might create unwanted current peaks.

`-res value`

Specifies series resistance. The default is 50.0Ohms.

`-r_off value`

Specifies the series resistance value when the interface element real value is set to `-hiz`. This should be a fairly high value to mimic a high impedance analog node. The default is 10G Ohm.

`-x2v value`

Specifies the voltage source value when the interface element real value is set to `realX`.

- If not specified, during DC, the interface element is 0.0V for a `realX` and during transient, the interface element holds its last value.
- If specified, during DC or transient, then interface element is forced to the specified value for a `realX`.

`-powernet 0 | 1`

Identifies an n2e node as an ideal voltage source on the analog side without the resistance value. You must use this option when Verilog drives analog power nets in order to remove the series resistance map resistors and to allow efficient partitioning of the analog circuit.

- 0 to not treat as an ideal voltage source (default).
- 1 to treat as an ideal voltage source.

`-library library_name`

Specifies a *library_name* to apply global n2e threshold options. Use this option to specify unique threshold options for all cells in the specified library. For more information see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies the cell and port names. The *cell_name* must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port to apply the voltage value to the ports referring to the target instance. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name. You can specify the asterisk (*) wildcard character in a node name.

`-type nettype_name`

Applies n2e (or e2n) command arguments to a specified nettype. For example:

```
n2e -min_delta 1e-09 -type voltage_r -node *;  
e2n -min_delta 1e-09 -type voltage_r -node *;  
n2e -min_delta 1e-10 -node top.g1.g4.a;  
e2n -gain 100 -node top.g1.g5.y;
```

These commands apply the n2e and e2n -min_delta argument value of 1e-09 for all n2e and e2n nettypes of voltage_r. Note that a specific n2e -min_delta value of 1e-10 applies to the top.g1.g4.a node and overrides the previous -min_delta value of 1e-09 value for a single n2e command.

Note:

The type *-nettype_name* argument limits the scope of the n2e (or e2n) command only; it does not change how the n2e (or e2n) interface element is modeled.

n2e for v_wire_avg, v_wire_sum, and v_wire_one Nettypes

Description

Controls the behavior of an n2e interface element for the v_wire_avg, v_wire_sum, and v_wire_one ntypes. For more information about ntypes, see [Support for Ntypes in Mixed-Signal Simulation](#).

Syntax

```
n2e [-node hierarchical_node_name]
    [-min_delta value]
    [-gain value]
    [-rf_rate value]
    [-res value]
    [-r_off value]
    [-x2v value]
    [-powernet 0 | 1]
    [-library library_name]
    [-cell cell_name -port port_name] |
    [-inst inst_name -port port_name] | [-node hier_name]
    [-type nettype_name]
```

Arguments

`-node hierarchical_node_name`

Specifies the hierarchical path to the n2e interface. You can specify wildcard characters (*) in the path.

`-min_delta value`

Specifies the absolute voltage threshold value for n2e events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-gain value`

Multiplies the analog value by the specified *value*. The *value* can be a negative number. The default value is 1.

`-rf_rate value`

Specifies how fast the voltage changes from the present value to the target value. Units are specified as second/volt, with the default set to 10ps/V. You should avoid rapid fast changes in voltage values that might create unwanted current peaks.

Appendix A: Mixed-Signal Control Commands

n2e for v_wire_avg, v_wire_sum, and v_wire_one Nettypes

`-res value`

Specifies series resistance. The default is 50.0 Ohms.

`-r_off value`

Specifies the series resistance value when the interface element real value is set to `-hiz`. This should be a fairly high value to mimic a high impedance analog node. The default is 10G Ohm.

`-x2v value`

Specifies the voltage source value when the interface element real value is set to `realX`.

- If not specified, during DC, the interface element is 0.0V for a `realX` and during transient, the interface element holds its last value.
- If specified, during DC or transient, then interface element is forced to the specified value for a `realX`.

`-powernet 0 | 1`

Identifies an n2e node as an ideal voltage source on the analog side without the resistance value. You must use this option when Verilog drives analog power nets in order to remove the series resistance map resistors and to allow efficient partitioning of the analog circuit.

- 0 to not treat as an ideal voltage source (default).
- 1 to treat as an ideal voltage source.

`-library library_name`

Specifies a `library_name` to apply global n2e threshold options. Use this option to specify unique threshold options for all cells in the specified library. For more information see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specify the cell and port names for connecting the n2e interface element. The `cell_name` must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port to apply the voltage value to the ports referring to the target instance. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name. You can specify the asterisk (*) wildcard character in a node name.

`-type nettype_name`

Applies `n2e` (or `e2n`) command arguments to a specified nettype. For example:

```
n2e -min_delta 1e-09 -type voltage_r -node *;  
e2n -min_delta 1e-09 -type voltage_r -node *;  
n2e -min_delta 1e-10 -node top.g1.g4.a;  
e2n -gain 100 -node top.g1.g5.y;
```

These commands apply the `n2e` and `e2n -min_delta` argument value of `1e-09` for all `n2e` and `e2n` nettypes of `voltage_r`. Note that a specific `n2e -min_delta` value of `1e-10` applies to the `top.g1.g4.a` node and overrides the previous `-min_delta` value of `1e-09` value for a single `n2e` command.

Note:

The `-type nettype_name` argument limits the scope of the `n2e` (or `e2n`) command only; it does not change how the `n2e` (or `e2n`) interface element is modeled.

netlist_commands_begin

Description

Use this command in the `vcsAD.init` mixed-signal simulation control file, along with the [netlist_commands_end](#) command, to pass simulator-specific commands to the PrimeSim XA, FineSim, or PrimeSim simulation engines.

Note:

For the PrimeSim XA simulation engine, you can also use the `xa_commands_begin` and `xa_commands_end` commands.

Syntax

```
netlist_commands_begin;  
simulator_specific_commands  
...  
netlist_commands_end;
```

Examples

See [Adding Netlist Commands to the Control File](#) for examples for the PrimeSim XA, FineSim, and PrimeSim simulation engines.

netlist_commands_end

Description

Use this command in the `vcsAD.init` mixed-signal simulation control file, along with the [netlist_commands_begin](#) command, to pass simulator-specific commands to the PrimeSim XA, FineSim, or PrimeSim simulation engines.

Note:

For the PrimeSim XA simulation engine, you can also use the `xa_commands_begin` and `xa_commands_end` commands.

Syntax

```
netlist_commands_begin;  
simulator_specific_commands  
...  
netlist_commands_end;
```

Examples

See [Adding Netlist Commands to the Control File](#) for examples for the PrimeSim XA, FineSim, and PrimeSim simulation engines.

optimize_shadowfile

Use the `optimize_shadowfile` command to optimize the shadow Verilog hierarchy generated for SPICE blocks. You can use the `-disable` and `-enable` options of the `optimize_shadowfile` command to change the default behavior.

Syntax

```
optimize_shadowfile -enable | -disable;
```

Description

To save compile time, this command directs the analog engine to avoid generating unnecessary Verilog dummy modules for blocks with the SPICE view. If references to nets in the transistor-level netlist (from the Verilog netlist) are made when this command is used, compilation errors related to the cross-module references (XMR) may occur. Be cautious when using this option with XMR.

Examples

In the following example, the PrimeSim XA, FineSim, and PrimeSim simulation engines are directed to avoid generating some of the Verilog dummy modules so the Verilog hierarchy under the SPICE view can be optimized.


```
optimize_shadowfile -enable | -disable;
```

Note:

This command prevents VCS from probing inside the SPICE hierarchy; this can prevent optimization of D2D through-nets and might increase the number of interface elements.

param_pass

Enables parameter passing between Verilog/VHDL and SPICE.

Syntax

```
param_pass -enable | -disable;
```

Description

This command uses the `-disable` and `-enable` options of the `param_pass` command to change the default behavior. [Table 11](#) shows the default behavior for parameter passing in different flows. To change the default behavior, use the `param_pass` command with the proper switch: `-enable` or `-disable`.

Table 11 Parameter Passing for Different Mixed-Signal Flows

Flow	Default	Description
Verilog-AMS-SPICE	<code>-enable</code>	Parameter passing between Verilog and SPICE is enabled by default.
Verilog-SPICE	<code>-disable</code>	Parameter passing between HDL and SPICE is disabled by default.
VHDL/Verilog-SPICE	<code>-disable</code>	Parameter passing between HDL and SPICE is disabled by default.

port_connect

Use this command when a SPICE cell is instantiated under a Verilog or VHDL parent and contains extra ports (usually VDD and VSS) that are not connected in the instantiation.

Syntax

```
port_connect -cell cell_name ([-inst inst_name] [real]  
    spice_port_name => hierarchical_net, ... |  
    spice_port_name => snps_open);
```

Arguments

`-cell cell_name`

Specifies the cell name.

`-inst inst_name`

If this option is used, only the specified instance is instantiated in SPICE. The [Full Hierarchical Path](#) to the instance is required.

`real`

Specifies that *hierarchical_net* is a real net in the digital domain.

`hierarchical_net`

Specifies the hierarchical path to the net; this net is used to connect the extra port.

Description

This command allows extra ports to connect to a hierarchical net in either the digital or the analog domain.

Note:

If a hierarchical net is at the top level, you must define it as a global net. For example, if you specify:

```
port_connect -cell inv ( inv_vdd => vdd , inv_vss => vss );
```

And `vdd` and `vss` are not declared as global (there is no such line: `.global vdd vss`), then the connection is not made.

Examples

- Connect the dangling SPICE ports `vdd` and `vss` to the ideal supply nets. Here the SPICE cell `inv1` is instantiated under a Verilog-top testbench:

```
inv1 i1 ( .y(y_wire), .a(a_wire) );
```

The SPICE cell has two extra ports called `vdd` and `vss` that are not present in the Verilog view of the cell and as a result are not connected in the instantiation above.

```
.subckt inv1 y a vdd vss
...
module inv1 (y, a);
...
```

Assuming that in the body of SPICE there are two ideal supplies defined as follows:

```
v3 vdd! 0 DC=3.0
v2 vss! 0 DC=0
```

The following mixed-signal command connects the dangling SPICE ports `vdd` and `vss` to the nets connected to the ideal supplies:

```
port_connect -cell inv1 ( vdd => vdd! , vss => vss! );
```

- Connect the dangling SPICE ports `vdd` and `vss` to the ideal supply nets driven by `top.i_power_blk`. In the following example, the SPICE cell `inv1` is instantiated under a Verilog-top testbench:

```
inv1 i1 (.y(y_wire), .a(a_wire) );
```

The cell has two extra ports, `vdd` and `vss`, that are not present in the Verilog view of the cell:

```
.subckt inv1 y a vdd vss
...
module inv1 (y, a);
...
```

Assuming that two ideal supplies are defined in subcircuit at the hierarchical path `top.i_power_blk` as:

```
v1 vdd 0 DC=1.2
v2 vss 0 DC=0
```

The following mixed-signal command connects the dangling SPICE ports `vdd` and `vss` to the nets connected to the ideal supplies:

```
port_connect -cell inv1 ( vdd => top.i_power_blk.vdd , vss =>
    top.i_power_blk.vss );
```

- Connect the SPICE ports `vdd` and `vss` to the Verilog real nets `top.my_vdd` and `top.my_vss`, respectively.

```
port_connect -cell test ( real vdd => top.my_vdd , real vss =>
    top.my_vss );
```

port_dir

Description

Declares port directions for SPICE subcircuit ports that cannot be derived from an equivalent multiple Verilog or VHDL view port direction.

Syntax

```
port_dir -cell cell_name [input|output|inout]
    [spice_port_name], [spice_port_name], ...;
[input|output|inout] [spice_port_name], ...);
```

Arguments

`-cell cell_name`

Specifies the cell name. You can specify the asterisk (*) wildcard character in a cell name.

`input|output|inout`

Specifies the port direction.

`spice_port_name`

Specifies the SPICE port name. You can specify the asterisk (*) wildcard character in a SPICE port name.

Examples

```
port_dir -cell invs1 (input a; output y);  
port_dir -cell invs1 (input a, b; output y);
```

print_ie_res

Prints out analog output resistance values for mixed-signal interface elements for which the driver strength calculation is enabled.

Syntax

```
print_ie_res [-limit value]  
-node hier_node_name | -cell cell_name -port port_name;
```

Arguments

`-limit value`

Specifies the upper limit for the resistance to be graphed in the output waveform file. The value can be expressed as an integer (such as 1000), metric (such as 1k or 1M or 250m), or scientific notation (such as 1e3). The default upper limit is 1MOhms.

`-node hier_node_name`

Specifies the hierarchical path to the interface element. The hierarchical path is taken from the `simv.msv/interface_element.rpt` file. You can specify a wildcard character (*) in the node name.

`-cell cell_name -port port_name`

Specifies the cell name and port name. You must specify both names, in which you can use wildcard characters (*).

Description

Bidirectional interface elements are common in mixed-signal designs. These interface elements usually model I/O elements that can assume one of the following two states during the circuit operation:

- Transmitter (driver)
- Receiver (high impedance (HiZ) input)

Two or more of such I/O elements are connected together to allow data exchange between multiple transmitters and receivers. Usually at each given time only one of the I/O elements is in transmission mode and the rest of the I/Os are in HiZ receiver mode. For this data exchange to occur, it is important that all the receivers stay in HiZ mode and do not drive the interface. Otherwise, they would interfere with the transmitter (active driver).

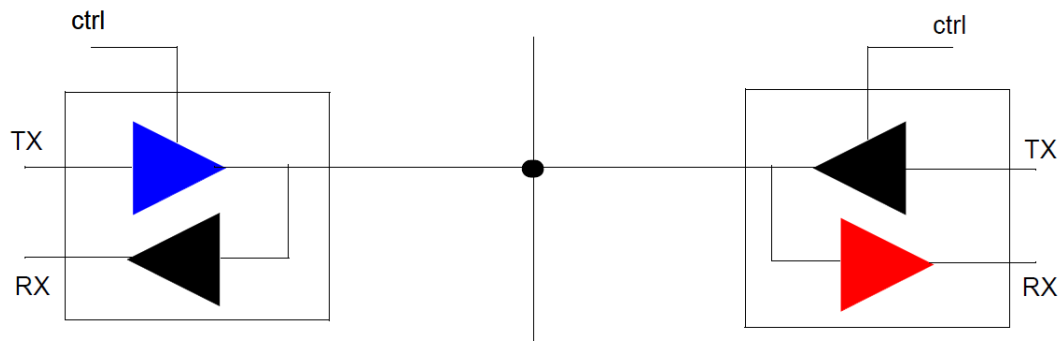
Note:

If you use `a2d -hiz_off`, the `print_ie_res` command is ignored for that interface element. If you use `a2d` without `-hiz_on` and the net is not bidirectional, the `print_ie_res` command is ignored for that interface element.

In a mixed-signal simulation, it is possible that one of the I/O elements is modeled in SPICE and another in RTL. In that case, it is important that when in the receiver mode, the SPICE I/O (blue device in [Figure 33](#)) must be in an HiZ state and does not drive, and potentially corrupt, the value driven by the digital driver (red device in [Figure 33](#)). Usually for that to happen, the resistance map for the bidirectional interface element (see [Figure 33](#)) for the SPICE I/O must be adjusted such that when the SPICE I/O is in the receiver mode, its output resistance satisfies the minimum resistance requirement for HiZ. Use the `rmap_file` command to specify the resistance map file.

For example, if the output resistance of the SPICE I/O is 1.2 kOhms when in receiver mode, the resistance map for this interface element must be modified such that the HiZ threshold is brought down from the default 90 kOhms to, for example, 1kOhms. That way when the SPICE I/O is in the receiver mode it exhibits an output resistance larger than the HiZ resistance threshold and the interface element goes into HiZ (not driving the interface net).

Figure 33 Bidirectional I/O Buffers Connecting to the Same Net

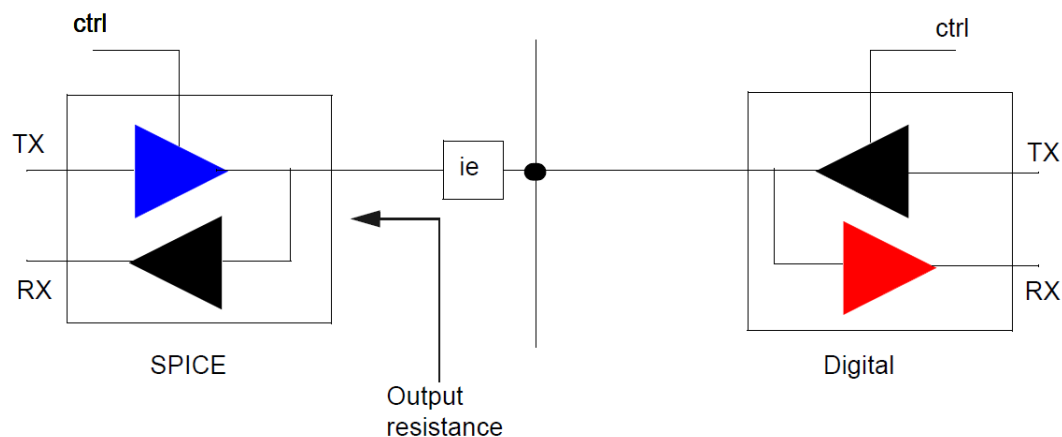


In this process, knowing the value of the output resistance exhibited by the SPICE I/O element, especially during the receiver mode, is crucial to proper adjustment of the resistance map for the interface element, as shown in [Figure 34](#).

The capability to view the output resistance of the analog drivers at the analog/digital boundary in the analog waveform file and decide how or whether to adjust the resistance map for that interface element is only applicable to interface elements for which analog drive strength calculations take place, which are:

- inout interface elements
- a2d interface elements for which drive strength calculation is enabled with the `-hiz_on` option of the a2d command

Figure 34 Output Resistance of the Analog Driver is used for Drive Strength Calculation of the Interface Element



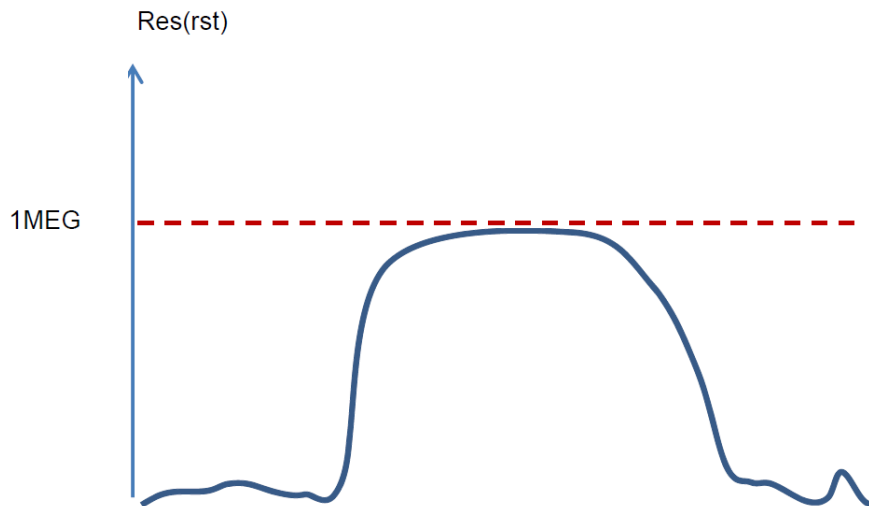
Examples

The following command enables dumping the output resistance for interface net

```
top.il.ckgen.rst:
```

```
print_ie_res -node top.il.ckgen.rst;
```

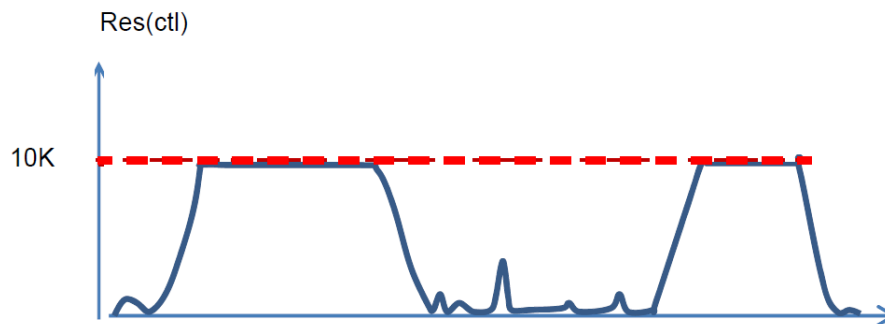
The output resistances bigger than 1MOhms is capped and displayed as 1MOhms:



The following command enables dumping the output resistance for interface net at port vbc of cell bandgap and limits the display of output resistances bigger than 10kOhms:

```
print_ie_res -limit 10k -cell bandgap -port vbc;
```

The output resistances bigger than 10kOhms is capped and displayed as 10kOhms:



r2e

Description

Controls the behavior of an r2e interface element. This command sets the threshold at which r2e events occur and also allows defining r2e as a voltage source (default) or a current source on the analog side of the interface.

Syntax

```
r2e [-type i]
    [-min_delta value]
    [-gain value] -node hierarchical_node_name [-rf_rate value]
    [-library library_name]
    [-cell cell_name -port port_name] |
    [-inst inst_name -port port_name] | [-node hier_name]
```

Arguments

`-type i`

Specifies whether the r2e is modeled as a voltage source (default) or as a current source (with `-type i`). When you specify `-type i`, the current direction follows the PrimeSim HSPICE rule: a positive value specifies that the current is flowing into the subcircuit and a negative value specifies that the current is flowing out of the subcircuit. You can change the sign of the `-gain` argument to change the current direction.

`-min_delta value`

Specifies the absolute voltage threshold value for r2e events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-gain value`

Multiplies the analog value by the specified *value*. The *value* can be a negative number. The default value is 1.

`-node hierarchical_node_name`

Specifies the hierarchical path to the r2e interface. You can specify wildcard characters (*) in the path.

`-rf_rate value`

Specifies how fast the voltage changes from the present value to the target value. The `-rf_rate` has a unit of second/volt, with the default set to 10ps/V.

You should avoid rapid changes in voltage values that might create unwanted current peaks.

`-library library_name`

Specifies a *library_name* to apply global `r2e` threshold options. Use this option to specify unique threshold options for all cells in the specified library. For more information see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specifies the cell and port names for the interface element. The *cell_name* must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port to apply the voltage value to the ports referring to the target instance. You can specify the asterisk (*) wildcard character in an instance name.

`-node hier_name`

Specifies the hierarchical node name for the interface element. You can specify the asterisk (*) wildcard character in a node name.

Examples

- The following example sets the `r2e` interface element at hierarchical path `top.i1.ictrl` to be a current type:

```
r2e -type i -node top.i1.ictrl;
```
- The following example sets the `-min_delta` to 1mV for the `r2e` interface at `test.pblk.clk`.

```
r2e -min_delta 1e-3 -node top.pblk.clk;
```

remove_d2a

Description

Removes a d2a (digital-to-analog) mixed-net from the digital-analog boundary and applies a constant DC voltage on the analog side source with a value defined by the `-dc` argument. The `-node` argument takes a mixed-net. The `-node` argument cannot take the asterisk wildcard character (*), such as `-node *`. However, a partial node search using the asterisk character is legal; for example, `-node top.din*` matches all signals with names starting with the `top.din` string.

Syntax

```
remove_d2a [-dc dc_voltage_source_value] -node node_name;
```

Arguments

`-dc dc_voltage_source_value`

Specified the constant DC *voltage* to apply on the analog side.

`-node hier_name`

Specifies the node from which to remove the d2a element. This argument cannot take the asterisk wildcard character (*), for example, `-node *`. However, a partially-specified node search using the asterisk character is accepted, such as `-node N*`.

Examples

- Remove the d2a interface element from node `top.VVDH` and replace it with a constant voltage of 1.8V.

```
remove_d2a -dc 1.8 -node top.VVDH;
```

- Remove all d2a interface elements from nodes in the top module that begin with `v` and replaces the interface elements with a constant voltage of 2.5V.

```
remove_d2a -dc 2.5 -node top.V* ;
```

Note:

When multiple `remove_d2a` commands are used for the same mixed-nets, the last `remove_d2a` command takes precedence.

report_option

Description

Use the `report_option` command to perform the following actions:

- Change the location of the runtime `msv` directory
- Generate concise or detailed information about interface elements in the `interface_element.rpt` file
- Generate limited or complete information about through-net elements in the `through_net.rpt` file

- Control the output of hierarchy information of cells and instances in the `hierarchy.bin` and `hierarchy.rpt` files
- Output additional information to the `interface_element.rpt` file for interface elements connected to `tranif`-type gates

Syntax

```
report_option [-msv_dir vcs | spice]
               [-interface_element concise | verbose]
               [-through_net concise | verbose]
               [-hierarchy ascii | binary | concise | suppress]
               [-tranif enable | disable];
```

Arguments

`-msv_dir vcs | spice`

Specifies the destination directory type, `vcs` or `spice`. The default is `vcs` (see [Example 46](#)).

If you specify `spice`, the runtime results are redirected to a directory as specified by `xa -o name` or `finestim -o name`, different from the compilation results directory (see [Example 47](#)).

By default, or if you specify `-msv_dir vcs`, the tool updates runtime results in the same compilation directory as specified by `vcs -o name`.

`-interface_element concise | verbose`

Specifies whether to generate the `interface_element.rpt` file as a concise report or as a report with additional information.

- The `concise` option has only one line for each interface element, which might not provide enough detail (see [Example 48](#)).
- The `verbose` option provides additional information about the interface elements in the `inout` section of the `interface_element.rpt` file. It also shows which supply element it found for the values of the interface element, such as `hiv`. It shows you the types of interface elements so you can copy and paste them in your `vcsAD.init` file (see [Example 49](#)).

`-through_net concise | verbose`

Specifies whether to generate the `through_net.rpt` file for all the through-nets that share the same connectivity or a maximum of ten through-nets sharing the same connectivity. This `-through_net` argument controls the through-net reporting only when the entries share the same connectivity; that is, it still shows all the remaining unique through-nets.

- `concise` is the default mode in which the `through_net.rpt` reports a maximum of ten through-nets that share the same connectivity. If the number of through-nets sharing the same connectivity is more than 10, the `through_net.rpt` shows additional information about how to report all the entries (see [Example 50](#)).
- `verbose` is the mode in which the `through_net.rpt` reports the complete list of the through-nets (see [Example 51](#)).

`-hierarchy ascii | binary | concise | suppress`

Specifies the mode to use when generating the `hierarchy.rpt` file as follows:

- `ascii` is the default mode and the hierarchy information of cells and instances is output in both binary format and textual format.
 - Hierarchy data in binary format is output as `<out>.daidir/hierarchy.bin`, for example, `simv.daidir/hierarchy.bin`.
 - Hierarchy data in textual format is output as `<out>.msv/hierarchy.rpt`, for example, `simv.msv/hierarchy.rpt`.
- `binary` mode enables the hierarchy data to be output only in non-readable binary format. The data is output to `<out>.msv/hierarchy.bin`, for example, `simv.msv/hierarchy.bin`.

Note:

You can use binary mode to improve performance by reducing compilation time and output file size. You can still generate the `hierarchy.rpt` text-format file from the binary `hierarchy.bin` file by using the [hierDump Utility](#).

- `concise` mode enables the hierarchy data to be output both in binary format and textual format but in a concise manner. Compared to `ascii` mode that generates the full hierarchy information of subcircuits and instances (see [Example 52](#)), `concise` mode only captures the hierarchical view of subcircuit relationships (see [Example 53](#)).

No instance names are involved, while indentation is used to show the parent-child relationship of two subcircuits. If two instances under the same parent use the same subcircuit, its definition only occurs one time.

- `suppress` mode disables the generation of both `hierarchy.bin` and `hierarchy.rpt`. This mode provides the fastest compile time and

requires the least disk space. Note the following conditions when using the `-hierarchy suppress` option:

- For testbench reuse, when `-hierarchy suppress` is specified, the tool forces `-hierarchy binary` to ensure that testbench reuse is working. A warning message is generated to prompt you for such change of behavior.
- Probing a digital-top design using wildcard and probe limit/level can be wrong in finding the correct levels. The level matching could be incorrect due to `hierarchy.rpt` being suppressed, which caused the tool only to recognize all digital top wrapped as one-level. However, probes values are always correct. For `suppress` mode, a warning message is generated to prompt you for potential incorrect probing level.

`-tranif enable | disable`

Writes out additional information to the `interface_element.rpt` file for interface elements connected to `tranif`-type gates (see [Example 54](#)). When this option is enabled, the tool writes an additional comment to the report file when an interface net is connected to a bidirectional pass switch (`tran`, `tranif1`, or `tranif0`).

Note:

You must specify the `-debug_access+drivers` option with the `vcs` command when using this argument.

hierDump Utility

You can use binary mode to improve performance by reducing compilation time and output file size. You can still generate the `hierarchy.rpt` text-format file from the binary `hierarchy.bin` file by using the `hierDump` utility included with each simulation engine:

- PrimeSim XA: `XA_HOME/bin/hierDump`
- FineSim: `FINESIM_HOME/bin/hierDump`
- PrimeSim: `PRIMESIM_HOME/bin/hierDump`

Run `hierDump` as follows:

```
hierDump -db binary_hier_file [-mode concise] -o file_name
```

You can use the `hierDump` utility to generate the `hierarchy.rpt` in either `ascii` mode (see [Example 55](#)) or `concise` mode (see [Example 56](#)).

Examples

This section provides examples for the different arguments of the `report_option` command and the `hierDump` utility.

Example 46 Runtime Results Directory Specified by Digital Simulator (msv_dir=vcs)

The following example saves the runtime results into VCS output directory as specified by the `vcs -o` option. By default, the directory name is `simv`.

```
// vcsAD.init
report_option -msv_dir vcs;
choose xa -hspice addr4.spi -c xa.cfg -o ./xal
```

Example 47 Runtime Results Directory Specified by Analog Simulator (msv_dir=spice)

The following example saves the runtime results into the `results/xal.msv` directory as specified by `-o` option of the `xa` command. Note that this is different from the compilation results directory, `simv.msv`.

```
// vcsAD.init
report_option -msv_dir spice;
choose xa -hspice addr4.spi -c xa.cfg -o results/xal;
```

Note:

In a mixed-signal simulation, the `simv.msv` directory contains the mixed-signal report files. Some of the report files are generated during compilation and some are generated during runtime. In cases when the runtime directory (where `simv` is executed) is different from the compilation directory (where `simv` is located), this command determines where the runtime files are written.

For more information about using this command, see [Running Concurrent Mixed-Signal Simulations From the Same Compiled Code](#).

Example 48 Concise Interface Element Report (interface_element=concise)

The following example shows the interface element section of the `interface_element.rpt` file for default `concise` reporting.

```
...
a2d -loth 1.65v -hith 1.65v -node top.dut.s_0;
a2d -loth 1.65v -hith 1.65v -node top.dut.s_1;
d2a -hiv 3.3v -lov 0v -node top.dut.a_0;
d2a -hiv 3.3v -lov 0v -node top.dut.a_1;
```

Example 49 Verbose Interface Element Report (interface_element=verbose)

The following example shows the interface element section of the `interface_element.rpt` file for `verbose` reporting.

```
...
# a2d interface elements:
# 2 total
a2d -loth 1.65v -hith 1.65v -node top.dut.s_0;
// loth and hith from node vdd value=3.3
```

Appendix A: Mixed-Signal Control Commands

report_option

```
// Top-Net
// top.s[-0]
// All Boundary Nets
// top.dut.s_0
a2d -loth 1.65v -hith 1.65v -node top.dut.s_1;
// loth and hith from node vdd value=3.3
// Top-Net
// top.s[-1]
// All Boundary Nets
// top.dut.s_1

# d2a interface elements:
# 2 total
d2a -hiv 3.3v -lov 0v -node top.dut.a_0;
// hiv and lov from node vdd value=3.3
// Top-Net
// top.dut.a_0
// All Boundary Nets
// top.dut.a_0
d2a -hiv 3.3v -lov 0v -node top.dut.a_1;
// hiv and lov from node vdd value=3.3
// Top-Net
// top.dut.a_1
// All Boundary Nets
// top.dut.a_1
```

Example 50 Concise Through-Net Report (through_net=concise)

The following example shows the `through_net.rpt` in the default `concise` mode. When more than ten through-nets share the same connectivity, the `through_net.rpt` limits the number of entries to 10 with a clear message on how to generate the complete list.

The through-nets (for example: `xtop.xia1.n*`) that share the same connectivity appear together while the ones that do not share the same connectivity are reported after a blank line.

```
xtop.xia1.temp1 A2A

xtop.xia1.temp4 D2D

# Only the first 10 ports forming the through-nets were reported below to
# limit the file size.
# To get the full list of all connectivity, use mixed-signal command
# report_option -through_net verbose;
xtop.xia1.n1 D2D
xtop.xia1.n2 D2D
xtop.xia1.n3 D2D
xtop.xia1.n4 D2D
xtop.xia1.n5 D2D
xtop.xia1.n6 D2D
xtop.xia1.n7 D2D
xtop.xia1.n8 D2D
```

Appendix A: Mixed-Signal Control Commands

report_option

```

xtop.xia1.n9 D2D
xtop.xia1.n10 D2D

xtop.xia1.temp2 D2D
xtop.xia1.temp3 D2D

xtop.xia7.t1 D2D

```

Example 51 Verbose Through-Net Report (through_net=verbose)

The following example shows the `through_net.rpt` in `verbose` mode in which the complete list of the through-nets is reported. There are more than ten through-nets that share the same connectivity and they all get reported in `verbose` mode.

```

xtop.xia1.temp1 A2A

xtop.xia1.temp4 D2D

xtop.xia1.n1 D2D
xtop.xia1.n2 D2D
xtop.xia1.n3 D2D
xtop.xia1.n4 D2D
xtop.xia1.n5 D2D
xtop.xia1.n6 D2D
xtop.xia1.n7 D2D
xtop.xia1.n8 D2D
xtop.xia1.n9 D2D
xtop.xia1.n10 D2D
xtop.xia1.n11 D2D
xtop.xia1.n12 D2D
xtop.xia1.n13 D2D
xtop.xia1.n14 D2D

xtop.xia1.temp2 D2D
xtop.xia1.temp3 D2D

xtop.xia7.t1 D2D

```

Example 52 Original Full Hierarchy in ASCII Mode (hierarchy=ascii)

The following example from the `hierarchy.rpt` file shows the full hierarchy information of subcircuits and instances of `ascii` mode.

```

top(top).inst1<inva>.x1<invb>
top(top).inst1<inva>.x2<invr>
top(top).inst2<inva>.x1<invb>
top(top).inst2<inva>.x2<invr>
top(top).inst3<inva>.x1<invb>
top(top).inst3<inva>.x2<invr>
top(top).inst4<inva>.x1<invb>
top(top).inst4<inva>.x2<invr>

```


Example 53 Hierarchy in Concise Mode (*hierarchy=concise*)

The following example from the `hierarchy.rpt` file shows the hierarchical view of subcircuit relationships of `concise` mode.

```
--(top)
  --<inva>
    --<invb>
      --<invr>
```

Example 54 Interface Element Report With tranif-type Gate Information (*tranif=enable*)

The following example writes additional information for interface elements on `tranif`-type gates.

```
// vcsAD.init
use_spice -cell vbb_pump ;
a2d -strength supply -node test.i4a.vpp ;
a2d -strength supply -node test.i5a.vpp ;
report_option -tranif enable;
```

When `report_option -tranif enable` is specified, the tool inserts additional information messages into the `interface_element.rpt` file as follows. Note that you must add the `-debug_access+drivers` option to the `vcs` command to produce this output.

```
// interface_element.rpt
inout -loth 0.9v -hith 0.9v -hiv 1.8v -lov 0v -node test.i2a.a;
  // from bufif1 , to tranif0 | tranif1
a2d -loth 1v -hith 1v -strength supply -node test.i4a.vpp; // to tranif0
a2d -loth 1v -hith 1v -strength supply -node test.i5a.vbb; // to tranif1
```

Example 55 ASCII Hierarchy Report From Binary File (*hierDump*)

The following example uses the `hierDump` utility to generate the `hierarchy.rpt` file in `ascii` mode from the binary file:

```
hierDump -db simv.msv/hierarchy.bin -o simv.msv/hierarchy.rpt
```

where the binary file is specified with the `-db` option and the direct output file is specified with the `-o` option.

Example 56 Concise Hierarchy Report From Binary File (*hierDump -mode concise*)

The following example uses the `hierDump` utility to generate the `hierarchy.rpt` file in `concise` mode from the binary file:

```
hierDump -db simv.msv/hierarchy.bin -mode concise -o simv.msv/hierarchy.rpt
```

resolve_x_inst_prefix

Description

Removes the SPICE *x* prefixes so that HDL testbenches do not have to be changed when multiple cell views are changed.

Syntax

```
resolve_x_inst_prefix -enable | -disable;
```

rmap_file

Specifies the resistance map file or path to the resistance map file. If this command is not specified, the tool reads the default `rmapAD.init` resistance map file from the PrimeSim XA, FineSim, and PrimeSim installation directory.

Syntax

```
[-file] resistance_map_file_name  
[-node node_name(s)] [-nodefile node_filename]  
[-inst instance_names(s)] [-inst instance_name -port port_name(s)]  
[-instfile instance_filename]  
[-subckt subckt_name(s)] [-subckt subckt_name -port port_name(s)]  
[-subcktfile subckt_filename];
```

Description

The `-node`, `-instance`, and `-subcircuit` options are mutually exclusive, and cannot be used with each other in the same command. However, these options can be used in separate `rmap_file` commands.

When more than one `rmap_file` command is used, the last command overrides the settings of the previous commands.

See [Table 12](#) for the four `rmap_file` command option categories.

Table 12 *rmap_file* Command Option Categories

Category	Command option
file	Custom resistance map file name.
node	<code>-node <i>node_name(s)</i></code> <code>-nodefile <i>node_filename</i></code>
instance	<code>-inst <i>instance_names(s)</i></code> <code>-inst <i>instance_name</i> <i>-port</i> <i>port_name(s)</i></code> <code>-instfile <i>instance_filename</i></code>

Table 12 *rmap_file Command Option Categories (Continued)*

Category	Command option
subcircuit	<code>-subckt subckt_name(s)</code> <code>-subckt subckt_name -port port_name(s)</code> <code>-subcktfile subckt_filename</code>

Examples

The following examples show different uses of the `rmap_file` command.

- Use the `resis_comp.map` file in the current directory as the resistance map file.
`rmap_file resis_comp.map;`
- Use the `resis_comp.map` file in the `/u/tools/xa` directory as the resistance map file.
`rmap_file /u/tools/xa/resis.map;`
- Apply the `r2.map` resistance map file to nodes `top.i1.wen` and `top.i2.ren`. The default resistance map file is applied to the remaining mixed nets.
`rmap_file r2.map -node top.i1.wen top.i2.ren;`
- Apply the `r3.map` resistance map file to all mixed nets connected to the `top.i1.abc` and `top.i2.xyz` instances. The default resistance map file is applied to the remaining mixed nets.
`rmap_file r3.map -inst top.i1.abc top.i2.xyz;`
- Apply the `r3.map` resistance map file to mixed nets connected to port `Z` on the `top.i1.abc` instance. The default resistance map file is applied to the remaining mixed nets.
`rmap_file r3.map -inst top.i1.abc -port Z;`
- Apply the `r4.map` resistance map file to all mixed nets connected to the `pad` subcircuit. The default resistance map file is applied to the remaining mixed nets.
`rmap_file r4.map -subckt pad;`
- Apply the `r4.map` resistance map file to the mixed net connected to the `IN` port on the `pad` subcircuit. The default resistance map file is applied to the remaining mixed nets.
`rmap_file r4.map -subckt pad -port IN;`
- Apply the `r5.map` file to the `top.i1.clk`, `top.i2.addr[0]`, `top.i2.addr[1]` and `top.i2.addr[2]` mixed nets. The nets are listed in the `n1.txt` file. The default resistance map file is applied to the remaining mixed nets.

```
rmap_file r5.map -nodefile n1.txt;
```

The `n1.txt` file contains the following node names:

```
top.i1.clk  
top.i2.addr[0]  
top.i2.addr[1]  
top.i2.addr[2]
```

- Apply the `r6.map` file to the nets connected to the `top.i1`, `top.i2`, `top.i3.abc` and `top.i4.xyz` instances. The nets are listed in the `inst1.txt` file. The default resistance map file is applied to the remaining mixed nets.

```
rmap_file r6.map -instfile inst1.txt;
```

The `inst1.txt` file contains the following node names:

```
top.i1  
top.i2  
top.i3.abc  
top.i4.xyz
```

- Apply the `r7.map` resistance map file to the `pad`, `abc`, and `xyz` subcircuits. The subcircuits are listed in the `s1.txt` file. The default resistance map file is applied to the remaining mixed nets.

```
rmap_file r7.map -subcktfile s1.txt;
```

The `s1.txt` file contains the following subcircuits.

```
pad  
abc  
xyz
```

- Apply the `r8.map` resistance map file to all mixed nets matched by `*addr*`. The default resistance map file is applied to the remaining mixed nets.

```
rmap_file r8.map -node *addr*;
```

- The following example ignores the `r1.map` resistance map file because the second `rmap_file` command includes the signal specified by the first command. When two or more `rmap_file` commands include the same nodes, the last command is used. In this example, the `r2.map` resistance map file is applied to the `top.i1.addr.n1` node and the default resistance map file is applied to the remaining mixed nets that are not covered by these commands.

```
rmap_file r1.map -node top.i1.addr.n1;  
rmap_file r2.map -node top.*addr*;
```

- The following example is invalid because the `-node` and `-inst` options cannot be used together in the same command. You must use two or more `rmap_file` commands to specify resistance mapping files for both nodes and instances.

```
// INVALID SPECIFICATION
rmap_file r1.map -node top.i1.addr -inst top.i1.abc;
```

- Apply the `r2.map` resistance map file to the mixed nets connected to the `top.i1.abc` instance. The `r1.map` resistance map file is applied to the `top.i1.addr.n1` node. The default resistance map file is applied to the remaining mixed nets.

```
rmap_file r1.map -node top.i1.addr.n1;
rmap_file r2.map -inst top.i1.abc;
```

If multiple `rmap_file` commands target the same mixed nets and each one of the commands uses one of the `-node`, `-instance`, or `-subcircuit` options, use the options with the `rmap_file` command in the following order:

1. Subcircuit-based
2. Instance-based
3. Node-based

The wildcard (*) character can be used in the port, node, instance, or subcircuit name.

The period (.) character is the required hierarchical separator. The string length must not exceed 4,096 characters.

Note:

When multiple `rmap_file` commands are used for the same mixed nets in the mixed-signal simulation control file, the last command takes precedence.

rt_a2d

Description

Controls all aspects of any runtime a2d interfaces created in the VCS PrimeSim AMS tool. If this command is not specified, all a2d events are triggered at 50% of the local VDD by default.

Syntax

```
rt_a2d [- loth lo_thrsh[V | %]]
      [-hith hi_thrsh[V | %]]
      -node hier_name;
```

Arguments

`-loth lo_thrsh [V | %], -hith hi_thrsh[V | %]`

Specifies the low and high a2d threshold values. They can be expressed either as absolute values (1.1V, 2.2) or as a percentage of the supply (10%, 90%).

`-node hier_name`

Because `rt_a2d` is instance based, you must specify the hierarchical path to the instance port. For example, `-node top.il.y;`

rt_d2a

Description

Controls all aspects of any runtime d2a interfaces created in the VCS PrimeSim AMS tool. The drive strength for `rt_d2a` defaults to the *supply* strength.

Syntax

```
rt_d2a [-rf_time slope_time] | [-rise_time rise_time]
[-fall_time fall_time]
[-x2v 0|1|2|3|4]
-hiv high_voltage [ V | % ]
-lov low_voltage [V | % ]
-node hier_name;
```

Arguments

`-rf_time slope_time`

Specifies the analog rise and fall times. The default time unit is in seconds, so specify the subunit with a value such as `-rf_time 1.5n`, for example. With this option, both rise and fall times are set to the same value.

`-rise_time rise_time`

Specifies the analog rise time. The default time unit is in seconds, so specify the subunit with a value such as `-fall_time 1n -rise_time 2n`, for example.

`-fall_time fall_time`

Specifies the analog fall time. The default time unit is in seconds, so specify the subunit with a value such as `-fall_time 1n -rise_time 2n`, for example.

```
-x2v 0|1|2|3|4
```

Sets the rule on how a logic X must be translated to a voltage level on the analog side. Use this option to manage the translation of the X to a voltage level with one of the following values:

- 0 to always set to the logic 0 voltage (default)
- 1 to always set to the logic 1 voltage
- 2 to set to (the logic 1 voltage + the logic 0 voltage)/2
- 3 to set to previous voltage
- 4 to set to the logic 1 voltage else; if logic = 1 set to the logic 0 voltage else get previous voltage

```
-hiv high_voltage [ V | % ]
```

Specifies the voltage that corresponds to the logic 1 state. Use this option to override the default value by either expressing an absolute voltage value (for example, 1.2V or 1.2) or as a percentage of the supply voltage (for example, 90%). You must use this option combination with the `-lov` option. By default, the logic 1 voltage value is the voltage of the local supply. If the tool cannot trace the `rt_d2a` net to an ideal supply, it assumes the same default value as the `rmap d2a`.

```
-lov low_voltage [V | % ]
```

Specifies the voltage that corresponds to the logic 0 state. Use this option to override the default value by either expressing an absolute voltage value (for example, 0.2V or 0.2) or as a percentage of the supply voltage (for example, 10%). You must use this option combination with the `-hiv` option. By default, the logic 0 voltage value is 0V.

```
-node hier_name
```

Because `rt_d2a` is instance based, you must specify the hierarchical path to the instance port. For example: `-node top.il.y;`

rt_e2n

Description

Controls the behavior of a runtime e2n interface element. This command sets the threshold at which e2n events occur and also allows e2n to convert analog voltage values (default) or current values to digital.

Syntax

```
rt_e2n [-min_delta value]
      [-set_type i]
      -node hierarchical_node_name;
```

Arguments

`min_delta value`

Specifies the absolute voltage/current threshold value for e2n events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum voltage or current value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-set_type i`

Enables the interface element to convert current to real. When the `$hdl_xmr` task has a SPICE block port as the source and a nettype variable as the destination, the tool places a `rt_e2n` to assign the port voltage value to the nettype variable. This interface element can be configured through the `rt_e2n` command. If you use the `-set_type i` option, the `rt_e2n` interface element no longer converts the voltage but instead converts the current pulled by the SPICE block port.

`-node hierarchical_node_name`

Specifies the hierarchical path to the e2n interface. You can specify wildcard characters (*) in the path.

rt_e2r

Description

Controls the behavior of a runtime e2r interface element. This command sets the threshold at which e2r events occur and also allows e2r to convert analog voltage values (default) or current values to digital.

Syntax

```
rt_e2r [-min_delta value]
      [-type i]
      -node hierarchical_node_name;
```


Arguments

`-min_delta value`

Specifies the absolute voltage/current threshold value for e2r events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum voltage or current value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-type i`

Enables the interface element to convert current to real. When the `$hdl_xmr` task has a SPICE block port as the source and a real variable as the destination, the tool places a `rt_e2r` to assign the port voltage value to the real variable. This interface element can be configured through the `rt_e2r` command. If you use the `type i` option, the `rt_e2r` interface element no longer converts the voltage but instead converts the current pulled by the SPICE block port.

`-node hierarchical_node_name`

Specifies the hierarchical path to the e2r interface. You can specify wildcard characters (*) in the path.

rt_n2e

Description

Controls the behavior of a runtime n2e interface element. This command sets the threshold at which n2e events occur and allows n2e to act as a voltage source (default) or a current source on the analog side of the interface.

Syntax

```
rt_n2e [-min_delta value]
        [-gain value]
        -node hierarchical_node_name
        [-x2v value];
```

Arguments

`-min_delta value`

Specifies the absolute voltage/current threshold value for n2e events. The *value* field is specified in the engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum voltage or current value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-gain value`

Multiplies the analog voltage or current by the specified *value*. The *value* can be a negative number. The default value is 1.

`-node hierarchical_node_name`

Specifies the hierarchical path to the n2e interface. You can specify wildcard characters (*) in the path.

`-x2v value`

Specifies the voltage source value when the interface element real value is set to `realX`.

- If not specified, during DC, the interface element is 0.0V for a `realX` and during transient, the interface element holds its last value.
- If specified, during DC or transient, then interface element is forced to the specified value for a `realX`.

rt_r2e

Description

Controls the behavior of a runtime r2e interface element. This command sets the threshold at which r2e events occur and also allows defining r2e as a voltage source (default) or a current source on the analog side of the interface.

Syntax

```
rt_r2e [-gain value]
       -node hierarchical_node_name;
```

Arguments

`-gain value`

Multiplies the analog voltage or current by the specified *value*. The multiplier can be a negative number. The default value is 1.

`-node hierarchical_node_name`

Specifies the hierarchical path to the r2e interface. You can specify wildcard characters (*) in the path.

shadow_file

Improves compile-time performance by hiding internal analog-side connections to the digital side controlled by the VCS tool.

Syntax

```
shadow_file -type compact;
```

Description

This command improves compile-time performance for designs with large analog cones and many analog ports. This command enables the analog simulator to hide the internal analog-side connections to the digital side controlled by VCS. This limits the associated computations and reduces runtime. VCS only sees the connection information at the interface (d2a / a2d and so on) boundary. However, the digital-to-digital through-net tunneling through analog domain is lost.

shadow_file_dir

Sets the `simv.daidir` directory which contains user-modified Verilog dummy module files.

Syntax

```
shadow_file_dir [-path] directory_path;
```

Description

In Verilog-SPICE and VHDL/Verilog-SPICE flows, mixed-signal simulation automatically generates Verilog dummy module files (called *shadow modules*) for all SPICE subcircuits in the design. These files are placed in the `simv.daidir` directory or, if the VCS switch `-o unique_name` is used, in the `unique_name.daidir` directory.

VCS then reads these files to construct the design hierarchy. If there are any problems in the Verilog dummy modules (usually caused by differences in the number of ports or their names between Verilog and SPICE views) VCS fails in the compilation phase.

This command allows you to work around such netlist disparity problems by following these steps:

1. Copy the `simv.daidir` directory (or `unique_name.daidir` if VCS option `-o unique_name` is used) to a new location.
2. Edit the shadow module files to resolve the Verilog/SPICE netlist disparity issues.
3. Run the compile again with the `shadow_file_dir` command in the mixed-signal simulation control file pointing to the new location.

Examples

The following example shows how to have the `shadow_file_dir` command point to the `/u/user1/work/wrapper_dir` directory, which contains the modified versions of `simv.daidir` files.

```
shadow_file_dir /u/user1/work/wrapper_dir;
```

skip_xmr_name_check

Description

When enabled, the compilation skips cross-module reference (XMR) checks. The default is `-disable`.

Syntax

```
skip_xmr_name_check -enable | -disable;
```

Examples

For example, a SPICE back-annotated netlist includes the net:

```
x1/x0/n36:1
```

while the Verilog testbench includes the system task:

```
$snps_force_volt(top.i0.x1.x0.\n36:1 ,3.0);
```

By default, the cross-module reference (XMR) check indicates a compilation error:

```
Error-[XMRE] Cross-module reference resolution error
```

Specifying the command:

```
skip_xmr_name_check -enable;
```

allows the compilation to skip this error and continue to compile and simulate.

spice_top

Description

Specifies that the top-level of the design is described in SPICE. This command is required for simulating a SPICE-top design.

Syntax

```
spice_top [-name top_name];
```

Arguments

`-name top_name`

Specifies the name (of your choice) to the top-level dummy module instance—see the following example. Specifying a `top_name` option is useful when you want the name of the top-level block in the design be something other than the default SPICE-top name: `snps_sptop`. This top-level name appears in the hierarchical signal names in the VCS-generated output file.

Examples

The following example specifies a SPICE-top design and names the top-level dummy module as `my_top`.

```
spice_top -name my_top;
```

In this example, a signal in the VCS-generated output file could be `my_top.x1.x2.cin`. However, if `my_top` had not been specified, the signal would be `snps_top.x1.x2.cin`.

transient_analysis

Specifies that the tool enters interactive mode when the transient analysis simulation is complete.

Syntax

```
transient_analysis -mode finish | stop;
```

Arguments

```
-mode finish
```

Runs transient simulation until it finishes (default)

```
-mode stop
```

Enters interactive mode and returns to the UCLI prompt at the end of transient simulation.

Description

By default, the mixed-signal simulation finishes at the end of the transient analysis, but if you specify `stop`, the simulation stops and returns to the interactive mode (UCLI prompt) after it reaches the end of the `.tran` time. You need to use the UCLI `quit` command to finish the simulation.

Examples

SPICE Netlist	transient_analysis \$finish \$stop	UCLI Command	finish task in Verilog	Behavior
.tran is not used	transient_analysis -mode finish;	run 10ns	Not used	Runs for 10ns and finishes.
.tran is not used	transient_analysis -mode stop;	run 10ns	Not used	Runs for 10ns and finishes. The use of transient_analysis -mode stop has no effect if .tran is not used in SPICE netlist.

Appendix A: Mixed-Signal Control Commands

transient_analysis

SPICE Netlist	transient_analysis \$finish \$stop	UCLI Command	finish task in Verilog	Behavior
.tran is not used	transient_analysis -mode finish;	run	#10 finish;	Runs for 10ns and finishes.
.tran is not used	transient_analysis -mode stop;	run	#10 finish;	Runs for 10ns and finishes. The use of transient_analysis -mode stop has no effect if .tran is not used in SPICE netlist.
.tran 1ns 30ns	transient_analysis -mode finish;	run	Not used	Runs for 30ns and then completes simulation.
.tran 1ns 30ns	transient_analysis -mode stop;	run	Not used	Runs for 30ns and then stops at UCLI prompt due to the transient_analysis -mode stop command.
.tran 1ns 25ns	transient_analysis -mode finish;	run 30ns	Not used	Runs for 25ns and then completes simulation.
.tran 1ns 25ns	transient_analysis -mode stop;	run 30ns	Not used	Runs for 25ns and then stops at UCLI prompt due to the transient_analysis -mode stop command.
.tran 1ns 30ns	transient_analysis -mode finish;	run 25ns; run	Not used	Runs for 25ns and then stops at UCLI prompt due to the use of run 25ns UCLI command. If run is issued, it runs and finishes the simulation.
.tran 1ns 30ns	transient_analysis -mode stop;	run 25ns	Not used	Runs for 25ns and then stops at UCLI prompt due to the use of run 25ns UCLI command. If run is issued, it runs and then returns to UCLI prompt due to the transient_analysis -mode stop command.
.tran 1ns 20ns	transient_analysis -mode finish;	run	#50 -mode finish;	Runs for 20ns and then completes simulation.

SPICE Netlist	transient_analysis \$finish \$stop	UCLI Command	finish task in Verilog	Behavior
.tran 1ns 20ns	transient_analysis -mode stop;	run	#50 -mode finish;	Runs for 20ns and then stops at UCLI prompt due to the transient_analysis -mode stop command.
.tran 1ns 80ns	transient_analysis -mode finish;	run	#50 -mode finish;	Runs for 50ns and then completes simulation.
.tran 1ns 80ns	transient_analysis -mode stop;	run	#50 -mode finish;	Runs for 50ns and then completes simulation.

u2e

Specifies the behavior of a u2e interface element. This interface element is used to drive a SPICE power net from a UPF supply net. Command options provide control over rise time rate, fall time rate, and scaling factor for output voltage.

Syntax

```
u2e [-node hierarchical_node_name | -cell cell_name -port port_name]
    [-rise_time_rate rise_time_rate]
    [-fall_time_rate fall_time_rate]
    [-gain value]
    [-min_delta value]
```

Arguments

`-node hierarchical_node_name`

Specifies the hierarchical node name. You can specify the asterisk (*) wildcard character in a node name.

`-cell cell_name -port port_name`

Specifies the cell and port names for the interface element. The `cell_name` must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-rise_time_rate rise_time_rate`

Specifies how fast the voltage changes from the present value to the higher target value. This argument has a unit of seconds/volt, with the default set to 10ps/V. You should avoid rapid changes in voltage values that might create unwanted current peaks.

`-fall_time_rate fall_time_rate`

Specifies how fast the voltage changes from the present value to the lower target value. This argument has a unit of seconds/volt, with the default set to 10ps/V. You should avoid rapid changes in voltage values that might create unwanted current peaks.

`-gain value`

Multiplies the analog value by the specified *value*. The *value* can be a negative number. The default value is 1.

`-min_delta value`

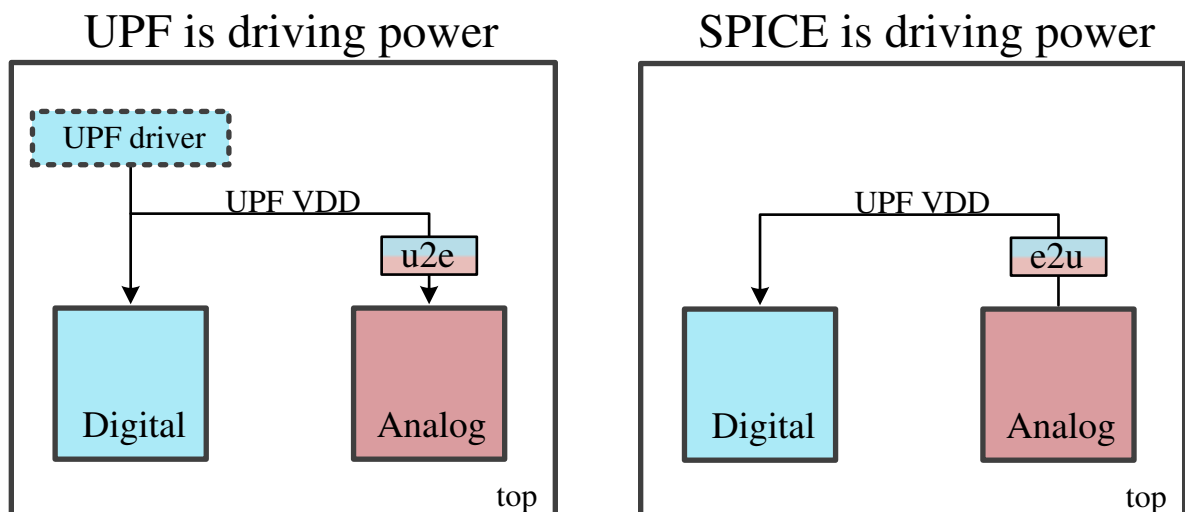
Specifies the absolute voltage threshold value for u2e events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of the maximum value. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

Description

The u2e interface element is used to convert a UPF supply net value to a SPICE power output signal. Use the u2e command to configure and change the default behavior for the u2e interface element placed at the UPF-to-SPICE interface. Use options to the command to add rise and fall times to the supply voltage transitions from UPF.

In the following figure, a u2e interface element converts the UPF supply net value for VDD to a SPICE power input signal.

Figure 35 Mixed-Signal Design With UPF



Examples

The following examples use command options to control the behavior of the u2e interface element:

- Specify a 1us rise time rate and 1us fall time rate. By default, the rise and fall transition times for the analog supply voltage would equal the time resolution for the analog engine:

```
u2e -node top.i1.vdd -rise_time_rate 1u -fall_time_rate 1u;
```

- Specify a gain of 2:

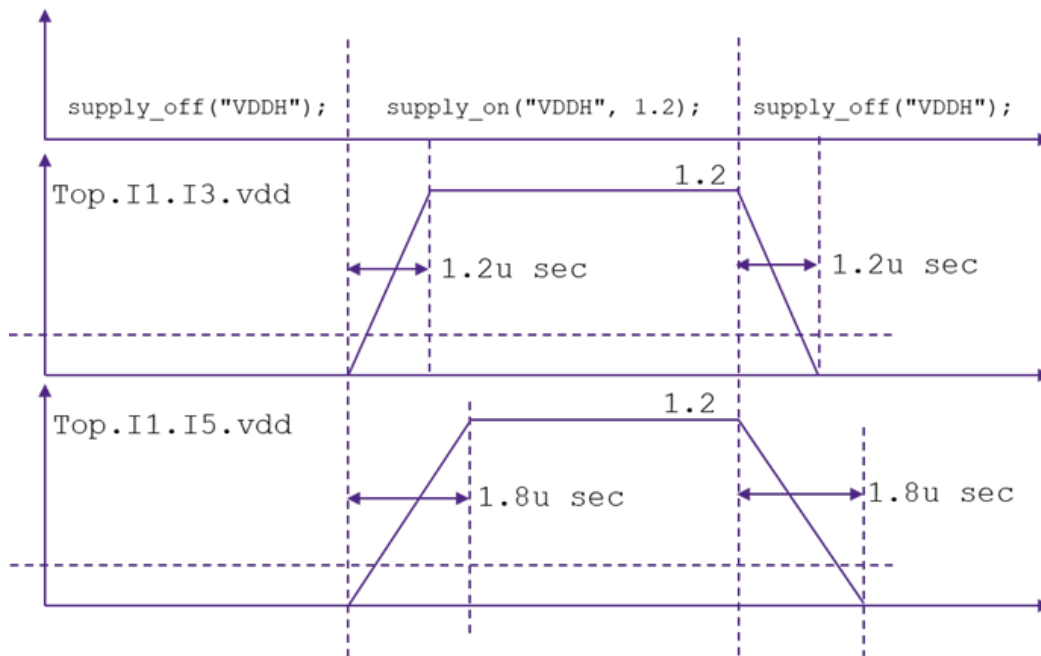
```
u2e -node top.i1.vdd -gain 2
```

In this example, the nets `Top.I1.I3.vdd` and `Top.I1.I5.vdd` are driven by the UPF supply net `VDDH`, so you set the following commands:

```
u2e -node Top.I1.I3.vdd -rise_time_rate 1.0u -fall_time_rate 1.0u
u2e -node Top.I1.I5.vdd -rise_time_rate 1.5u -fall_time_rate 1.5u
```

The output of the u2e interface elements placed by the tool results in the following:

Figure 36 Example of `rise_time_rate` and `fall_time_rate`



As shown in the preceding figure:

- With the rise time rate set to 1.0us/V, it takes 1.2us for net `Top.I1.I3.vdd` to rise to 1.2V.
- With the fall time rate set to 1.0us/V, it takes 1.2us for net `Top.I1.I3.vdd` to fall from 1.2V.
- With the rise time rate set to 1.5us/V, it takes 1.8us for net `Top.I1.I5.vdd` to rise to 1.2V.
- With the fall time rate set to 1.5us/V, it takes 1.8us for net `Top.I1.I5.vdd` to fall from 1.2V.

udn_bidir

Description

Controls the behavior of an `udn_bidir` interface element. This command sets a Verilog-A module for a specified user-defined nettype on a specific node.

Syntax

```
udn_bidir
  [-type value]
  [-min_delta value]
  [-max_delta value]
  [-module value]
  [-library library_name]
  [-cell cell_name -port port_name] |
  [-inst inst_name -port port_name] | [-node hierarchical_node_name]
```

Arguments

`-type value`

Specifies the user-defined nettype for the interface element.

`-min_delta value`

Specifies the absolute current threshold value for bidir events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of possible maximum value of bidir, that stabilizes after several bidir conversions. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-max_delta value`

Sets the voltage difference between two bidir events to be less than or equal to the specified value, which must be greater than 0. The `-max_delta` option

does not disable the `-min_delta` option. Therefore it is recommended to use both the arguments together for precise control over bidir filtering. For example, if you specify only `-max_delta`, the default `-min_delta` (1% of possible maximum value of bidir) is still applied. And if `-max_delta` is less than the default `-min_delta`, then the `-min_delta` takes precedence, and `-max_delta` is ignored. Note that if you specify a very small `-max_delta` value it can slow down a simulation. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-module value`

Specifies the module name of the Verilog-A model.

`-library library_name`

Specifies a `library_name` to apply global `udn_bidir` threshold options. Use this option to specify unique threshold options for all cells in the library. For more information, see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specify the cell and port names for the interface element. The `cell_name` must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

`-inst inst_name -port port_name`

Specifies the hierarchical path to the instance port to apply the voltage value to the ports referring to the target instance. You can specify the asterisk (*) wildcard character in an instance name.

`-node hierarchical_node_name`

Specifies the hierarchical node name for the interface element. You can specify the asterisk (*) wildcard character in a node name.

udn_e2n

Description

Controls the behavior of a `udn_e2n` interface element. This command sets a Verilog-A module for a specified user-defined nettype on a specific node.

Syntax

```
udn_e2n
[-type value]
[-min_delta value]
[-max_delta value]
[-module value]
[-library library_name]
```

```
[-cell cell_name -port port_name] |  
[-inst inst_name -port port_name] [-node hierarchical_node_name]
```

Arguments

-type value

Specifies the user-defined nettype for the interface element.

-min_delta value

Specifies the absolute current threshold value for e2n events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of possible maximum value of e2n, that stabilizes after several e2n conversions. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

-max_delta value

Sets the voltage difference between two e2n events to be less than or equal to the specified value, which must be greater than 0. The *-max_delta* option does not disable the *-min_delta* option. Therefore it is recommended to use both the arguments together for precise control over e2n filtering. For example, if you specify only *-max_delta*, the default *-min_delta* (1% of possible maximum value of e2n) is still applied. And if *-max_delta* is less than the default *-min_delta*, then the *-min_delta* takes precedence, and *-max_delta* is ignored. Note that if you specify a very small *-max_delta* value it can slow down a simulation. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

-module value

Specifies the module name of the Verilog-A model.

-library library_name

Specifies a *library_name* to apply global udn_e2n threshold options. Use this option to specify unique threshold options for all cells in the specified library. For more information, see [Using Library-Based Interface Element Options](#).

-cell cell_name -port port_name

Specifies the cell and port names for the interface element. The *cell_name* must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

-inst inst_name -port port_name

Specifies the hierarchical path to the instance port to apply the voltage value to the ports referring to the target instance. You can specify the asterisk (*) wildcard character in an instance name.

`-node hierarchical_node_name`

Specifies the hierarchical node name. You can specify the asterisk (*) wildcard character in a node name.

udn_n2e

Description

Controls the behavior of an `udn_n2e` interface element. This command sets a Verilog-A module for a specified user-defined nettype on a specific node.

Syntax

```
udn_n2e
  [-type value]
  [-min_delta value]
  [-module value]
  [-library library_name]
  [-cell cell_name -port port_name] |
  [-inst inst_name -port port_name] | [-node hierarchical_node_name]
```

Arguments

`-type value`

Specifies the user-defined nettype for the interface element.

`-min_delta value`

Specifies the absolute current threshold value for n2e events. The *value* field is specified by using engineering notation, such as 1e-09 and must be nonnegative. When the value is set to 0, nothing is filtered at the boundary. The default is 1% of possible maximum value of n2e, that stabilizes after several n2e conversions. For more information about this argument, see [Controlling the Number of Time Points at the Analog-Digital Boundary](#).

`-module value`

Specifies the module name of the Verilog-A model.

`-library library_name`

Specifies a *library_name* to apply global `udn_n2e` threshold options. Use this option to specify unique threshold options for all cells in the library. For more information, see [Using Library-Based Interface Element Options](#).

`-cell cell_name -port port_name`

Specify the cell and port names for the interface element. The *cell_name* must be the same as the name used in the [use_spice](#), [use_verilog](#), or [use_vhdl](#) commands. You can specify the asterisk (*) wildcard character in a cell name.

```
-inst inst_name -port port_name
```

Specifies the hierarchical path to the instance port to apply the voltage value to the ports referring to the target instance. You can specify the asterisk (*) wildcard character in an instance name.

```
-node hierarchical_node_name
```

Specifies the hierarchical node name for the interface element. You can specify the asterisk (*) wildcard character in a node name.

upf_port_connect

Description

When used in an analog-top configuration where a digital block with power and ground network defined by UPF must be connected to an external SPICE block (the SPICE supplies, either local or global nets, must be connected to the power and ground network defined in the UPF file), the `upf_port_connect` command:

- Connects the supplies generated from SPICE to the UPF-created supply ports by placing `e2u` interface elements.
- Connects the supplies generated from UPF to an external SPICE block by placing `u2e` interface elements.

Syntax

```
upf_port_connect -cell cell_name  
  ([-inst inst_name] upf_port_name => hier_node_name, ... )
```

Arguments

```
-cell cell_name
```

Specifies the digital cell name which must be the same as the name used in the `use_verilog` or `use_vhdl` commands.

```
-inst inst_name
```

Specifies the full hierarchical instance path. If this option is used, only the specified instance is used for the port connection. The full hierarchical path to the instance is required.

```
upf_port_name
```

Specifies the UPF-created supply port.

```
hier_node_name, ...
```

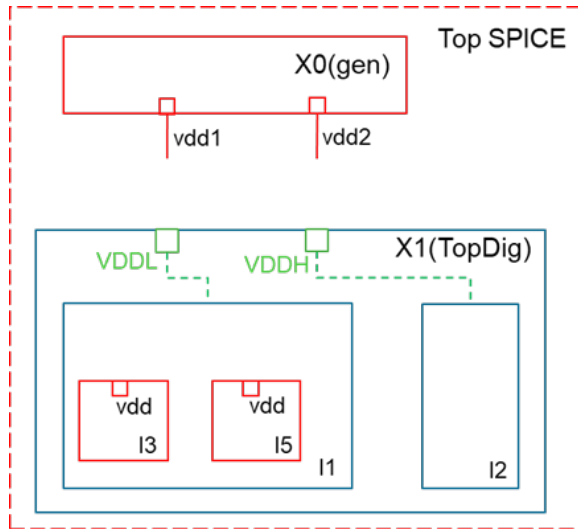
Specifies the name of SPICE net that is connected to the UPF-created supply port.

Examples

Consider the scenario represented in [Figure 37](#) where:

- vdd1 and vdd2 are SPICE nets
- VDDL and VDDH are UPF supply ports

Figure 37 Before Using upf_port_connect

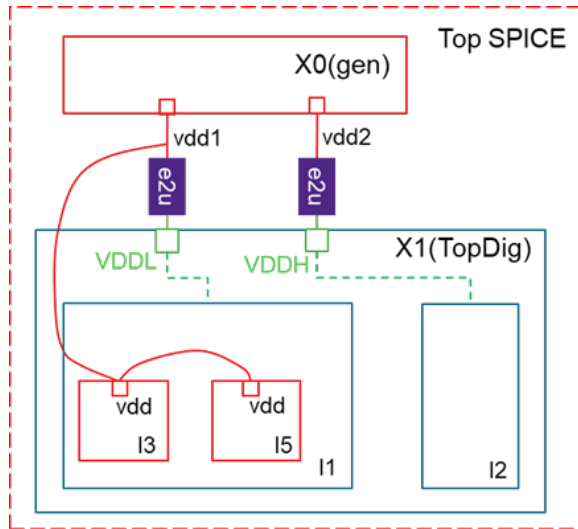


The following command specifies the connections of vdd1 to VDDL and vdd2 to VDDH:

```
upf_port_connect -cell TopDig (VDDL => vdd1, VDDH => vdd2)
```

Assuming that vdd1 and vdd2 are driving VDDL and VDDH, the tool places two `e2u` interface elements as shown in [Figure 38](#). Moreover, if this command is used in combination with the VCS option `-power ams_tunnel_thru_upf`, the supplies of SPICE blocks X1.I1.I3 and X1.I1.I5 are directly connected to vdd1 through tunneling as shown in [Figure 38](#).

Figure 38 After Using upf_port_connect



upgrade_to_error

Changes a warning message to an error message.

Syntax

```
upgrade_to_error -id message_ID [message_ID];
```

Description

You can upgrade warning messages to error messages for the [MSV-IE-OPTTNE], [MSV-RTIE-OPT-TNE] and [MSV-MV-PND-POD] message identifiers.

The [MSV-IE-OPT-TNE] message identifier applies to the following error/warning message,

[MSV-IE-OPT-TNE]: Interface Element command, Option Target Not Found

It applies to the following interface element command arguments:

- -node incorrect path/node name
- -cell incorrect cell name
- -port incorrect port name
- -inst incorrect path or instance name
- -vdd incorrect path or vdd name
- -vss incorrect path or vss name

Note:

The `-vdd` and `-vss` arguments do not apply to the runtime interface element commands.

The `[MSV-RTIE-OPT-TNF]` message identifier applies to the following message:

```
[MSV-RTIE-OPT-TNF]: Runtime Interface Element command, Option Target Not Found
```

It applies to the following runtime interface element command arguments:

- `-node` incorrect path or node name
- `-cell` incorrect cell name
- `-port` incorrect port name
- `-inst` incorrect path or instance name

Examples

The following example has this entry in the `vcsAD.init` file.

```
d2a -hiv 1.8 -lov 0.0 -node top.fail;
```

If this command generates the following warning message:

```
Warning: [MSV-IE-OPT-TNF]: Interface Element command, Option Target Not Found.  
In the "vcsAD.init" file line no. 2, "d2a" command, option target  
"node=top.fail" was not found, the command will be ignored.  
To resolve the problem please modify the command to contain the  
correct "node=<path to node>.<node name>" for the Interface Element.
```

You can upgrade this type of warning message to an error message with:

```
upgrade_to_error -id MSV-IE-OPT-TNF
```

use_spice

Uses the SPICE view of a multi-view cell for a child cell under a Verilog or VHDL parent.

Syntax

```
use_spice -cell subcircuit_name  
[-inst instance_name ]  
[port_map (port_map_list)]  
[port_index_order (port_name_list)];
```

Arguments

`-cell subcircuit_name`

Specifies the cell to replace with a SPICE view. The substitution is done on a cell basis for all instances of the cell. The asterisk (*) wildcard character can be used as part of the subcircuit name.

`-inst instance_name`

Specifies the instance to replace with the SPICE view. The [Full Hierarchical Path](#) to the instance is required. If you use the VCS `-adopt wildcard wildcard` command-line switch, the instance name can also contain a wildcard (*) character. Note that using the VCS `-adopt wildcard` switch and specifying a wildcard (*) character in the instance name increases the compilation time by

- Forcing the tool to elaborate the design one time so all the hierarchical instance names in the design are known, then
- Applying the `use_spice` command with the wildcard (*) character in the instance name and elaborating the design for a second time

`port_map (port_map_list)`

Specifies the mapping between Verilog port names and SPICE port names as follows:

- Map the Verilog/VHDL port name to SPICE. Use this option when the ports in the SPICE view of a multi-view cell have a different name compared to their Verilog or VHDL view. The `port_map_list` can have one or more of the following mappings, each separated by a comma (,):

```
verilog/vhdl_port_name => spice_port_name
```

- Map the Verilog/VHDL port name to SPICE using this alternate syntax for single dimensional buses. The port name could also be a bus identifier.

```
verilog/vhdl_port_name => spice_port_name[range]
```

If you do not specify a range, the members with the same index number are mapped together:

```
verilog/vhdl_port_name[0] => spice_port_name[0]  
verilog/vhdl_port_name[1] => spice_port_name[1]  
...
```

But if you specify the SPICE bus range, the leftmost member of the Verilog/VHDL bus is mapped to the leftmost member of the SPICE port, and so on. You can specify the range for SPICE bus to, for example, flip the bus order in SPICE so that SPICE MSB maps to Verilog/VHDL LSB and vice versa.

- Create an open connection for a Verilog/VHDL port without a corresponding SPICE port. The top-level Verilog/VHDL net connected to that port remain open/dangling.

```
verilog/vhdl_port_name => snps_open
```

- Map all Verilog/VHDL ports (or all remaining Verilog/VHDL ports that have not been mapped yet) to SPICE ports by position. The first port declared in Verilog/VHDL is mapped to the first port declared in the SPICE subcircuit, and so on.

```
* => snps_by_position
```

- Map all Verilog/VHDL ports (or all remaining Verilog/VHDL ports that have not been mapped yet) to SPICE ports by name. Ports that have the same name in SPICE and Verilog/VHDL are mapped together.

```
*=> snps_by_name
```

- Create an open connection for all Verilog/VHDL ports (or all remaining unmapped Verilog/VHDL ports). The top-level Verilog/VHDL nets connecting to those Verilog ports stay open/dangling when the Verilog/VHDL view of the cell is substituted with the SPICE view.

```
* => snps_open
```

```
port_index_order (port_name_list)
```

Set the order of the port indexes to make them continuous. When the indexes of the SPICE ports of a single-view cell are not continuous (they are intermingled) you can use this argument to change the ordering.

Note:

This argument only works with a single SPICE view of a cell.

Description

By default, mixed-signal simulation chooses the view for the child cell that is the same as the parent cell view. For example, if the parent of a multi-view cell uses a Verilog view, the same Verilog view is picked for the child cell by default. Use this command to override the default view selection and explicitly select the SPICE view for a given cell or instance of a cell.

Note:

If the `use_spice` command targets a cell under a VHDL parent, the Verilog view of the SPICE must also be available, otherwise, the `use_spice` command does not take effect. The Verilog view of the SPICE can be an empty module (no content) as long as it contains port definitions. Such a Verilog view for SPICE can be generated by the `spicean` utility.

Also, note that an instance-based `use_spice` command does work for an instance inside a `generate/endgenerate` block and instance array. For example:

```
genvar i;
generate
  for(i=0; i<=3; i=i+1) begin : genb
    inv1 nd (sb[i], s[i]);
  end
endgenerate

module inv1 (output y, input a);
...
endmodule
```

The following command targets only one instance to SPICE, the third one with the `2` field.

```
use_spice -cell inv1 -inst top.genb[2].nd;
```

Examples

The following examples illustrate different uses of the `use_spice` command:

- Use the SPICE view for all instances of the `memory` cell. This command can be used when both the `memory` Verilog module and the `memory` SPICE subcircuit are available.

```
use_spice -cell memory;
```

- Use the SPICE view for the `inv2` cell only for instances `top.i1.u1` and `top.i2.u5.inv5`.

```
use_spice -cell inv2 -inst top.i1.u1 top.i2.u5.inv5 ;
```

- Replace the Verilog view of a multi-view cell with its SPICE counterpart and map the Verilog ports to SPICE ports when they have different names.

```
module inv1 (y, a);
...
.subckt inv1 i o
...
use_spice -cell inv1 port_map ( y => o, a => i );
```

- Map Verilog ports to SPICE ports when a few of the ports have different names in Verilog and SPICE but the rest have identical names in both views and can be mapped by name.

```
module inv1 (z, n, a, b, c);
...
.subckt inv1 y m a b c
...
use_spice -cell inv1 port_map ( z => y, n => m, * => snps_by_name);
```

- Map Verilog ports to SPICE ports when some Verilog ports have no corresponding port in SPICE, some ports have different names in Verilog and SPICE, and the remaining ports have identical names.

```
module inv1 (z, n, a, b, c);
...
.subckt inv1 y a b c
...
use_spice -cell inv1 port_map ( z => y, n =>snps_open, * =>
    snps_by_name);
```

- Map a Verilog bus to its SPICE counterpart when there is a mismatch between the Verilog and SPICE bus port names:

```
module data_path (data, ...);
input [2:0] data;
...

.subckt data_path d_2 d_1 d_0 ...
...
use_spice -cell data_path port_map ( data => {d_2, d_1, d_0}, *=>
    snps_by_name);
```

Another way to map the bus ports is to use the bus identifiers to map the ports:

```
use_spice -cell data_path port_map (data => d, *=> snps_by_name);
```

Another way is to use bus notation to declare the bus:

```
use_spice -cell data_path port_map (data => d[2:0], *=> snps_by_name);
```

The leftmost member of data is mapped to the leftmost member of d and so on:

```
data[2] => d[2]
data[1] => d[1]
data[0] => d[0]
```

Or reverse the order as follows:

```
use_spice -cell data_path port_map (data => d[0:2], *=> snps_by_name);
```

Which performs the following mapping:

```
data[2] => d[0]
data[1] => d[1]
data[0] => d[2]
```

- Specify the instance name by using the wildcard (*) character:

```
use_spice -cell ph_det -inst test.i1.*.u1;
use_spice -cell serdes -inst test.idata1.i_s*;
use_spice -cell div -inst test.i3.u2.*;
```

The wildcard (*) character in the instance name is only supported if the `-adopt wildcard` switch is added to the VCS command line as follows:

```
vcs -ad -adopt wildcard ...
```

Note that all ports must be mapped. That is why the wildcard (*) character in the `*=>snps_by_name` port map command is required to map any port that is not explicitly mapped.

Note that the instance name (under the SPICE view) begins with the letter `x`. To ensure that you are using the correct hierarchical path to the instance, use the hierarchical names listed in the `interface_element.rpt` file under the `simv.msv` directory as a guide.

- Replace a Verilog module with a SPICE subcircuit of a different name:

```
use_spice -cell Verilog_module_name:SPICE_subcircuit_name [-inst
  instance_name ] [port_map (port_map_list) ] ;
```

You can apply the `use_spice` command in the same way as when the names of the two views are the same.

- Replace the Verilog module `CPU` with a SPICE subcircuit `SPICE_CPU`.

```
use_spice -cell CPU:SPICE_CPU;
```

- Replace the module `CPU` with a SPICE subcircuit `SPICE_CPU` in instance `blk1`, but replaces module `CPU` with SPICE subcircuit `SPICE_CPU2` in instance `blk2`.

```
use_spice -cell CPU:SPICE_CPU -inst top.dut.blk1;
use_spice -cell CPU:SPICE_CPU2 -inst top.dut.blk2;
```

- Use the `port_index_order` argument to fix the order of the indexes of the SPICE ports for a single-view cell.

```
.subckt addr4 s_3 a_2 a_1 a_0
+b_3 b_2 b_1 b_0 cin
+a_3 s_2 s_1 s_0 cout
...
```

These ports are discontinuous; the ports `s` and `a` are intermingled such that `s_3` appears in place of `a_3`.

```
use_spice -cell addr4 port_index_order (*=>same, a=>dec, s=>inc);
```

In this example, the `port_index_order` argument specifies the order of the indexes of the ports of `addr4` as follows: from left to right, port `a` has the indexes arranged in decreasing order (`a_3 a_2 a_1 a_0`). Port `s` appears in the increasing order (`s_0 s_1 s_2 s_3`) while the remaining ports follow the original order.

use_verilog

Use the `use_verilog` command to use the Verilog view of a cell with multiple views for a child cell under a SPICE parent.

Syntax

```
use_verilog| use_vcs
-cell module_name:logical_library.module_name |
-module module_name:logical_library.module_name
[-inst instance_name]
[-exclude instance_name]
[port_map (port_map_list)];
```

Arguments

`-cell module_name:logical_library.module_name`

Replaces a SPICE subcircuit with a Verilog module. The module name cannot contain an asterisk wildcard (*) character. You can also select a module of a specific library. The `-cell` and `-module` options perform the same function.

`-module module_name:logical_library.module_name`

Replaces a SPICE subcircuit with a Verilog module. The module name cannot contain an asterisk wildcard (*) character. You can also select a module of a specific library. The `-cell` and `-module` options perform the same function.

`-inst instance_name`

Instantiates only the specified Verilog instances under SPICE. You must specify the [Full Hierarchical Path](#) to the instance. If the instance name contains special characters, you must escape the characters by inserting a backslash character (\) and terminating the escaped string with a space as follows:

```
// WRONG!
use_verilog -cell monitor -inst i1<1>.i2;

// Correct
use_verilog -cell monitor -inst \i1<1> .i2;
```

`-exclude instance_name`

Excludes from partition the specified Verilog instances under SPICE. You must specify the [Full Hierarchical Path](#) to the instance. If the instance name contains special characters, you must escape the characters by inserting a backslash character (\) and terminating the escaped string with a space as shown in the `-inst` argument.

```
port_map (port_map_list)
```

Specifies the mapping between port names in SPICE and Verilog as follows:

- Map the SPICE name to the Verilog name. Use this option when the ports in the Verilog view of a multi-view cell have a different name compared to their SPICE view. The `port_map_list` can have one or more of the following mappings, each separated by a comma (,):

```
spice_port_name => verilog_port_name
```

- Declare a SPICE port without a corresponding Verilog port. The top-level SPICE net connected to that port remains open and dangling.

```
spice_port_name => snps_open
```

- Map all SPICE ports (or all remaining SPICE ports that have not been mapped yet) to Verilog ports by position. The first port declared in the subckt definition is mapped to the first port declared in Verilog module definition, and so on.

```
* => snps_by_position
```

- Map all SPICE ports (or all remaining SPICE ports that have not been mapped yet) to Verilog ports by name. Ports that have the same name in SPICE and Verilog are mapped together.

```
*=> snps_by_name
```

- Disconnect all SPICE ports (or all remaining SPICE ports that have not been mapped yet). All top-level SPICE nets that connect to those SPICE ports stay open and dangling when the SPICE view of the cell is replaced with the Verilog view.

```
* => snps_open
```

Description

This command explicitly selects the Verilog view for the specified cell or instance of a cell. By default, mixed-signal simulation chooses the view for the child cell that is the same as the parent cell view. For example, if the parent of a multi-view cell uses a SPICE view, the same SPICE view is picked for the child cell by default.

Support for Wildcard Character and the -exclude Option

The `-inst` option in the `use_verilog` command specifies the instances to use the Verilog view. The wildcard (*) character is supported only in the `use_spice` command and not in the `use_verilog` command.

The `-exclude` option is supported in the `use_verilog` command, which can be used to provide the instances that must not be partitioned. Also, the wildcard (*) character is

supported in the hierarchical path of the `-inst` and `-exclude` options in the `use_verilog` mixed-signal control command. This feature is useful when there are large number of instances in the design.

The syntax of the `-exclude` option with the wildcard character is as follows:

```
use_verilog -module inverter -inst top.*.* -exclude top.a0.*;
```

Note:

The wildcard (*) character in the instance name is only supported if the `-adopt wildcard` option is added to the VCS command line as follows:

```
% vcs -adopt wildcard ...
```

For example:

```
% vcs -sverilog example.v -ad example.init -adopt wildcard -full64
```

Note:

Using the `-adopt wildcard` option and the wildcard (*) character in the `use_verilog` command increases (sometimes almost doubles) the compilation time.

Examples

The following example illustrates usage of `-exclude` option in the `use_verilog` command.

```
* example.spi

subckt inv i zn
xi0 i zn inverter
.ends
.subckt inverter in out
mn1 out in gnd gnd n w=1u l=1u
mp1 out in vdd vdd p w=4u l=1u
,ends

// vcsAD.init

choose xa -hspice example.spi;

use_spice -cell inv;
use_verilog -module inverter -inst top.*.* -exclude top.a0.i0;
resolve_x_inst_prefix enable;
```

This feature has the following limitations:

- A wildcard character is restricted to only one layer of design hierarchy. For example, if `top.*.b1` is searched in the following design hierarchies:

```
top.a1.b1
top.p0.a1.b1
```

Then the wildcard character is applied only to the design `top.a1.b1`.

use_veriloga

Use the `use_veriloga` command to explicitly instruct the tool to use the Verilog-A view of a multi-view cell for a child cell under a Verilog parent.

Syntax

```
use_veriloga -cell module_name | -module module_name
[-inst instance_name]
[port_map (port_list)];
```

Arguments

`-cell module_name`

Specifies the cell to replace with a Verilog-A view. The substitution is done on a cell basis for all instances of the cell.

`-module module_name`

Specifies the cell to replace with a Verilog-A view. The substitution is done on a cell basis for all instances of the cell.

`-inst instance_name`

If you use this argument, only the specified Verilog-A instances are instantiated under SPICE. You must specify the [Full Hierarchical Path](#) to the instance.

`port_map (port_map_list)`

Specifies the mapping between Verilog port names and Verilog-A port names as follows:

- Map the Verilog/VHDL port name to Verilog-A. Use this option when the ports in the Verilog-A view of a multi-view cell have a different name compared to their Verilog or VHDL view. The `port_map_list` can have one or more of the following mappings, each separated by a comma (,):

```
verilog/vhdl_port_name => veriloga_port_name
```

- Map the Verilog/VHDL port name to Verilog-A using this alternate syntax for single dimensional buses. The port name could also be a bus identifier.

```
verilog/vhdl_port_name => veriloga_port_name[range]
```

If you do not specify a range, the members with the same index number are mapped together:

```
verilog/vhdl_port_name[0] => veriloga_port_name[0]
verilog/vhdl_port_name[1] => veriloga_port_name[1]
...
```

But if you specify the Verilog-A bus range, the leftmost member of the Verilog/VHDL bus is mapped to the leftmost member of the Verilog-A port, and so on. You can specify the range for Verilog-A bus to, for example, flip the bus order in Verilog-A so that Verilog-A MSB maps to Verilog/VHDL LSB and vice versa.

- Create an open connection for a Verilog/VHDL port without a corresponding Verilog-A port. The top-level Verilog/VHDL net connected to that port remain open/dangling.

```
verilog/vhdl_port_name => snps_open
```

- Map all Verilog/VHDL ports (or all remaining Verilog/VHDL ports that have not been mapped yet) to Verilog-A ports by position. The first port declared in Verilog/VHDL is mapped to the first port declared in the Verilog-A subcircuit, and so on.

```
* => snps_by_position
```

- Map all Verilog/VHDL ports (or all remaining Verilog/VHDL ports that have not been mapped yet) to Verilog-A ports by name. Ports that have the same name in Verilog-A and Verilog/VHDL are mapped together.

```
*=> snps_by_name
```

- Create an open connection for all Verilog/VHDL ports (or all remaining unmapped Verilog/VHDL ports). The top-level Verilog/VHDL nets connecting to those Verilog ports stay open/dangling when the Verilog/VHDL view of the cell is substituted with the Verilog-A view.

```
* => snps_open
```

Description

By default, mixed-signal simulation chooses the view for the child cell that is the same as the parent cell view. For example, if the parent of a multi-view cell uses a Verilog view, the same Verilog view is picked for the child cell by default. Use this command to override the default behavior and explicitly select the Verilog-A view for the given cell or instance of a cell.

Note:

Also, note that an instance-based `use_veriloga` command does work for an instance inside a `generate/endgenerate` block and instance array. For example:

```
genvar j;
generate
  for(j=0; j<=3; j=j+1) begin : genva
    invva nc (sc[j], sb[j]);
  end
endgenerate

module invva (output y, input a);
...
endmodule
```

The following command targets only one instance to Verilog-A, the third one with the 2 field.

```
use_veriloga -cell invva -inst top.genva[2].nc;
```

Examples

The following examples illustrate different uses of the `use_veriloga` command:

- Use the Verilog-A view for all instances of the `memory` cell. This command can be used when both the `memory` Verilog module and the `memory` Verilog-A subcircuit are available.

```
use_veriloga -cell memory;
```

- Use the Verilog-A view for the `inv2` cell only for instances `top.i1.u1` and `top.i2.u5.inv5`.

```
use_veriloga -cell inv2 -inst top.i1.u1 top.i2.u5.inv5 ;
```

- Replace the Verilog view of a multi-view cell with its Verilog-A counterpart and map the Verilog ports to Verilog-A ports when they have different names.

```
module inv1 (y, a);
...
.subckt inv1 i o
...
use_veriloga -cell inv1 port_map ( y => o, a => i );
```

- Map Verilog ports to Verilog-A ports when a few of the ports have different names in Verilog and Verilog-A but the rest have identical names in both views and can be mapped by name.

```
module inv1 (z, n, a, b, c);
...

```

Appendix A: Mixed-Signal Control Commands

use_veriloga

```
.subckt inv1 y m a b c
...
use_veriloga -cell inv1 port_map ( z => y, n =>m, * => snps_by_name);
```

- Map Verilog ports to Verilog-A ports when some Verilog ports have no corresponding port in Verilog-A, some ports have different names in Verilog and Verilog-A, and the remaining ports have identical names.

```
module inv1 (z, n, a, b, c);
...
.subckt inv1 y a b c
...
use_veriloga -cell inv1 port_map ( z => y, n =>snps_open, * =>
    snps_by_name);
```

- Map a Verilog bus to its Verilog-A counterpart when there is a mismatch between the Verilog and Verilog-A bus port names:

```
module data_path (data, ...);
input [2:0] data;
...

.subckt data_path d_2 d_1 d_0 ...
...
use_veriloga -cell data_path port_map ( data => {d_2, d_1, d_0}, *=>
    snps_by_name);
```

Another way to map the bus ports is to use the bus identifiers to map the ports:

```
use_veriloga -cell data_path port_map (data => d, *=> snps_by_name);
```

Another way is to use bus notation to declare the bus:

```
use_veriloga -cell data_path port_map (data => d[2:0], *=>
    snps_by_name);
```

The leftmost member of data is mapped to the leftmost member of d and so on:

```
data[2] => d[2]
data[1] => d[1]
data[0] => d[0]
```

Or reverse the order as follows:

```
use_veriloga -cell data_path port_map (data => d[0:2], *=>
    snps_by_name);
```

Which performs the following mapping:

```
data[2] => d[0]
data[1] => d[1]
data[0] => d[2]
```

- Replace a Verilog module with a Verilog-A subcircuit of a different name:

```
use_veriloga -cell Verilog_module_name:SPICE_subcircuit_name [-inst
instance_name ] [port_map (port_map_list) ] ;
```

You can apply the `use_veriloga` command in the same way as when the names of the two views are the same.

- Replace the Verilog module `CPU` with a Verilog-A subcircuit `VA_CPU`.

```
use_veriloga -cell CPU:VA_CPU;
```

- Replace the module `CPU` with a Verilog-A subcircuit `VA_CPU` in instance `blk1`, but replaces module `CPU` with Verilog-A subcircuit `VA_CPU2` in instance `blk2`.

```
use_veriloga -cell CPU:VA_CPU -inst top.dut.blk1;
use_veriloga -cell CPU:VA_CPU2 -inst top.dut.blk2;
```

use_vhdl

Description

The `use_vhdl` command selects the VHDL view of a multi-view cell instantiated in a SPICE block.

Note:

This command applies only to VHDL/Verilog-SPICE flow.

Syntax

```
use_vhdl -cell cell_name
[-inst instance_name]
[port_map (port_map_list)];
```

Where *cell_name*::=

```
entity_name |
entity_name:library_name.entity_name |
entity_name:entity_name(architecture_name) |
entity_name:library_name.entity_name(architecture_name) |
entity_name:configuration_name |
entity_name:library_name.configuration_name
```

Arguments

`-cell cell_name`

Replaces a SPICE subcircuit with a VHDL cell. The entity name cannot contain a wildcard (*) character. You can also select a cell of a:

- Specific library
- Specific architecture
- VHDL architecture from a specific library
- VHDL configuration name
- VHDL configuration from a specific library

`-inst instance_name`

If this option is used, only the specified instances are instantiated in VHDL. The [Full Hierarchical Path](#) to the instance is required.

If the instance name contains special characters, you must escape the characters by inserting a backslash character (\) and terminating the escaped string with a space as follows:

```
// WRONG!
use_vhdl -module monitor -inst i1<1>.i2;

// Correct
use_vhdl -module monitor -inst \i1<1> .i2;
```

`port_map (port_map_list)`

Specifies the mapping between port names in SPICE and VHDL as follows:

- Map the SPICE port name to the VHDL port name. Use this option when the ports in the VHDL view of a multi-view cell have a different name compared to their SPICE view. The `port_map_list` can have one or more of the following mappings, each separated by a comma (,):

```
spice_port_name => vhdl_port_name
```

- Declare a SPICE port without a corresponding VHDL port. The top-level SPICE net connected to that port remains open and dangling.

```
spice_port_name => snps_open
```

- Map all SPICE ports (or all remaining SPICE ports that have not been mapped yet) to VHDL ports by position. The first port declared in the subckt definition is mapped to the first port declared in VHDL module definition, and so on.

```
* => snps_by_position
```

- Map all SPICE ports (or all remaining SPICE ports that have not been mapped yet) to VHDL ports by name. Ports that have the same name in SPICE and VHDL are mapped together.

```
*=> snps_by_name
```

- Disconnects all SPICE ports (or all remaining SPICE ports that have not been mapped yet). All top-level SPICE nets that connect to those SPICE ports stay open and dangling when the SPICE view of the cell is replaced with the VHDL view.

```
* => snps_open
```

Examples

- Use the VHDL view for inv cell instances `inv2`, `xi1.xu1`, and `xi2.xu5.xinv5`.

```
use_vhdl -cell inv2 -inst xi1.xu1 xi2.xu5.xinv5;
```

- Map SPICE ports `o` and `i` to VHDL ports `y` and `a`, respectively.

```
.subckt inv1 i o
...
```

```
entity inv1 is port (y, a : out STD_LOGIC);
...
```

```
use_vhdl -cell inv1 port_map ( o => y, i => a );
```

- Map SPICE ports to VHDL ports when a few of the ports have different names in VHDL and SPICE but the rest have identical names in both views and can be mapped by name. Note that all ports must be mapped, which is why the wildcard (*) character in `*=> snps_by_name` is required to map any port that is not explicitly mapped.

```
.subckt inv1 y m a b c
...
```

```
entity inv1 is port (z, n, a, b, c : out STD_LOGIC);
...
```

```
use_vhdl -cell inv1 port_map ( y => z, m => n, * => snps_by_name);
```

- Map SPICE ports to VHDL ports when some SPICE ports have no corresponding port in VHDL, a few of the ports have different names in VHDL and SPICE, and the rest have identical names:

```
.subckt inv1 y n a b c
...
```


Appendix A: Mixed-Signal Control Commands

use_vhdl

```
entity inv1 is port (z, a, b, c : out STD_LOGIC);
...
```

```
use_vhdl -cell inv1 port_map ( y => z, n =>snps_open, * =>
  snps_by_name);
```

- Select the library to use when multiples libraries exist. For example, if you have compiled two different VHDL descriptions for the same `invvhd` cell in two different libraries `vhdl_lib` and `vhdl_lib2`:

```
vhdlan -work vhdl_lib -nc vhdl/inv.vhd -full64
vhdlan -work vhdl_lib2 -nc vhdl/inv2.vhd -full64
```

- Specify a cell in a particular library

```
use_vhdl -cell invvhd:vhdl_lib2.invvhd;
```

- Select an architecture if multiple architectures exist in your VHDL description. For example, to choose the `arch2` architecture of the entity `cell1`:

```
architecture arch1 of cell1 is
begin
...
end arch1;
architecture arch2 of cell1 is
begin
...
end arch2;
```

```
use_vhdl -cell cell1:cell1(arch2);
```

- Specify both multiple libraries and multiple architectures.

```
use_vhdl -cell invvhd:vhdl_lib2.invvhd(arch2);
```

- Select the configuration to simulate if you have multiple configurations in your VHDL description. The following example selects the `cg1` configuration of the entity `cell1`:

```
configuration cfg1 of cell1 is
for structural
end for;
end configuration cfg1;
```

```
configuration cfg2 of cell1 is
for structural2
end for;
end configuration cfg2;
```

```
use_vhdl -cell cell1:cfg1;
```

- Select a configuration if you have both multiple libraries and multiple configurations:

```
use_vhdl -cell invvhd:vhdl_lib2.cfg2;
```

B

Using Hierarchical Aliases for Interface Elements in Mixed-Signal Commands

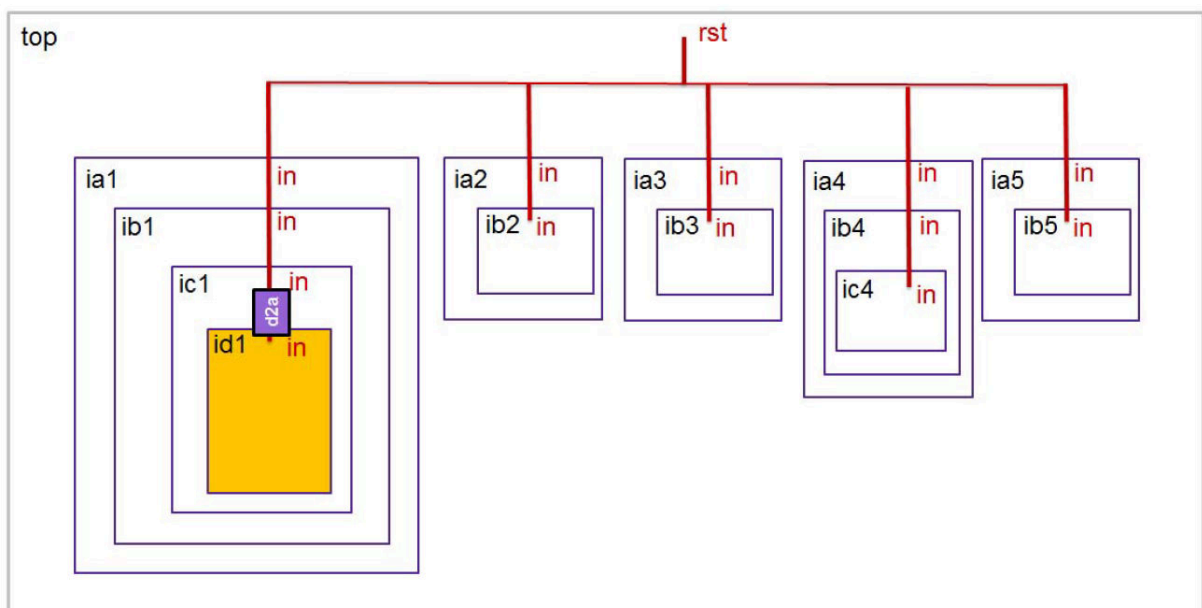
This appendix explains how to use hierarchical aliases for interface elements.

After the tool compiles a mixed-signal design, the tool lists all the interface elements with their hierarchical locations to the `simv.msv/interface_element.rpt` report file. By default, the hierarchical names listed in the `interface_element.rpt` file are the names that must be used in any mixed-signal command. Starting with the N-2017.12 release, in PrimeSim XA and FineSim mixed-signal simulations, you can specify the hierarchical name specified in the `interface_element.rpt` file or any other hierarchical alias of that net in a mixed-signal command.

For example, the mixed-signal design shown in [Figure 39](#) contains the following d2a interface element listed in the `interface_element.rpt` file.

```
d2a -hiv 1.2 -lov 0 -node top.ia1.ib1.ic1.id1.in;
```

Figure 39 Interface Element at the Instantiation of an Analog Block Under a Digital Parent



Starting with the N-2017.12 release of the PrimeSim XA and FineSim tools, you can specify nodes by using any hierarchical alias for the net. For example, any of the following d2a specifications refer to the same net and result in the same d2a interface element:

```
d2a -hiv 1.0 -lov 0.2 -node top.rst;
d2a -hiv 1.0 -lov 0.2 -node top.ia1.in;
d2a -hiv 1.0 -lov 0.2 -node top.ia1.ib1.in;
d2a -hiv 1.0 -lov 0.2 -node top.ia1.ib1.ic1.in;
d2a -hiv 1.0 -lov 0.2 -node top.ia1.ib1.ic1.id1.in;
d2a -hiv 1.0 -lov 0.2 -node top.ia3.ib3.in;
d2a -hiv 1.0 -lov 0.2 -node top.ia5.ib5.in;
```

Limitation

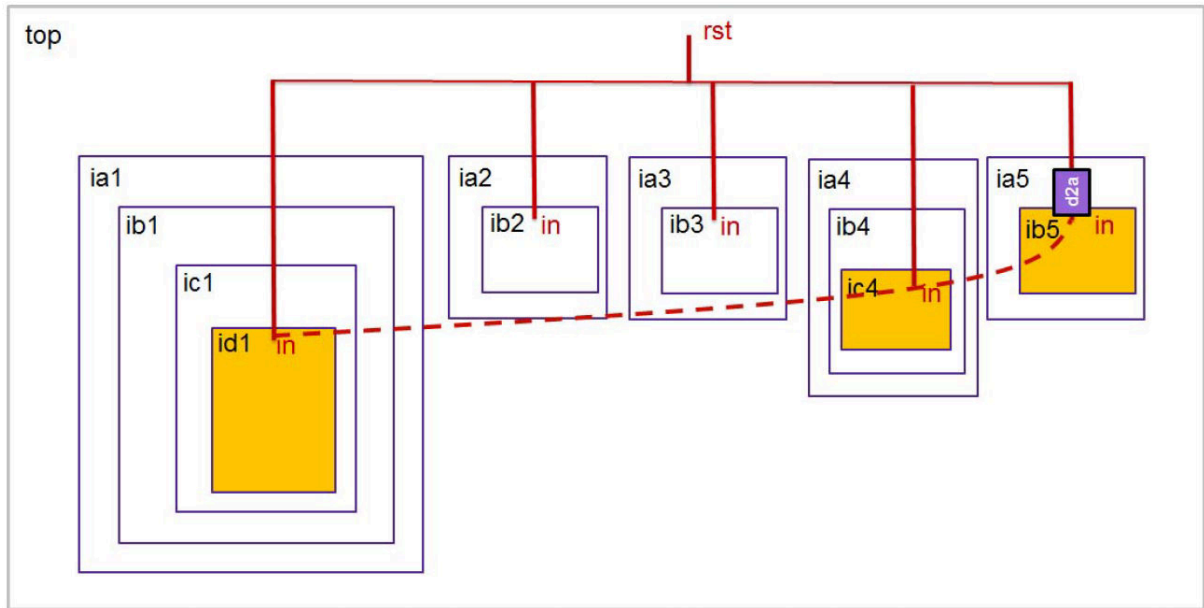
You can use the hierarchical alias of an interface element as long as that net name is directly connected to the interface element through SPICE nets or Verilog wires, and no net splitting is done by the simulator. Nets are split when the connectivity is made through SystemVerilog `logic`, `reg`, VHDL `signal`, or due to UPF level shifters/isolation cells, or SDF back-annotation of a digital net, and so on.

Note:

The same concept and support for all aliases of the interface element in mixed-signal commands apply if the white blocks in [Figure 39](#) represent analog and the yellow block represents digital. Since the tool always puts the interface element in the analog domain, the location of the interface element is `top.ia1.ib1.ic1.in`.

Another example for mixed-signal design hierarchy is shown in [Figure 40](#), which is similar to [Figure 39](#) but uses three analog cell views used instead of one. The `in` ports of the three analog blocks are connected to each other within the parent digital block. As a result, all three analog ports are shorted together by the tool and connect to the same interface element. In this case, the selects picks the port location with the shortest hierarchical path, `top.ia5.ib5.in`, when assigning the node for the interface element. As in [Figure 39](#), you can use `top.ia5.ib5.in`, `top.rst`, `top.ia1.in`, or any other hierarchical alias for the net when creating the interface element by using the `vcsAD.init` mixed-signal simulation control file.

Figure 40 Interface Element at the Instantiation of Three Analog Blocks, all Connected to the Same Net in the Digital Parent



In both [Figure 39](#) and [Figure 40](#), any hierarchical alias can be used instead of `top.ia5.ib5.in` to place the interface element, assuming that the connectivity between those aliases and the interface net `top.ia5.ib5.in` is through Verilog wires or SPICE nets, and those connections are not split because of SDF back-annotation, low power isolation/level shifter cells, or the use of SystemVerilog `logic`, `reg`, or VHDL `signal` nets that automatically split a net.

Note:

This feature is currently available only in the PrimeSim XA simulation engine and is not supported in the FineSim or PrimeSim simulation engines.

C

Mixed-Signal Simulation for Three-Dimensional Integrated Circuits (3DIC)

This appendix describes how the 3DIC methodology allows the integration of different chips in a mixed-signal flow with different technologies, in the same netlist, without modifying the netlist, models, or parameters of the individual chips.

This appendix describes:

- [Introduction to the 3DIC Mixed-Signal Flow](#)
- [How 3DIC Works in the Standalone PrimeSim XA Tool](#)
- [Basic Mixed-Signal 3DIC Implementation](#)
- [Enhanced Mixed-Signal 3DIC Implementation](#)

Introduction to the 3DIC Mixed-Signal Flow

Use the 3DIC methodology to:

- Maintain separate scopes for the different chips.
- Use devices, models and parameters with same names in different chips.
- Define global nets and environmental parameters locally for each chip.
- Maintain separate corners and parametric controls for different chips.

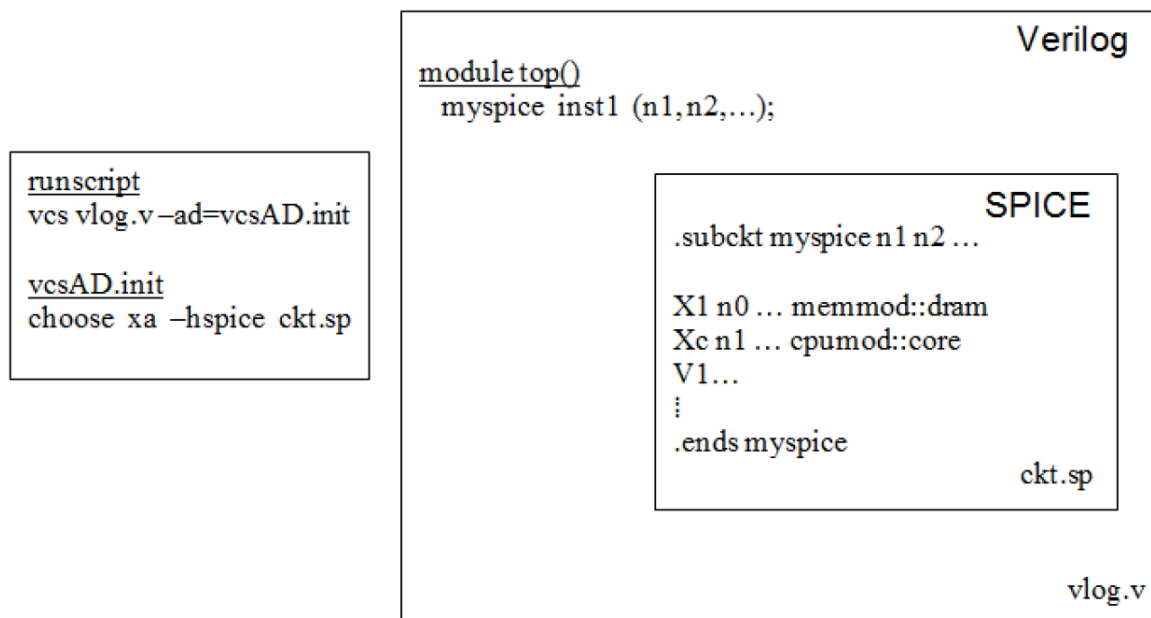
How 3DIC Works in the Standalone PrimeSim XA Tool

For details about how the 3DIC technology works, see Three-Dimensional Integrated Circuit (3DIC) Modularization in the *PrimeSim XA User Guide*.

Basic Mixed-Signal 3DIC Implementation

A netlist containing a 3DIC design can be included in a Verilog-Top mixed-signal simulation as long as the interface between HDL and SPICE does not cross the 3DIC domain boundary. For example, in [Figure 41](#), the instantiation in Verilog is made for the `mypspice` subcircuit, which is defined outside of the scope of the 3DIC module.

Figure 41 SPICE Subcircuit Instantiated in Verilog



Enhanced Mixed-Signal 3DIC Implementation

The enhanced mixed-signal 3DIC implementation allows:

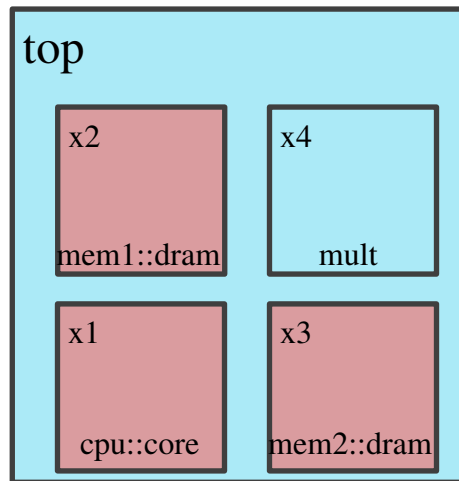
- Including SPICE cells with a 3DIC scope in a mixed-signal simulation without modification of the netlist.
- Cell based references to be distinguishable for their 3DIC scope using mixed-signal commands.

For this implementation, replacements involving 3DIC scope require the cell name, 3DIC scope, and instance.

Specifying a Verilog-Top With SPICE Leaf

[Figure 42](#) show a Verilog-Top design with SPICE leaf instantiations.

Figure 42 Verilog-Top With SPICE Leaf Instantiations



You can select a SPICE subcircuit with a 3DIC scope using the `use_spice` command. To use this command to instantiate a 3DIC SPICE cell, you must specify the cell and icmodule. For example:

```
use_spice -cell core -icmodule cpu -inst top.x1;
```

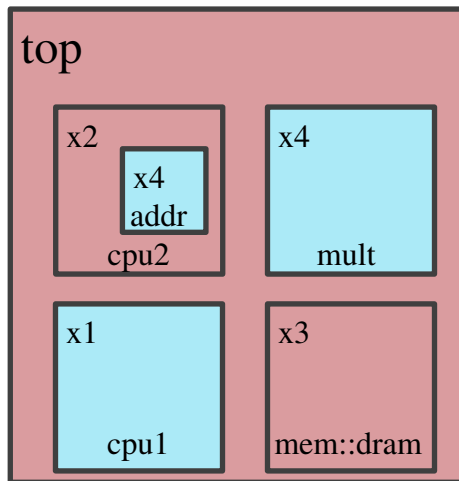
You can replace any cell with a SPICE view with a 3DIC scope:

```
use_spice -cell and_gt -icmodule cpu -inst top.x4.x5.x1;
use_spice -cell dram -icmodule mem1 -inst top.x2;
use_spice -cell dram -icmodule mem2 -inst top.x3;
```

Specifying a SPICE-Top With Verilog or VHDL Leaf

Figure 43 show a SPICE-Top design with Verilog leaf instantiations.

Figure 43 SPICE-Top With HDL Leaf Instantiations



You can select a Verilog or VHDL module with a 3DIC scope using the `use_verilog` or `use_vhdl` command. To use this command to instantiate a 3DIC SPICE cell, you must specify the cell and icmodule. For example:

```
use_vhdl -cell mult;
use_verilog -cell cpu1;
use_verilog -cell addr:VLOG.addr -icmodule cpu2;
//use_vhdl -cell addr:VHDL.addr -icmodule cpu2;
```

Cell-Based Mixed-Signal Commands Affected by 3DIC Scope

The port-related commands such as:

```
d2a -cell cellname -port portname
```

And the `port_dir` and `port_connect` commands are not supported.

Instead, the [a2d](#), [d2a](#), [e2r](#), and [r2e](#) commands at the interface to a 3DIC module are applied by node. For example:

```
d2a -node top.il.n1
```

In order to define the direction of the ports of a SPICE subcircuit instantiated in Verilog, a Verilog module view of that cell needs to be provided.

Support for Verilog-A

Verilog-A instantiated by `.hdl` within the SPICE block is supported.

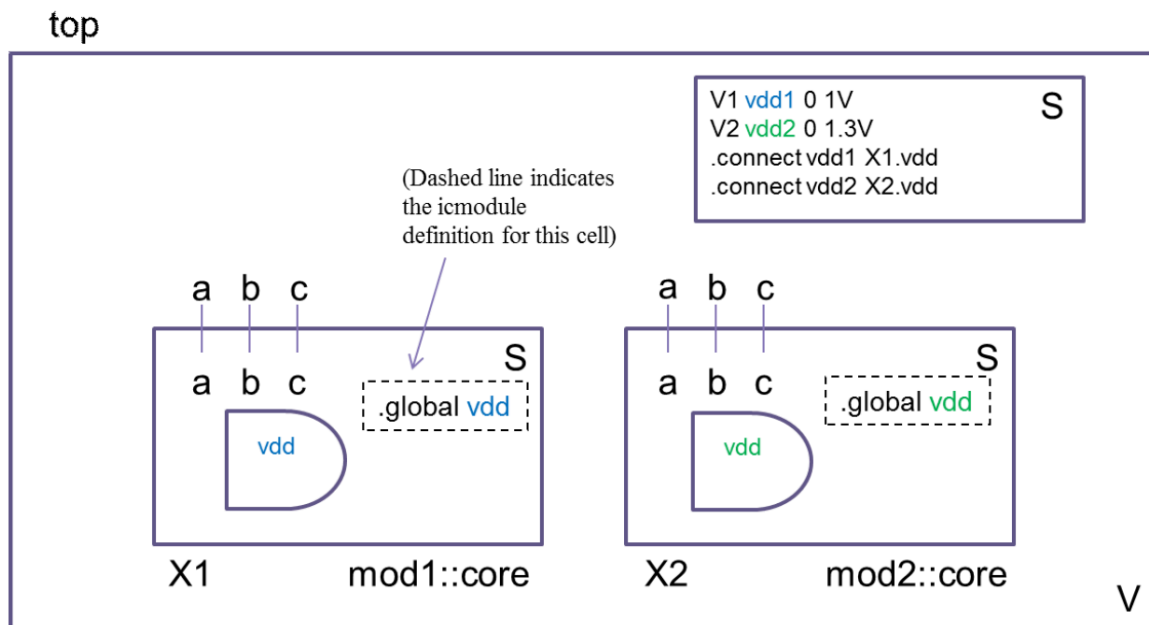
Wildcard Support

Instance-based wildcards that refer to blocks inside a 3DIC module are allowed. For example: `top.x1.ou*`.

Standalone 3DIC Global Supplies

As in standalone 3DIC, global supplies may be defined independently through an instance-based connection. See [Figure 44](#).

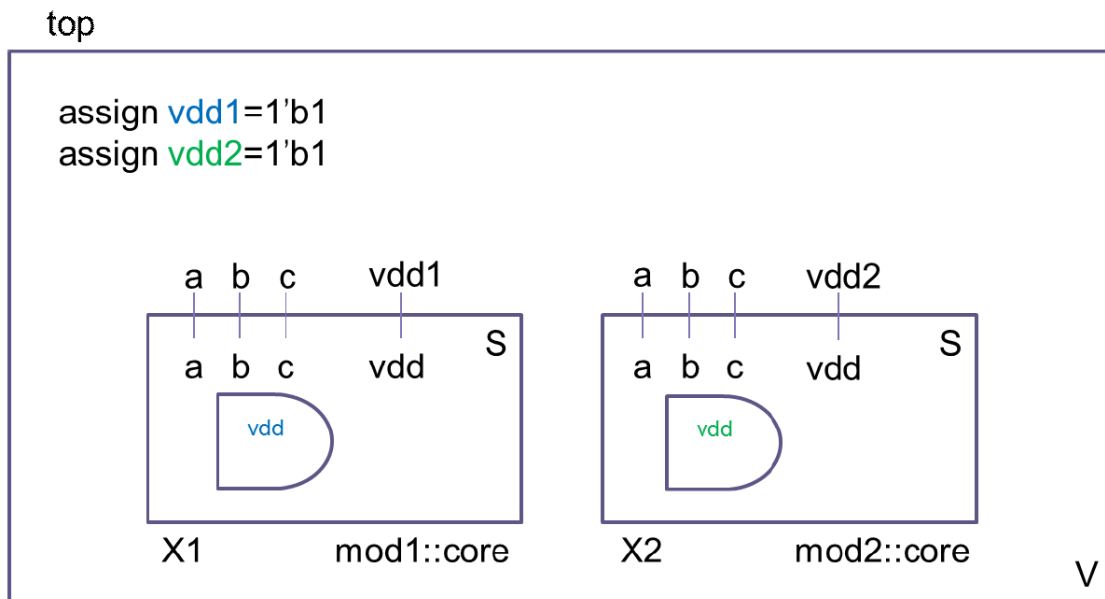
Figure 44 Standalone Global Supplies Example



Direct Supply Connections Through the Mixed-signal Interface

[Figure 45](#) shows direct supply connections.

Figure 45 Direct Supply Connections



Node-based connections from Verilog are supported, allowing for separate supplies:

```

d2a -powernet 1 ... -node top.vdd1 -vih 1.0
d2a -powernet 1 ... -node top.vdd2 -vih 1.2

```

If a cell- or port-based reference is made, it is applied to all cells of that name, regardless of 3DIC scope. For example:

```

d2a -powernet 1 -cell cellname -port n1

```

D

Unified Power Format (UPF) in Mixed-Signal Simulation

This appendix discusses how mixed-signal simulation with UPF allows designs that contain HDL descriptions, SystemVerilog or VHDL, SPICE, and UPF to be simulated together. Information from UPF is automatically passed to SPICE for the supplies and signals.

This appendix describes:

- [Overview of UPF in Mixed-Signal Simulation](#)
- [Preparing a Mixed-Signal Simulation With UPF](#)
- [Running a Mixed-Signal Simulation With UPF](#)

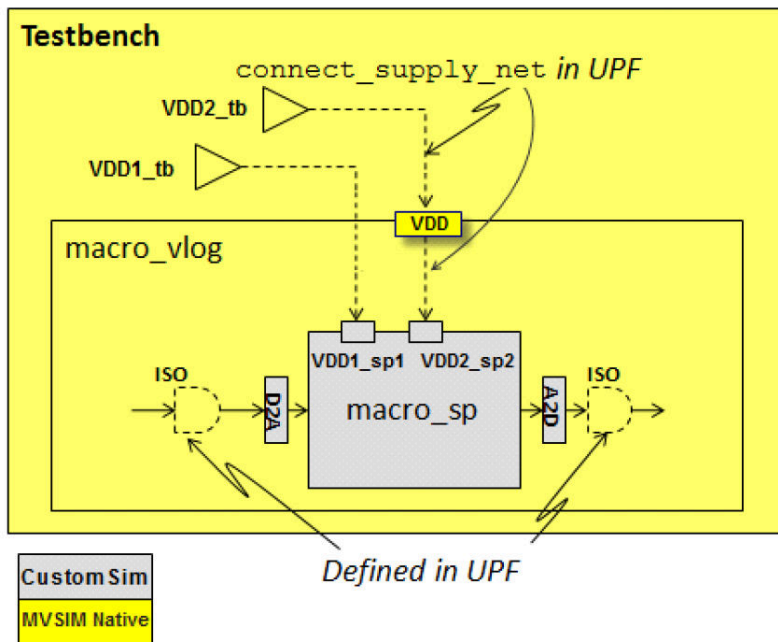
Overview of UPF in Mixed-Signal Simulation

The VCS PrimeSim AMS tool allows a design to have:

- UPF files that define a supply network.
- A system described in SystemVerilog or VHDL.
- A transistor design in SPICE.

The SPICE portion is treated as a black box. The power supply for the SPICE block is directly controlled from the UPF. See [Figure 46](#).

Figure 46 Power Supplies Controlled by UPF



Preparing a Mixed-Signal Simulation With UPF

To enable mixed-signal simulation with UPF, you need to first ensure that native low power (NLP) runs on the digital only design. This guarantees the UPF and HDL files are consistent. Then you need to replace blocks with SPICE representations with the [use_spice](#) command. These cells need to power connections in the HDL representation. For example:

```
module inv_spi (input logic in, output logic out,
               input logic vdd, input logic gnd);
```

The SPICE representation must also have the power ports. Use the UPF command `connect_supply_net` to connect the UPF net to the SPICE port, as shown below:

```
connect_supply_net VDD -ports {I1/vdd}
```

Running a Mixed-Signal Simulation With UPF

The `-upf` and `-ad` VCS command-line options are used together to run the simulation. The `-upf` option reads in the UPF files that define the power network. The `-ad` option reads the mixed-signal simulation control file that defines the analog simulation.

Running a mixed-signal simulation with UPF is described further in the following sections:

- [SPICE/HDL Boundary](#)
- [SPICE/HDL Boundary for Signals](#)
- [SPICE/HDL Boundary for Supplies](#)
- [Tunneling SPICE Supply Ports Through the UPF Supply Network](#)
- [UPF Power-Aware Interface Elements](#)
- [Examples](#)
- [Limitations](#)

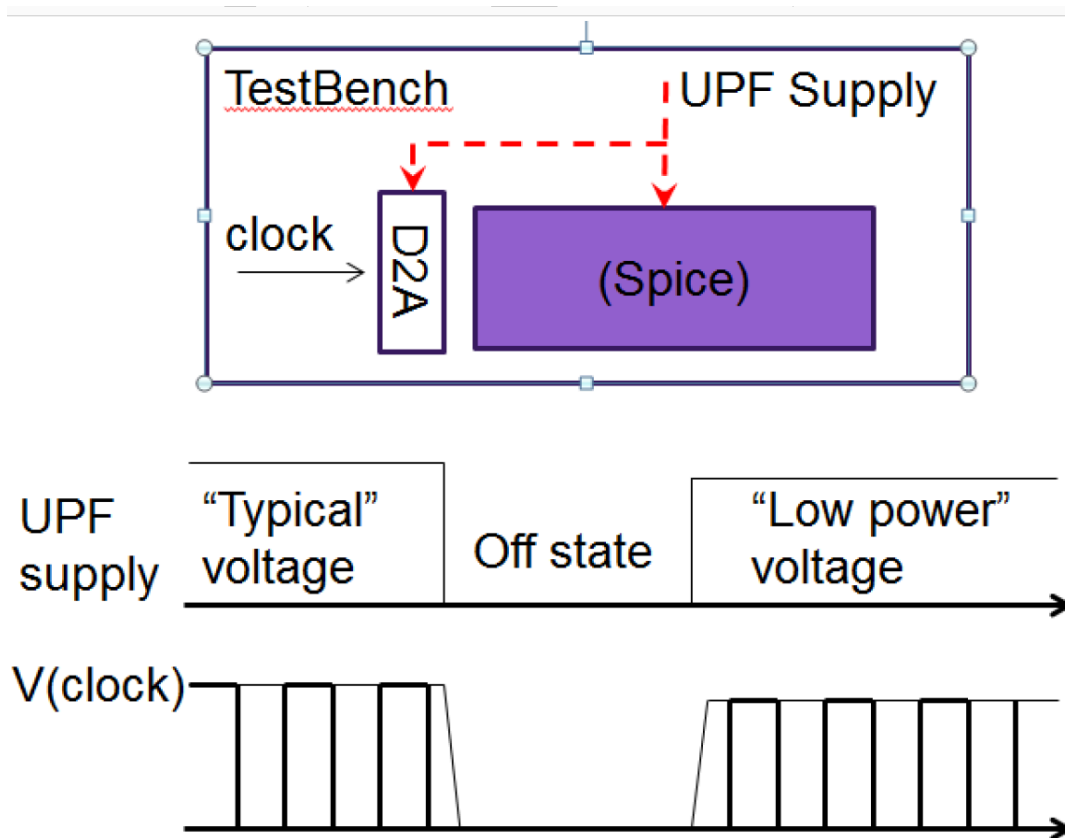
SPICE/HDL Boundary

The boundary between SPICE and HDL has two types of interfaces: a2d and d2a interface elements for signals and r2e interface elements for supplies. The tool automatically detects whether the boundary net is a signal or supply and automatically inserts the correct interface element.

SPICE/HDL Boundary for Signals

a2d and d2a interface elements are automatically inserted following the traditional simulation model. These interface elements have the `-vdd` dynamic supply feature enabled, and the vdd node is determined from the `connect_supply_net` command. As the supply changes from the HDL testbench, the interface element characteristics also change. For example, the analog voltage of a d2a tracks the HDL supply as shown in [Figure 47](#).

Figure 47 SPICE/HDL Boundary Signals

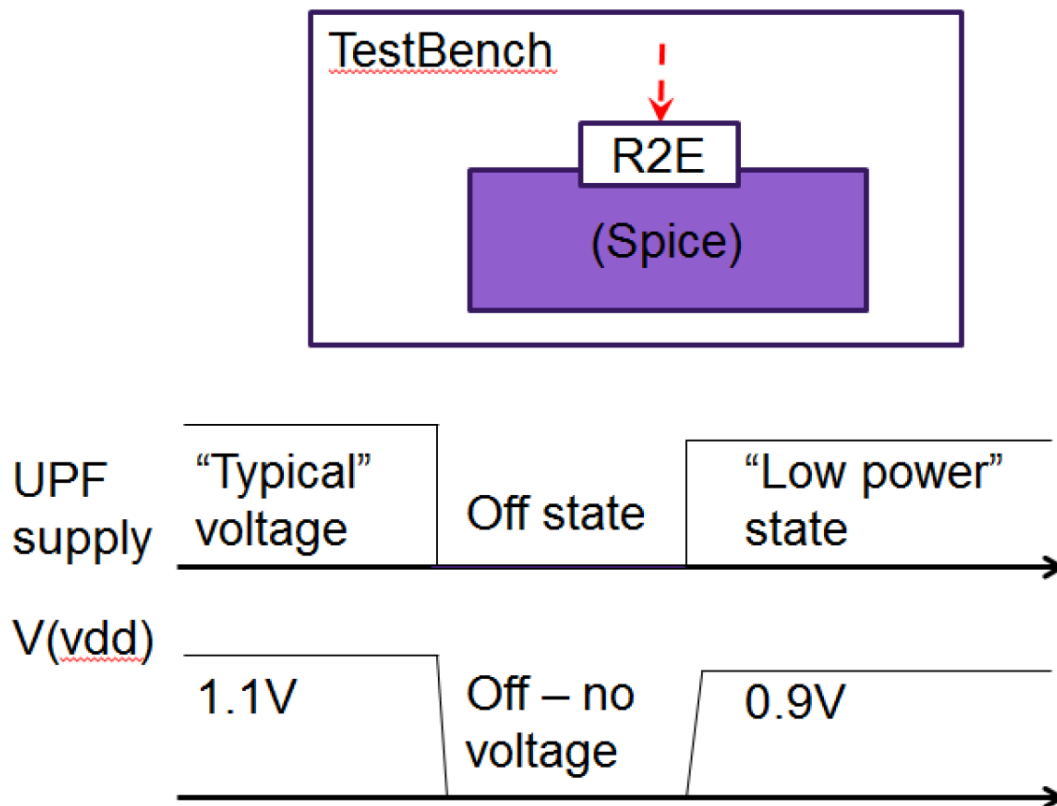


For signals coming from SPICE to HDL, the a2d tracks the supply for the proper threshold voltages.

SPICE/HDL Boundary for Supplies

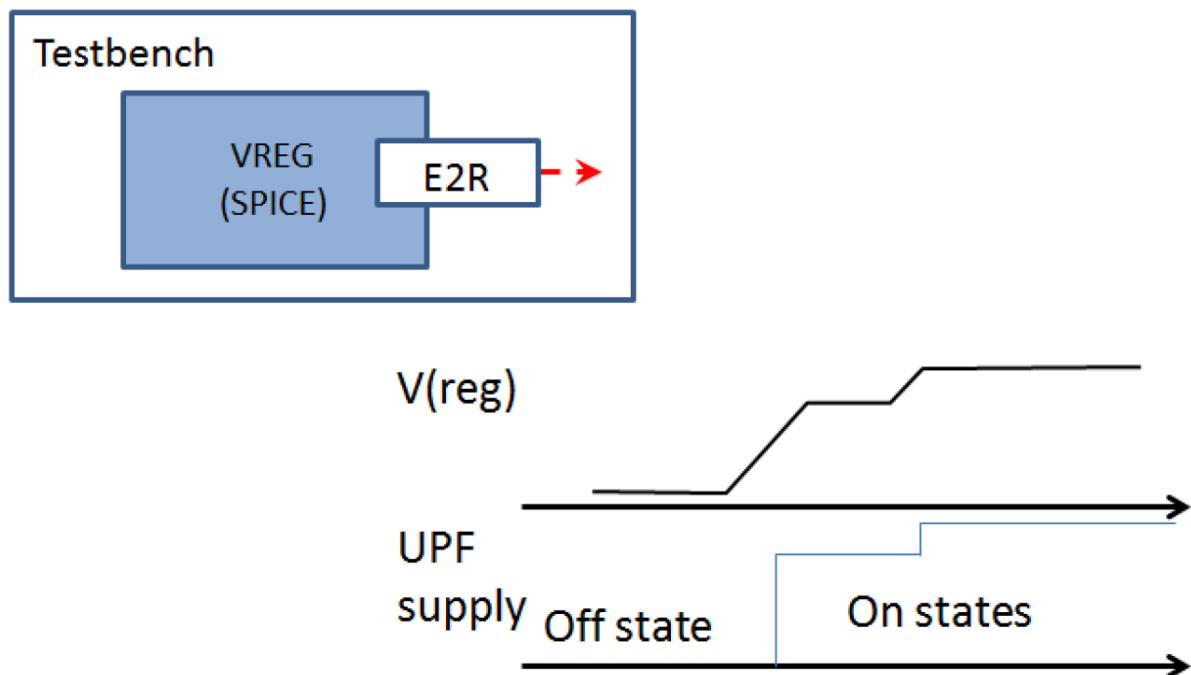
r2e interface elements are automatically inserted between the `connect_supply_net` signal and the SPICE input ports. The analog signal tracks the signal changes in VCS testbench with the `supply_on()` statements, as shown in [Figure 48](#).

Figure 48 SPICE/HDL Supply Signals



e2r interface elements are automatically inserted between the `connect_supply_net` signal and SPICE output ports. As the analog voltage rises and falls, this value is set in the VCS testbench as a UPF signal that contains the state, either on or off, and the value as a real number, as shown in [Figure 49](#).

Figure 49 On and Off States



Example Design

The following example shows a SystemVerilog file that describes an inverter targeted to SPICE.

```
`timescale 1ns/1ns
module tb ();
import UPF::*;
logic clk;
wire clkcb;
inv_spi I1 (clk, clkcb, ,);
always #6 clk = ~clk;
initial begin
    supply_on("VDD", 3.3);
    supply_on("VSS", 0.0);
    clk = 1'b0;
    #40 supply_on("VDD", 3.0);
    #40 supply_on("VDD", 3.3);
    #40 $finish();
end
endmodule

module inv_spi (input logic in, output logic out,
    input logic vdd, input logic gnd);
endmodule
```


Appendix D: Unified Power Format (UPF) in Mixed-Signal Simulation Running a Mixed-Signal Simulation With UPF

This file creates a power domain, D1, and power nets VDD and VSS. These nets are connected with `connect_supply_net` to the SPICE inverter power ports.

The SPICE netlist defines the transistors for the inverter:

```
* SUBCKT inverter
.model n nmos level=54
.model p pmos level=54
.subckt inv_spi in out vdd gnd
mn1 out in gnd gnd n w=1.00u l=0.35u
mp1 out in vdd vdd p w=2.00u l=0.35u
.ends
.end
```

The SPICE subcircuit for the inverter has the same four ports as the SystemVerilog module. The SPICE netlist does not contain `.global` or voltage sources for power and ground. This information comes from the SystemVerilog testbench via the UPF file.

The VCS run script enables mixed-signal with UPF:

```
vcs -lca -ad=vcsAD.init -upf <options>
```

The `-lca` option is needed as this is a "Limited Customer Availability" feature. The VCS tool compiles and runs and generates analog waveforms as shown in [Figure 50](#).

Figure 50 Analog Waveforms



Tunneling SPICE Supply Ports Through the UPF Supply Network

Mixed-signal simulation with the VCS PrimeSim AMS tool allows tunneling through Verilog. That is, several ports of analog instances which are connected through Verilog are considered as connected inside analog, and do not get A2D/D2A interface elements inserted.

When several SPICE supply ports are connected to the same UPF supply net (in the same hierarchy or different hierarchies), the SPICE supply ports are connected to this UPF supply net through E2U/U2E interface elements.

You can use the `-power=ams_tunnel_thru_upf` option at compile time to enable tunneling of SPICE supply ports through the UPF supply network. In this way, the SPICE supply ports are considered as connected inside analog, and do not get the U2E/E2U interface elements inserted. This tunneling mechanism replaces the connection with U2E/E2U interface elements by a direct connection inside the analog database.

The following is the syntax:

```
% vcs -power=ams_tunnel_thru_upf -ad=vcsAD.init -upf upf_file.upf  
    -sverilog design_files
```

This feature improves the SPICE simulation accuracy and performance by creating a single E2U or U2E interface element (if needed) to communicate the supply value between the VCS tool and the VCS PrimeSim AMS analog simulation engine.

Tunneling Use Model

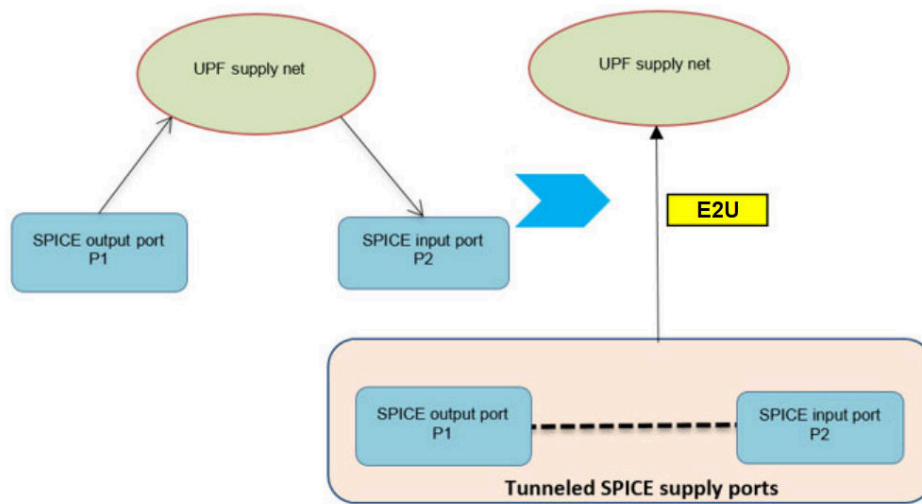
The following sections describe this feature in detail:

- [SPICE Supply Port Driving a UPF Supply Net](#)
- [UPF Supply Net Driving Multiple SPICE Supply Ports](#)
- [Multiple SPICE Supply Ports Driving the Same UPF Supply Net](#)

SPICE Supply Port Driving a UPF Supply Net

If a SPICE supply port (for example, `P1`) drives a UPF supply net, and if the same supply net drives another SPICE supply port (for example, `P2`), then these two SPICE ports (`P1` and `P2`) are tunneled and a single E2U interface element is created (instead of creating both E2U and U2E elements) as shown in [Figure 51](#).

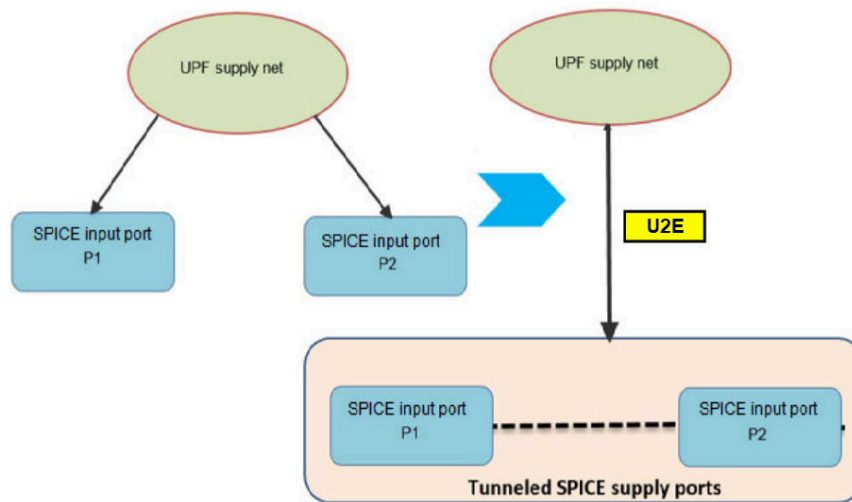
Figure 51 SPICE Supply Port Driving a UPF Supply Net



UPF Supply Net Driving Multiple SPICE Supply Ports

If a UPF supply net drives multiple SPICE supply ports, then all those SPICE ports are tunneled and a single U2E interface element is created (instead of creating separate U2E elements for each of these SPICE input ports) as shown in Figure 52.

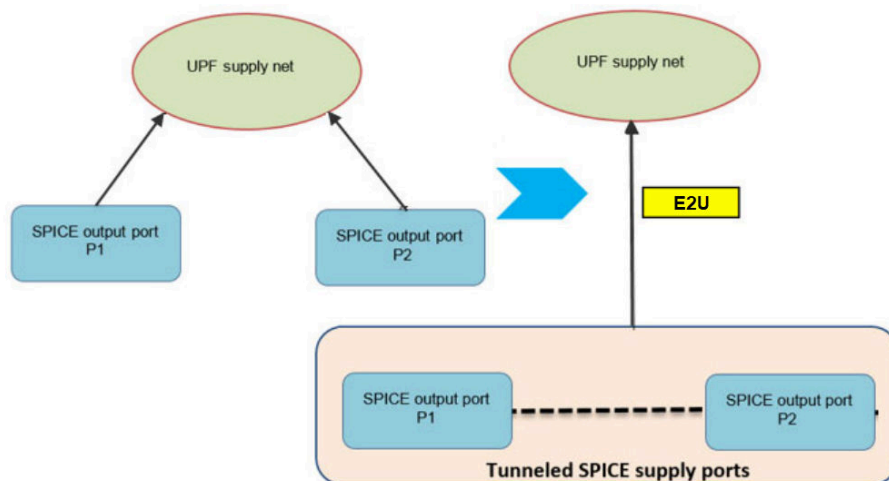
Figure 52 UPF Supply Net Driving Multiple SPICE Supply Ports



Multiple SPICE Supply Ports Driving the Same UPF Supply Net

If multiple SPICE supply ports drive the same UPF supply net, then all those SPICE ports are tunneled and a single E2U interface element is created (instead of creating separate E2U elements for each of these SPICE output ports) as shown in Figure 53.

Figure 53 Multiple SPICE Supply Ports Driving the Same Supply Net



Tunneling Limitations

The limitations are as follows:

- If a UPF supply net is connected to a SPICE inout port, then the inout port is treated either as an input port or as an output port.

UPF Power-Aware Interface Elements

UPF power-aware interface elements are supported in the mixed-signal simulation environment. To enable these interface elements, use the `-power=supply_aware_ie` option with the `vcs` command to read UPF supply information for creating interface elements for the power supplies. The following example uses the `-power=supply_aware_ie` option to enable UPF power-aware interface elements:

```
% vcs -power=supply_aware_ie -ad=vcsAD.init -upf file.upf \
    -sverilog file.v
```

During simulation, VCS Native Low Power (NLP) monitors each UPF supply. AMS uses the UPF supply registered for an a2d signal to get the logic from voltage value and a d2a signal to get the voltage value of logic. Use this feature to simulate the interface elements inserted by the simulation tool at the analog-digital boundaries in the mixed-signal simulation environment.

UPF power-aware interface elements are described further in the following sections:

- [Boundary SPICE Port With set_port_attributes Specified in the UPF](#)
- [Boundary SPICE Port With set_related_supply_net Specified in the UPF](#)

- [SPICE Cell With Liberty Information](#)
- [SPICE Cell Without `set\_port\_attributes`, `set\_related\_supply\_net`, and Liberty Information](#)

Boundary SPICE Port With `set_port_attributes` Specified in the UPF

If the UPF file specifies the boundary SPICE port with the `set_port_attributes` command,

- A d2a interface element on a SPICE input port uses the `-receiver_supply` option of the `set_port_attributes` command
- A a2d interface element on a SPICE output port uses the `-driver_supply` option of the `set_port_attributes` command

Boundary SPICE Port With `set_related_supply_net` Specified in the UPF

If the boundary SPICE port has the `set_related_supply_net` command specified and the `set_port_attributes` command is also specified on the same SPICE port, then the `set_port_attributes` command overrides the `set_related_supply_net` command. Otherwise, the supply information associated with the `set_related_supply_net` command is used.

SPICE Cell With Liberty Information

For a SPICE cell with Liberty information:

- The UPF supply connected to the related power pin and ground pin of the boundary SPICE port is used.
- If the `connect_supply_net` command is specified on the related power and ground pins, the UPF supply information associated with the `connect_supply_net` is used.
- The following rules are used to connect the supply to the appropriate power and ground pins:
 - If the SPICE cell is an isolation DB cell, then the UPF supply of the corresponding UPF isolation strategy is used.
 - If the associated isolation strategy does not have any isolation supply specified, then the default isolation supply of the power domain is considered.
 - When the DB cell is not an isolation cell and does not have an associated isolation policy, the primary supply of the power domain in which the SPICE cell is instantiated is used.

SPICE Cell Without `set_port_attributes`, `set_related_supply_net`, and Liberty Information

If no `set_port_attributes`, `set_related_supply_net`, and Liberty information is specified for a SPICE cell, then AMS assumes that the supply of the digital net is the same as the supply of the power domain where the SPICE cell is instantiated.

Examples

Running a mixed-signal simulation with UPF is demonstrated in the following examples:

- [Example 1](#)
- [Example 2](#)

Example 1

The following example uses the `set_related_supply_net` command for a SPICE input port. Therefore, the supply associated with the pin is an SRSN power supply.

UPF

```
create_supply_set SS_srsn -function {power VDD_srsn} \  
    -function {ground VSS_srsn}  
set_related_supply_net -object_list { m/m/sp1/in1 } \  
    -power VDD_srsn -ground VSS_srsn
```

vcsAD.init

```
use_spice -cell sp1;
```

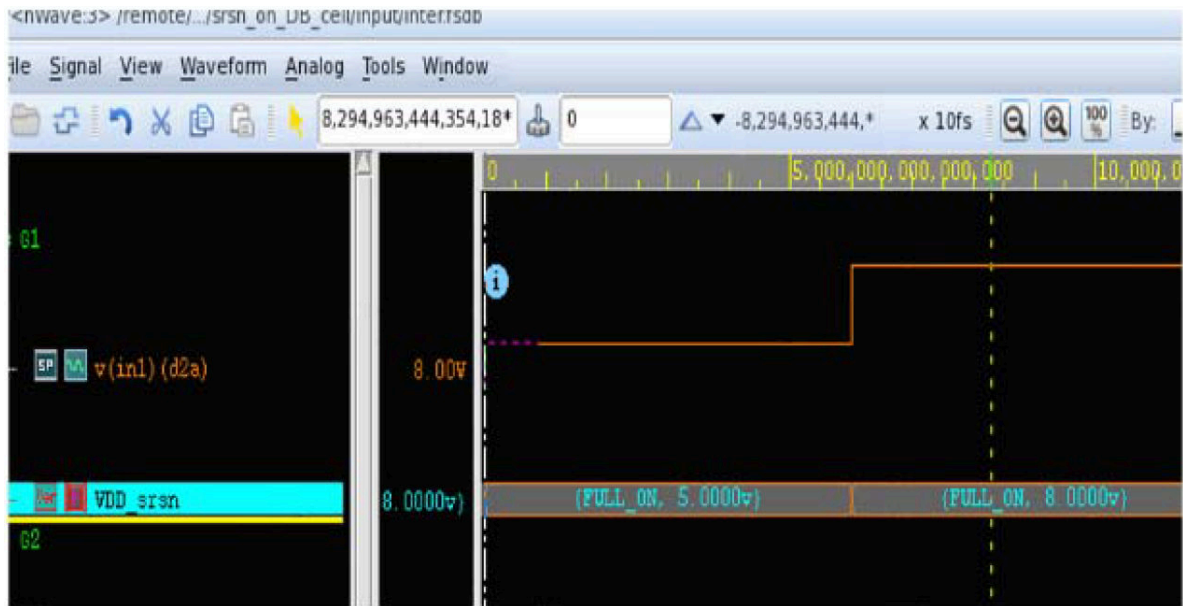
test.spi

```
* Simple inverter  
.subckt sp1 in1 out1 vdd vss  
mn1 out1 in1 vss vss n w=20.00u l=0.35u  
mp1 out1 in1 vdd vdd p w=40.00u l=0.35u  
.ends
```

Testbench

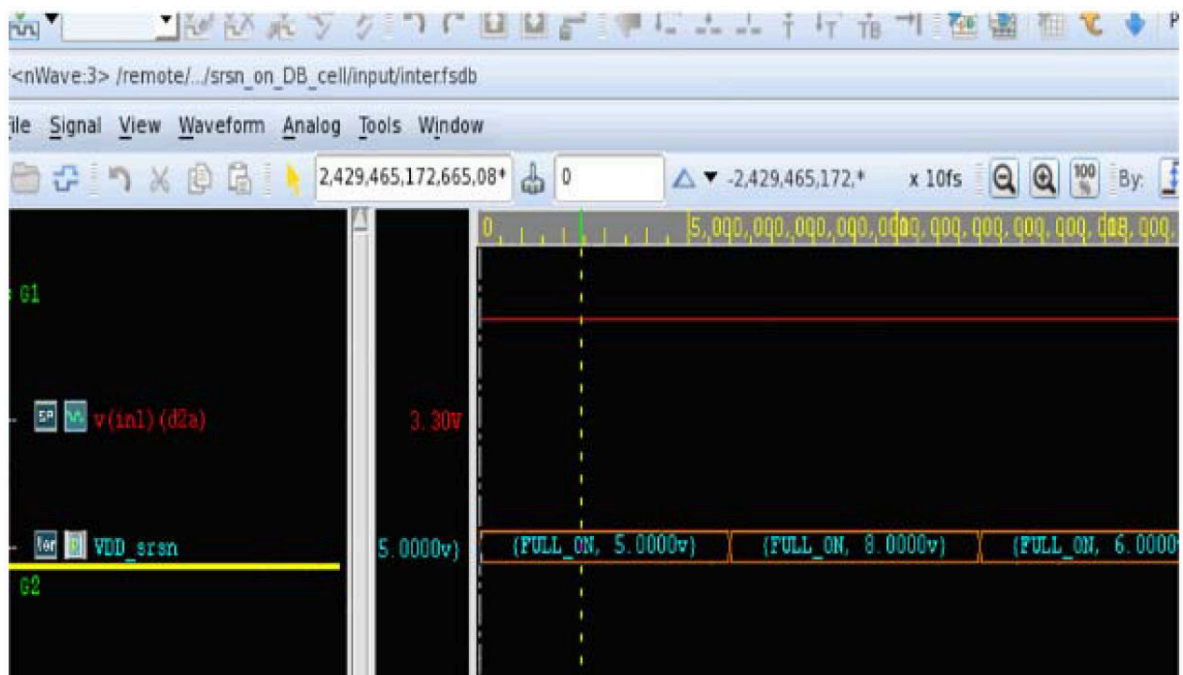
```
supply_on("VDD_srsn",5.0);  
#60 ; supply_on("VDD_srsn",8.0);
```

Figure 54 Simulation Result With the `-power=supply_aware_ie` Option



In the preceding figure, the `VDD_srsn` and the SPICE input have the same voltage values. When `VDD_srsn` changes, the SPICE input value also changes.

Figure 55 Simulation Result Without the `-power=supply_aware_ie` Option



In the preceding figure, SPICE values are not supply-aware. When the `set_related_supply_net` supply value changes, the SPICE voltage value does not change and holds a constant 3.3 volts; this is the default d2a voltage.

Reporting

If you run without the `-power=supply_aware_ie` option, the tool generates the following interface element as reported in the `interface_element.rpt` file:

```
# without -power=supply_aware_ie
d2a hiv=3.3v lov=0v node=tb.top_unit.m.m.sp1.in1;
```

If you run with the `-power=supply_aware_ie` option, VCS NLP determines the actual driver of the signal, identifies the supply, and generates the following d2a interface element:

```
# with -power=supply_aware_ie
d2a -hiv 100% -lov 0% -node tb.top_unit.m.m.sp1.in1
    -vss tb/top_unit/VSS_srsn
    -vdd tb/top_unit/VDD_srsn;
```

Example 2

UPF

```
create_supply_set SS_srsn -function {power VDD_srsn} \
    -function {ground VSS_srsn}
set_related_supply_net -object_list {m/m/sp1/out1} \
    -power VDD_srsn -ground VSS_srsn
```

Note that the `set_related_supply_net` command is defined for the SPICE output port. Therefore, the supply associated with the pin is the `set_related_supply_net` power supply.

vcsAD.init

```
use_spice -cell sp1;
```

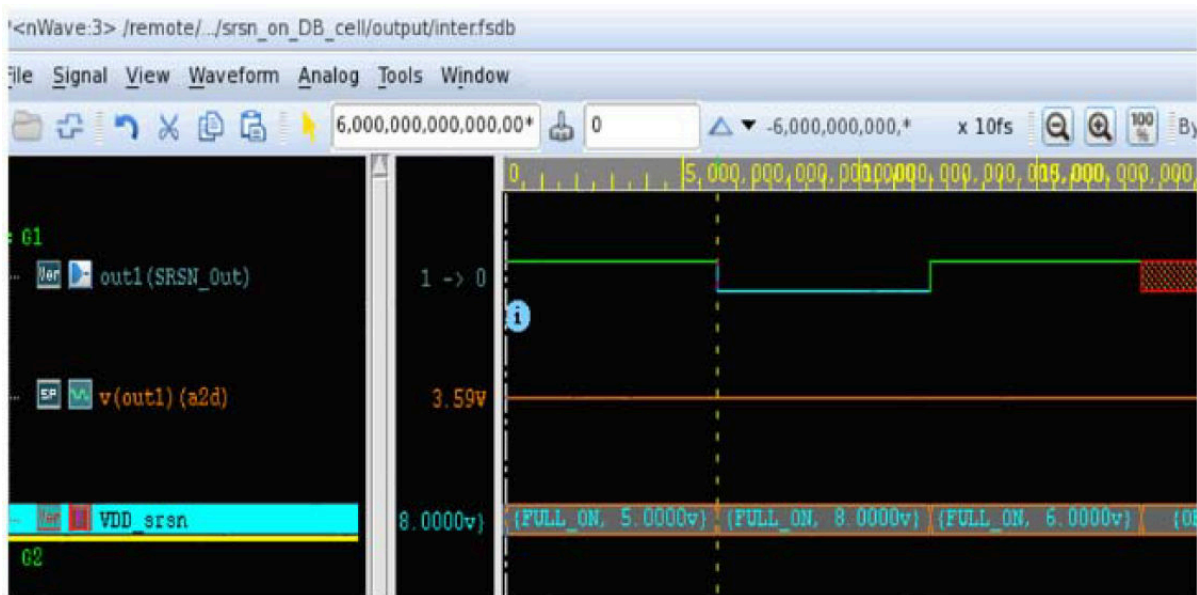
test.spi

```
* Simple voltage divider out1=3.6V
.subckt sp1 in1 out1 vdd vss
R1 temp1 temp 2k
R2 temp 0 3k
R3 temp out1 3k
V1 temp1 0 6V
.ends
```

Testbench

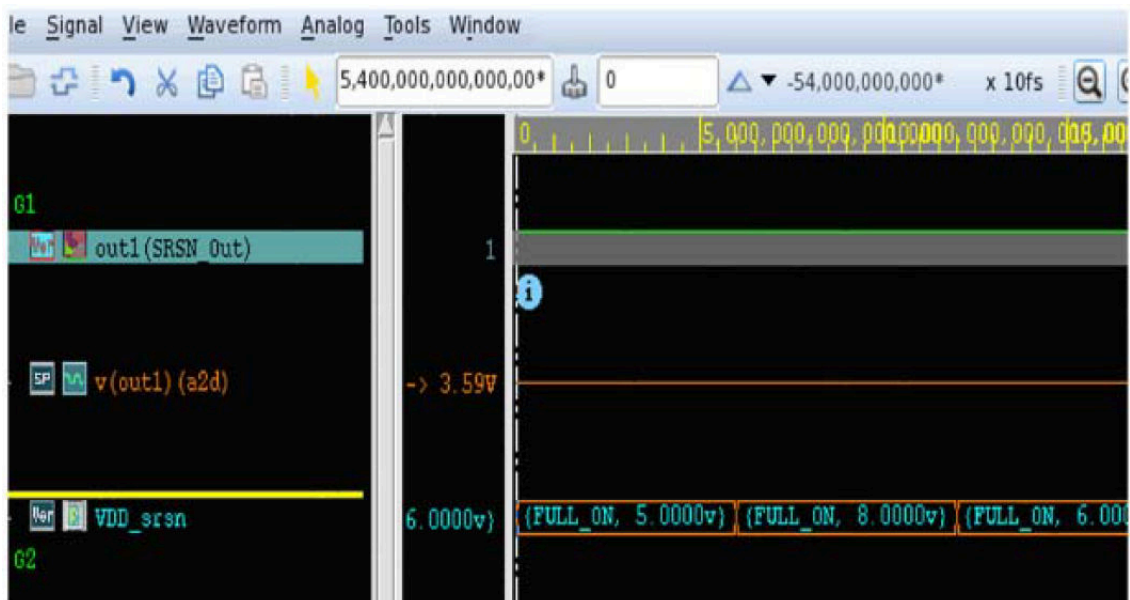
```
supply_on("VDD_srsn",5.0);
#60 ; supply_on("VDD_srsn",8.0);
```


Figure 56 Simulation Result With the `-power=supply_aware_ie` Option



In the preceding figure, when `VDD_srsn` changes to 8.0, `out1` changes to 0. In AMS, the higher and lower threshold values for the a2d interface element are 50% of the a2d supply voltage. When `VDD_srsn` is 8.0, the higher threshold value is 4.0, which is greater than the SPICE output voltage. Therefore, the corresponding digital value is 0.

Figure 57 Simulation Result Without the `-power=supply_aware_ie` Option



In the preceding figure, when the `set_related_supply_net` supply value changes, the digital value does not change and holds at 1.

Reporting

If you run with the `-power=supply_aware_ie` option, the tool generates the following a2d interface element as reported in the `interface_element.rpt` file:

```
a2d -loth 50% -hith 50% -node tb.top_unit.m.m.sp1.out1
    -minv 0.1 -minv_logic x
    -vss tb/top_unit/VSS_srsn
    -vdd tb/top_unit/VDD_srsn;
```

Limitations

- Verilog-AMS is not supported.
- The voltage aware flow is not supported. No 30-70 (VIH-VIL) related corruptions are done in MVSIM native mode at the MVSIM Native and PrimeSim XA, FineSim, or PrimeSim boundaries. Signals going in or coming out of the SPICE instance are assumed to be correctly level-shifted.
- SystemVerilog nettype is not supported.
- Merged VPD mode is not supported.
- MVSIM native mode does not support the following hierarchy for mixed-signal verification: Verilog -> SPICE -> Verilog -> SPICE. The SPICE subcircuit cannot have VHDL as either parent or child.
- You cannot specify UPF for the nested Verilog instance inside the SPICE instance.
- UPF must not refer to any object in the nested Verilog instance under the SPICE instance.
- `connect_supply_net` is not allowed in UPF either from SPICE to SPICE or SPICE to UPF.
- SPICE port of inout direction is not supported.
- Supply net connection through XMRs, aliases, and jumper ports in the RTL is not supported.

E

Support for Nettypes in Mixed-Signal Simulation

This appendix describes support for nettypes in mixed-signal simulation. As defined in the IEEE Standard SystemVerilog LRM 2012, SystemVerilog supports nettypes. Use this capability to describe general abstract values for a net with an optional resolution function.

This chapter describes:

- [Overview of Nettypes in the VCS PrimeSim AMS Tool](#)
- [Predefined Nettypes](#)
- [Use Model for Predefined Nettypes](#)
- [User-Defined Nettypes](#)
- [Arrays of SystemVerilog Nettypes](#)

Overview of Nettypes in the VCS PrimeSim AMS Tool

Mixed-signal simulation flows in the PrimeSim XA, FineSim, and PrimeSim simulation engines support the interface between SystemVerilog Nettype and SPICE and are described later in this chapter. This feature has the following capabilities:

- Interface elements between the predefined nettypes and analog components are inserted automatically, enabling the transfer of values from the nettype to the analog domain and vice-versa. These interface elements are called e2n (unidirectional electrical to nettype), n2e (unidirectional nettype to electrical), or bidirectional.
- Analog tunneling through a nettype, which takes place between two SPICE ports if they are hierarchically connected through a nettype.

If multiple SPICE ports are connected through the same nettype net, those SPICE ports are collapsed to a single SPICE node (tunneled or shorted together).

Caution:

SystemVerilog nettypes are only supported in the three-step flow (which requires separate steps of *analysis*, *elaboration*, and *run*) just as is the case with designs that contain VHDL.

The Verilog-AMS flow is not supported in the nettype flow.

Note:

All predefined nettypes described in this manual, with the exception of `current_r`, support *inout* interface elements. The *inout* interface element is disabled for `current_r` is because a more complex interface element is required. To correctly distribute the current from analog and digital drivers to analog and digital loads, an interface element with extra fields to model the impedance seen at the interface on the digital side is required.

Predefined Nettypes

Predefined nettypes let you exchange electrical voltage, current, or Thevenin information among multiple analog and digital blocks.

The following predefined nettypes are available in the `$VCS_HOME/etc/snps_msv/snps_msv_nettype_pkg.svp` package:

```
nettype i_wiret i_wire      with i_wire_rf;
nettype r_wiret r_wire;
nettype real    current_r    with rf_sum_of_drivers;
nettype real    voltage_r    with rf_l_driver;
nettype real    wrealldriver  with wreal_rf_def;
nettype real    wreal4state  with wreal_rf_4state;
nettype real    wrealavg     with wreal_rf_avg;
nettype real    wrealmax     with wreal_rf_max;
nettype real    wrealmin     with wreal_rf_min;
nettype real    wrealsum     with wreal_rf_sum;
nettype th_wiret th_wire    with th_wire_rf;
nettype v_wiret v_wire_4state with v_wire_rf_4state;
nettype v_wiret v_wire_avg   with v_wire_rf_avg;
nettype v_wiret v_wire_max   with v_wire_rf_max;
nettype v_wiret v_wire_min   with v_wire_rf_min;
nettype v_wiret v_wire_one   with v_wire_rf_one;
nettype v_wiret v_wire_sum   with v_wire_rf_sum;
```

Predefined nettypes are described in the following sections:

- [current_r Nettype](#)
- [i_wire Nettype](#)
- [voltage_r Nettype](#)
- [th_wire Nettype](#)
- [v_wire_avg Nettype](#)

- [v_wire_one Nettype](#)
- [v_wire_sum Nettype](#)

current_r Nettype

The `current_r` nettype resolves multiple driver current values. The resolved current value on this nettype is the sum of the active drivers. This nettype has one member, a real type and is defined as:

```
nettype real current_r with rf_sum_of_drivers;
```

In the following example, `ib` is declared as a nettype of type `current_r`:

```
current_r ib;
```

i_wire Nettype

The `i_wire` nettype can resolve multiple driver current values. The resolved current value on this nettype is the sum of the active drivers. This nettype has two members, one real type (`i`) and one logic type (`active`).

```
typedef struct {  
    real i;  
    logic active;  
} i_wiret;
```

In this structure, `i` holds the driver value, and `active` determines if the driver is active or not. The driver current is counted when this field is set to 1. Otherwise, the value of `i` is ignored.

This nettype is defined as:

```
nettype i_wiret i_wire with i_wire_rf;
```

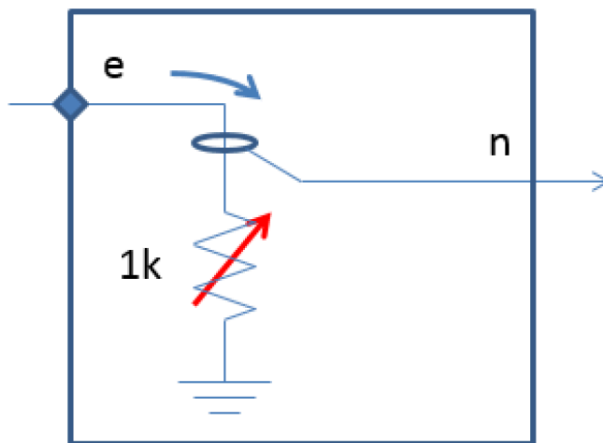
In the following example, `iw` is declared as a nettype of type `i_wire`:

```
i_wire iw;
```

e2n Interface Element for i_wire and current_r

An e2n interface element is inserted between an electrical output and an `i_wire` or `current_r` input. To accurately mimic digital loading on the connection, a resistor is used in the interface element to measure the current as shown in [Figure 58](#). The value of the resistance must be consistent with the loading effect of the digital block. By default, the resistor in the interface element is internally grounded.

Figure 58 e2n Example for current_r

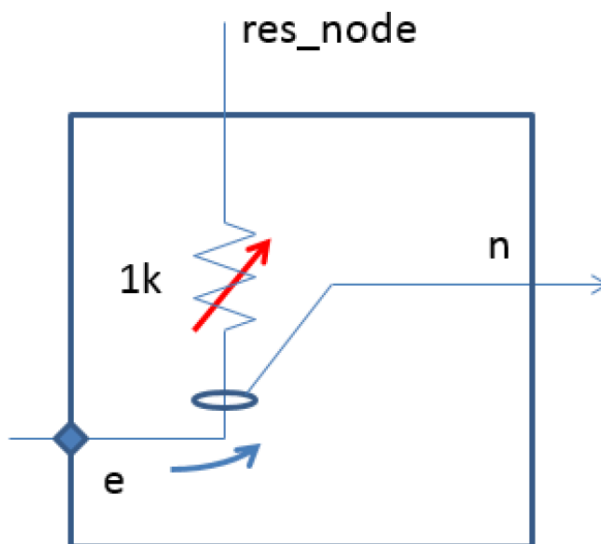


Changes to analog current cause the e2n interface element to generate an event and pass the analog current to the digital side as a real value. The value of the resistance can be changed by setting the `res` option with the `e2n` command. For example:

```
e2n -res 2k -node top.iw4;
```

To model a pull-up resistor, use the `res_node` argument. The value of the argument is an analog power source, such as `vdd`.

Figure 59 e2n Example for current_r With Pull-Up Resistor



In the following example the resistor of the interface element is connected between nodes `top.net1` and `vdd`.

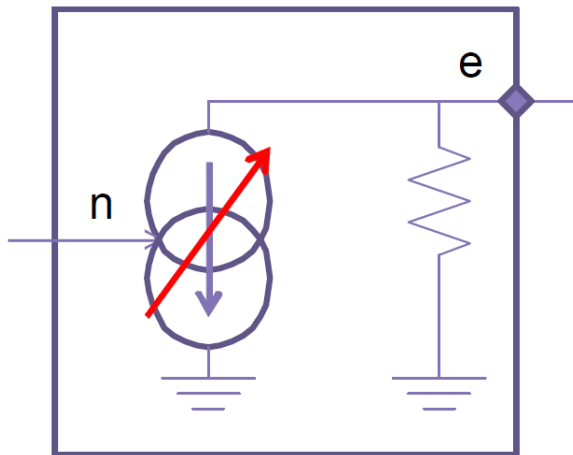
```
e2n -res 100k -res_node vdd -node top.net1;
```

The convention for the current direction is as follows: when current flows into the interface element, a negative value is sent to the VCS tool, and when current flows out of the interface element, a positive value is sent to the VCS tool.

n2e Interface Element for i_wire and current_r

An n2e interface element is inserted between an i_wire_t or current_r and an electrical input. This interface element is effectively a current source in the analog simulator and contains parameters to control its electrical characteristics.

Figure 60 n2e Example for current_r



The resolved value in the VCS simulator is the value of the current source. A parallel resistor ensures convergence of the analog simulator. You can change arguments for the interface element with the n2e command. For example:

```
n2e -rf_rate 25p -node top.rw3;
```

The current direction follows the SPICE convention. For example, a positive value from the VCS tool is converted into a current going into the interface element and a negative value from the VCS tool is converted into a current coming out of the interface.

voltage_r Nettype

The voltage_r nettype can resolve multiple drivers if only one driver value is active and the remaining drivers are realZ. If there is more than one active driver, the resolved value is realX and all driver values.

This nettype has one member, a real type, and is defined as:

```
nettype real voltage_r with rf_1_driver;
```

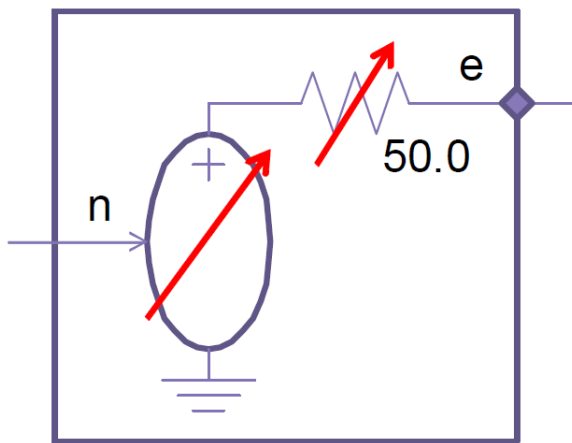
In the following example, `va` is declared as a nettype of type `voltage_r`:

```
voltage_r va;
```

n2e Interface Element for `v_wire_avg`, `v_wire_sum`, `v_wire_one`, and `voltage_r`

An `n2e` interface element is inserted between a `v_wire_avg`, `v_wire_sum`, `v_wire_one`, or `voltage_r` output and an electrical input. This interface element is effectively a voltage source in the analog simulator, and it has some parameters that control the electrical characteristics, as shown in [Figure 61](#).

Figure 61 *n2e Example for `voltage_r`*



The resolved value in the VCS simulator is the value of the voltage source, and a nominal series resistance ensures convergence of the analog simulator. The value of the resistance can be changed with the `n2e` command. For example:

```
n2e -res 125.0 -node top.rw1;
```

e2n Interface Element for `v_wire_avg`, `v_wire_sum`, `v_wire_one`, and `voltage_r`

An `e2n` interface element is inserted between an `v_wire_avg`, `v_wire_sum`, `v_wire_one`, or `voltage_r` output and a `v_wire` input. Changes to the analog voltage translate to an `e2n` event generation in which the new analog voltage is passed to the digital side as a real value.

You can change the value of the `min_delta` argument with the `e2n` command. For example:

```
e2n -min_delta 0.02 -node top.rw2;
```

th_wire Nettype

The `th_wire` nettype models Thevenin resistance and voltage and can resolve multiple drivers.

```
typedef struct {  
    real vth;  
    real rth;  
    logic active;  
} th_wiret;
```

The `th_wire` nettype consists of the following three members:

- `vth` is a real type member that holds the voltage value.
- `rth` is a real type member that holds the resistance value
- `active` determines whether the driver is active. The driver voltage is counted when this field is set to 1. Otherwise, the values of the fields `vth` and `rth` are ignored.

This nettype is defined as:

```
nettype th_wiret th_wire with th_wire_rf;
```

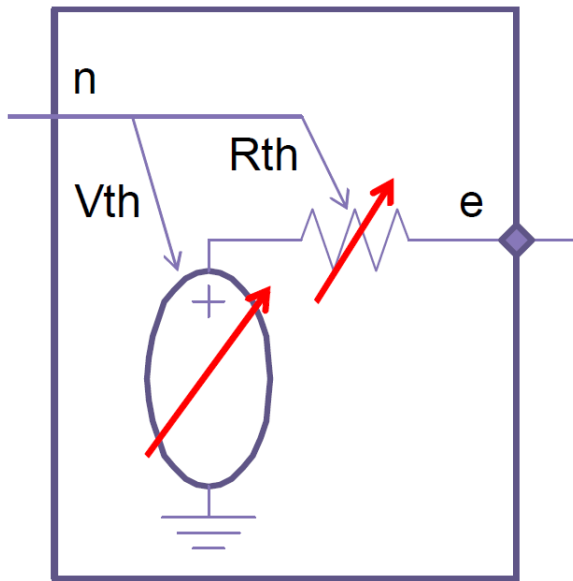
In the following example, `wire1` is declared as a nettype of type `th_wire`:

```
th_wire wire1;
```

n2e Interface Element for th_wire

An `n2e` interface element is inserted between a `th_wire`, output and an electrical input. This interface element uses the `vth` and `rth` fields from the `th_wire`, as resolved in the VCS tool, as the values of the internal voltage source and series resistance presented to the analog simulator. See [Figure 62](#).

Figure 62 *n2e Example for th_wire*



Arguments to the interface element can be changed with the `n2e` command. For example:

```
n2e -rf_rate 25p -node top.rth3;
```

e2n Interface Element for th_wire

An `e2n` interface element is inserted between an electrical output and a `th_wire` net. This interface element monitors the electrical voltage and sets the `Vth` field;

It also dynamically calculates the effective electrical resistance and sets the `Rth` field. You can change the value of the `min_delta` argument with the `e2n` command. For example:

```
e2n -min_delta 0.02 -node top.iw4;
```

v_wire_avg Nettype

The `v_wire_avg` nettype can resolve multiple driver values. In the case of multiple drivers, the resolved voltage value on this nettype is the average of the voltages of the active drivers.

```
typedef struct {
    real v;
    logic active;
} v_wiret;
```

This nettype has two members, one real type named as `v`, and one logic type named `active`:

- `v` holds the driver value.
- `active` determines whether the driver is active. The driver voltage is counted when this field is set to 1. Otherwise, the value of the `v` field is ignored.

This nettype is defined as:

```
nettype v_wiret v_wire_avg with v_wire_rf_avg;
```

In the following example, `va` is declared as a nettype of type `v_wire_avg`:

```
v_wire_avg va;
```

v_wire_one Nettype

The `v_wire_one` nettype can resolve multiple drivers. In the case of multiple drivers, only one driver `active` value should be 1. If there is more than one active driver, the `active` field of the resolved value is `x` (meaning that all driver values have been ignored).

```
typedef struct {  
    real v;  
    logic active;  
} v_wiret;
```

This nettype has two members, one real type named `v`, and one logic type named `active`:

- `v` holds the driver value.
- `active` determines whether the driver is active. The driver voltage is counted when this field is set to 1. Otherwise, the value of the `v` field is ignored.

This nettype is defined as:

```
nettype v_wiret v_wire_one with v_wire_rf_one;
```

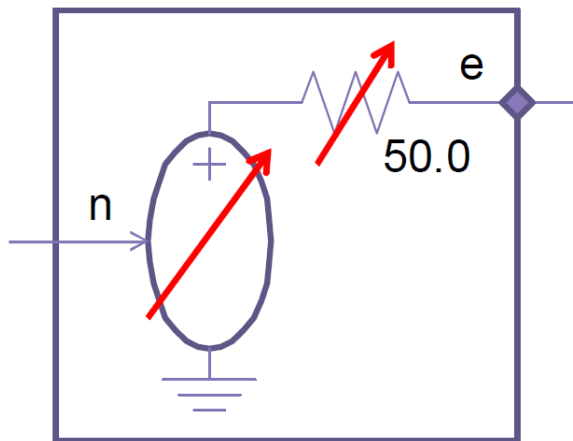
In the following example, `vo` is declared as a nettype of type `v_wire_one`:

```
v_wire_one vo;
```

n2e Interface Element for v_wire_avg, v_wire_sum, and v_wire_one

An `n2e` interface element is inserted between a `v_wire_avg`, `v_wire_sum`, or `v_wire_one` output and an electrical input. This interface element is effectively a voltage source in the analog simulator, and it has some parameters that control the electrical characteristics, as shown in [Figure 63](#).

Figure 63 *n2e Example for v_wire_one*



The resolved value in the VCS simulator is the value of the voltage source, and a nominal series resistance ensures convergence of the analog simulator. The value of the resistance can be changed with the `n2e` command. For example:

```
n2e -res 125.0 -node top.rw1;
```

e2n Interface Element for v_wire_avg, v_wire_sum, and v_wire_one

An e2n interface element is inserted between an electrical output and a `v_wire` input. Changes to the analog voltage translate to an e2n event generation in which the new analog voltage is passed to the digital side as a real value.

You can change the value of the `min_delta` argument with the `e2n` command. For example:

```
e2n -min_delta 0.02 -node top.rw2;
```

v_wire_sum Nettype

The `v_wire_sum` nettype can resolve multiple drivers. In the case of multiple drivers, the resolved voltage value on this nettype is the sum of the voltages of the active drivers.

```
typedef struct {
    real v;
    logic active;
} v_wiret;
```

The nettype has two members, one real type named `v`, and one logic type named `active`:

- `v` holds the driver value.
- `active` determines whether the driver is active. The driver voltage is counted when this field is set to 1. Otherwise, the value of the `v` field is ignored.

This nettype is defined as:

```
nettype v_wiret v_wire_sum with v_wire_rf_sum;
```

In the following example, `vs` is declared as a nettype of type `v_wire_sum`:

```
v_wire_sum vs;
```

Use Model for Predefined Nettypes

During the elaboration step, enable mixed-signal simulation using the `-ad=initfile` elaboration option using the following command:

```
% vcs -sverilog -ad=initfile [elab_options] \  
top_entity/module/config
```

The following example illustrates the use of predefined nettypes at the SystemVerilog-SPICE boundary.

Verilog File - test.v

```
`timescale 1ns/10ps  
import snps_msv_nettype_pkg::*;  
  
module sv_top;  
  v_wiret nt_wire;  
  v_wire_sum in, out;  
  
  initial begin  
    #1; nt_wire.v = 0.0; nt_wire.active = 1;  
    #10; nt_wire.v = 3.3; nt_wire.active = 1;  
    #10; nt_wire.v = 0.0; nt_wire.active = 1;  
    #10; nt_wire.v = 3.3; nt_wire.active = 1;  
    #10; nt_wire.v = 0.0; nt_wire.active = 0;  
    #10; nt_wire.v = 3.3; nt_wire.active = 0;  
    #10; nt_wire.v = 0.0; nt_wire.active = 0;  
    #10; nt_wire.v = 3.3; nt_wire.active = 1;  
    #10; nt_wire.v = 0.0; nt_wire.active = 1;  
    #10; nt_wire.v = 3.3; nt_wire.active = 1;  
    #10; nt_wire.v = 1.3; nt_wire.active = 1;  
    #100 $finish;  
  end  
end
```

```
assign in = nt_wire;

sp1 g1 (in, n1);
sp2 g2 (n1, out);
endmodule

module sp1 (input v_wire_sum a, output v_wire_sum y);endmodule

module sp2 (input v_wire_sum a, output v_wire_sum y);
endmodule
```

SPICE File - test.spi

```
.include 'cmos35t.mod'
.global vdd gnd 0

.subckt sp1 a y
m1 y a gnd gnd nch L=0.4u W=8u
m2 y a vdd vdd pch L=0.4u W=8u
.ends

.subckt sp2 a y
m1 y a gnd gnd nch L=0.4u W=8u
m2 y a vdd vdd pch L=0.4u W=8u
.ends

v1 vdd 0 3.3
v2 gnd 0 0.0
.end
```

Mixed-Signal Simulation Control File - vcsAD.init

```
choose xa -hspice test.sp -c xa.cfg -o xa/xa;
use_spice -cell sp1;
use_spice -cell sp2;
```

PrimeSim XA Control File - xa.cfg

```
probe_waveform_voltage * -port 1
```

vcs Run Script - run

```
rm -r work
mkdir work
rm -r xa
mkdir xa

rm -r simv* csrc *.vpd
```

```
vlogan \  
-full64 \  
-work work \  
-sverilog \  
test.v \  
-l vlogan.log  
  
vcs \  
-full64 \  
sv_top \  
-lca \  
-debug_access+all \  
-xlm coerce_nettype \  
-ad=vcsAD.init \  
-ad_iereport \  
-l vcs.log  
  
simv \  
-ucli \  
-do xaVcs.ucli \  
-l simv.log
```

VCS ucli Control File - xaVcs.ucli

```
dump -file vcs.vpd -type vpd  
dump -add / -depth 0 -scope "."  
run 100ns  
quit
```

User-Defined Nettypes

User-defined nettypes are nettypes other than the predefined nettypes that are defined in the Synopsys package.

There are two methods to have user-defined nettypes connect to SPICE: with Synopsys-defined interface elements and with user-defined interface elements written in Verilog-A:

- [User-Define Nettypes With Predefined Interface Elements](#)
- [User-Defined Nettypes With Custom Interface Elements](#)

User-Define Nettypes With Predefined Interface Elements

The user-defined nettype should be mapped to a predefined nettype so that it can connect to SPICE. The mapping to the nettype ID is made as follows:

```
volt_u, volt: Unresolved and resolved "voltage" values.
current_u, current: Unresolved and resolved "current" values.
iv_u, iv: Unresolved and resolved "voltage" and "current" values.
```

All these user-defined nettypes are in a SystemVerilog package. The user-defined nettype should be mapped to a predefined nettype so that it can connect to SPICE. The mapping to the nettype ID is made as follows:

`vcsAD.init` file:

```
nettype_map <library>.<package>.
<nettype information> => <nettype alias name>;
```

For example, the `nettype_map` command in the `vcsAD.init` mixed-signal simulation control file can be:

```
nettype_map msv_lib.msv_types.volt_u      => msv_ie_v_wire_av_r;
nettype_map msv_lib.msv_types.volt        => msv_ie_v_wire_av_r;
nettype_map msv_lib.msv_types.volt_u      => msv_ie_voltage_r;
nettype_map msv_lib.msv_types.volt        => msv_ie_voltage_r;
nettype_map msv_lib.msv_types.current_u   => msv_ie_i_wire_r;
nettype_map msv_lib.msv_types.current     => msv_ie_i_wire_r;
nettype_map msv_lib.msv_types.current_u   => msv_ie_current_r;
nettype_map msv_lib.msv_types.current     => msv_ie_current_r;
nettype_map msv_lib.msv_types.iv_u        => msv_ie_th_wire_r;
nettype_map msv_lib.msv_types.iv          => msv_ie_th_wire_r;
```

If you map a nettype as shown in the following example, then mixed-signal simulation knows that usage of `current_u` is same as `i_wire`:

```
nettype_map msv_lib.msv_types.current_u => msv_ie_i_wire_r;
```

You can use the following interface element names for all predefined types:

Interface Element Name	Predefined Nettype Name
<code>msv_ie_v_wire_av_r</code>	<code>v_wire_avg</code>
<code>msv_ie_v_wire_sum_r</code>	<code>v_wire_sum</code>
<code>msv_ie_v_wire_one_r</code>	<code>v_wire_one</code>
<code>msv_ie_i_wire_r</code>	<code>i_wire</code>
<code>msv_ie_th_wire_r</code>	<code>th_wire</code>

Interface Element Name	Predefined Nettype Name
msv_ie_voltage_r	voltage_r
msv_ie_current_r	current_r

The following example illustrates how the `nettype_map` command is added in the `vcsAD.init` mixed-signal simulation control file:

01_test.v

```
`timescale 1ns/10ps
package MP;
  typedef struct {
    real v;
    logic active;
  } nt_v;

  function automatic nt_v nt_v_rf_s(input nt_v driver[]);
    nt_v_rf_s.v = 0.0;
    nt_v_rf_s.active = 1'b0;
    foreach (driver[i]) begin
      if (driver[i].active) begin
        nt_v_rf_s.active = 1'b1;
        nt_v_rf_s.v += driver[i].v;
      end
    end
  endfunction

  nettype nt_v nt_v_s with nt_v_rf_s;

endpackage : MP;

import snps_msv_nettype_pkg::*;
import MP::*;

module sv_top;
  nt_v nt_wire;
  nt_v_s sp_in1;

  initial begin
    #1;
    nt_wire.v = 0.0; // #1
    nt_wire.active = 1;
    #10;
    nt_wire.v = 2.0; // #11
    nt_wire.active = 1;
    #10;
    nt_wire.v = 1.0; // #21
    nt_wire.active = 1;
  end
endmodule
```

Appendix E: Support for Nettypes in Mixed-Signal Simulation User-Defined Nettypes

```

        #10;nt_wire.v = 2.0; // #31
        nt_wire.active = 1;
        #10;
        nt_wire.v = 1.0; // #41
        nt_wire.active = 1;
        #100 $finish;
    end

    assign sp_in1 = nt_wire;
    nt_v_s nt_tmp;

    sp1 SP1 (sp_in1, nt_tmp );
    sp2 SP2 (sp_in1, nt_tmp );

endmodule

module sp1 (input nt_v_s sp_in, output nt_v_s sp_out);
endmodule

module sp2 (input nt_v_s sp_in, output nt_v_s sp_out);
endmodule

```

01_test.spi

```

* SUBCKT inverter
.inc '01_test.mod'
.subckt sp1 sp_in sp_out
mn1 sp_out sp_in gnd gnd n w=1.00u l=0.35u
mp1 sp_out sp_in vdd vdd p w=2.00u l=0.35u
.ends

.subckt sp2 sp_in sp_out
mn1 sp_out sp_in gnd gnd n w=1.00u l=0.35u
mp1 sp_out sp_in vdd vdd p w=2.00u l=0.35u
.ends

.global vdd gnd
vsu vdd gnd 3.3
.end

```

01_test.init

```

choose xa -hspice 01_test.spi;
use_spice -cell sp1;
use_spice -cell sp2;
nettype_map work.MP.nt_v_s => msv_ie_v_wire_sum_r;

```

Command Line

```

% vlogan 01_test.v -sverilog -full64
% vcs sv_top -ad=01_test.init -full64

```

User-Defined Nettypes With Custom Interface Elements

Custom user-defined nettypes (UDN) are also supported at the mixed signal boundaries. These nettypes can contain several fields for example, voltage field, current field, and so on.

Use Model

This section provides examples to explain the use model for setting up the custom UDN at the mixed-signal boundaries as follows:

Placement of the Verilog-A Interface Elements

A Verilog-A model is placed on the mixed-signal net, closest to the SPICE net. This model serves as the interface between the nettype and the SPICE electrical net. As it is written in Verilog-A, each field of the nettype becomes a separate port. Therefore, a uni-directional Verilog-A model contains $N+1$ total ports, while a bidirectional model contains $2N+1$ total ports, where N is the number of fields in the nettype. Separate Verilog-A models are written for `n2e`, `e2n`, and bidirectional scenarios. These models are called with the `.hdl` lines in the SPICE netlist.

The nettype connects to the Verilog-A model by position, not name.

- For `udn_n2e`, the ports are connected by `field1`, `field2`, `field3`, and so on, electrical.
- For `udn_e2n`, the ports are connected by electrical, `field1`, `field2`, `field3`, and so on.
- For `udn_bidir`, the ports are connected by `field1_in`, `field2_in`, `field3_in`, electrical, `field1_out`, `field2_out`, `field3_out`, and so on.

Resolution Function

Mixed-signal nets with UDN still employ the SystemVerilog (SV) resolution function to resolve all the drivers into a single resolved value. This value is transmitted from VCS to the analog solver. The Verilog-A module then converts these individual fields into the correct analog values.

For bidirectional nets, SV drives are resolved, sent to SPICE for solving, and the solved values are returned to SV receivers.

Waveform Names

The simulated SPICE netlist contains the instantiated Verilog-A model, named identically to the boundary net with an appended underscore. For example, if the boundary net is `ni`, the Verilog-A instance is named `ni_`.

The SPICE nets for each of the nettype uni-directional interface elements is named by appending an underscore and the field name to the nettype. For instance, if the boundary net is `ni` and the `udn` has field `r1`, the SPICE net is named as `ni_r1`.

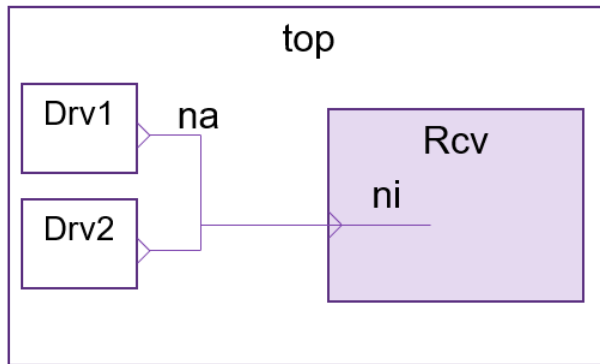
Example of Unidirectional and Bidirectional Interface Elements

The following example has a nettype with two fields, `r1` and `r2`, both declared as `real`, as shown in following figures.

```
typedef struct { real r1; real r2; } nettl;
```

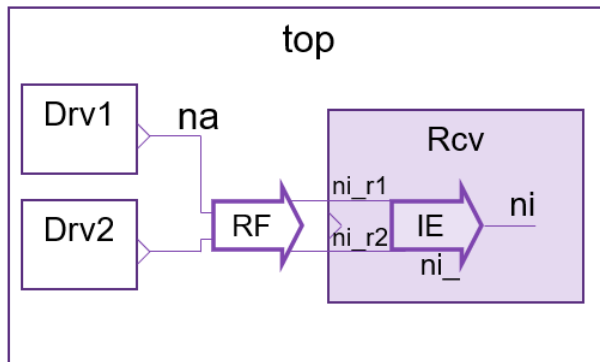
The first design has nettype `na` connected between `Drv1`, `Drv2`, and `Rcv` as shown in [Figure 64](#).

Figure 64 Original Design



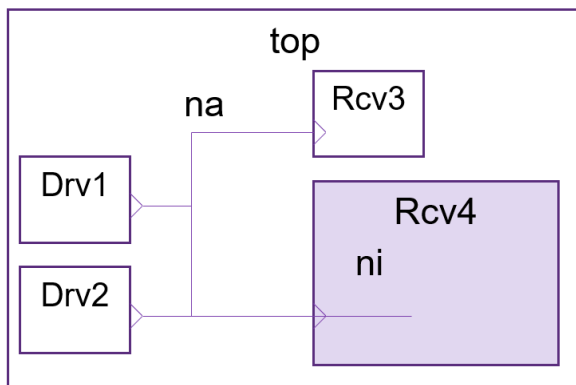
Cell `Rcv` is targeted to SPICE, so net `ni` is a mixed-signal net and an interface element is placed on this SPICE net. The SystemVerilog resolution function resolves the two drivers and transmits these fields to the analog solver. The Verilog-A module for the interface element has three ports, inputs `r1` and `r2` and output `nettlelec`. The effective simulated design is shown in [Figure 65](#).

Figure 65 Design with Resolution Function and Interface Element Insertion



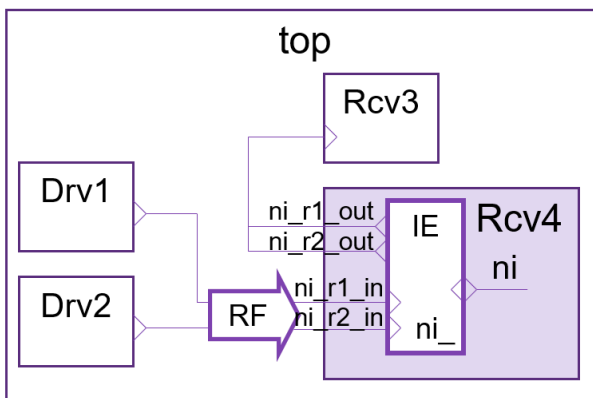
The second design has nettype `na` connected between `Drv1`, `Drv2`, `Rcv3`, and `Rcv4`, as shown in [Figure 66](#).

Figure 66 Original Design



Cell `Rcv4` is targeted to SPICE, so net `ni` is a mixed-signal net and an interface element is placed on this SPICE net. The SystemVerilog resolution function resolves the two drivers and transmits these fields to the analog solver. The Verilog-A module for the interface element has five ports, inputs `r1_in` and `r2_in`, inout `nettlelec`, outputs `r1_out`, and `r2_out`. The effective simulated design is shown in Figure 67.

Figure 67 Design with Resolution Function and Interface Element Insertion



Commands to Change the Arguments of the Interface Elements

There are three separate commands, `udn_n2e`, `udn_e2n`, and `udn_bidir` to control the arguments of the interface elements. Other interface elements (`a2d/d2a`, `e2r/r2e`, `e2n/n2e`) use only two commands, while bidirectional elements use both. The UDN or electrical interface elements have three commands, with `udn_bidir` as a separate command.

Mixed-Signal Control Command

In the mixed-signal simulation control file such as, `vcsAD.init`, additional commands must be added to map a user-defined nettype to its interface elements as follows:

```
udn_n2e -type <UDN_name> -module <va_module>;  
udn_e2n -type <UDN_name> -module <va_module>;  
udn_bidir -type <UDN_name> -module <va_module>;
```

For example:

```
udn_n2e -type mynettype -module my_n2e_va;  
udn_e2n -type mynettype -module my_e2n_va;  
udn_bidir -type mynettype -module my_bidir_va;
```

Where `mynettype` is the name of the user-defined nettype that appears at mixed-signal boundary. The Verilog-A module `my_n2e_va` is used to send the nettype value to SPICE. The last port of this module connects to SPICE and the previous ports connect to the respective fields of the user-defined nettype `mynettype`. The Verilog-A module `my_e2n_va` is used for the reverse communication, that is, to send SPICE value to user-defined nettype `mynettype`. The first port of this Verilog-A module connects to SPICE and the following remaining ports connect to the respective fields of this user-defined nettype.

Parameter values can also be passed to a Verilog-A interface element module using `<field>=<value>` syntax, as follows:

```
udn_e2n -type mynettype -module va_e2n -param1 1.1 -param2 2.2;
```

Here `param1=1.1` and `param2=2.2` are used to set the values of the parameters `param1` and `param2` of the Verilog-A module `va_e2n` to 1.1 and 2.2, respectively.

These Verilog-A interface element modules are included in the `interface_element.rpt` file.

Example

The following example shows how to use custom interface elements at the mixed-signal boundary.

```
02_test.v  
02_test.spi  
vt.va
```

02_test.v

```
package my_nettype;  
  typedef struct {  
    logic l;  
    real volt;  
    real tt;  
  } udn_vt;
```

Appendix E: Support for Nettypes in Mixed-Signal Simulation

User-Defined Nettypes

```

function automatic udn_vt udn_vt_rf_1(input udn_vt driver[]);
    udn_vt_rf_1.l = 1'b0;
    udn_vt_rf_1.volt = 0.0;
    udn_vt_rf_1.tt = 1.0e-12;
    foreach (driver[i]) begin
        udn_vt_rf_1.l = driver[i].l;
        udn_vt_rf_1.volt += driver[i].volt;
        udn_vt_rf_1.tt += driver[i].tt;
    end
endfunction

nettype udn_vt udn_vt_1 with udn_vt_rf_1;

endpackage : my_nettype;

`timescale 1ps/1ps
import my_nettype::*;
module testbench (output logic clk_out, output udn_vt_1 vt_out);

// Logic output driving d2a IE to Spice
reg clk;
initial clk = 1'b0;
always #3000 clk = ~clk;
assign clk_out = clk;

// UDN output driving udn2e IE to Spice
logic l1;
real v1;
real tt1;
initial l1 = 1'b0;
always #4000 l1 = ~l1;
initial begin
    v1 = 1.1;
    tt1 = 4.0e-12;
    #60_000
        v1 = 1.0;
    #60_000
        v1 = 1.1;
    #60_000
        tt1 = 400.0e-12;
    #60_000
        tt1 = 2.0e-9;
    #60_000
        tt1 = 1.0e-9;
    #10_000 $finish;
end

assign vt_out = '{l1, v1, tt1};
endmodule

`timescale 1ps/1ps
import my_nettype::*;
module top;

```

Appendix E: Support for Nettypes in Mixed-Signal Simulation User-Defined Nettypes

```

    wire clk; // Logic signal
    interconnect clk1; // Becomes udn_vt_1 type

    inv2 dut0 (clk, clk1);
    testbench tb0 (clk, clk1);

endmodule

// Empty module - targeted to Spice
module inv2 (input logic clk_l_in, input udn_vt_1 clk_u_in);
endmodule

```

02_test.spi

```

*** top
** Read in Verilog-A module for UDN2E
.hdl "vt.va"

.subckt inv_sp y a
mn1 y a gnd gnd n w=5.00u l=0.35u
mp1 y a vdd vdd p w=10.00u l=0.35u
.ends inv_sp

.subckt inv2 clk_l_in clk_u_in
** Traditional "logic" path with d2a IE
xl01 cl1 clk_l_in inv_sp
xl02 cl2 cl1 inv_sp

** New nettype path with udn2e IE
xn03 cn3 clk_u_in inv_sp
xn04 cn4 cn3 inv_sp
.ends inv2

vsu vdd 0 1.1
vgd gnd 0 0.0
.global vdd gnd
.option post probe
.model n nmos level=54
.model p pmos level=54

.end

```

vt.va

```

`include "disciplines.vams"
module vt1 (l, volt, tt, out);
input l, volt, tt;
output out;
electrical volt, tt, out;
logic l;

real r_volt, r_tt;

```



```
real r_l;  
  
analog begin  
    r_volt = V(volt);  
    r_tt = V(tt);  
    if (l == 1'b1) r_l = 1.0; else r_l = 0.0;  
    V(out) <+ r_volt * transition(r_l,0.0,r_tt);  
end  
endmodule
```

Command Line

```
% vlogan 02_test.v -sverilog -full64  
% vcs top -ad -full64
```

Limitations

This feature has the following limitations.

- The cosimulation flows of the FineSim and PrimeSim simulation engines do not support parameter passing to a Verilog-A interface element module.
- Fields in the user-defined nettype are always scalar such as real, logic, bit, and so on. A user-defined nettype with a field of type vector, struct, union, and array is not supported.

Arrays of SystemVerilog Nettypes

For each of the nettypes supported at the mixed-signal boundary as described in this chapter, arrays of those nettypes are also supported. Appropriate interface elements (n2e/e2n) are inserted at the digital/analog interface for each element of the array.

Note:

Note that since array of real and array of wreal are not supported across the mixed-signal interface, the appropriate mechanism of transferring an array of reals through the mixed-signal interface is through an array of nettypes.

In [Table 13](#), a two-element array of the `voltage_r` nettype connects between two drivers and two loads. The connection is made through SystemVerilog interconnect. If `buf_s1` (containing the drivers) is replaced by SPICE, two e2n interface elements are inserted. If `buf_s2` (containing the receivers) is replaced by SPICE, two n2e interface elements are inserted.

Table 13 Two Element Array of voltage_r Nettype Example

Driver	Receiver
<pre>module buf_s1 (a, net_1); input a; output voltage_r net_1[0:1]; real oval[0:1]; assign net_1=oval; always @(a) begin if (a) oval= {1.0,2.0}; else oval= {0.0,0.0}; endendmodule</pre>	<pre>module buf_s2 (net_1, out); input voltage_r net_1[0:1]; output voltage_r out[0:1]; real midval[0:1]; assign midval[0:1] = net_1[0:1]; assign out[0:1] = midval[0:1];endmodule</pre>
Connection	
<pre>import snps_msv_nettype_pkg::*;module top;reg cin;interconnect net_1, net_2;buf_s1 drvvr (.a(cin), .net_1(net_1));buf_s2 rcvr (.net_1(net_1), .out(net_2));...endmodule</pre>	

The following SPICE netlists could be substituted for the driver or receiver:

```
.subckt buf_s1 a net_d_0 net_d_1
X0 a net_d_0 buf_1
X1 a net_d_1 buf_1
.ends.subckt buf_s2 net_r_0 net_r_1 out_0 out_1
X0 net_r_0 out_0 buf_2
X1 net_r_1 out_1 buf_2
.ends
```

For this example, a bus_format command must be issued in the mixed-signal simulation control file. The command for this example is:

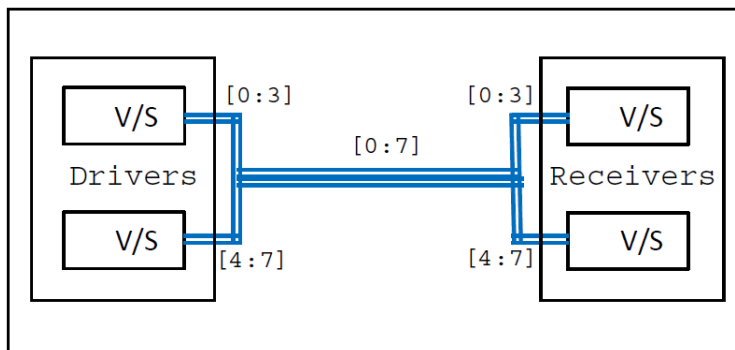
```
bus_format %_d;
```

Connecting Bits

Each of the bits of a nettype array may drive loads located in different blocks. Each of the load blocks may be either SPICE or Verilog. The driving blocks could also be SPICE or Verilog.

These connections are generalized in [Figure 68](#). This example shows an 8-bit array of nettype whose elements may connect to SPICE or Verilog.

Figure 68 Connecting Bit of a nettype Array Example



In this case several drivers and receivers can each be SPICE or Verilog. When an element of the array is connected between two SPICE blocks an electrical connection is made between them, and no interface element is placed between them. Similarly, if an element of the array is connected between two Verilog blocks, there is also no interface element inserted.

For any array element connecting SPICE to Verilog or Verilog to SPICE, the appropriate interface element (e2n/n2e) is inserted.

Rules and Limitations

Only one, unpacked dimension is allowed. Multi-dimensional arrays of nettype are not supported.

Each bit of a nettype-array port should be present in the port list on exactly one time. For example, the following format is not allowed:

```
.subckt mycell a[0] b[0] a[1] b[1]
...
.ends
```

The index for each dimension should start from the left bound of the dimension to the right bound in ascending or descending order.

F

Verilog/VHDL/PrimeSim XA VPI Mixed-Signal Simulation

This appendix provides information on the Verilog/VHDL/PrimeSim XA VPI mixed-signal simulation works.

This appendix describes:

- [Running a Verilog/VHDL/PrimeSim XA VPI Mixed-Signal Simulation](#)
- [Setting Up System Environment Variables for Mixed-Signal Simulation](#)
- [Mixed-Signal Simulation With Verilog as the Top Instance](#)
- [Instance-Based Instantiation With Verilog Configuration](#)
- [Mixed-Signal Simulation With VHDL as the Top Instance](#)
- [Mixed-Signal Simulation With SPICE as the Top Instance](#)
- [Donut Partitioning With Verilog as the Top Instance \(V-S-V\)](#)
- [Donut Partitioning With SPICE as the Top Instance \(S-V-S\)](#)
- [Save-Restart in Mixed-Signal Simulation](#)
- [Configuration File Commands](#)
- [Automatic Voltage Level Detection](#)
- [Mixed-Signal Simulation Interactive Mode](#)
- [Verilog System Tasks for Mixed-Signal Simulation](#)
- [Mixed-Signal Simulation Setup Guidelines](#)
- [Partitioning Guidelines](#)
- [Strength Table Setup Guidelines](#)
- [Mixed-Signal Simulation With ModelSim](#)

- [PrimeSim XA Features Not Supported by Mixed-Signal Simulation](#)
- [References](#)

Running a Verilog/VHDL/PrimeSim XA VPI Mixed-Signal Simulation

This appendix presents a solution to Verilog/VHDL/PrimeSim XA VPI mixed-signal simulation. The chapter is structured in the following order:

- System environment variable setup
- Analog/digital partitioning flows and examples
- Configuration commands
- Setup and partitioning guidelines

Verilog/VHDL/PrimeSim XA mixed-signal simulation allows a design to be partitioned into digital and analog blocks and simulated as one. The digital partition is in Verilog/VHDL and simulated by a Verilog/VHDL simulator. PrimeSim XA simulates the analog partitions with SPICE netlist format. The mixed-signal simulation interfaces synchronize both the Verilog/VHDL simulator and PrimeSim XA as well as passing and translating signal values back and forth between these two simulators.

The Synopsys Verilog/VHDL/PrimeSim XA mixed-signal simulation offers three analog/digital partitioning flows to fit into different design and verification methodologies:

- Verilog/VHDL netlist on top with leaf instances assigned to SPICE
- SPICE netlist on top with leaf instances assigned to Verilog or VHDL
- Integration with Virtuoso® Analog Design Environment with analog and digital netlists generated by 3rd-party/Verilog mixed-signal simulation

Cadence® NC-Verilog®/VHDL^[1] and Mentor Graphics ModelSim® are supported. Cadence Verilog-XL^[2] also works with mixed-signal simulation, but it is not fully tested. Other PLI 2.0 compliant Verilog/VHDL simulators may also work with PrimeSim XA mixed-signal simulation but have not been fully tested.

Mixed-signal simulation uses Verilog Procedural Interface (VPI) or Programming Language Interface (PLI) 2.0, to interact with ncsim, NC-Verilog/VHDL simulator. The mixed-signal simulation executable is a shared library including VPI code and the PrimeSim XA engine. Mixed-signal simulation starts with ncsim as the master simulator which dynamically links with the mixed-signal simulation library and invokes the PrimeSim XA engine. The single process combines the ncsim, mixed-signal simulation interface, and PrimeSim XA engine. The interactions between ncsim and PrimeSim XA go through

VPI function calls. This approach does not need any communication backplane and interprocess communication (IPC). Thus, best performance can be achieved.

Setting Up System Environment Variables for Mixed-Signal Simulation

Before running mixed-signal simulation, the system must be set up using the following steps.

1. Set the NC-Verilog/VHDL executables path as shown in the following example:

```
set path=($path /usr/local/vendors/cadence/ldv40/tools/bin)
```

2. Add the NC-Verilog/VHDL library path to the `LD_LIBRARY_PATH` environment variable as shown in the following syntax example:

```
setenv LD_LIBRARY_PATH  
${LD_LIBRARY_PATH}/bin:/usr/local/vendors/cadence/ldv40/tools/inca/li  
b:/usr/local/vendors/cadence/ldv40/tools/lib
```

3. Add the directory containing `libxa_vpi.so` to `LD_LIBRARY_PATH`, where `libxa_vpi.so` is the VPI shared library in the PrimeSim XA installation directory as shown in the following example:

```
setenv LD_LIBRARY_PATH  
/global/apps/xa_2020.12/linux64/interfaces:$LD_LIBRARY_PATH
```

Mixed-Signal Simulation With Verilog as the Top Instance

This section describes the following:

- [High-Level Mixed-Signal Simulation Instructions](#)
- [Detailed Mixed-Signal Simulation Instructions](#)

High-Level Mixed-Signal Simulation Instructions

High-level mixed-signal simulation provides the steps necessary to run Verilog/PrimeSim XA mixed-signal simulation without using the cell view approach.

Note:

Syntax Convention: A backslash character (\) in syntax examples marks a line continuation. Where there is no space before the \, the line continues unbroken. If there is a space prior to the \, a space exists in the syntax.

To perform high-level mixed-signal simulation without using the cell view approach, do the following:

1. For those Verilog modules to be simulated by PrimeSim XA, replace their module body with only one line as shown in the following syntax example:

```
initial $snps_module();
```

2. Provide a SPICE netlist for Verilog modules that have the same module or subcircuit and port names.
3. Recompile the Verilog source code using `ncvlog`. Proper `hdl.var` and `cds.lib` are required for `ncvlog`.
4. Insert `ncelab` with an additional command-line option where `libxa_vpi.so` is the VPI share library shipped to Synopsys customers as shown in the following syntax example:

```
-loadvpi libxa_vpi.so:snps_vpi_startup
```

5. Specify PrimeSim XA parameters such as the netlist file name in the `cosim.cfg` file.
6. Run `ncsim` with additional command-line option as shown in the following syntax example:

```
-loadvpi libxa_vpi.so:snps_vpi_startup +snps+./cosim.cfg
```

Detailed Mixed-Signal Simulation Instructions

The information contained in this section provides details on running Verilog/PrimeSim XA mixed-signal simulation using the cell view approach.

To perform detailed mixed-signal simulation using the cell view approach, do the following:

1. Create new Verilog source files for the modules to be simulated in PrimeSim XA. It is not necessary to modify the original Verilog source code. The modules should have the same module name and port name as the original Verilog modules. The module body contains only the following syntax line:

```
initial $snps_module();
```

Use a new file name extension, such as `.cs`, to compile the new files into a new view such as `cosim` view.

2. Modify `hdl.var` to define a new view as shown in the following syntax example:

```
DEFINE VIEW_MAP (.cs => cosim)
```

3. The selected view described in the previous syntax is cell-based. For an instance-based view selection, insert the following compilation directive before the instance in the original Verilog source code as shown in the following syntax:

```
`uselib lib=cosim_lib view=cosim
```

4. Create a SPICE netlist for the Verilog modules. Make sure to have the same subcircuit name as the Verilog module name and port names as well.

Note:

If a subcircuit name is different from the module name, use `map_subckt_name` to associate them.

5. Compile the new Verilog files into a new view as specified in `hdl.var`.
6. Prepare the mixed-signal simulation configuration file, for example, `cosim.cfg`, with PrimeSim XA parameters and mixed-signal simulation parameters.
7. Run the NC-Verilog command with additional `-loadvpi` command option. NC-Verilog is a Cadence product that requires three steps to run a simulation: compilation, elaboration, and simulation. The related commands are:

- a. Compilation—Use the `ncvlog` command. The syntax is :

```
% ncvlog top.v gate.cs
```

- b. Elaboration—Use the `ncelab` command. The syntax is:

```
% ncelab -loadvpi libxa_vpi.so:snps_vpi_startup -access \
+rw -LIBNAME cosim_lib cosim_lib.top -snapshot \
cosim_lib.top:cosim
```

- c. Simulation—Use the `ncsim` command. The syntax is:

```
% ncsim -loadvpi libxa_vpi.so:snps_vpi_startup \
+snps+"cosim.cfg" cosim_lib.top:cosim
```

where `libxa_vpi.so` is the VPI share library are shipped with the product.

Note:

The `ncsim` command-line option `+snps+` is used to pass the `cosim.cfg` configuration file name to mixed-signal simulation. If the `+snps+` option is not specified, the default configuration file is `cosim.cfg`.

The following example illustrates a simple inverter chain with five inverters of which two are in analog and three in digital. The `inv` module is shown in both the `top.v` and `gate.cs` files. Because `hdl.var` defines the `.cs` file as the `cosim` view with a higher precedence over the default module view, the `inv` module is simulated in PrimeSim XA. Its equivalent subcircuit is defined in the `inv.spi` file.

Sample files for the example include the following:

- `top.v`

Verilog source code that contains the default `inv` module. This file is compiled into the default module view.

- `gate.cs`

Verilog source code containing an `inv` module to be simulated by PrimeSim XA. This file is compiled into the cosim view.

- `hdl.var`

Verilog configuration file that specifies a cosim view, directs the compiler to compile `*.cs` source files into the cosim view, and directs the elaborator to pick cells with the cosim view whenever available.

- `cds.lib`

Verilog configuration file that defines design libraries. The physical directory for a design library must already exist. Refer to the Cadence *NC-Verilog User Manual* for details.

- `inv.spi` and `test.spi`

SPICE netlist with `inv` subcircuit simulated by PrimeSim XA.

- `cosim.cfg`

Mixed-signal simulation configuration file that specifies both PrimeSim XA and mixed-signal simulation parameters such as the SPICE netlist file name.

Example 57 `top.v`

```
// Top cell with 5 chain inverters, but pushing through
// one more level of hierarchy by my_buf
`timescale 1ns / 10ps

module top;
  wire z1, z2, z3;
  testbench tb(z1, z2, z3, a);
  chain main(a, z1, z2, z3);
endmodule

module testbench(z1, z2, z3, a0);
  input  z1, z2, z3;
  output a0;
  reg    a0;
  always #25 a0=~a0;
  initial begin
    a0=1'b1;
    $monitor($time,, a0,, z1,, z2,, z3);
  end
endmodule
```

Appendix F: Verilog/VHDL/PrimeSim XA VPI Mixed-Signal Simulation Mixed-Signal Simulation With Verilog as the Top Instance

```

        #200;
        $finish;
    end
endmodule

module chain (a, z1, z2, z3);
    input  a;
    output z1, z2, z3;
    my_buf x1 (a, z1);
    my_inv x2 (z1, z2);
    my_buf x3 (z2, z3);
endmodule

module inv (a, z);
    input a;
    output z;
    assign z=~a;
endmodule

module my_inv (a, z);
    input  a;
    output z;
    assign z=~a;
endmodule

module my_buf (a, z);
    input  a;
    output z;
    wire  t;
    my_inv IV1(a, t);
    inv IV2(t, z);
endmodule

```

Example 58 *gate.cs*

```

module inv (a, z);
    input  a;
    output z;
    initial $snps_module();
endmodule

```

Example 59 *inv.spi*

```

.subckt inv a z
m1 z a vdd vdd p l=0.5u w=5u as=1.0e-10 ad=1.0e-10
+ ps=0 pd=0
m2 z a 0 0 n l=0.5u w=3u as=1.0e-10 ad=1.0e-10
+ ps=0 pd=0
.ends

```

Example 60 *test.spi*

```

.param VDDVAL=3v
* global nodes

```

```
.global vdd vss gnd
* supplies
vvdd vdd 0 dc VDDVAL
vgnd gnd 0 dc 0v
.inc models
.inc inv.spi
.print v(*)
.end
```

Example 61 Other files

```
===== cosim.cfg =====
set_args test.spi
===== hdl.var =====
DEFINE WORK cosim_lib
DEFINE VIEW_MAP (.cs => cosim)
===== cds.lib =====
DEFINE cosim_lib ./cosim_lib
```

Instance-Based Instantiation With Verilog Configuration

NC-Verilog 5.1 supports instance-based instantiation by using Verilog configurations in accordance with the IEEE standard, IEEE 1364-2001. A library map file contains the binding rules. This feature is invoked using `ncvlog` and `ncelab` with the `-libmap` command-line option to specify the library map file.

Multiple implementations of the same module can be compiled into different design libraries. Using Verilog configurations, `ncelab` searches design libraries to bind instances as shown in the following example.

```
===== File: top.v =====
module top();
  chain a1(...);
  chain a2(...);
endmodule

module chain(...);
  inv i1(...);
  inv i2(...);
endmodule

module inv(in, out);
  input in;
  output out;
  assign out = ~in;
endmodule

===== File: inv.cs =====
module inv(in, out);
  input in;
  output out;
```

Appendix F: Verilog/VHDL/PrimeSim XA VPI Mixed-Signal Simulation Instance-Based Instantiation With Verilog Configuration

```

    initial $snps_module();
endmodule

===== Library map file: lib.map =====
library rtlLib top.v;
library cosimLib inv.cs;

config cfg;
  design rtlLib.top;
  default liblist rtlLib cosimLib;
  instance top.a2.i1 liblist cosimLib;
endconfig

```

To compile the design units, invoke `ncvlog` using the following syntax:

```
% ncvlog -libmap lib.map top.v inv.cs
```

This compiles the design units into the appropriate libraries as follows:

Design Units	Library
top	rtlLib
chain	rtlLib
inv (from <code>inv.cs</code>)	cosimLib
inv (from <code>top.v</code>)	rtlLib

In the `lib.map` (library map) file, the Verilog configuration `cfg` specifies an instance-based instantiation for instance `top.a2.i1`. To elaborate the top design, use the following command:

```
% ncelab -libmap lib.map cfg -loadvpi \
    libxa_vpi.so:snps_vpi_startup -access +rwc
```

The instance `top.a2.i1` is bound to the design unit `inv` in `cosimLib` while the remaining three `inv` instances are bound to the design unit `inv` in `rtlLib`. During mixed-signal simulation, the instance `top.a2.i1` is partitioned to the analog simulator and the others are simulated by Verilog simulator. With the Verilog configuration, analog and digital partitioning for mixed-signal simulation can be accomplished in an instance-based fashion.

Refer to the *NC-Verilog User Manual* and *IEEE 1364-2001 Standard* for further details on instance-based instantiation.

Mixed-Signal Simulation With VHDL as the Top Instance

The Cadence `ncsim` tool can simulate pure Verilog, pure VHDL, and mixed Verilog/VHDL designs. This Synopsys mixed-signal simulation uses VPI to interact with `ncsim`. However, VPI can only access Verilog objects. In order to simulate with VHDL, PrimeSim XA needs Verilog as the media to interact with VHDL indirectly. Therefore, a Verilog wrapper is required.

Use the following steps to run mixed-signal simulation with VHDL designs.

1. Create a Verilog module with the same port definition as the VHDL entity to be partitioned to SPICE. This Verilog module contains only one statement as the module body and functions as a wrapper around the SPICE block as shown below:

```
initial $snps_module();
```

2. Modify the original VHDL code to instantiate the new Verilog module.

3. Compile the VHDL code with `ncvhd1` as shown in the following syntax:

```
% ncvhd1 top.vhd
```

4. Compile the Verilog code with `ncvlog` as shown in the following syntax:

```
% ncvlog gate.cs
```

5. Elaborate the design with `ncelab` as shown in the following syntax:

```
% ncelab -loadvpi libxa_vpi.so:snps_vpi_startup -access +rwc top:a
```

6. Prepare SPICE netlist for the SPICE block.

7. Setup mixed-signal simulation control file.

8. Run mixed-signal simulation with `ncsim` as shown in the following syntax:

```
% ncsim -loadvpi libxa_vpi.so:snps_vpi_startup +snps+cosim.cfg top
```

Note:

Bidirectional port in VHDL/PrimeSim XA mixed-signal simulation is not supported.

The following example shows a VHDL on top design of an inverter chain with two leaf inverters assigned to SPICE. The VHDL entity `inv` is replaced by a SPICE subcircuit `inv` for mixed-signal simulation. A Verilog module `inv` is created as the wrapper of the SPICE subcircuit.

Appendix F: Verilog/VHDL/PrimeSim XA VPI Mixed-Signal Simulation Mixed-Signal Simulation With VHDL as the Top Instance

This example uses the following sample files:

- `top.vhd`: The VHDL design
- `gate.cs`: The Verilog wrapper for SPICE subcircuit `inv`
- `test.spi`: SPICE netlist
- `inv_sub.spi`: SPICE netlist
- `cosim.cfg`: Mixed-signal simulation configuration file
- `hdl.var`: NC-Verilog configuration file
- `cds.lib`: NC-Verilog configuration file

```
===== File: top.vhd =====
library ieee;
use ieee.std_logic_1164.all;
library std;
use std.textio.all;
entity top is
end top;
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_textio.all;
library std;
use std.textio.all;
architecture A of top is
    component
        testbench port (z1, z2, z3: in std_logic;
                        a: out std_logic);
        end component;
    component
        cut port (a: in std_logic;
                 z1, z2, z3: out std_logic);
        end component;
    signal a, z1, z2, z3: std_logic;
begin
    tb: testbench PORT MAP (z1, z2, z3, a);
    main: cut PORT MAP (a, z1, z2, z3);
    process (a, z1, z2, z3)
        VARIABLE I: LINE;
        begin
            write( I, now, left, 15);
            write( I, a , right, 3);
            write( I, z1, right, 3 );
            write( I, z2 , right, 3);
            write( I, z3 , right, 3);
            writeline(output, I);
        end process;
end;
library ieee;
```

Appendix F: Verilog/VHDL/PrimeSim XA VPI Mixed-Signal Simulation Mixed-Signal Simulation With VHDL as the Top Instance

```

use ieee.std_logic_1164.all;
entity testbench is
    port (z1, z2, z3: in std_logic;
          a: out std_logic);
end testbench;
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_textio.all;
library std;
use std.textio.all;
architecture behav of testbench is
    signal tick: std_logic;
begin
    process
        variable i: std_logic := '0';
        variable initial: integer := 0;
    begin
        a <= i;
        tick <= i after 3 ns;
        if (now >= 250 ns) then
            wait;
        end if;
        wait for 25 ns;
        i := NOT i;
    end process;
    process(tick)
        variable error: STRING (1 to 7) := "ERROR: ";
    VARIABLE I: LINE;
    begin
        if (now > 0 ns) then
            if (tick /= z1) or
               (tick = z2 ) or
               (tick = z3 ) then
                write( I, error, left, 7);
                write( I, now, left, 15);
                write( I, tick, right, 3);
                write( I, z1, right, 3);
                write( I, z2, right, 3);
                write( I, z3, right, 3);
                writeline(output, I);
            end if;
        end if;
    end process;
end behav;
library ieee;
use ieee.std_logic_1164.all;
entity cut is
    port (a: in std_logic;
          z1, z2, z3: out std_logic);
end cut;
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_textio.all;

```

Appendix F: Verilog/VHDL/PrimeSim XA VPI Mixed-Signal Simulation Mixed-Signal Simulation With VHDL as the Top Instance

```

library std;
use std.textio.all;
architecture gate of cut is component
  my_buf port (a: in std_logic;
              z: out std_logic);
  end component;
  component
  my_inv port(a: in std_logic;
             z: out std_logic);
  end component;
  signal m1, m2, m3: std_logic;
begin
  x1: my_buf PORT MAP (a, m1);
  x2: my_inv PORT MAP (m1, m2);
  x3: my_buf PORT MAP (m2, m3);
  z1 <= m1;
  z2 <= m2;
  z3 <= m3;
  -- process (a, m1, m2, m3)
  -- VARIABLE I: LINE;
  -- begin
  --   write( I, now, left, 15);
  --   write( I, a , right, 3);
  --   write( I, m1, right, 3 );
  --   write( I, m2 , right, 3);
  --   write( I, m3 , right, 3);
  --   writeline(output, I);
  -- end process;
end;
library ieee;
use ieee.std_logic_1164.all;
entity my_inv is
  port(a: in std_logic;
       z: out std_logic);
end my_inv;
library ieee;
use ieee.std_logic_1164.all;
architecture behav of my_inv is
begin
  -- z <= NOT a after 1 ns;
  z <= NOT a;
end;
--library ieee;
--use ieee.std_logic_1164.all;
--entity inv is
--  port(a: in std_logic;
--       z: out std_logic);
--end inv;
--library ieee;
--use ieee.std_logic_1164.all;
--architecture behav of inv is
--begin
--  z <= NOT a after 1 ns;

```



```
--      z <= NOT a;
--end;
library ieee;
use ieee.std_logic_1164.all;
entity my_buf is
    port (a: in std_logic;
          z: out std_logic);
end my_buf;
library work;
use work.all;
architecture gate of my_buf is
    component
        my_inv port(a: in std_logic;
                    z: out std_logic);
    end component;
    component
        inv port(a: in std_logic;
                 z: out std_logic);
    end component;
    signal t: std_logic;
begin
    IV1: my_inv PORT MAP (a, t);
    IV2: inv PORT MAP (t, z);
end gate;

===== File: gate.cs =====
module inv (a, z);
    input  a;
    output z;
    initial $snps_module();
endmodule

===== File: test.spi =====
.param VDDVAL=3v
* global nodes
.global vdd vss gnd
* supplies
vdd vdd 0 dc VDDVAL
vss vss 0 dc 0v
.inc models
.inc inv_sub.spi
.print v(*)
.end

===== File: inv_sub.spi =====
.subckt inv a z
m1 z a vdd vdd p l=0.5u w=5u as=1.0e-10 ad=1.0e-10 ps=0 pd=0
m2 z a 0 0 n l=0.5u w=3u as=1.0e-10 ad=1.0e-10 ps=0 pd=0
.ends
.subckt invs a z vcc
m1 z A vdd vdd p l=0.35u w=400u as=1.0e-10 ad=1.0e-10 ps=0 pd=0
m2 z a 0 0 n l=0.35u w=200u as=1.0e-10 ad=1.0e-10 ps=0 pd=0
.ends

===== File: cosim.cfg =====
set_args spice/test.spi
```

```
===== File: hdl.var =====  
DEFINE WORK cosim_lib  
DEFINE VIEW_MAP (`.cs => cosim, .vhd => vhd, .v => module )  
===== File: cds.lib =====  
INCLUDE /rmnt/tools/cadence/LDV51QSR1/tools/inca/files/cds.lib  
DEFINE cosim_lib ./cosim_lib
```

Mixed-Signal Simulation With SPICE as the Top Instance

In this design flow, the SPICE netlist is the design top instance. Verilog instances are instantiated from the SPICE netlist. In general, circuit designers have the whole SPICE netlist and would like to replace certain digital blocks with Verilog instances.

To run mixed-signal simulation, the `cosim.v` Verilog interface file is automatically generated by executing a PrimeSim XA command against the original SPICE netlist. The `cosim.v` file contains the Verilog top module instantiating Verilog instances to replace SPICE subcircuits. Then mixed-signal simulation is conducted against `cosim.v`, other Verilog source files for digital instances, and the original SPICE netlist.

The procedure to run mixed-signal simulation for this design flow is described as follows:

1. Create a configuration file such as `cosim.cfg` for both PrimeSim XA and mixed-signal simulation containing the following commands:
 - `set_args`: Used with PrimeSim XA command-line options including the SPICE netlist to run PrimeSim XA.
 - `digital_cell` or `digital_cell_inst`: Specifies the Verilog instances in the SPICE netlist.
 - `verilog_file`: Specifies the Verilog file containing the Verilog module definitions.

Here is an example `cosim.cfg` file:

```
set_args      spice/test.spi  
digital_cell  invd  
verilog_file  verilog/invd.v
```

In this example, `invd.v` defines the Verilog inverter and `digital_cell` defines the digital partitions instantiated in `cosim.v` which is generated by PrimeSim XA.

2. Run PrimeSim XA with the configuration file. PrimeSim XA stops simulation after generating `cosim.v` as shown in the following example:

```
% xa -cscfg cosim.cfg
```

After `cosim.v` is generated, you can skip step 2 in future mixed-signal simulation runs if analog and digital partitioning and the Verilog port analog and digital interface definitions remain unchanged.

3. Use the Verilog compiler to compile `cosim.v` together with other Verilog source files.
4. Start mixed-signal simulation from the top Verilog module defined in `cosim.v` and the SPICE netlist. PrimeSim XA skips simulating the SPICE subcircuits specified in `digital_cell` or `digital_cell_inst` commands.

The following example presents a simple inverter chain with a SPICE netlist on top and two leaf inverters partitioned to Verilog. Sample files for include the following:

- `cosim.v`: Verilog top module that instantiates two Verilog inverters. This digital interface file is automatically generated by PrimeSim XA.
- `test.spi`: SPICE top netlist of an inverter chain.
- `inv.spi`: SPICE netlist for an inverter.
- `invd.spi`: SPICE netlist of the inverter to be partitioned to Verilog.
- `buf.spi`: A SPICE inverter chain.
- `invd.v`: A Verilog inverter module.

The sample files are as follows:

Example 62 `cosim.v`

```
`timescale 1ns / 10ps
module top;
  wire w1; // x1.n1
  wire \x1.n1 = w1;
  wire w2; // out1
  wire \out1 = w2;
  wire w3; // x3.n1
  wire \x3.n1 = w3;
  wire w4; // out
  wire \out = w4;
  // Instance section
  invd \x1.x2 (w1, w2);
  invd \x3.x2 (w3, w4);
  // interface nodes
  initial begin
    $snps_a2d_node(w1, "x1.n1");
    $snps_d2a_node(w2, "out1");
    $snps_a2d_node(w3, "x3.n1");
    $snps_d2a_node(w4, "out");
  end
  initial $snps_module(1);
  // By default, spiceflow mixed-signal simulation will
  // use .tran time for the simulation time
  // To specify the simulation time from verilog, add
  // the "spice_finish 0" command
  // in the cosim config file
  // initial begin
```

```
// #100 $finish;
// end
endmodule
```

Example 63 *test.spi*

```
.param VDDVAL=3v
* global nodes
.global vdd vss gnd
* supplies
vvdv vdd 0 dc VDDVAL
vgnd gnd 0 dc 0v
* top level netlist
x1 in out1 buf
x2 out1 out2 inv
x3 out2 out buf
x4 out dummy inv
.inc models
.inc inv.spi
.inc invd.spi
.inc buf.spi
vin in 0 pwl 0n 0v 1n 0v 1.1n 3v 6n 3v 6.2n 0v r
.print v(*)
.tran 0.1n 100n
.end
```

Example 64 *inv.spi*

```
.subckt inv a z
m1 z a vdd vdd p l=0.5u w=5u as=1.0e-10 ad=1.0e-10 ps=0 pd=0
m2 z a 0 0 n l=0.5u w=3u as=1.0e-10 ad=1.0e-10 ps=0 pd=0
.ends
```

Example 65 *invd.spi*

```
.subckt invd a z
m1 z a vdd vdd p l=0.5u w=5u as=1.0e-10 ad=1.0e-10 ps=0 pd=0
m2 z a 0 0 n l=0.5u w=3u as=1.0e-10 ad=1.0e-10 ps=0 pd=0
.ends
```

Example 66 *buf.spi*

```
.subckt buf in out
x1 in n1 inv
x2 n1 out invd
.ends buf
```

Example 67 *invd.v*

```
`timescale 1ns/10ps
module invd (a, z);
input a;
output z;
assign z =~a;
endmodule
```

Note:

To simulate a VHDL block in a SPICE-top flow, create a Verilog wrapper for the VHDL entity with the same port definition and instantiate the VHDL entity in the Verilog wrapper.

Donut Partitioning With Verilog as the Top Instance (V-S-V)

PrimeSim XA VPI does not support V-S-V donut partitioning.

Donut Partitioning With SPICE as the Top Instance (S-V-S)

This section describes the following:

- [Using SPICE-on-Top Partitioning](#)
 - [First Run Example](#)
 - [Second Run Example](#)
-

Using SPICE-on-Top Partitioning

In the S-V-S partitioning flow, the top instance is in SPICE and Verilog instances are instantiated from the SPICE netlist. This is similar to the normal SPICE netlist-on-top flow. Within the Verilog module, subblocks can be simulated in PrimeSim XA. The Verilog module containing analog cells is similar to the top Verilog module in the normal standalone Verilog netlist-on-top flow.

To use SPICE-on-top donut partitioning, perform the following steps:

1. Specify the cosim view of the subblock in the Verilog module to be simulated in PrimeSim XA. This is similar to the Verilog netlist-on-top flow where the `hdl.var` file should contain the following syntax specifying the cosim view:

```
DEFINE VIEW_MAP (.cs => cosim )
```

2. Replace the module body of the subblock in the Verilog module to be simulated in PrimeSim XA with the following syntax line:

```
initial $snps_module();
```

3. Specify the following in the mixed-signal simulation configuration file:

```
set_args spice/test.spi  
digital_cell buffer  
verilog_file verilog/buf.v
```

where `buffer` is the Verilog module name, and `verilog/buf.v` is the Verilog file name.

4. Run PrimeSim XA with the configuration file. PrimeSim XA stops simulation after generating `cosim.v` as shown in the following example:

```
% xa -cscfg cosim.cfg
```

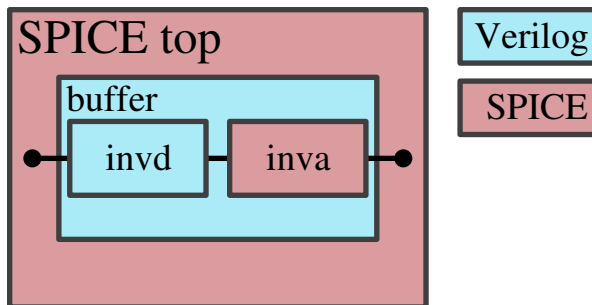
Note:

After `cosim.v` is generated, step 4 can be skipped in future mixed-signal simulation runs if analog and digital partitioning and the Verilog port analog and digital interface definitions remain unchanged.

5. The Verilog compiler is used to compile `cosim.v` together with other Verilog source files.
6. Start mixed-signal simulation from the top Verilog module defined in `cosim.v` and the SPICE netlist. PrimeSim XA skips simulating the SPICE subcircuits specified in `digital_cell` commands while partitioning the subblock specified in step 3 into PrimeSim XA.

Figure 69 is an example of donut partitioning with a SPICE top. Within the SPICE top, the `buffer` is partitioned to Verilog while one of its subblocks, `inva`, is partitioned to be simulated in PrimeSim XA.

Figure 69 SPICE Top Mixed-Signal Simulation Working in Donut Partitioning



First Run Example

```
% xa -cscfg cosim.cfg
```

Second Run Example

```
% ncvlog verilog/cosim.v verilog/buf.v verilog/gate.cs
% ncelab -loadvpi libxa_vpi.so:snps_vpi_startup -access +rwc \
  -LIBNAME cosim_lib cosim_lib.top -SNAPSHOT cosim_lib.top:cosim
```

```
% ncsim -loadvpi libxa_vpi.so:snps_vpi_startup +snps+"cosim.cfg" \
cosim_lib.top:cosim
```

Verilog and SPICE Files

```
===== test.spi =====
.param VDDVAL=3v

* global nodes
.global vdd vss gnd

* supplies
vvdd vdd 0 dc VDDVAL
vgnd gnd 0 dc 0v

* top level netlist
x1 in out1 buffer

.inc models
.inc inva.spi
.inc invd.spi
.inc buf.spi

vin in 0 pwl 0n 0v 1n 0v 1.1n 3v 6n 3v 6.2n 0v
.print v(*)
.tran 0.1n 10n

.end

===== inva.spi =====
.subckt inva a z
m1 z a vdd vdd p l=0.5u w=5u as=1.0e-10 ad=1.0e-10 ps=0 pd=0
m2 z a 0 0 n l=0.5u w=3u as=1.0e-10 ad=1.0e-10 ps=0 pd=0
.ends

===== invd.spi =====
.subckt invd d_a d_z
m1 d_z d_a vdd vdd p l=0.5u w=5u as=1.0e-10 ad=1.0e-10 ps=0 pd=0
m2 d_z d_a 0 0 n l=0.5u w=3u as=1.0e-10 ad=1.0e-10 ps=0 pd=0
.ends

===== buf.spi =====
.subckt buffer in out
x1 in n invd
x2 n out inva
.ends buffer

===== buf.v =====
'timescale 1ns/10ps

module buffer (in, out);
    input    in;
```

```

    output    out;
    wire      n;
    invd inst_invd (in, n);
    inva inst_inva(n, out);
endmodule

module invd (d_a, d_z);
    input    d_a;
    output   d_z;

    assign    d_z = ~d_a;
    always @ (d_a) $display("%t invd : d_z = %v", $time, d_z);
endmodule

===== gate.cs =====
'timescale 1ns/10ps

module inva (a, z);
    input    a;
    output   z;

endmodule

```

Save-Restart in Mixed-Signal Simulation

Mixed-signal simulation allows you to save the complete simulation state. Simulation can be restarted at a later time by loading the simulation state and continued from where it was saved. The simulation state is saved to a Verilog snapshot and a PrimeSim XA save file, *.save.ic. You can restart the simulation by invoking the Verilog snapshot.

To save a simulation state, you can get into `ncsim` interactive mode and apply the `save` command as shown in the following example:

```

ncsim> run clean
ncsim> save snapshot_name

```

To restart the simulation, use the `ncsim` command with the saved snapshot as shown in the following example:

```
% ncsim snapshot_name
```

Restart from within `ncsim` interactive mode is not supported.

Configuration File Commands

This section lists the configuration file commands used in Verilog/VHDL/PrimeSim XA mixed-signal simulation.

The value sets of some common configuration command arguments are as follows:

<bool>

0, 1

<positive number>

1, 2, 3, and so on

<double>

Floating point number

<time>

Floating point number plus time unit. For example, 100p and 1n stand for 100 pico seconds and 1 nano second, respectively.

<file>

File name

For details on the configuration file commands, see the following:

- [analog_cell](#)
- [define_print_variable](#)
- [define_strength](#)
- [digital_cell](#)
- [digital_cell_inst](#)
- [dump_interface](#)
- [dump_port_prop](#)
- [dump_setting](#)
- [map_subckt_name](#)
- [map_unfound_port](#)
- [set_args](#)
- [set_intr_mode](#)
- [set_port_prop](#)
- [verilog_file](#)

analog_cell

Generates Verilog module templates containing the `$snps_module()` statement for analog partitions in the Verilog as the top instance flow.

Syntax

```
analog_cell [-ext <file name extension>] [-dir <directory>] <cell 1>  
  <cell 2> -vmod <verilog sub-module name>...
```

Arguments

`-ext`

Specifies the file name extension for the generated Verilog module templates.
The default file name extension is `.cs`.

`-dir`

Specifies the directory to put the generated Verilog module template. The default directory is the current working directory.

`cell name`

Can be a wildcard.

`-vmod`

Specifies the submodule that remains in Verilog. Do not use wildcards.

Note:

When using the `-vmod` option, only one cell within an `analog_cell` command may be used.

Description

The `analog_cell` command generates Verilog module templates containing the `$snps_module()` statement for analog partitions in the Verilog as the top instance flow. If the design module of an analog partition does not exist in the design library, mixed-signal simulation stops after the template is generated. Then this new file must be compiled in order to start mixed-signal simulation. If the design module of an analog partition already exists in the design library, `analog_cell` does not generate the module template.

define_print_variable

Defines a print variable used as a reference voltage in the `set_port_prop` command.

Syntax

```
define_print_variable <print variable name> = <expression>
```

Description

This command defines a print variable used as a reference voltage in the `set_port_prop` command. The print variable is added to the `snps_cosim.sp` netlist file with a SPICE `.print` statement.

Note:

The syntax for the print variable in `define_print_variable` is identical to the `.print` statement syntax.

define_strength

Defines a strength table with resistances mapped to the seven Verilog strength levels.

Syntax

```
define_strength <strength table name> [<double>] [-<strength option>  
<double>] [-<strength option> <double>] ...
```

Description

This command defines a strength table with resistances mapped to the seven Verilog strength levels.

Each `-<strength option>` is used to map to the corresponding Verilog strength level and can be any of the following:

- `-supply`
- `-strong`
- `-pull`
- `-weak`
- `-large`
- `-medium`
- `-small`

The value inserted after `-<strength option>` is a strength resistor's resistance. If a value does not have an associated `-<strength option>`, it is set as the default value for the remaining strength levels not specified using the `-<strength option>`.

`<strength table name>` is used in the `-strength` port property of the `set_port_prop` command for strength resolution at inout ports. Verilog inputs are applied through the resistor with respect to the Verilog strength level and PrimeSim XA resolves contributions of both the Verilog- and SPICE-sides in order to obtain the final bidirectional net value.

digital_cell

Specifies the SPICE subcircuit to be partitioned to Verilog.

Syntax

```
digital_cell <sub-circuit name>
```

Description

In a SPICE flow, specifies the SPICE subcircuit to be partitioned to Verilog.

digital_cell_inst

Specifies the SPICE instance to be partitioned to Verilog.

Syntax

```
digital_cell_inst <SPICE instance name>
```

Description

In a SPICE flow, specifies the SPICE instance to be partitioned to Verilog.

dump_interface

Produces a report file showing the mapping result between analog and digital ports.

Syntax

```
dump_interface [0|1|2]
```

Arguments

0

Do not dump the `.csintf` file.

1

Generates the `.csintf` file that lists all interface nodes and properties.

2

(Default) Generates the `.csintf` file at the end of mixed-signal simulation that lists all interface nodes, properties, and the number of interface events for each interface node.

Description

This command produces a report file showing the mapping result between analog and digital ports.

Examples

Here is a `.csintf` file example.

```
a2d main.out2 xmain.out2 node=out2 vhi=2.1 vlo=0.9
d2a main.out1 xmain.out1 node=out1 logichv=3 logiclv=0 rise=1000
    fall=1000 rm_glitch=1000
```

- Column two lists the Verilog ports.
- Column three lists the PrimeSim XA ports.

dump_port_prop

Outputs port properties associated with matching ports.

Syntax

```
dump_port_prop <file>
```

Description

Outputs what port properties have been associated with the matching ports.

dump_setting

Outputs configuration command settings to the PrimeSim XA log file.

Syntax

```
dump_setting <bool>
```

Description

Outputs configuration command settings to the PrimeSim XA log file. Default for `<bool>` is 0.

map_subckt_name

Maps a module name to the correct subcircuit definition in the SPICE instantiation.

Syntax

```
map_subckt_name <module_name> <subckt_name>
```

Description

If the module name is different from the subcircuit name, this command maps it to the correct subcircuit definition in the SPICE instantiation.

map_unfound_port

Maps an unfound port to the specified SPICE node name.

Syntax

```
map_unfound_port [-cell <pattern>] <map_node> <unfound_port> ...
```

Description

When writing the interface netlist file, if a subcircuit has more ports than inst module ports, this command maps the unfound port to the specified SPICE node name.

The search priority is in a top-down order as follows:

1. Exact cell name.
 2. Match cell pattern.
 3. Match unfound port list for rules without the `-cell` argument.
-

set_args

Passes the regular PrimeSim XA command-line argument to the PrimeSim XA tool.

Syntax

```
set_args <snps_args> ...
```

Description

This command passes the regular PrimeSim XA command-line argument to the PrimeSim XA tool. For example: `set_args test.spi` directs the PrimeSim XA tool to accept `test.spi` as an input netlist.

set_intr_mode

Sets the interactive mode.

Syntax

```
set_intr_mode <bool>
```

Description

By default, Ctrl+C stops simulation in the Verilog simulator's interactive mode. To move between the interactive modes of the Verilog simulator and the PrimeSim XA tool, use the following commands:

- `call snps_intr_mode`: Leaves the Verilog simulator's interactive mode and enters the PrimeSim XA interactive mode.
- `quit`: Leaves the PrimeSim XA interactive mode and returns to the Verilog simulator's interactive mode.

If `set_intr_mode` is set to 1, Ctrl+C stops the simulation in PrimeSim XA's interactive mode instead of the Verilog simulator's interactive mode. PrimeSim XA interactive commands can be applied to debug the simulation. In this case, Verilog's interactive mode cannot be entered by users. Default for `<bool>` is 0.

set_port_prop

Description

Applies specified properties to matched cells or instances and their ports.

The port properties apply to the matched cells or instances and their ports.

- `-cell` is used for cell based port properties.
- `-inst` is used for instance-based port properties.
- Port names match Verilog port definitions that are case-sensitive.
- This specified value overrides any default value.
- If more than one rule is found for a particular property, the last rule is used.
- Without any cell or port pattern specified, the default value is used.

The options for port properties are listed below under **Arguments**.

Syntax

```
set_port_prop [-cell <pattern>|-inst <pattern>]  
[-port <pattern>] <-port property1> <value1> <-port property2> <value2> ...  
[-follow_ov_param] [-no_a2d <bool>] [-no_d2a <bool>]  
[-cap capacitance_value] [-res resistance_value] [-linear <bool>]  
[-csrc <bool>]
```

Arguments

`-logichv <double>|<output variable>`

[default] `v(vdd)` where `vdd` is the detected voltage supply node. If no detected `vdd`, the default is 3.

Set port logic1 voltage.

Its value can be a double number or an output variable which is a string identifier starting with an alphabetic letter.

The output variable is defined with a `.print` statement to represent a voltage expression. For example:

```
.print logichv=par('0.7 * v(vdd)')
```

where `logichv` is the output variable and `'0.7 * v(vdd)'` is the voltage expression.

`-logiclv <double>|<output variable>`

[default] 0

Set port logic0 voltage.

Its value can be a double number or an output variable which is a string identifier starting with an alphabetic letter.

`-logicxv <double>|<output variable>`

[default] `(logichv - logiclv) * 0.5`

Set port logic X voltage.

Its value can be a double number or an output variable which is a string identifier starting with an alphabetic letter.

`-vhi <double>|<output variable>`

[default] `(logichv - logiclv) * 0.7`

Set port logic1 threshold voltage.

Its value can be a double number or an output variable which is a string identifier starting with an alphabetic letter.

`-vlo <double> <output variable>`

[default] `(logichv - logiclv) * 0.3`

Set port logic0 threshold voltage.

Its value can be a double number or an output variable which is a string identifier starting with an alphabetic letter.

`-timex <time>`

[default] No state X report.

Report X when the output port voltage stays between `-vlo` and `-vhi` longer than the specified `<time>`.

`-slope <time>`

[default] 1p

Set port rising and falling time.

`-rise <time>`

[default] 1p

Set port rising time.

`-fall <time>`

[default] 1p

Set port falling time.

`-delay <time>`

[default] 0

Set port delay. This delays the signal output to Verilog.

Only allow positive delay.

`-delay1 <time>`

[default] 0

Apply port delay to the rising edge only.

`-delay0 <time>`

[default] 0

Apply port delay to the falling edge only.

`-delay_hz2st <time>`

[default] 0

Apply port delay to signal changes from a Hi-Z state to a strong state.

`-rm_glitch <time>`

[default] `-slope` value

Remove glitches within `<time>` after Verilog input changes.

Apply to inout port.

`-strength <strength name>`

[default] no strength

`<strength name>` is a string identifier defined in `define_strength`.

Apply to inout port for strength resolution with the resistor specified in `<strength name>`.

`-vsrc <bool>`

[default] 0

Model input as a voltage source. It is partitioned into a smaller block and results in a faster simulation runtime.

Only inputs without Hi-Z can use this option, otherwise, the simulation may be incorrect.

`-vprint <bool>`

[default] 0

Insert a `.print` statement to print voltage value.

`-lprint <bool>`

[default] 0

Insert an `.lprint` statement to print voltage logic.

`-follow_ov_param`

Allows a user-defined expression to be dynamically evaluated along two digital-to-analog (d2a) events (as opposed to being evaluated at the final value). Because this option allows a dynamic change on the logic values of d2a interface elements, you can model complex voltage-controlled voltage sources, ideal level shifters, interface elements, and modulated voltage sources.

`-no_a2d <bool>`

[default] 0

Skips the a2d interface element insertion on the specified interface node.

`-no_d2a <bool>`

[default] 0

Skips the d2a interface element insertion on the specified interface node.

`-cap capacitance_value`

[default] 0

Adds node capacitance value to the interface node. The capacitance value is in double precision and has the unit of Farad.

```
-res resistance_value
```

```
[default] 1
```

Adds node resistance value to the interface node. The resistance value is in double precision and has the unit of Ohm. This value is used only when `-linear` option is 0.

```
-linear <bool>
```

```
[default] 1
```

Generates a linear slope for d2a interface.

```
-csrc <bool>
```

```
[default] 0
```

Model interface as constant `vsrc`.

verilog_file

Specifies Verilog source file containing Verilog module definitions for [digital_cell](#) or [digital_cell_inst](#).

Syntax

```
verilog_file <Verilog source file name>
```

Description

In a SPICE flow, this command specifies the Verilog source file containing the Verilog module definitions for `digital_cell` or `digital_cell_inst`. `verilog_file` can be applied multiple times for different Verilog sources.

```
set_args xa_top.sp
set_rise_step 10
set_fall_step 6
set_port_prop -cell top -port outport1 outport2 \
  outport3 -vhi 2.64 -vlo 0.66
set_port_prop -cell top -port inport* \
  -logichv 3.3 -logiclv 0 -slope 100ps
```

Automatic Voltage Level Detection

Mixed-signal simulation can automatically identify voltage levels at interface nodes thereby reducing the need for user intervention. The rules described in [Voltage Setting Rules](#) have the following precedence: Rule 1 and Rule 2.

Voltage Setting Rules

Rule 1

The `set_port_prop` configuration command provides the flexibility to overwrite both default and automatically detected voltages.

Rule 2

Search through channel connected voltage sources. The voltage levels of the voltage sources are applied to the interface nodes. Rule 1 takes higher priority.

Mixed-Signal Simulation Interactive Mode

The mixed-signal simulation interactive commands add to the command set described in the *PrimeSim XA Command Reference: PrimeSim XA Interactive Commands*. The mixed-signal simulation interactive mode allows information to be obtained on both interface elements and interface activity history. It also permits watchpoints to be set on interface node activities to catch a specific event in the interactive debugging mode.

Mixed-signal simulation interactive commands are used with `XA>`, the PrimeSim XA interactive mode prompt. The *PrimeSim XA Command Reference: PrimeSim XA Interactive Commands* provides information on how to get into the PrimeSim XA interactive mode.

To get into PrimeSim XA interactive mode from the `ncsim>`, the NC-Verilog interactive prompt, use the following command:

```
call snps_intr_mode
```

To continue simulation in NC-Verilog, issue the command `cont` from the PrimeSim XA interactive mode prompt `XA>` and simulation continues. If you are prompted with `ncsim>` after issuing the `cont` command at `XA>`, type `run` and NC-Verilog continues.

Note:

Currently only NC-Verilog is supported in mixed-signal simulation interactive debugging.

[Table 14](#) lists the commands used in mixed-signal simulation interactive debugging.

Table 14 *Mixed-Signal Simulation Interactive Mode Commands*

Command	Function
<code>csli</code>	List Interface Nodes
<code>csh</code>	Print Global Interface History in Time
<code>csnh</code>	Print Interface Node History by Node Name
<code>csinh</code>	Print Interface Node History by Node Index
<code>csnph</code>	Set the Number of Entries Printed by <code>csnh</code> and <code>csinh</code>
<code>csnw</code>	Set Watchpoint to Interface Node by Node Name
<code>csinw</code>	Set Watchpoint to Interface Node by Node Index
<code>csdnw</code>	Delete Watchpoint by Node Name
<code>csdinw</code>	Delete Watchpoint by Node Index

List Interface Nodes

csli

```
csli <pattern> <-a2d|-d2a|-biput>
```

`csli` lists all mixed-signal simulation interface nodes if no option is specified. A pattern can be used to search for certain names. You can choose to list a certain type of interface node by specifying `-a2d`, `-d2a`, or `-biput`.

Table 15 *List Interface Nodes: csli Syntax Descriptions*

Parameter	Description
<i>pattern</i>	Pattern used to search for certain interface node names. Pattern matching is based on the Tool Command Language (Tcl) API.
<code>-a2d</code>	Lists only a2d (PrimeSim XA to Verilog) interface nodes.
<code>-d2a</code>	Lists only d2a (Verilog to PrimeSim XA) interface nodes.
<code>-biput</code>	Lists only bidirectional interface nodes.

Table 16 *PrimeSim XA Example*

XA> csl	Prints all interface nodes.
XA> csl *addr* -d2a	Prints d2a interface nodes with names matching the pattern *addr*.

A typical result of `csl` command is shown in the following example. Note that `<=>` denotes bidirectional ports:

```
XA> csl
cosim interface nodes:
-----
id      type      node
-----
7       <=>a2d      b[3]
6       <=>a2d      db[2]
5       <=>a2d      db[1]
4       <=>a2d      db[0]
10      d2a        pch2
12      d2a        rd2
15      d2a        wr2
7       <=>d2a      db[3]
3       d2a        addr[2]
6       <=>d2a      db[2]
2       d2a        addr[1]
5       <=>d2a      db[1]
1       d2a        addr[0]
4       <=>d2a      db[0]
8       d2a        en2
-----
```

Note:

In this example, the bidirectional ports have both a2d and d2a interface nodes:
7 <=>a2d db[3] and 7 <=>d2a db[3].

Print Global Interface History in Time

csh

`csh <number of entries (default is 10)>`

`csh` prints the global interface activity history in chronological order. If no argument is specified, `csh` prints the maximum number of entries available up to a maximum of 10 entries. If the number of entries is specified, `csh` prints up to the specified number of entries. The maximum number of global history entries is set to 10000 by default.

The number can be changed by using the `max_history` command in the mixed-signal simulation configuration file as follows:

```
max_history <max # of global history entries>
```

Table 17 *PrimeSim XA Example*

XA> csh	Prints 10 global interface activity history entries.
XA> csh 20	Prints 20 global interface activity history entries.

Print Interface Node History

csnh, csinh

```
csnh <name>
csinh <id>
```

`csnh` and `csinh` print the activity of a specified interface node if available. Entries for the specified node stored in the history buffer are printed in chronological order. Both a2d and d2a history are printed if available. The maximum number of entries printed each time by `csnh` and `csinh` can be set by the command `csnph`. The default is 10.

The `id` corresponds to the `id` field in the output of the `csli` command. This `id` can also be used in other PrimeSim XA interactive commands.

Table 18 *PrimeSim XA Example*

XA> csnh db[3]	Prints activity history of interface node on db[3].
XA> csinh 10	Prints activity history of interface node with index 10.

Set the Number of Entries Printed by csnh and csinh

csnph

```
csnph <number of entries>
```

`csnph` reports the current setting if no argument is given. If an argument is specified, the number of entries to be printed by the `csnh` and `csinh` commands are set. The number is limited between `max_history` and 0.

Table 19 *PrimeSim XA Example*

XA> csnph	Prints current csnph setting.
XA> csnph 20	Sets the maximum number of entries to be printed in each csnh and csinh call to 20.

Set Watchpoint to Interface Node

csnw, csinw

```
csnw <name> <-a2d|-d2a|-hz>
csinw <id> <-a2d|-d2a|-hz>
```

csnw and csinw set a watchpoint to the specified interface node. If no additional option is given, any activity on the interface node triggers the watchpoint and you enter the XA> prompt. Use -a2d, -d2a, or -hz to catch a specific type of interface activity. If no argument is given to csnw and csinw, a list of current watchpoints is printed. Previous watchpoint settings are overridden by the new setting.

Table 20 *Set watchpoint to interface node: csnw csinw Syntax Descriptions*

Parameter	Description
name	Interface node name to which the watchpoint is set.
id	Interface node ID to which the watchpoint is set.
-a2d	Watch for a2d (PrimeSim XA to Verilog) activity only.
-d2a	Watch for d2a (Verilog to PrimeSim XA) activity only.
-hz	Watch for Hi-Z event only.

Table 21 *PrimeSim XA Example*

XA> csnw	Prints the list of currently set watchpoints.
XA> csnw addr[2]	Sets watchpoint on interface node addr[2].
XA> csinw 5 -hz	Sets watchpoint on interface node with id 5 to watch for Hi-Z events.
XA> csnw db[0] -d2a	Sets watchpoint on d2a part of interface node db[0].

Delete Watchpoint

`csdnw` and `csdinw` delete the watchpoint specified by name or ID, or delete all watchpoints if `-a` or `-all` option is used. If no argument is given, `csdnw` and `csdinw` print the list of currently set watchpoints.

`csdnw`, `csdinw`

```
csdnw <name|-a|-all>
csdinw <id|-a|-all>
```

Table 22 *PrimeSim XA Example*

XA> <code>csdinw</code>	Prints the list of currently set watchpoints.
XA> <code>csdnw db[1]</code>	Deletes the watchpoint on <code>db[1]</code> .
XA> <code>csdinw 4</code>	Deletes the watchpoint on node with id 4.

Verilog System Tasks for Mixed-Signal Simulation

The following system tasks are available for interactions between Verilog and SPICE. They are incorporated into Verilog source code to pass data to or retrieve data from analog blocks. System tasks should be put into the initial block of a Verilog module.

```
$snps_a2d_node (net, "SPICE node name")
```

Creates an A-to-D interface element between the Verilog net and SPICE node. This system task behaves like a continuous assignment from a SPICE internal node to the Verilog net.

```
$snps_add_cap (net, variable)
```

This system task adds capacitance to the SPICE node connecting to the interface net. It requires two arguments, one Verilog net and one Verilog variable, constant, or parameter of real type. The Verilog net has to be an interface connecting to a SPICE node. The second argument specifies capacitance in Farad and represents the lumped sum capacitance of Verilog side components connecting to the SPICE node.

```
$snps_d2a_node (net, "SPICE node name")
```

Creates a D-to-A interface element between the Verilog net and SPICE node. This allows Verilog to connect the net directly to a SPICE internal node, instead of going through port mapping. For example,

```
initial $snps_d2a_node (test, "xi1.xi2.sync");
```

where `test` is a Verilog net in the module containing this system task and `xi1.xi2.sync` is the hierarchical path name to identify a SPICE node.

```
$snps_get_volt (net, variable)
```

Requires two arguments, one Verilog net and one Verilog variable of real type. The Verilog net has to be an interface connecting to a SPICE node. This system task retrieves the analog voltage of the SPICE node at current time and assigns the voltage to the variable.

```
$snps_inout_node (net, "SPICE node name")
```

Creates one D-to-A interface element and one A-to-D interface element between the Verilog net and SPICE node. This is equivalent to one `$snps_d2a_node()` and one `$snps_a2d_node()` combined.

```
$snps_module()
```

Designates the current module to be partitioned into an SPICE subcircuit. The module body should contain nothing but only one initial block of this system task.

```
$snps_save_waveform(obj1 [, level1], obj2 [, level2], ...)
```

Allows Verilog object waveforms to be saved to the PrimeSim XA waveform file. The Verilog objects can be net, register, net bit, register bit, and module instance. The optional level argument is valid only for module instance objects and specifies all nets under the design hierarchy level. Its default value is 1. A 0-level means the full hierarchy of the given instance.

```
$snps_set_volt (net, variable)
```

Requires two arguments, one Verilog net and one Verilog variable, constant, or parameter of real type. The Verilog net has to be an interface connecting to a SPICE node. This system task assigns the value of the second argument to the SPICE node as an analog voltage at current time.

Mixed-Signal Simulation Setup Guidelines

There are several areas that require special attention during mixed-signal simulation setup in order to help simulation performance and avoid common mistakes:

- [Map Correct Port Voltages](#)
- [Define Clear Port Direction](#)
- [Set Input Ports As Voltage Sources If Possible](#)
- [Define SPICE Netlist Bus Notation](#)
- [Handle Bidirectional Ports](#)

Map Correct Port Voltages

This is especially true for `logicxv`: Verilog assigns logic X value to the nets which are not initialized. For input ports from Verilog to PrimeSim XA, PrimeSim XA takes port voltages for DC initialization and simulation. It is important to map a correct analog voltage for logic X value at input ports. Some circuits require logic X to have the same analog voltage as logic0, while some circuits require it to be the middle voltage between logic1 and logic0.

Define Clear Port Direction

If a port direction is known to be unidirectional for the SPICE block, its corresponding mixed-signal simulation view Verilog module should clearly define an input or output port rather than an inout port. This reduces the number of interface elements and improve simulation performance.

Set Input Ports As Voltage Sources If Possible

If the input from Verilog to PrimeSim XA never becomes HiZ, this input can be treated as a voltage source to the SPICE block. This improves PrimeSim XA simulation. Use the `-vsrc` option of the `set_port_prop` configuration command to set ports as voltage sources.

Define SPICE Netlist Bus Notation

Usually, Verilog defines vector nets at ports. The SPICE netlist only has bit-level port definition. A bus notation is required to map each individual bit-level port back to Verilog vector ports. The default bus notation for SPICE netlist is square brackets `[]`. Other bus

notations can be set by using a PrimeSim XA netlist option in the SPICE netlist as shown in the following example:

```
.opt XA_BUS_FORMAT = "<%d>"  
.opt XA_BUS_FORMAT = "%d"
```

Handle Bidirectional Ports

If strength fighting occurs at bidirectional interface ports, use the `-strength` option for `set_port_prop` and `define_strength` to map Verilog strengths to the proper resistance values.

Partitioning Guidelines

This section describes the following:

- [Partition Boundary With Clear Digital Behavior](#)
- [Avoid Partitioning at Timing Sensitive Signals](#)
- [Avoid Reach-in Signals in Analog Partitions](#)
- [Avoid Partitioning at Bidirectional Signals Involved Strength Fighting and Pass Switches](#)
- [Avoid Fine Grain Partitioning](#)

Partition Boundary With Clear Digital Behavior

In mixed-signal simulation, digital and analog signals are presented on two sides of a partition boundary. In order to reduce the loss of accuracy and to maintain correct functionality, the boundary signals should have clear digital behavior and should not be voltage sensitive.

Avoid Partitioning at Timing Sensitive Signals

The signal conversion from analog to digital depends on high and low threshold voltages. If the circuit design is timing sensitive at the interface signals, functionality errors may occur due to timing shift by a slight change in threshold voltages. There should be certain timing error margin for the interface signals. Also, the timing representation in Verilog may not match the exact timing in SPICE. It is recommended not to partition at timing sensitive signals.

Avoid Reach-in Signals in Analog Partitions

Verilog elaboration fails when the Verilog netlist contains a reach-in signal partitioned into an analog partition whose internal objects are not visible to Verilog elaborator.

Reach-in signals can be replaced with a new Verilog net using either of the following system tasks; depending on the direction of the original reach-in signal.

```
$snps_a2d_node()  
$snps_d2a_node(),
```

The system task associates the new Verilog net to the SPICE node that is equivalent to the original reach-in signal.

Avoid Partitioning at Bidirectional Signals Involved Strength Fighting and Pass Switches

Bidirectional interface signals are supported. However, the signal value set by VPI at one terminal of a pass switch (the primitive gate tranif0 and tranif1) cannot be propagated to the other end. A Verilog n-MOSFET gate is added in between the pass switch and the interface signal to allow signal value passing through the pass switch. If the bidirectional interface signals involve strength fighting, the final signal value is resolved by PrimeSim XA. A resistor is added to incorporate the contribution of the digital signal in resolving the final value. Special attention is required to map the resistance value, specified by the `set_port_prop` configuration command, to its corresponding digital strength.

Avoid Fine Grain Partitioning

Fine granularity partitioning creates many small analog and digital blocks and introduces many interface signals which decrease mixed-signal simulation performance. Frequent and unnecessary analog and digital signal conversion may also introduce functionality errors.

Strength Table Setup Guidelines

Multiple drivers can drive the same net using different values. The final value of the net depends on the strengths of the drivers. Strength fighting may occur at bidirectional nets or inout ports of digital and analog partitions. Verilog defines seven strength levels and rules to resolve strength fights. PrimeSim XA models Verilog strength as a resistor. The Verilog signal input is applied through the resistor and PrimeSim XA resolves both Verilog and SPICE contributions to obtain the final values of the bidirectional nets.

A strength table defines a set of resistance strength values that are mapped to Verilog seven strength levels for use in strength resolution at inout ports. If Verilog-side signals

always win during strength fighting or there is no strength fighting at inout ports, it is not necessary to introduce strength resistors.

The `define_strength` configuration command specifies the resistances of strength resistors to map to the Verilog seven strength levels. The syntax is shown in the following example.

```
define_strength strength_tbl -supply 10 -strong 100 \  
-pull 1000 -large 10000 -weak 100000 \  
-medium 1000000 -small 10000000
```

The resistance presents a Verilog strength relative to a lumped sum SPICE impedance at the bidirectional net. The SPICE impedance depends on the transistor model, technology, and process used in the design. Therefore, a default strength table does not satisfy the requirement because the relative resistances are both design-dependent and port-dependent.

Data flow direction must be available in order to select proper resistances. If the Verilog-side signal wins the strength fight, the strength resistor's resistance must be significantly smaller than SPICE-side impedance. Conversely, if the SPICE-side signal wins the strength fight, the resistance of the strength resistor must be significantly larger than the SPICE-side impedance. The following two examples show how this works:

Examples

Assume the following for this example:

- Strength fighting occurs at port Y
- Simulation time is 10 ns
- The Verilog-side presents a weak logic0
- The SPICE-side has 3V and logic1 before strength resolution
- Data flows from the SPICE- to the Verilog-side at the 10 ns mark indicating that the SPICE-side driving strength is stronger.

In this example, the final value at port Y should be logic1.

If the impedance at the SPICE-side is 1000 Ohms, then the proper resistance of the strength resistor can be 10000 Ohms; in which case PrimeSim XA does the following:

- Resolves that the voltage at port Y to be 2.8V
- Sets port Y to logic1

In this case, the weak Verilog strength is mapped to a 1000 Ohm strength resistor.

- Later, at simulation time 20 ns:
- The Verilog-side presents a strong logic0 at port Y

- SPICE-side voltage is 3V with a driving impedance of 1000 Ohms before strength resolution
- Data flows from the Verilog- to SPICE-side
- The final value at port Y should be logic0

The proper resistance of the strength resistor may be 100 Ohms for PrimeSim XA to resolve the strength fight and produce a final value of 0.5V and a logic0. Therefore, strong Verilog strength is mapped to a 100 Ohm strength resistor.

The preceding two examples show why it is important to know the data flow direction to select proper resistance values. Designers specify strength resolution as follows:

- Full Verilog netlist: Data flow direction is specified by setting different Verilog strength levels that drive the same net.
- Full SPICE netlist: Data flow direction is determined by different size transistors connecting to the same net.
- Mixed-signal Simulation netlist: Data flow information is not available. Signal strength tables are constructed from information provided by designers to determine data flow directions.

The report_port_resistance configuration command creates a report that details the SPICE-side resistance or impedance from interface nodes to voltage sources. Strength tables can be constructed from the data flow directions in a circuit design and the SPICE-side path resistance.

Note:

The resistance difference between two consecutive Verilog strength levels can be one (1) order of magnitude such that if 100 Ohms is described a strong level then the pull level can be 1,000 Ohms.

Mixed-Signal Simulation With ModelSim

Verilog/VHDL is used with PrimeSim XA to simulate mixed Verilog/VHDL and SPICE-based designs that contain both digital and analog partitions. This is accomplished by using the Verilog/VHDL simulator to simulate the digital netlist; while PrimeSim XA simulates the analog SPICE netlist. When complete, analog and/or digital simulation results are available for designers to verify their designs.

In addition to Cadence NCSIM, mixed-signal simulation with Mentor Graphics ModelSim is now supported:

- [ModelSim/PrimeSim XA Integration](#)
- [Running ModelSim/PrimeSim XA Mixed-Signal Simulation With Standalone ModelSim](#)
- [Running ModelSim/PrimeSim XA Mixed-Signal Simulation Under the ADMS Environment](#)

ModelSim/PrimeSim XA Integration

The `libxa_vpi.so` mixed-signal simulation library supports ModelSim integration with either of the following:

- Standalone ModelSim
- In the ADMS environment

In both Verilog and SPICE design partitioning flows, most mixed-signal simulation features and limitations for NC-Verilog/VHDL are applicable to ModelSim. Because the cell view and Verilog configurations for instance-based instantiation are not available in ModelSim, users must modify the original Verilog source files to add the `$snps_module()` system task to designate analog partitions.

Note:

`save-restart` is not supported in ModelSim/PrimeSim XA mixed-signal simulation.

Running ModelSim/PrimeSim XA Mixed-Signal Simulation With Standalone ModelSim

Use the ModelSim commands shown in the following steps to run ModelSim in a standalone ModelSim environment.

1. Create a design library using the following syntax. In this example, the design library is named `work`.

```
% vlib work
```

2. Compile the Verilog source code using the following syntax:

```
% vlog top.v
```

3. Use the `-pli` command-line option to link `libxa_vpi.so` into ModelSim for simulation using the following syntax. In this example, `top` is the name of the top design module.


```
% vsim -c -pli libxa_vpi.so +snps+cosim.cfg top
```

To enter PrimeSim XA interactive mode from ModelSim, press Ctrl+C during simulation to pause at ModelSim interactive prompt, and use the command:

```
% vsim(paused) snps_intr_mode
```

Running ModelSim/PrimeSim XA Mixed-Signal Simulation Under the ADMS Environment

Use the ADMS commands shown the following steps to run ModelSim/PrimeSim XA mixed-signal simulation under the ADMS environment.

1. Create a design library using the following syntax. In this example, the design library is named `work`.

```
% valib work
```

2. Compile the Verilog source code using the following ModelSim syntax. In this example, `-ms` invokes the ModelSim compiler:

```
% valog top.v -ms
```

3. Invoke ModelSim to run the simulation using the following syntax. In this example, `top` is the name of the top design module.

```
% vasim top -ms -pli libxa_vpi.so
```

Note:

The `-c` command-line option does not work for mixed-signal simulation in batch mode.

PrimeSim XA Features Not Supported by Mixed-Signal Simulation

Mixed-signal simulation is designed for transient analysis. The following PrimeSim XA features and parameters are not supported:

- DC Analysis
- AC Analysis
- Monte Carlo Analysis
- Parameter Sweeping Analysis

References

- [1] NC-Verilog/VHDL is a functional verification tool from Cadence Design Systems, Inc.
- [2] Verilog-XL is functional verification tool from Cadence Design Systems, Inc.

G

FineSim VPI Cosimulation

This appendix describes the FineSim Pro tool support for mixed-mode simulation with Verilog simulators using the Verilog Programming Interface (VPI) API.

This chapter covers the following topics:

- [Mixed-Mode Simulation](#)
- [FineSim Pro Tasks](#)
- [Configuration Commands](#)
- [Automatic Verilog Instance Generation](#)
- [Circuit Example: \(test.sp\)](#)
- [Parallel Cosimulation](#)
- [Common Cosimulation Problems](#)

Mixed-Mode Simulation

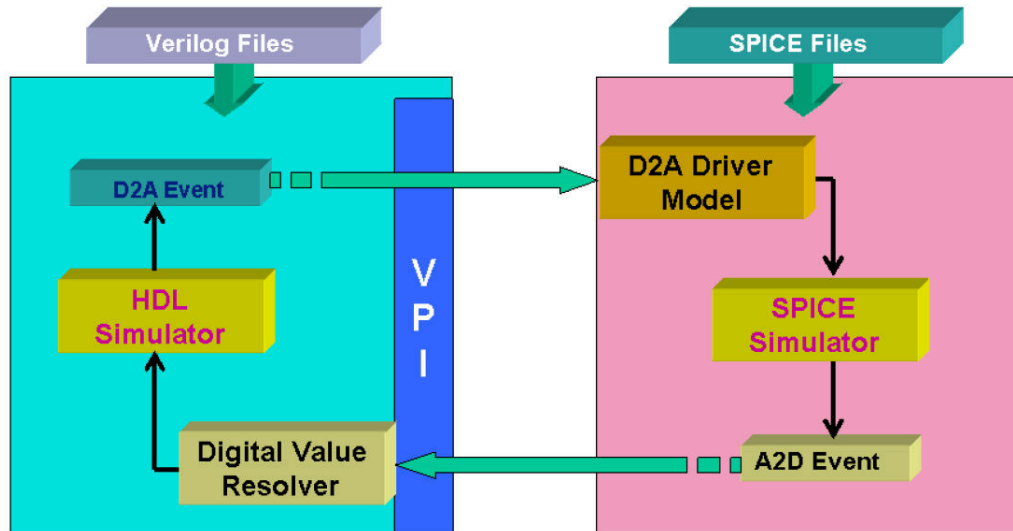
Mixed mode simulation involves combining digital and analog simulators in various ways. However, it has been difficult to find efficient methods for synchronization between the two domains. This is because the analog simulator uses dynamic time step control while the digital simulator uses an event driven paradigm. In the FineSim Pro tool, efficient synchronization algorithms have been developed and applied to mixed-mode simulation. The FineSim Pro tool supports mixed-mode simulation with Verilog simulators using the Verilog Programming Interface (VPI) API. Using this standard API allows the FineSim Pro tool to be used with any vendor's Verilog simulator.

Verilog Cosimulation

The FineSim Pro tool supports mixed-mode simulation with Verilog simulators. This is referred to as Verilog cosimulation. The FineSim Pro tool includes a dynamic library, `finesim.so`, which provides this support. When the Verilog simulator starts it loads

`finesim.so` and interacts with the FineSim Pro tool through the VPI API. The following figure shows the interaction of the digital and analog simulators.

Figure 70 Verilog Cosimulation Flow



Running Verilog Cosimulation

Because the FineSim Pro tool provides Verilog cosimulation through a VPI library, `finesim.so`, the Verilog simulator must load the VPI library. How this is done depends on the Verilog simulator, but generally, the Verilog simulator must be advised of the location of the VPI library, the name of the library, and the name of the start-up function in the library.

Because `finesim.so` is a dynamic library that is loaded by the Verilog simulator, its location usually must be added to the `LD_LIBRARY_PATH` environment variable. This allows the Verilog simulator to find the library.

The FineSim VPI interface has been updated to compile with GCC 4.5. To avoid an environment issue, you should set `$FINESIM_HOME/lib/Linux##` as part of the `LD_LIBRARY_PATH` environment variable before loading `finesim.so` into your Verilog simulator.

The FineSim tool now also includes `finesim.so` in the respective `lib` folder. The previous requirement of setting `LD_LIBRARY_PATH` to `$FINESIM_HOME/finesim/platform/Linux##` is no longer required. Here is an example of starting 64-bit cosimulation with VCS:

```
% source finesim.cshrc
% setenv LD_LIBRARY_PATH $FINESIM_HOME/lib/Linux64:${LD_LIBRARY_PATH}
% vcs -R -full64 +vpi -load finesim.so:finesim_startup \
  +cli+3 ms1.v
```

After the location of the library is set by using the `LD_LIBRARY_PATH`, the name of the library, `finesim.so`, and the name of the startup function, `finesim_startup`, are generally passed as command-line options to the Verilog simulator.

Verilog Simulator Commands

FineSim VPI supports most of the common Verilog simulator commands. Below is some example syntax needed to start Verilog simulation:

ModelSim

```
vsim -c -pli finesim.so
```

NCSim

```
ncverilog +access+rw +loadvpi=finesim.so:finesim_startup ....
```

VCS

```
vcs -R +vpi -load finesim.so:finesim_startup +cli+3 ....
```

VerilogXL

```
verilog +access+rw +loadvpi=finesim.so:finesim_startup
```

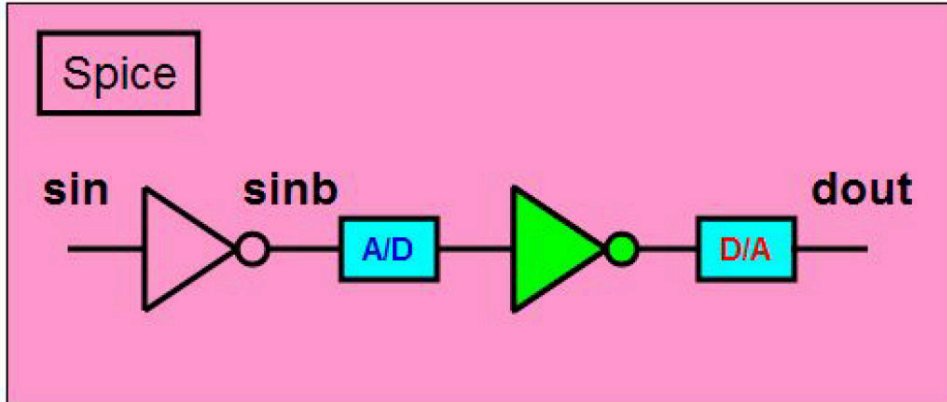
Examples

In general, analog and digital mixed circuits can be categorized into two kinds of circuit styles. One style has the analog netlist as a design top instance, and digital instances are instantiated from the analog netlist. The other style has the digital netlist as a design top instance, and analog instances are instantiated from the digital netlist.

In this section, simple examples with the analog (SPICE) netlist and digital (Verilog) netlist on top are given to show how the two circuit styles can be simulated with the FineSim Pro tool.

SPICE Top Example

Figure 71 Block Diagram of Lab1



In this example the main circuit is the SPICE netlist. The SPICE netlist includes both analog and digital subcircuits. The first inverter is in SPICE, the second is in Verilog format. In SPICE, *sin* is the input of the first inverter. The output of the first inverter, *sinb* goes to the input of the second inverter, which is an instance of a Verilog module. The output of the second inverter then goes to *dout*. Because analog and digital signals are mixed, the FineSim Pro tool automatically inserts A2D and D2A blocks to convert the signals from analog to digital and back again.

The following examples can be found in Lab 1_Spice_Top.

Example 68 *ms1.sp Netlist*

```

* ms1.sp
* Mixed Sim - SPICE Top Example
.option post
vdd VDD 0 dc 2.5
vss VSS 0 dc 0
.global VSS VDD
vin sin 0 pulse (0 2.5 1.0n 1.0n 1.0n 4.0n 10n)
.inc ./model.inc

** Top Netlist **
X1 sin sinb sinv
X2 sinb dout vinv

.subckt sinv in out
mp0 out in VDD VDD p l=0.25u w=3u
mn0 out in VSS VSS n l=0.25u w=1.5u
.ends
.subckt vinv in out
.ends
.tran 1p 100ns
.end
  
```

Example 69 *ms1.v*

```
// ms1.v
`timescale 1ns/1ps
module top;
    vinv I1 (sinb,dout);
    initial begin
        $finesim_config( ,
            // show the progress of FineSim Pro
            ".option progress=0",
            // run FineSim Pro command
            ".finesim -o ms1 ms1.sp"
        );
        $finesim_instance(I1,"X2"); // mapping each instance
        $monitor(" %10.3f sinb= %b dout= %b", $realtime, sinb, dout);
        $dumpfile("ms1.vcd");
        $dumpvars(0, top);
    end
endmodule

module vinv (i,o);
    input i;
    output o;
    not #1 (o,i);
endmodule
```

Note that the SPICE netlist includes a subcircuit wrapper for the digital block, `vinv`, which includes the port list. This digital block is defined and instantiated in the Verilog netlist file.

In addition, the Verilog netlist file contains tasks for configuring the FineSim Pro simulation, such as `$finesim_config`, `$finesim_instance`, in the top module. See [FineSim Pro Tasks](#) for detailed information about the tasks used in the Verilog cosimulation.

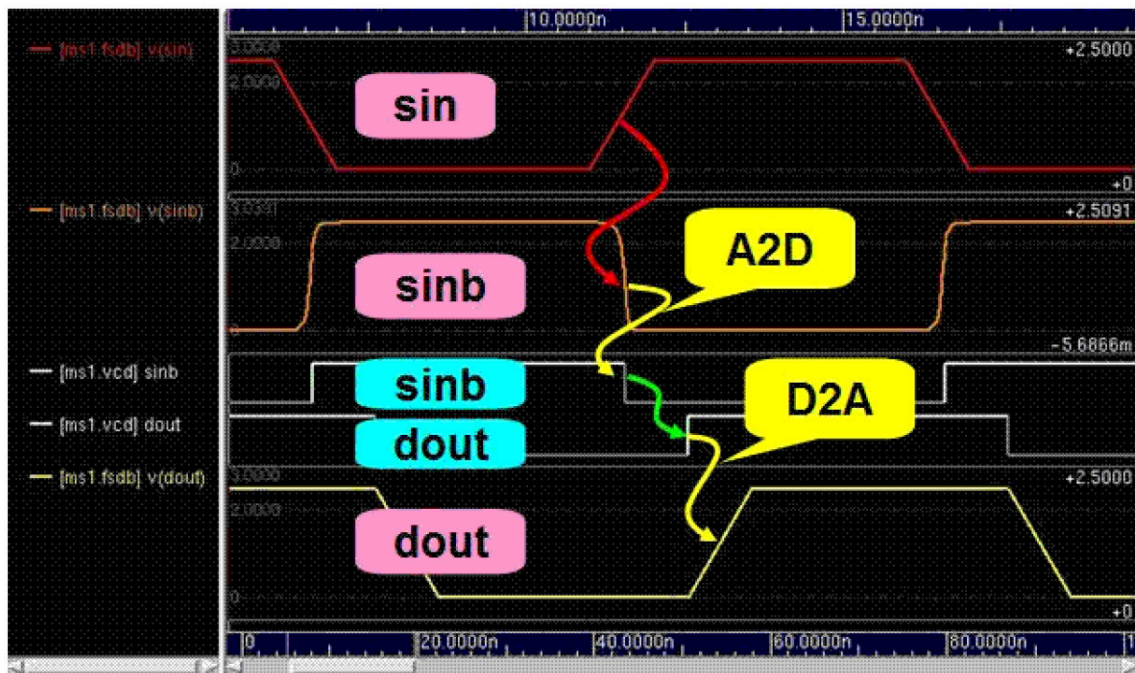
Although in this example the top-level netlist is the SPICE netlist, it should be noted that the Verilog simulator reads the Verilog netlist, as can be seen in the examples below. The tasks in the Verilog netlist configure the FineSim Pro tool and start the top-level SPICE simulation.

Run

```
% verilog +access+rcw +loadvpi=finesim.so:finesim_startup ms1.v
% ncverilog +access+rcw +loadvpi=finesim.so:finesim_startup ms1.v
```

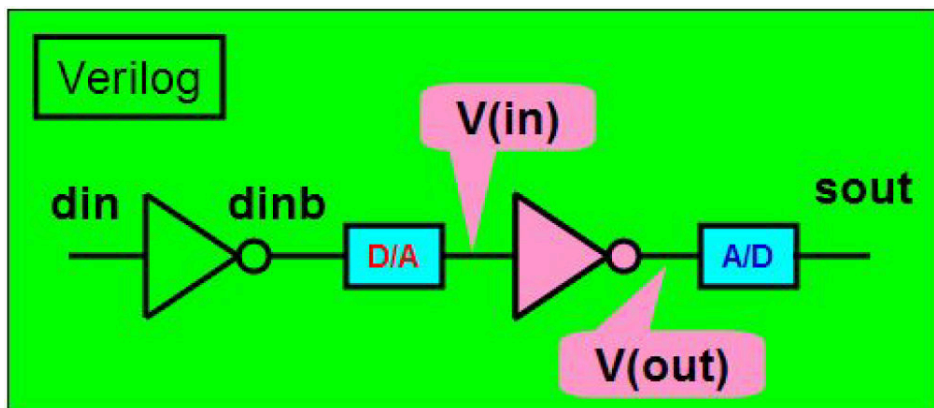
The simulation result waveforms are shown in the following figure.

Figure 72 Lab1 Simulation Results



Verilog Top Example

Figure 73 Block Diagram of Lab2



In the example the main circuit is the Verilog netlist. The Verilog netlist includes both analog and digital subcircuits. The first inverter is in Verilog, the second is in SPICE format. In Verilog, `din` is the input of the first inverter. The output of the first inverter, `dinb` goes to the input of the second inverter, which is an instance of a SPICE subcircuit. The output of the second inverter then goes to `sout`. Because analog and digital signals are mixed, the FineSim Pro tool automatically inserts D2A and A2D blocks to convert the signals from digital to analog and back again.

The following example can be found in Lab 2_Verilog_Top.

Example 70 ms2.v

```
// ms2.v
`timescale 1ns/1ps
module top;
  reg din;
  wire dinb, sout;
  vinv I1 (din,dinb);
  sinv I2 (dinb,sout);
  initial begin
    $finesim_config( , ".finesim -o ms2 ms2.sp" );
    $monitor(" %10.3f din=%b dinb=%b sout=%b", $realtime, din, dinb, sout);
    $dumpfile("ms2.vcd");
    $dumpvars(0);
    din=0;
    repeat(10) #10 din= ~din;
    $finish;
  end
endmodule

module vinv (i,o);
  input i;
  output o;
  not #1 (o,i);
endmodule

module sinv (in,out);
  input in;
  output out;
  reg out;
  initial $finesim_module;
endmodule
```

Example 71 ms2.sp

```
* ms2.sp
* Mixed Sim - Verilog Top Example
vvd VDD 0 dc 2.5
vvs VSS 0 dc 0
.global VSS VDD
.inc ./model.inc
.inc './finemix.sp' $ Automatically generated SPICE instance netlist

.subckt sinv in out
mp0 out in VDD VDD p l=0.5u w=1u
mn0 out in VSS VSS n l=0.5u w=1u
.ends

.option post
.tran 1p 100ns
.end
```

In Verilog top netlist style, the analog block in the Verilog netlist file is defined as a module wrapper with port declarations and the `$finesim_module` task. The `$finesim_module` task causes the FineSim Pro tool to generate a SPICE instance netlist and save it in the file named `finemix.sp`. These analog blocks are defined and instantiated in the SPICE netlist file. The `finemix.sp` should also be included in the SPICE netlist file.

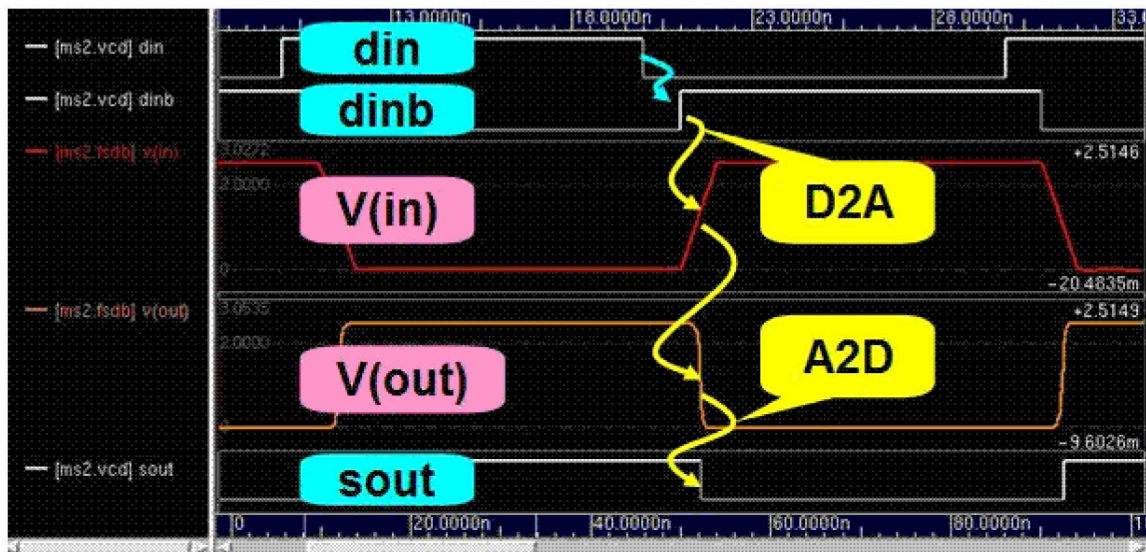
See [FineSim Pro Tasks](#) for detailed information about the tasks used in the Verilog cosimulation.

Run

```
% verilog +access+rwc +loadvpi=finesim.so:finesim_startup ms2.v
% ncverilog +access+rwc +loadvpi=finesim.so:finesim_startup ms2.v
```

Result

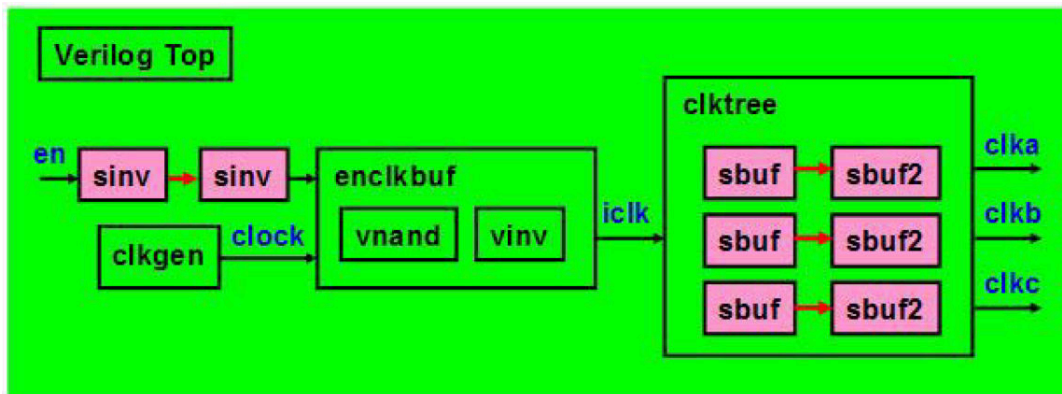
Figure 74 Lab2 Simulation Results



Verilog Top Example With (Gate Level Netlist + TR Level Cell)

This lab can be found in Lab 3_VT_Gate_TR.

Figure 75 Block Diagram of Lab3



This example has a more complicated hierarchy. The top-level netlist is in Verilog, but it contains eight instances of spice subcircuits. The SPICE netlist for these instances is again automatically generated and is found in `finemix_ms3.sp`. Note that all the nets are connected in the Verilog domain, including the ones colored red in the diagram above that connect two analog blocks. Because these signals go through the Verilog domain, which is digital, they are converted from analog to digital and then back again.

Verilog Offset Time in the FineSim Cosimulation Flow

The FineSim tool supports Verilog offset time (`#xx`) in the cosimulation flow.

For example:

```
initial begin
#50
$finesim_config( , ".finesim -o input input.sp", ".option dump_ie=1" );
// to run FineSim
```

The FineSim cosimulation flow disables D2As and A2Ds before the simulation of the specified offset time and then enables them after the specified offset time.

The FineSim simulation engine issues the following messages:

```
Message! D2As and A2Ds are not enabled at 0s.
Starting Transient Analysis ...
48.6ns (48.5 %)
Message! D2As and A2Ds are enabled at 50ns.
```

FineSim Pro Tasks

You can control the FineSim Pro tool and Verilog cosimulation using tasks in the Verilog netlist, such as `$finesim_config` and `$finesim_instance`. This section describes the tasks supported by the FineSim Pro tool.

This section describes the following:

- [\\$finesim_config](#)
- [\\$finesim_input](#)
- [\\$finesim_output](#)
- [\\$finesim_inout](#)
- [\\$finesim_module](#)
- [\\$finesim_instance](#)

\$finesim_config

Description

With this task, a variety of configuration commands can be defined. Configuration commands can directly be specified, and/or configuration file name containing those commands can be specified.

Syntax

```
$finesim_config( ["config_file_name"] [, "config_command"] ... );
```

Arguments

Argument	Description
<i>config_command</i>	The configuration command.
<i>config_file_name</i>	File that contains the configuration commands.

Examples

```
$finesim_config( "test.cfg" );  
$finesim_config( , ".finesim test.sp");  
$finesim_config( , ".option progress=0 accurate=1",  
".a2d vdd25 vl=1.25 vh=1.25",  
".d2a vdd25 vl=0 vh=2.5 vx=1.25 tr=0.3n tf=0.3n",  
".finesim -out test test.sp");
```

In the first example, the configuration file name is specified. In the other example, configuration commands are given directly.

\$finesim_input

Description

With this task, you can connect a Verilog input to a SPICE output through an A2D module. This task has the same meaning as the configuration command `.INPUT`.

Syntax

```
$finesim_input( net_name, "spice_node_name" [, "a2d_model_name" ] );
```

Arguments

Argument	Description
<i>a2d_model_name</i>	Name of model used for A/D conversion.
<i>net_name</i>	Input node name to the Verilog module.
<i>spice_node_name</i>	Output node name from the SPICE module.

Examples

```
$finesim_input( TOP.IN , "XI1.out" , "default" );
```

\$finesim_output

Description

With this task, you can connect a Verilog output to a SPICE input through a D2A module. This task has the same meaning as the configuration command `.OUTPUT`.

Syntax

```
$finesim_output( net_name, "spice_node_name" [, "d2a_model_name" ] );
```

Arguments

Argument	Description
<i>d2a_model_name</i>	Name of model used for D/A conversion.
<i>net_name</i>	Output node name from the Verilog module.
<i>spice_node_name</i>	Input node name to the SPICE module.

Examples

```
$finesim_output( TOP.OUT , "XI1.in" , "default" );
```

\$finesim_inout

Description

With this task, you can connect a Verilog inout port to a SPICE port through D2A and/or A2D modules. This task has the same meaning as the configuration command `.INOUT`.

Syntax

```
$finesim_inout( net_name, "spice_node_name" \
[, "D2A=d2a_model_name" ] [, "A2D=a2d_model_name"] );
```

Arguments

Argument	Description
<i>a2d_model_name</i>	Name of model used for A/D conversion.
<i>d2a_model_name</i>	Name of model used for D/A conversion.
<i>net_name</i>	Bidirectional node name to/from the Verilog module.
<i>spice_node_name</i>	Bidirectional node name from/to the SPICE module.

Examples

```
$finesim_inout( TOP.DATA , "XI1.data" , "D2A=default", "A2D=default" );
```

\$finesim_module

Description

This task enables you to define a module as a SPICE subcircuit. If the subcircuit name is not specified in the argument, the name of the Verilog module in which this task is defined is used. If the subcircuit name is different from the module name, you can specify the name. When this task is included in a Verilog module, the FineSim Pro tool automatically generates a SPICE instance netlist which is saved in the file `finemix.sp` by default, and internal A/D and D/A conversion modules. The save file name can be changed by using `.OPTION` command within the `$finesim_config` task.

The generated SPICE file should be included in the SPICE netlist. The model used for the A/D and D/A conversion can be specified by using Verilog statement `defparam` or `parameter` with the keywords of `finesim_a2d` and `finesim_d2a` within the module which `$finesim_module` task is included. If it is not specified, the `DEFAULT` model is used.

See [\\$finesim_config](#) for details.

Syntax

```
$finesim_module [ ( "spice_subckt_name" ) ];
```

Arguments

Argument	Description
<i>spice_subckt_name</i>	SPICE subcircuit name to be used in the generation of the SPICE instance netlist.

Examples

```
$finesim_module;
```

or:

```
$finesim_module("inv");
```

\$finesim_instance

Description

With this task, you can map a Verilog instance to a SPICE instance. The SPICE instance should be an instance of a subcircuit wrapper that just includes the port definitions. This task causes the Verilog instance to be simulated instead of the SPICE instance. Normally it is used when the top-level netlist is a SPICE netlist. The model used for the A/D and D/A conversion can be specified by using Verilog statement `defparam` or `parameter` with the keywords of `finesim_a2d` and `finesim_d2a` within the instance module. If it is not specified, the `DEFAULT` model is used.

See [finesim_a2d / finesim_d2a Parameters](#) for the details.

Syntax

```
$finesim_instance( instance_name, "spice_instance_name" );
```

Arguments

Argument	Description
<i>instance_name</i>	Instance name in the Verilog netlist.
<i>spice_instance_name</i>	Instance name in the SPICE netlist.

Examples

```
$finesim_instance( I1 , "X1" );
```

Configuration Commands

As described in the [\\$finesim_config](#) section, a variety of configuration commands can be given using the task `$finesim_config`. These commands can either be put in a configuration file specified in the `$finesim_config` task or included directly in the `$finesim_config` task. In this section, each of those command statements is described in detail.

This section describes the following:

- [.RESISTANCE](#)
- [.A2D](#)
- [.D2A](#)
- [.SCOPE](#)
- [.INPUT](#)
- [.OUTPUT](#)
- [.INOUT](#)
- [.OPTION](#)
- [.FINESIM](#)
- [finesim_a2d / finesim_d2a Parameters](#)

.RESISTANCE

Description

This command is used to define a model of signal strengths. In Verilog, net value is represented by logic and strength. Verilog defines these different signal strengths: supply, strong, pull, large, weak, medium, small, highz.

The `.RESISTANCE` command is used to specify the equivalent resistances that correspond to these signal strengths. A resistance model is part of the specification of an A2D or D2A converter.

Consider D2A, S being the signal strength:

- if (S == supply) apply R7 as resistance of the Thevenin equivalent circuit.
- if (S == strong) apply R6 as resistance of the Thevenin equivalent circuit.
- if (S == pull) apply R5 as resistance of the Thevenin equivalent circuit.
- if (S == large) apply R4 as resistance of the Thevenin equivalent circuit.
- if (S == weak) apply R3 as resistance of the Thevenin equivalent circuit.
- if (S == medium) apply R2 as resistance of the Thevenin equivalent circuit.
- if (S == small) apply R1 as resistance of the Thevenin equivalent circuit.
- if (S == highz) apply R0 as resistance of the Thevenin equivalent circuit.

Consider A2D, R being the driven resistance of the SPICE net:

- if (R <= R7) apply supply strength in the Verilog net.
- else if (R <= R6) apply strong strength in the Verilog net.
- else if (R <= R5) apply pull strength in the Verilog net.
- else if (R <= R4) apply large strength in the Verilog net.
- else if (R <= R3) apply weak strength in the Verilog net.
- else if (R <= R2) apply medium strength in the Verilog net.
- else if (R <= R1) apply small strength in the Verilog net.
- else apply highz strength in the Verilog net.

The default signal strengths for both a2d and d2a are "1 3k 4k 5k 50k 70k 90k 10g".

Syntax

```
.RESISTANCE res_model_name R7 R6 R5 R4 R3 R2 R1 R0
```

Arguments

Argument	Description
R7 ... R0	Signal strengths which is corresponding to Strength7 ~ Strength0.
<i>res_model_name</i>	Resistance model name. The default is "DEFAULT".

Examples

```
.resistance default 1 3k 4k 5k 50k 70k 90k 10g
.resistance VDD33R 1 3k 5k 10k 50k 80k 100k 20g
```

The following example illustrates how the `.RESISTANCE`, `.A2D`, and `.D2A` commands are used together:

```
.resistance default 1 3k 4k 5k 50k 70k 90k 10g
.A2D default VL=1.25 VH=1.25 TX=1n R=default
.D2A default VL=0 VH=2.5 VX=1.25 TR=1n TF=1n R=default
```

In the `.A2D` example above:

- If net is 2.5 volts and driven resistance R is 3k ohm in the SPICE, this net value is converted to {strong 1} in Verilog because $(R=3k) \leq (R6=3k)$ and $(V=2.5) \geq (VH=1.25)$.
- If net is 0.5 volts and driven resistance R is 4k ohm in the SPICE, this net value is converted to {pull 0} in Verilog because $(R=4k) \leq (R5=4k)$ and $(V=0.5) \leq (VL=1.25)$.

In the `.D2A` example above:

- Net value {strong, 1} in Verilog is converted to the Thevenin equivalent circuit with $V=2.5$ volts and $R=3k$ ohm in SPICE.
- Net value {pull, 0} in Verilog is converted to the Thevenin equivalent circuit with $V=0$ volts and $R=4k$ ohm in SPICE.

.A2D

Description

This command is used to define a model for A/D conversion. Different models can be defined for different input conditions.

Syntax

```
.A2D a2d_model_name [VL=real_value] [VH=real_value] \
[TX=real_value] [R=res_model_name]
```

Arguments

Argument	Description
<code>a2d_model_name</code>	The name of the model.
<code>R=res_model_name</code>	The RESISTANCE model name for signal strengths. The default is DEFAULT.

Argument	Description
<code>TX=real_value</code>	Show unknown state ("X") if the signal is between VL and VH for longer than TX. The default is 1ns.
<code>VH=real_value</code>	Logic "1" threshold voltage. The default is 1.25.
<code>VL=real_value</code>	Logic "0" threshold voltage. The default is 1.25.

Examples

```
.A2D default VL=1.25 VH=1.25 TX=1n R=default
.A2D VDD33 VL=1.65 VH=1.65 TX=0n R=VDD33R
```

.D2A

Description

This command is used to define a model for D/A conversion. Different models can be defined for different conditions.

Syntax

```
.D2A d2a_model_name [VL=real_value] [VH=real_value] [VX=real_value] \
[TR=real_value] [TF=real_value] [T0X=real_value] [TX1=real_value] \
[T1X=real_value] [TX0=real_value] [R=res_model_name]
```

Arguments

Argument	Description
<code>d2a_model_name</code>	The name of the model.
<code>R=res_model_name</code>	The RESISTANCE model name which is defined in the RESISTANCE command. The default is <code>DEFAULT</code> .
<code>T0X=real_value</code>	0 to X transition time. The default is rising time (TR).
<code>T1X=real_value</code>	X to 1 transition time. The default is rising time (TR).
<code>TF=real_value</code>	Falling time. The default is 1ns.
<code>TR=real_value</code>	Rising time. The default is 1ns.
<code>TX0=real_value</code>	X to 0 transition time. The default is falling time (TF).
<code>TX1=real_value</code>	X to 1 transition time. The default is falling time (TF).
<code>VH=real_value</code>	Logic "1" voltage. The default is 2.5.

Argument	Description
<code>VL=<i>real_value</i></code>	Logic "0" voltage. The default is 0.
<code>VX=<i>real_value</i></code>	Logic "x" voltage. The default is 1.25.

Examples

```
.D2A default VL=0 VH=2.5 VX=1.25 TR=1n TF=1n R=default
.D2A VDD33 VL=0 VH=3.3 VX=1.65 TR=0.5n TF=0.5n R=VDD33R
```

.SCOPE

Description

This command is used to define the naming scope for nets. The scope is a hierarchy name separated by a delimiter dot(.). It is prepended to Verilog or SPICE net names to create a full net name.

Syntax

```
.SCOPE [VERILOG=scope_name] [SPICE=scope_name]
```

Arguments

Argument	Description
<code>SPICE=<i>scope_name</i></code>	Hierarchy name applied for SPICE net name.
<code>VERILOG=<i>scope_name</i></code>	Hierarchy name applied for Verilog net name.

Examples

```
.SCOPE VERILOG=TOP.I1.I2 SPICE=XI1.XI2
.INPUT in out
```

As with this example, it is treated as follows:

```
".INPUT TOP.I1.I2.in XI1.XI2.out"
```

.INPUT

Description

This command is used to connect a Verilog input to a SPICE output through an A2D module. This command has the same meaning as the `$finesim_input` task.

Syntax

```
.INPUT verilog_net_name spice_net_name [A2D=a2d_model_name]
```

Arguments

Argument	Description
<i>A2D=a2d_model_name</i>	A2D name of the model to be used for A/D conversion.
<i>spice_net_name</i>	Input net name to the SPICE module from the Verilog module.
<i>verilog_net_name</i>	Output net name from the Verilog module to the SPICE module.

Examples

```
.INPUT TOP.IN XI1.out A2D=default
```

.OUTPUT

Description

This command is used to connect a Verilog output to a SPICE input through a D2A module. This command has the same meaning as the `$finesim_output` task.

Syntax

```
.OUTPUT verilog_net_name spice_net_name [D2A=d2a_model_name]
```

Arguments

Argument	Description
<i>D2A=d2a_model_name</i>	Name of the model to be used for D/A conversion.
<i>spice_net_name</i>	Output net name from the SPICE module to the Verilog module.
<i>verilog_net_name</i>	Input net name to the Verilog module from the SPICE module.

Examples

```
.OUTPUT TOP.OUT XI1.in D2A=default
```

.INOUT

Description

This command is used to connect a Verilog inout port to a SPICE port through A2D and/or D2A modules. This command has the same meaning as the `$finesim_inout` task.

Syntax

```
.INOUT verilog_net_name spice_net_name [A2D=a2d_model_name]
[D2A=d2a_model_name]
```

Arguments

Argument	Description
<code>A2D=a2d_model_name</code>	Name of the model to be used for A/D conversion.
<code>D2A=d2a_model_name</code>	Name of the model to be used for D/A conversion.
<code>spice_net_name</code>	Bidirectional node name from/to the SPICE module.
<code>verilog_net_name</code>	Bidirectional node name to/from the Verilog module.

Examples

```
.INOUT TOP.data XI1.data D2A=default A2D=default
```

.OPTION

Description

By using this `.OPTION` command, you can specify various options.

Syntax

```
.OPTION [INST_FILE=file_name] [PROGRESS=0|1] [IMAX=integer_value] \
[IMODE=0|1] [ACCURATE=0|1] [dump_ie=0|1] [minimize_ie=0|1|2]
[bus_format="<%d>"] [port_map_by_name=0|1]
```

Arguments

Argument	Description
<code>INST_FILE=file_name</code>	Sets the file name for the SPICE instance netlist. If a Verilog module has a <code>\$finesim_module</code> task, a SPICE instance is automatically generated in this file. The default file name is <code>finemix.sp</code> .

Argument	Description
PROGRESS=0 1	Sets the mode for showing the status of the simulation progress. The default value is 1 (on mode).
IMAX= <i>integer_value</i>	Sets the maximum iteration count for DC solving in FineSim/Verilog cosimulation. The default value is 5.
IMODE=0 1	Sets the interrupt mode for FineSim Pro execution. When IMODE=1, a "Ctrl+C" causes the FineSim simulation engine to stop running. The default value is 0 (off mode).
ACCURATE=0 1	Sets the re-calculation mode for A/D conversion. A value of 1 results in more accurate timing, but causes the simulation to run more slowly. The default value is 0 (off mode).
dump_ie=0 1	Sets to dump out A2D and D2A information into *.dumpie file. The default value is 0 (off mode).
minimize_ie=0 1 2	Sets to avoid unnecessary A2D and D2A conversion. If two nodes are connected within analog domain and it is not defined as a register type, then minimize_ie=1 removes D2A but keeps A2D for waveform probing; minimize_ie=2 removes both A2D and D2A if digital signal of A2D does not drive primitive gate or continuous assignment. This option only applies to ports defined under \$finesim_module task. The default value is 0 (off mode).
bus_format="<%d>"	Specifies bus format, where %d is mandatory, representing the bits. The brackets ("<" ">") indicate the bus characters, which can be modified to fit the bus character requirements.
port_map_by_name 0 1	Defines whether the FineSim simulation engine uses the port order or port name to map between the Verilog module and SPICE subcircuit. The default is 0, which maps each cosimulation connection based on the order the ports are defined. When set to 1, the FineSim simulation engine maps the connection based on the name of the port, so the port order in Verilog versus SPICE can be different, but the name of the port has to be identical.

.FINESIM

Description

With this command, users can define the FineSim Pro command and arguments. The command arguments and syntax are the same as those for the standalone FineSim Pro tool.

Syntax

```
.FINESIM finesim_command_arguments
```

Arguments

Argument	Description
<code>finesim_command_arguments</code>	See the <i>FineSim User Guide: Pro and SPICE Reference</i> in for details.

Examples

```
.FINESIM -out finesimout input.sp
```

finesim_a2d / finesim_d2a Parameters

Description

The `finesim_a2d` and `finesim_d2a` parameters can be used to change the model to be used for the A/D and D/A conversion. The “DEFAULT” model is used by default. The designation of specific signals is delimited by “\$” character, like “`finesim_d2a$signal`”.

Syntax

```
defparam I2.finesim_a2d      = "vdd_33_1";
parameter finesim_a2d        = "vdd_33";
parameter finesim_d2a$A33    = "vdd33";
parameter \finesim_a2d$DA[0] = "vdd_25";
```

Examples

The first example overrides the A/D conversion model name with “`vdd_33_1`” for instance I2. The second specifies the A/D model name of “`vdd_33`” for all signals used within the module. The third specifies the D/A model name of “`vdd33`” only for a signal A33. The fourth specifies the A/D model name of “`vdd_25`” only for the first signal of bus DA.

Automatic Verilog Instance Generation

The FineSim Pro tool provides the support of automatic Verilog instance file generation when SPICE Top structure cosimulation is used. This flow introduces the `-genv` command option and some new options, such as `finesim_verilog_file`, `finesim_verilog_module`, and `finesim_verilog_subckt_file`.

This section describes the flow supported by the FineSim Pro tool.

- `-genv`
- `finesim_bus_format`
- `finesim_port_map_by_name`

- [finesim_verilog_file](#)
 - [finesim_verilog_instance](#)
 - [finesim_verilog_module](#)
 - [finesim_verilog_module_file](#)
 - [finesim_verilog_subckt_file](#)
-

-genv

Description

If this command option is given, Verilog instance is automatically generated according the subcircuit definition in the deck file.

Syntax

```
finesim -genv[=1|2] deck_file
```

Examples

```
finesim -genv test.sp  
finesim -genv=1 test.sp  
finesim -genv=2 test.sp
```

The first example is the same as the second example, only the Verilog instance file is created. The third example generates both a Verilog instance file and SPICE subcircuit definition file.

finesim_bus_format

Description

Specifies bus format, where %d is mandatory, representing the bits. "<" ">" indicates the bus characters, which can be modified to fit the bus characters.

Syntax

```
finesim_bus_format="<%d>"
```

finesim_port_map_by_name

Description

This option defines whether the FineSim tool uses the port order or port name to map between the Verilog module and SPICE subckt. The default is 0, which maps each cosimulation connection is based on the order the ports are defined. When set to 1, the

FineSim tool maps the connection based on the name of the port, so the port order in Verilog versus SPICE can be different, but the name of the port must be identical.

Syntax

```
finesim_port_map_by_name=1
```

finesim_verilog_file

Description

Sets the file name for Verilog instance file. Default file name is "finemix.v".

Syntax

```
.option finesim_verilog_file=file_name
```

Examples

```
.option finesim_verilog_file="inv.v"
```

finesim_verilog_instance

Description

Specifies a SPICE instance to be replaced by a Verilog module. Note that this option can only work with the command option `-genv`. If the subckt name is the same as the module name, the module name can be omitted.

Syntax

```
.option finesim_verilog_instance="instance_name:module_name ..."
```

Examples

```
.option finesim_verilog_instance="x1.x2.x3:inv x1.x2.x4:buf"
```

In this example, SPICE instances `x1.x2.x3` and `x1.x2.x4` are replaced by the Verilog module `inv` and `buf`, respectively.

finesim_verilog_module

Description

Specifies a SPICE subcircuit to be replaced by a Verilog module. If the subcircuit name is that same as the module name, the module name can be omitted. This option can be specified multiple times.

Syntax

```
.option finesim_verilog_module="subckt_name:module_name ... "
```

Examples

```
.option finesim_verilog_module="inv buf:BUF nand2:Nand2"
```

The example means the cosimulation uses the `inv`, `BUF`, and `Nand2` Verilog modules on the digital side, not the `inv`, `buf`, and `nand2` subcircuits in the analog netlist.

finesim_verilog_module_file

Description

This option specifies the Verilog file containing the Verilog module definitions. This option is used when the Verilog instance file is generated by `finesim -genv`.

Syntax

```
finesim_verilog_module_file="filename"
```

finesim_verilog_subckt_file

Description

Sets the file name for a new SPICE subcircuit definition file. If command option `-genv=2` is given, a new SPICE subcircuit definition file is generated. This subcircuit is empty because this subcircuit is replaced by a Verilog module. Default file name is `"finemix_subckt.sp"`.

Syntax

```
.option finesim_verilog_subckt_file=file_name
```

Examples

```
.option finesim_verilog_subckt_file="pll.sp"
```

Circuit Example: (test.sp)

```
.option post
.global 0
vdd VDD 0 dc 2.5
vss VSS 0 dc 0
.global VSS VDD
vin sin 0 pulse (0 2.5 1.0n 1.0n 1.0n 4.0n 10n)
.inc ./model.inc
```

```

** Top Netlist **
X1 sin sinb sinv
X2 sinb dout vinv

.subckt sinv in out
mp0 out in VDD VDD p l=0.25u w=3u
mn0 out in VSS VSS n l=0.25u w=1.5u
.ends

.subckt vinv in out
mp0 out in VDD VDD p l=0.25u w=3u
mn0 out in VSS VSS n l=0.25u w=1.5u
.ends

.tran 1p 100ns
.end

```

In this example, suppose you want to replace the `vinv` subcircuit with the `vinv` module on the Verilog side. First, add the `.option finesim_verilog_module="vinv"` to the `test.sp` deck file, and then execute command `finesim -genv=2 test.sp` in the terminal. Then the `finemix.v` and `finemix_subckt.sp` files are automatically generated. Third, include `finemix.v` in the Verilog file and include `finemix_subckt.sp` in the `test.sp` deck file separately. Now, the automatic Verilog instance generation flow is done and you can continue to run the cosimulation.

Parallel Cosimulation

The FineSim Pro tool has applied its parallel simulation technology to cosimulation. It solves the performance bottleneck of slower transistor level simulation in a cosimulation environment. To use this feature, modify the `finesim` command to run the parallel FineSim Pro tool with the `-np` switch, as shown in the following example:

```

//You can change a 1CPU serial run of
.finesim -spice -o 1CPU input.spi
//to an 8CPU run of
.finesim -np 8 -spice -o 8CPU input.spi

```

Currently, the maximum number of `-np` in parallel cosimulation is 8.

Common Cosimulation Problems

Because every Verilog simulator behaves a bit differently with VPI, sometimes unexpected problems are encountered. This section goes through some of these common cosimulation problems and the workarounds for fixing them.

Error while reading shared library symbols, cannot find new threads: generic error

You can workaround the problem by setting:

```
setenv LD_ASSUME_KERNEL"
```

ERROR: VPI NOFORCB: vpi_put_value() cannot force a bit of an unexpanded vector net

Some of the Verilog simulation does not expand out the output bus, so you cannot use `.input/.output/.inout` to drive the signal.

For the output [MSB:LSB] name that is causing the issue, add "wire scalared [MSB:LSB] name". Alternatively, you can set the signal as "output reg".

H

PrimeSim VPI Cosimulation

This appendix describes the PrimeSim Pro tool support for mixed-mode simulation with Verilog simulators using the Verilog Programming Interface (VPI) API.

This chapter covers the following topics:

- [Mixed-Mode Simulation](#)
- [PrimeSim Pro Tasks](#)
- [Configuration Commands](#)
- [Automatic Verilog Instance Generation](#)
- [Circuit Example: \(test.sp\)](#)
- [Parallel Cosimulation](#)
- [Common Cosimulation Problems](#)

Mixed-Mode Simulation

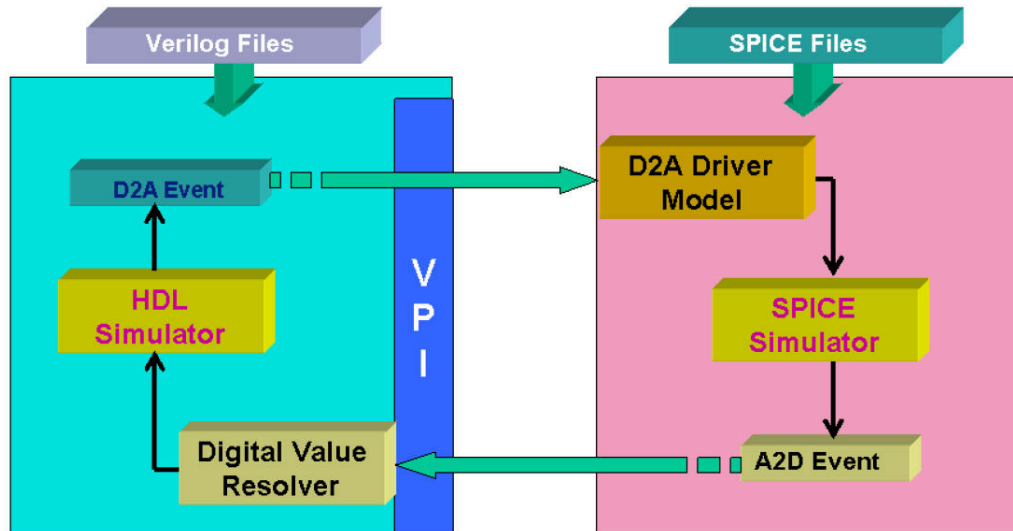
Mixed mode simulation involves combining digital and analog simulators in various ways. However, it has been difficult to find efficient methods for synchronization between the two domains. This is because the analog simulator uses dynamic time step control while the digital simulator uses an event driven paradigm. In the PrimeSim Pro tool, efficient synchronization algorithms have been developed and applied to mixed-mode simulation. The PrimeSim Pro tool supports mixed-mode simulation with Verilog simulators using the Verilog Programming Interface (VPI) API. Using this standard API allows the PrimeSim Pro tool to be used with any vendor's Verilog simulator.

Verilog Cosimulation

The PrimeSim Pro tool supports mixed-mode simulation with Verilog simulators. This is referred to as Verilog cosimulation. The PrimeSim Pro tool includes a dynamic library, `primesim.so`, which provides this support. When the Verilog simulator starts it loads

`primesim.so` and interacts with the PrimeSim Pro tool through the VPI API. The following figure shows the interaction of the digital and analog simulators.

Figure 76 Verilog Cosimulation Flow



Running Verilog Cosimulation

Because the PrimeSim Pro tool provides Verilog cosimulation through a VPI library, `primesim.so`, the Verilog simulator must load the VPI library. How this is done depends on the Verilog simulator, but generally, the Verilog simulator must be advised of the location of the VPI library, the name of the library, and the name of the start-up function in the library.

Because `primesim.so` is a dynamic library that is loaded by the Verilog simulator, its location usually must be added to the `LD_LIBRARY_PATH` environment variable. This allows the Verilog simulator to find the library.

The PrimeSim VPI interface has been updated to compile with GCC 4.5. To avoid an environment issue, you should set `$PRIMESIM_HOME/lib/Linux##` as part of the `LD_LIBRARY_PATH` environment variable before loading `primesim.so` into your Verilog simulator.

The PrimeSim tool now also includes `primesim.so` in the respective `lib` folder. The previous requirement of setting `LD_LIBRARY_PATH` to `$PRIMESIM_HOME/primesim/platform/Linux##` is no longer required. Here is an example of starting 64-bit cosimulation with VCS:

```
% source primesim.cshrc
% setenv LD_LIBRARY_PATH $PRIMESIM_HOME/lib/Linux64:${LD_LIBRARY_PATH}
% vcs -R -full64 +vpi -load primesim.so:primesim_startup \
  +cli+3 ms1.v
```

After the location of the library is set by using the `LD_LIBRARY_PATH`, the name of the library, `primesim.so`, and the name of the startup function, `primesim_startup`, are generally passed as command-line options to the Verilog simulator.

Verilog Simulator Commands

PrimeSim VPI supports most of the common Verilog simulator commands. Below is some example syntax needed to start Verilog simulation:

ModelSim

```
vsim -c -pli primesim.so
```

NCSim

```
ncverilog +access+rw +loadvpi=primesim.so:primesim_startup ....
```

VCS

```
vcs -R +vpi -load primesim.so:primesim_startup +cli+3 ....
```

VerilogXL

```
verilog +access+rw +loadvpi=primesim.so:primesim_startup
```

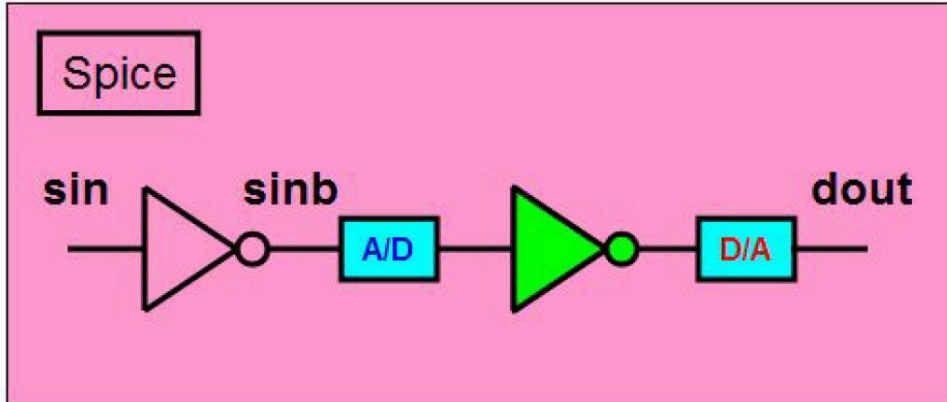
Examples

In general, analog and digital mixed circuits can be categorized into two kinds of circuit styles. One style has the analog netlist as a design top instance, and digital instances are instantiated from the analog netlist. The other style has the digital netlist as a design top instance, and analog instances are instantiated from the digital netlist.

In this section, simple examples with the analog (SPICE) netlist and digital (Verilog) netlist on top are given to show how the two circuit styles can be simulated with the PrimeSim Pro tool.

SPICE Top Example

Figure 77 Block Diagram of Lab1



In this example the main circuit is the SPICE netlist. The SPICE netlist includes both analog and digital subcircuits. The first inverter is in SPICE, the second is in Verilog format. In SPICE, *sin* is the input of the first inverter. The output of the first inverter, *sinb* goes to the input of the second inverter, which is an instance of a Verilog module. The output of the second inverter then goes to *dout*. Because analog and digital signals are mixed, the PrimeSim Pro tool automatically inserts A2D and D2A blocks to convert the signals from analog to digital and back again.

The following examples can be found in Lab 1_Spice_Top.

Example 72 *ms1.sp* Netlist

```
* ms1.sp
* Mixed Sim - SPICE Top Example
.option post
vdd VDD 0 dc 2.5
vss VSS 0 dc 0
.global VSS VDD
vin sin 0 pulse (0 2.5 1.0n 1.0n 1.0n 4.0n 10n)
.inc ./model.inc

** Top Netlist **
X1 sin sinb sinv
X2 sinb dout vinv

.subckt sinv in out
mp0 out in VDD VDD p l=0.25u w=3u
mn0 out in VSS VSS n l=0.25u w=1.5u
.ends
.subckt vinv in out
.ends
.tran 1p 100ns
.end
```

Example 73 *ms1.v*

```
// ms1.v
`timescale 1ns/1ps
module top;
    vinv I1 (sinb,dout);
    initial begin
        $primesim_config( ,
            // show the progress of PrimeSim Pro
            ".option progress=0",
            // run PrimeSim Pro command
            ".primesim -o ms1 ms1.sp"
        );
        $primesim_instance(I1,"X2"); // mapping each instance
        $monitor("%10.3f sinb= %b dout= %b", $realtime, sinb, dout);
        $dumpfile("ms1.vcd");
        $dumpvars(0, top);
    end
endmodule

module vinv (i,o);
    input i;
    output o;
    not #1 (o,i);
endmodule
```

Note that the SPICE netlist includes a subcircuit wrapper for the digital block, `vinv`, which includes the port list. This digital block is defined and instantiated in the Verilog netlist file.

In addition, the Verilog netlist file contains tasks for configuring the PrimeSim Pro simulation, such as `$primesim_config`, `$primesim_instance`, in the top module. See [PrimeSim Pro Tasks](#) for detailed information about the tasks used in the Verilog cosimulation.

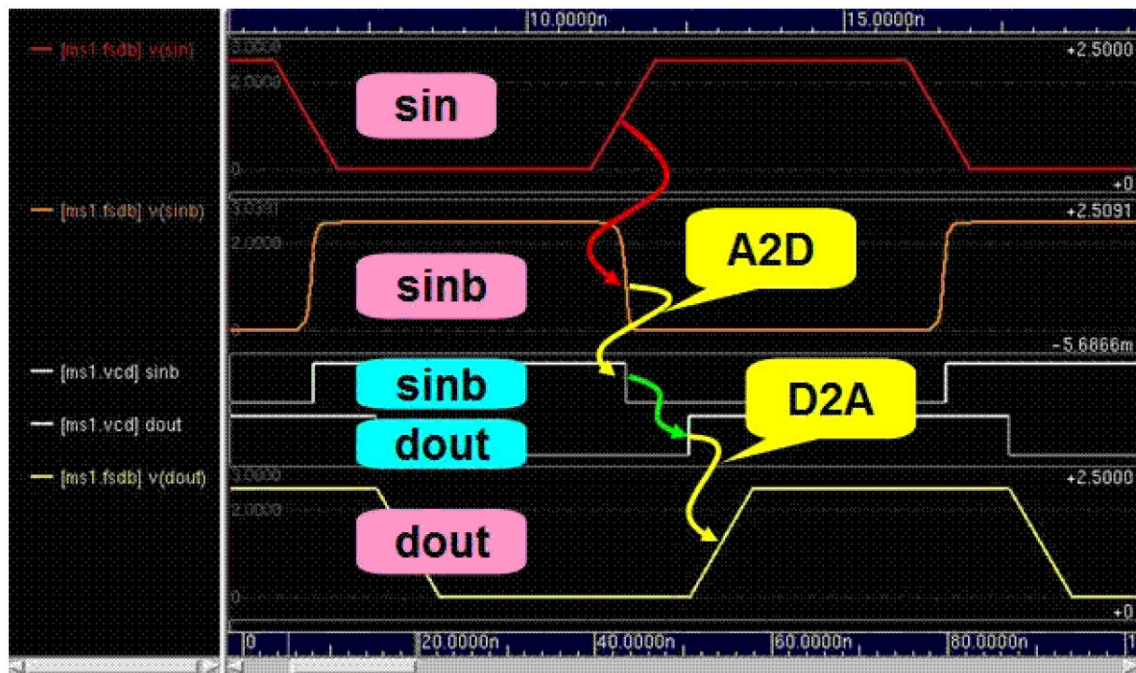
Although in this example the top-level netlist is the SPICE netlist, it should be noted that the Verilog simulator reads the Verilog netlist, as can be seen in the examples below. The tasks in the Verilog netlist configure the PrimeSim Pro tool and start the top-level SPICE simulation.

Run

```
% verilog +access+rw +loadvpi=primesim.so:primesim_startup ms1.v
% ncverilog +access+rw +loadvpi=primesim.so:primesim_startup ms1.v
```

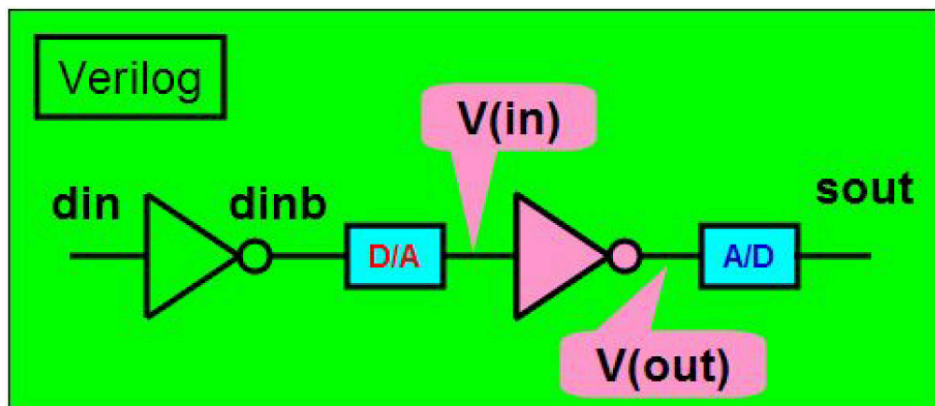
The simulation result waveforms are shown in the following figure.

Figure 78 Lab1 Simulation Results



Verilog Top Example

Figure 79 Block Diagram of Lab2



In the example the main circuit is the Verilog netlist. The Verilog netlist includes both analog and digital subcircuits. The first inverter is in Verilog, the second is in SPICE format. In Verilog, `din` is the input of the first inverter. The output of the first inverter, `dinb` goes to the input of the second inverter, which is an instance of a SPICE subcircuit. The output of the second inverter then goes to `sout`. Because analog and digital signals are mixed, the PrimeSim Pro tool automatically inserts D2A and A2D blocks to convert the signals from digital to analog and back again.

The following example can be found in Lab 2_Verilog_Top.

Example 74 ms2.v

```
// ms2.v
`timescale 1ns/1ps
module top;
  reg din;
  wire dinb, sout;
  vinv I1 (din,dinb);
  sinv I2 (dinb,sout);
  initial begin
    $primesim_config( , ".primesim -o ms2 ms2.sp" );
    $monitor(" %10.3f din=%b dinb=%b sout=%b", $realtime, din, dinb, sout);
    $dumpfile("ms2.vcd");
    $dumpvars(0);
    din=0;
    repeat(10) #10 din= ~din;
    $finish;
  end
endmodule

module vinv (i,o);
  input i;
  output o;
  not #1 (o,i);
endmodule

module sinv (in,out);
  input in;
  output out;
  reg out;
  initial $primesim_module;
endmodule
```

Example 75 ms2.sp

```
* ms2.sp
* Mixed Sim - Verilog Top Example
vdd VDD 0 dc 2.5
vss VSS 0 dc 0
.global VSS VDD
.inc ./model.inc
.inc './primemix.sp' $ Automatically generated SPICE instance netlist

.subckt sinv in out
mp0 out in VDD VDD p l=0.5u w=1u
mn0 out in VSS VSS n l=0.5u w=1u
.ends

.option post
.tran 1p 100ns
.end
```

In Verilog top netlist style, the analog block in the Verilog netlist file is defined as a module wrapper with port declarations and the `$primesim_module` task. The `$primesim_module` task causes the PrimeSim Pro tool to generate a SPICE instance netlist and save it in the file named `primemix.sp`. These analog blocks are defined and instantiated in the SPICE netlist file. The `primemix.sp` should also be included in the SPICE netlist file.

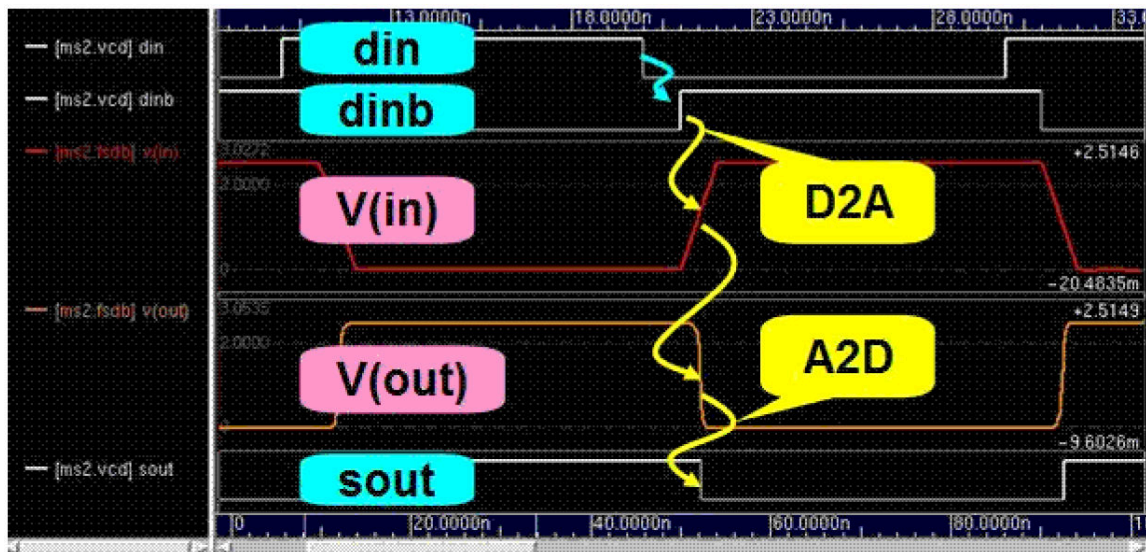
See [PrimeSim Pro Tasks](#) for detailed information about the tasks used in the Verilog cosimulation.

Run

```
% verilog +access+rw +loadvpi=primesim.so:primesim_startup ms2.v
% ncverilog +access+rw +loadvpi=primesim.so:primesim_startup ms2.v
```

Result

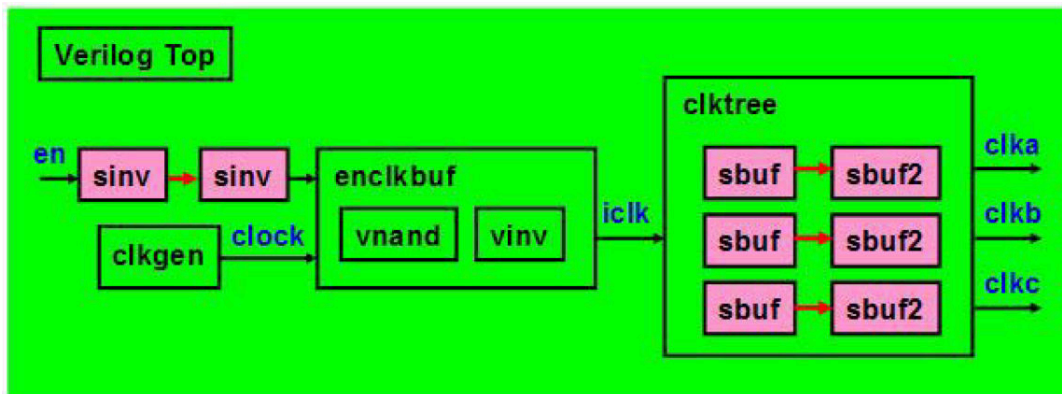
Figure 80 Lab2 Simulation Results



Verilog Top Example With (Gate Level Netlist + TR Level Cell)

This lab can be found in Lab 3_VT_Gate_TR.

Figure 81 Block Diagram of Lab3



This example has a more complicated hierarchy. The top-level netlist is in Verilog, but it contains eight instances of spice subcircuits. The SPICE netlist for these instances is again automatically generated and is found in `primemix_ms3.sp`. Note that all the nets are connected in the Verilog domain, including the ones colored red in the diagram above that connect two analog blocks. Because these signals go through the Verilog domain, which is digital, they are converted from analog to digital and then back again.

Verilog Offset Time in the PrimeSim Cosimulation Flow

The PrimeSim tool supports Verilog offset time (`#xx`) in the cosimulation flow.

For example:

```
initial begin
  #50
  $primesim_config( , ".primesim -o input input.sp", ".option
  dump_ie=1" ); // to run PrimeSim
```

The PrimeSim cosimulation flow disables D2As and A2Ds before the simulation of the specified offset time and then enables them after the specified offset time.

The PrimeSim simulation engine issues the following messages:

```
Message! D2As and A2Ds are not enabled at 0s.
Starting Transient Analysis ...
  48.6ns (48.5 %)
Message! D2As and A2Ds are enabled at 50ns.
```

PrimeSim Pro Tasks

You can control the PrimeSim Pro tool and Verilog cosimulation using tasks in the Verilog netlist, such as `$primesim_config` and `$primesim_instance`. This section describes the tasks supported by the PrimeSim Pro tool.

This section describes the following:

- [\\$primesim_config](#)
- [\\$primesim_input](#)
- [\\$primesim_output](#)
- [\\$primesim_inout](#)
- [\\$primesim_module](#)
- [\\$primesim_instance](#)

\$primesim_config

Description

With this task, a variety of configuration commands can be defined. Configuration commands can directly be specified, and/or configuration file name containing those commands can be specified.

Syntax

```
$primesim_config( ["config_file_name"] [, "config_command"] ... );
```

Arguments

Argument	Description
<i>config_command</i>	The configuration command.
<i>config_file_name</i>	File that contains the configuration commands.

Examples

```
$primesim_config( "test.cfg" );  
$primesim_config( , ".primesim test.sp");  
$primesim_config( , ".option progress=0 accurate=1",  
".a2d vdd25 vl=1.25 vh=1.25",  
".d2a vdd25 vl=0 vh=2.5 vx=1.25 tr=0.3n tf=0.3n",  
".primesim -out test test.sp");
```

In the first example, the configuration file name is specified. In the other example, configuration commands are given directly.

\$primesim_input

Description

With this task, you can connect a Verilog input to a SPICE output through an A2D module. This task has the same meaning as the configuration command `.INPUT`.

Syntax

```
$primesim_input( net_name, "spice_node_name" [, "a2d_model_name" ] );
```

Arguments

Argument	Description
<i>a2d_model_name</i>	Name of model used for A/D conversion.
<i>net_name</i>	Input node name to the Verilog module.
<i>spice_node_name</i>	Output node name from the SPICE module.

Examples

```
$primesim_input( TOP.IN , "XI1.out" , "default" );
```

\$primesim_output

Description

With this task, you can connect a Verilog output to a SPICE input through a D2A module. This task has the same meaning as the configuration command `.OUTPUT`.

Syntax

```
$primesim_output( net_name, "spice_node_name" [, "d2a_model_name" ] );
```

Arguments

Argument	Description
<i>d2a_model_name</i>	Name of model used for D/A conversion.
<i>net_name</i>	Output node name from the Verilog module.
<i>spice_node_name</i>	Input node name to the SPICE module.

Examples

```
$primesim_output( TOP.OUT , "XI1.in" , "default" );
```

\$primesim_inout

Description

With this task, you can connect a Verilog inout port to a SPICE port through D2A and/or A2D modules. This task has the same meaning as the configuration command `.INOUT`.

Syntax

```
$primesim_inout( net_name, "spice_node_name" \
[, "D2A=d2a_model_name" ] [, "A2D=a2d_model_name"]);
```

Arguments

Argument	Description
<i>a2d_model_name</i>	Name of model used for A/D conversion.
<i>d2a_model_name</i>	Name of model used for D/A conversion.
<i>net_name</i>	Bidirectional node name to/from the Verilog module.
<i>spice_node_name</i>	Bidirectional node name from/to the SPICE module.

Examples

```
$primesim_inout( TOP.DATA , "XI1.data" , "D2A=default", "A2D=default" );
```

\$primesim_module

Description

This task enables you to define a module as a SPICE subcircuit. If the subcircuit name is not specified in the argument, the name of the Verilog module in which this task is defined is used. If the subcircuit name is different from the module name, you can specify the name. When this task is included in a Verilog module, the PrimeSim Pro tool automatically generates a SPICE instance netlist which is saved in the file `primemix.sp` by default, and internal A/D and D/A conversion modules. The save file name can be changed by using `.OPTION` command within the `$primesim_config` task.

The generated SPICE file should be included in the SPICE netlist. The model used for the A/D and D/A conversion can be specified by using Verilog statement `defparam` or `parameter` with the keywords of `primesim_a2d` and `primesim_d2a` within the module

which `$primesim_module` task is included. If it is not specified, the `DEFAULT` model is used.

See [\\$primesim_config](#) for details.

Syntax

```
$primesim_module [ ( "spice_subckt_name" ) ] ;
```

Arguments

Argument	Description
<i>spice_subckt_name</i>	SPICE subcircuit name to be used in the generation of the SPICE instance netlist.

Examples

```
$primesim_module;
```

or:

```
$primesim_module("inv");
```

\$primesim_instance

Description

With this task, you can map a Verilog instance to a SPICE instance. The SPICE instance should be an instance of a subcircuit wrapper that just includes the port definitions. This task causes the Verilog instance to be simulated instead of the SPICE instance. Normally it is used when the top-level netlist is a SPICE netlist. The model used for the A/D and D/A conversion can be specified by using Verilog statement `defparam` or `parameter` with the keywords of `primesim_a2d` and `primesim_d2a` within the instance module. If it is not specified, the `DEFAULT` model is used.

See [primesim_a2d / primesim_d2a Parameters](#) for the details.

Syntax

```
$primesim_instance( instance_name, "spice_instance_name" );
```

Arguments

Argument	Description
<i>instance_name</i>	Instance name in the Verilog netlist.
<i>spice_instance_name</i>	Instance name in the SPICE netlist.

Examples

```
$primesim_instance( I1 , "X1" );
```

Configuration Commands

As described in [\\$primesim_config](#), a variety of configuration commands can be given using the task `$primesim_config`. These commands can either be put in a configuration file specified in the `$primesim_config` task or included directly in the `$primesim_config` task. In this section, each of those command statements is described in detail.

This section describes the following:

- [.RESISTANCE](#)
- [.A2D](#)
- [.D2A](#)
- [.SCOPE](#)
- [.INPUT](#)
- [.OUTPUT](#)
- [.INOUT](#)
- [.OPTION](#)
- [.PRIMESIM](#)
- [primesim_a2d / primesim_d2a Parameters](#)

.RESISTANCE

Description

This command is used to define a model of signal strengths. In Verilog, net value is represented by logic and strength. Verilog defines these different signal strengths: supply, strong, pull, large, weak, medium, small, highz.

The `.RESISTANCE` command is used to specify the equivalent resistances that correspond to these signal strengths. A resistance model is part of the specification of an A2D or D2A converter.

Consider D2A, S being the signal strength:

- if (S == supply) apply R7 as resistance of the Thevenin equivalent circuit.
- if (S == strong) apply R6 as resistance of the Thevenin equivalent circuit.

- if (S == pull) apply R5 as resistance of the Thevenin equivalent circuit.
- if (S == large) apply R4 as resistance of the Thevenin equivalent circuit.
- if (S == weak) apply R3 as resistance of the Thevenin equivalent circuit.
- if (S == medium) apply R2 as resistance of the Thevenin equivalent circuit.
- if (S == small) apply R1 as resistance of the Thevenin equivalent circuit.
- if (S == highz) apply R0 as resistance of the Thevenin equivalent circuit.

Consider A2D, R being the driven resistance of the SPICE net:

- if (R <= R7) apply supply strength in the Verilog net.
- else if (R <= R6) apply strong strength in the Verilog net.
- else if (R <= R5) apply pull strength in the Verilog net.
- else if (R <= R4) apply large strength in the Verilog net.
- else if (R <= R3) apply weak strength in the Verilog net.
- else if (R <= R2) apply medium strength in the Verilog net.
- else if (R <= R1) apply small strength in the Verilog net.
- else apply highz strength in the Verilog net.

The default signal strengths for both a2d and d2a are "1 3k 4k 5k 50k 70k 90k 10g".

Syntax

```
.RESISTANCE  res_model_name  R7 R6 R5 R4 R3 R2 R1 R0
```

Arguments

Argument	Description
R7 ... R0	Signal strengths which is corresponding to Strength7 ~ Strength0.
res_model_name	Resistance model name. The default is "DEFAULT".

Examples

```
.resistance default 1 3k 4k 5k 50k 70k 90k 10g
.resistance VDD33R 1 3k 5k 10k 50k 80k 100k 20g
```

The following example illustrates how the `.RESISTANCE`, `.A2D`, and `.D2A` commands are used together:

```
.resistance default 1 3k 4k 5k 50k 70k 90k 10g
.A2D default VL=1.25 VH=1.25 TX=1n R=default
.D2A default VL=0 VH=2.5 VX=1.25 TR=1n TF=1n R=default
```

In the `.A2D` example above:

- If net is 2.5 volts and driven resistance R is 3k ohm in the SPICE, this net value is converted to {strong 1} in Verilog because $(R=3k) \leq (R6=3k)$ and $(V=2.5) \geq (VH=1.25)$.
- If net is 0.5 volts and driven resistance R is 4k ohm in the SPICE, this net value is converted to {pull 0} in Verilog because $(R=4k) \leq (R5=4k)$ and $(V=0.5) \leq (VL=1.25)$.

In the `.D2A` example above:

- Net value {strong, 1} in Verilog is converted to the Thevenin equivalent circuit with V=2.5 volts and R=3k ohm in SPICE.
- Net value {pull, 0} in Verilog is converted to the Thevenin equivalent circuit with V=0 volts and R=4k ohm in SPICE.

.A2D

Description

This command is used to define a model for A/D conversion. Different models can be defined for different input conditions.

Syntax

```
.A2D a2d_model_name [VL=real_value] [VH=real_value] \
[TX=real_value] [R=res_model_name]
```

Arguments

Argument	Description
<i>a2d_model_name</i>	The name of the model.
<i>R=res_model_name</i>	The RESISTANCE model name for signal strengths. The default is DEFAULT.
<i>TX=real_value</i>	Show unknown state ("x") if the signal is between VL and VH for longer than TX. The default is 1ns.
<i>VH=real_value</i>	Logic "1" threshold voltage. The default is 1.25.
<i>VL=real_value</i>	Logic "0" threshold voltage. The default is 1.25.

Examples

```
.A2D default VL=1.25 VH=1.25 TX=1n R=default
.A2D VDD33 VL=1.65 VH=1.65 TX=0n R=VDD33R
```

.D2A

Description

This command is used to define a model for D/A conversion. Different models can be defined for different conditions.

Syntax

```
.D2A d2a_model_name [VL=real_value] [VH=real_value] [VX=real_value] \
[TR=real_value] [TF=real_value] [T0X=real_value] [TX1=real_value] \
[T1X=real_value] [TX0=real_value] [R=res_model_name]
```

Arguments

Argument	Description
d2a_model_name	The name of the model.
R=res_model_name	The RESISTANCE model name which is defined in the RESISTANCE command. The default is DEFAULT.
T0X=real_value	0 to X transition time. The default is rising time (TR).
T1X=real_value	X to 1 transition time. The default is rising time (TR).
TF=real_value	Falling time. The default is 1ns.
TR=real_value	Rising time. The default is 1ns.
TX0=real_value	X to 0 transition time. The default is falling time (TF).
TX1=real_value	X to 1 transition time. The default is falling time (TF).
VH=real_value	Logic "1" voltage. The default is 2.5.
VL=real_value	Logic "0" voltage. The default is 0.
VX=real_value	Logic "x" voltage. The default is 1.25.

Examples

```
.D2A default VL=0 VH=2.5 VX=1.25 TR=1n TF=1n R=default
.D2A VDD33 VL=0 VH=3.3 VX=1.65 TR=0.5n TF=0.5n R=VDD33R
```

.SCOPE

Description

This command is used to define the naming scope for nets. The scope is a hierarchy name separated by a delimiter dot(.). It is prepended to Verilog or SPICE net names to create a full net name.

Syntax

```
.SCOPE [VERILOG=scope_name] [SPICE=scope_name]
```

Arguments

Argument	Description
SPICE= <i>scope_name</i>	Hierarchy name applied for SPICE net name.
VERILOG= <i>scope_name</i>	Hierarchy name applied for Verilog net name.

Examples

```
.SCOPE VERILOG=TOP.I1.I2 SPICE=XI1.XI2
.INPUT in out
```

As with this example, it is treated as follows:

```
".INPUT TOP.I1.I2.in XI1.XI2.out"
```

.INPUT

Description

This command is used to connect a Verilog input to a SPICE output through an A2D module. This command has the same meaning as the \$primesim_input task.

Syntax

```
.INPUT verilog_net_name spice_net_name [A2D=a2d_model_name]
```

Arguments

Argument	Description
A2D= <i>a2d_model_name</i>	A2D name of the model to be used for A/D conversion.
<i>spice_net_name</i>	Input net name to the SPICE module from the Verilog module.
<i>verilog_net_name</i>	Output net name from the Verilog module to the SPICE module.

Examples

```
.INPUT TOP.IN XI1.out A2D=default
```

.OUTPUT

Description

This command is used to connect a Verilog output to a SPICE input through a D2A module. This command has the same meaning as the `$primesim_output` task.

Syntax

```
.OUTPUT verilog_net_name spice_net_name [D2A=d2a_model_name]
```

Arguments

Argument	Description
<code>D2A=d2a_model_name</code>	Name of the model to be used for D/A conversion.
<code>spice_net_name</code>	Output net name from the SPICE module to the Verilog module.
<code>verilog_net_name</code>	Input net name to the Verilog module from the SPICE module.

Examples

```
.OUTPUT TOP.OUT XI1.in D2A=default
```

.INOUT

Description

This command is used to connect a Verilog inout port to a SPICE port through A2D and/or D2A modules. This command has the same meaning as the `$primesim_inout` task.

Syntax

```
.INOUT verilog_net_name spice_net_name [A2D=a2d_model_name]
[D2A=d2a_model_name]
```

Arguments

Argument	Description
<code>A2D=a2d_model_name</code>	Name of the model to be used for A/D conversion.
<code>D2A=d2a_model_name</code>	Name of the model to be used for D/A conversion.

Argument	Description
<i>spice_net_name</i>	Bidirectional node name from/to the SPICE module.
<i>verilog_net_name</i>	Bidirectional node name to/from the Verilog module.

Examples

```
.INOUT TOP.data XI1.data D2A=default A2D=default
```

.OPTION

Description

By using this `.OPTION` command, you can specify various options.

Syntax

```
.OPTION [INST_FILE=file_name] [PROGRESS=0|1] [IMAX=integer_value] \
[IMODE=0|1] [ACCURATE=0|1] [dump_ie=0|1] [minimize_ie=0|1|2]
[bus_format="<%d>"] [port_map_by_name=0|1]
```

Arguments

Argument	Description
<code>INST_FILE=<i>file_name</i></code>	Sets the file name for the SPICE instance netlist. If a Verilog module has a <code>\$primesim_module</code> task, a SPICE instance is automatically generated in this file. The default file name is <code>primemix.sp</code> .
<code>PROGRESS=0 1</code>	Sets the mode for showing the status of the simulation progress. The default value is 1 (on mode).
<code>IMAX=<i>integer_value</i></code>	Sets the maximum iteration count for DC solving in PrimeSim/Verilog cosimulation. The default value is 5.
<code>IMODE=0 1</code>	Sets the interrupt mode for PrimeSim Pro execution. When <code>IMODE=1</code> , a "Ctrl+C" causes the PrimeSim simulation engine to stop running. The default value is 0 (off mode).
<code>ACCURATE=0 1</code>	Sets the re-calculation mode for A/D conversion. A value of 1 results in more accurate timing, but causes the simulation to run more slowly. The default value is 0 (off mode).
<code>dump_ie=0 1</code>	Sets to dump out A2D and D2A information into <code>*.dumpie</code> file. The default value is 0 (off mode).

Argument	Description
<code>minimize_ie=0 1 2</code>	Sets to avoid unnecessary A2D and D2A conversion. If two nodes are connected within analog domain and it is not defined as a register type, then <code>minimize_ie=1</code> removes D2A but keeps A2D for waveform probing; <code>minimize_ie=2</code> removes both A2D and D2A if digital signal of A2D does not drive primitive gate or continuous assignment. This option only applies to ports defined under <code>\$primesim_module</code> task. The default value is 0 (off mode).
<code>bus_format="<%d>"</code>	Specifies bus format, where <code>%d</code> is mandatory, representing the bits. The brackets (" <code><</code> " " <code>></code> ") indicate the bus characters, which can be modified to fit the bus character requirements.
<code>port_map_by_name 0 1</code>	Defines whether the PrimeSim simulation engine uses the port order or port name to map between the Verilog module and SPICE subcircuit. The default is 0, which maps each cosimulation connection based on the order the ports are defined. When set to 1, the PrimeSim simulation engine maps the connection based on the name of the port, so the port order in Verilog versus SPICE can be different, but the name of the port has to be identical.

.PRIMESIM

Description

With this command, users can define the PrimeSim Pro command and arguments. The command arguments and syntax are the same as those for the standalone PrimeSim Pro tool.

Syntax

```
.PRIMESIM primesim_command_arguments
```

Arguments

Argument	Description
<code>primesim_command_arguments</code>	See the "Running the PrimeSim Tool" chapter in the <i>PrimeSim User Guide: Pro and SPICE</i> in for details.

Examples

```
.PRIMESIM -out primesimout input.sp
```

primesim_a2d / primesim_d2a Parameters

Description

The `primesim_a2d` and `primesim_d2a` parameters can be used to change the model to be used for the A/D and D/A conversion. The “DEFAULT” model is used by default. The designation of specific signals is delimited by “\$” character, like “`primesim_d2a$signal`”.

Syntax

```
defparam I2.primesim_a2d      = "vdd_33_1";  
parameter primesim_a2d       = "vdd_33";  
parameter primesim_d2a$A33   = "vdd33";  
parameter \primesim_a2d$DA[0] = "vdd_25";
```

Examples

The first example overrides the A/D conversion model name with “`vdd_33_1`” for instance I2. The second specifies the A/D model name of “`vdd_33`” for all signals used within the module. The third specifies the D/A model name of “`vdd33`” only for a signal A33. The fourth specifies the A/D model name of “`vdd_25`” only for the first signal of bus DA.

Automatic Verilog Instance Generation

The PrimeSim Pro tool provides the support of automatic Verilog instance file generation when SPICE Top structure cosimulation is used. This flow introduces the `-genv` command option and some new options, such as `primesim_verilog_file`, `primesim_verilog_module`, and `primesim_verilog_subckt_file`.

This section describes the flow supported by the PrimeSim Pro tool.

- `-genv`
- `primesim_bus_format`
- `primesim_port_map_by_name`
- `primesim_verilog_file`
- `primesim_verilog_instance`
- `primesim_verilog_module`
- `primesim_verilog_module_file`
- `primesim_verilog_subckt_file`

-genv

Description

If this command option is given, Verilog instance is automatically generated according the subcircuit definition in the deck file.

Syntax

```
primesim -genv[=1|2] deck_file
```

Examples

```
primesim -genv test.sp  
primesim -genv=1 test.sp  
primesim -genv=2 test.sp
```

The first example is the same as the second example, only the Verilog instance file is created. The third example generates both a Verilog instance file and SPICE subcircuit definition file.

primesim_bus_format

Description

Specifies bus format, where %d is mandatory, representing the bits. "<" ">" indicates the bus characters, which can be modified to fit the bus characters.

Syntax

```
primesim_bus_format="<%d>"
```

primesim_port_map_by_name

Description

This option defines whether the PrimeSim tool uses the port order or port name to map between the Verilog module and SPICE subckt. The default is 0, which maps each cosimulation connection is based on the order the ports are defined. When set to 1, the PrimeSim tool maps the connection based on the name of the port, so the port order in Verilog versus SPICE can be different, but the name of the port must be identical.

Syntax

```
primesim_port_map_by_name=1
```

primesim_verilog_file

Description

Sets the file name for Verilog instance file. Default file name is "primemix.v".

Syntax

```
.option primesim_verilog_file=file_name
```

Examples

```
.option primesim_verilog_file="inv.v"
```

primesim_verilog_instance

Description

Specifies a SPICE instance to be replaced by a Verilog module. Note that this option can only work with the command option `-genv`. If the subckt name is the same as the module name, the module name can be omitted.

Syntax

```
.option primesim_verilog_instance="instance_name:module_name ..."
```

Examples

```
.option primesim_verilog_instance="x1.x2.x3:inv x1.x2.x4:buf"
```

In this example, SPICE instances `x1.x2.x3` and `x1.x2.x4` are replaced by the Verilog module `inv` and `buf`, respectively.

primesim_verilog_module

Description

Specifies a SPICE subcircuit to be replaced by a Verilog module. If the subcircuit name is that same as the module name, the module name can be omitted. This option can be specified multiple times.

Syntax

```
.option primesim_verilog_module="subckt_name:module_name ... "
```

Examples

```
.option primesim_verilog_module="inv buf:BUF nand2:Nand2"
```

The example means the cosimulation uses the `inv`, `BUF`, and `Nand2` Verilog modules on the digital side, not the `inv`, `buf`, and `nand2` subcircuits in the analog netlist.

primesim_verilog_module_file

Description

This option specifies the Verilog file containing the Verilog module definitions. This option is used when the Verilog instance file is generated by `primesim -genv`.

Syntax

```
primesim_verilog_module_file="filename"
```

primesim_verilog_subckt_file

Description

Sets the file name for a new SPICE subcircuit definition file. If command option `-genv=2` is given, a new SPICE subcircuit definition file is generated. This subcircuit is empty because this subcircuit is replaced by a Verilog module. Default file name is `"primemix_subckt.sp"`.

Syntax

```
.option primesim_verilog_subckt_file=file_name
```

Examples

```
.option primesim_verilog_subckt_file="pll.sp"
```

Circuit Example: (test.sp)

```
.option post
.global 0
vvd VDD 0 dc 2.5
vss VSS 0 dc 0
.global VSS VDD
vin sin 0 pulse (0 2.5 1.0n 1.0n 1.0n 4.0n 10n)
.inc ./model.inc

** Top Netlist **
X1 sin sinb sinv
X2 sinb dout vinv

.subckt sinv in out
mp0 out in VDD VDD p l=0.25u w=3u
mn0 out in VSS VSS n l=0.25u w=1.5u
.ends
```

```
.subckt vinv in out
mp0 out in VDD VDD p l=0.25u w=3u
mn0 out in VSS VSS n l=0.25u w=1.5u
.ends

.tran 1p 100ns
.end
```

In this example, suppose you want to replace the `vinv` subcircuit with the `vinv` module on the Verilog side. First, add the `.option primesim_verilog_module="vinv"` to the `test.sp` deck file, and then execute command `primesim -genv=2 test.sp` in the terminal. Then the `primemix.v` and `primemix_subckt.sp` files are automatically generated. Third, include `primemix.v` in the Verilog file and include `primemix_subckt.sp` in the `test.sp` deck file separately. Now, the automatic Verilog instance generation flow is done and you can continue to run the cosimulation.

Parallel Cosimulation

The PrimeSim Pro tool has applied its parallel simulation technology to cosimulation. It solves the performance bottleneck of slower transistor level simulation in a cosimulation environment. To use this feature, modify the `primesim` command to run the parallel PrimeSim Pro tool with the `-np` switch, as shown in the following example:

```
//You can change a 1CPU serial run of
.primesim -spice -o 1CPU input.spi
//to an 8CPU run of
.primesim -np 8 -spice -o 8CPU input.spi
```

Currently, the maximum number of `-np` in parallel cosimulation is 8.

Common Cosimulation Problems

Because every Verilog simulator behaves a bit differently with VPI, sometimes unexpected problems are encountered. This section goes through some of these common cosimulation problems and the workarounds for fixing them.

Error while reading shared library symbols, cannot find new threads: generic error

You can workaround the problem by setting:

```
setenv LD_ASSUME_KERNEL"
```

ERROR: VPI NOFORCB: vpi_put_value() cannot force a bit of an unexpanded vector net

Some of the Verilog simulation does not expand out the output bus, so you cannot use `.input/.output/.inout` to drive the signal.

For the output [MSB:LSB] name that is causing the issue, add "wire scaled [MSB:LSB] name". Alternatively, you can set the signal as "output reg".

I

Reserved Keywords

This appendix describes the reserved keywords. When using the Verilog-SPICE, VHDL/Verilog-SPICE or Verilog-AMS-SPICE flows, or Verilog-AMS-SPICE, these terms are treated as keywords in the Verilog modules. If any of the following keywords are used as net names, port names, instance names or module names in the Verilog (D) modules, you must rename these keywords to avoid compilation errors.

This appendix describes:

- [Reserved Keywords for Verilog-SPICE and VHDL/Verilog-SPICE](#)
- [Reserved Keywords for Verilog-AMS-SPICE](#)

Reserved Keywords for Verilog-SPICE and VHDL/Verilog-SPICE

[Table 23](#) for an alphabetical listing of the reserved keywords for NanoSim VCS:

Table 23 *Reserved Keywords for Verilog-SPICE and VHDL/Verilog-SPICE*

abs	absdelay	abstol	access
acos	acosh	ac_stim	always
analog	analysis	and	asin
asinh	assign	atan	atan2
atanh	begin	branch	buf
bufif0	bufif1	capacitor	case
casex	casez	ceil	cmos
connect	connectrules	continuous	cos
cosh	cross	ddt	ddt_nature
deassign	default	defparam	disable

Appendix I: Reserved Keywords
Reserved Keywords for Verilog-SPICE and VHDL/Verilog-SPICE

Table 23 *Reserved Keywords for Verilog-SPICE and VHDL/Verilog-SPICE (Continued)*

discipline	discrete	domain	driver_update
edge	else	end	enddiscipline
endcase	endconnectrules	endmodule	endfunction
endnature	endprimitive	endspecify	endtable
endtask	event	exclude	exp
final_step	flicker_noise	floor	flow
for	force	forever	fork
from	function	generate	genvar
ground	highz0	highz1	hypot
inductor	idt	idtmod	idt_nature
iexp	ipulse	ipwl	isine
if	ifnone	inf	initial
intial_step	inout	input	integer
join	laplace_nd	laplace_np	laplace_zd
laplace_zp	large	last_crossing	limexp
ln	log	macromodule	max
medium	merged	min	module
nand	nature	negedge	net_resolution
nmos	noise_table	nor	not
notif0	notif1	or	output
parameter	pmos	posedge	potential
pow	primitive	pull0	pull1
pullup	pulldown	rcmos	real
realtime	reg	release	repeat
resolveto	resistor	rnmos	rpmos
rtran	rtranif0	rtranif1	scalared

Appendix I: Reserved Keywords
Reserved Keywords for Verilog-SPICE and VHDL/Verilog-SPICE

Table 23 *Reserved Keywords for Verilog-SPICE and VHDL/Verilog-SPICE (Continued)*

sin	sinh	slew	small
specify	specparam	split	sqrt
strong0	strong1	supply0	supply1
table	tan	tanh	task
time	timer	tline	tran
tranif0	tranif1	transition	tri
tri0	tri1	triand	trior
trireg	units	vcvs	vccs
vectored	vexp	vpulse	vpwl
vsine	wait	wand	weak0
weak1	while	white_noise	wire
wor	wreal	xnor	xor
zi_nd	zi_np	zi_zd	zi_zp

Reserved Keywords for Verilog-AMS-SPICE

See [Table 24](#) for an alphabetical listing of the reserved keywords for Verilog-AMS-SPICE:

Table 24 *Reserved Keywords for Verilog-AMS-SPICE*

above	abs	absdelay	acos
acosh	ac_stim	aliasparam	always
analog	analysis	and	asin
asinh	assign	atan	atan2
atanh	begin	branch	buf
bufif0	bufif1	case	casex
casez	ceil	cmos	connectrules
cos	cosh	cross	ddt
ddx	deassign	default	defparam
disable	discipline	driver_update	edge
else	end	enddiscipline	endcase
endconnectrules	endmodule	endfunction	endnature
endparamset	endprimitive	endspecify	endtable
endtask	event	exclude	exp
final_step	flicker_noise	floor	flow
for	force	forever	fork
from	function	generate	genvar
ground	highz0	highz1	hypot
idt	idtmod	if	ifnone
inf	initial	intial_step	inout
input	integer	join	laplace_nd
laplace_np	laplace_zd	laplace_zp	large
last_crossing	limexp	In	localparam

Table 24 *Reserved Keywords for Verilog-AMS-SPICE (Continued)*

log	macromodule	max	medium
min	module	nand	nature
negedge	net_resolution	nmos	noise_table
nor	not	notif0	notif1
or	output	parameter	paramset
pmos	posedge	potential	pow
primitive	pull0	pull1	pullup
pulldown	rcmos	real	realtime
reg	release	repeat	rnmos
rpmos	rtran	rtranif0	rtranif1
scalared	sin	sinh	slew
small	specify	specparam	sqrt
string	strong0	strong1	supply0
supply1	table	tan	tanh
task	time	timer	tran
tranif0	tranif1	transition	tri
tri0	tri1	triand	trior
trireg	vectored	wait	wand
weak0	weak1	while	white_noise
wire	wor	wreal	xnor
xor	zi_nd	zi_np	zi_zd

Glossary

A2A through-net

A net that is used only for port connections between two SPICE subcircuits in a Verilog view.

A2D

An analog-to-digital interface element.

BA (back-annotation)

Back-annotation (BA) is a process of stitching the parasitic RCs back to the design netlist through connectivity information (net name, instance name, pin name) inside the parasitic file.

bidirectional switch

A device that conducts in both directions. In such cases, signals on either side of the device can be the driver signal. A *bidirectional switch* is typically used to enable isolation between buses or signals.

D2A

A digital-to-analog interface element.

D2D through-net

A net that is only used for port connections between two Verilog modules in a SPICE view.

donut configuration

A description of the design using different views across different levels of hierarchy. For example: Verilog-SPICE-Verilog or SPICE-VHDL-SPICE are considered *donut configurations*.

DSPF

A detailed standard parasitic format (DSPF) output netlist format is generated by an extraction tool, and describes interconnect information. Actual net parasitic resistance and capacitance component information is contained in this format.

GUI

A graphical user interface (GUI).

HAR

Hierarchical array reduction (HAR) that speeds-up the simulation for memory designs (DRAM and SRAM).

instantiation

The process of creating an instance from a module definition or simulator primitive, and defining the connectivity and parameters of that instance.

mixed-net

A net that connects the discrete domain (digital) to the continuous domain (analog). All nodes that exist at the boundary between VCS and PrimeSim XA are considered *mixed-nets*.

mixed-signal

A circuit containing analog- and digital-style components.

multiple view

In a given design, at a particular hierarchy, if more than one representation is available for simulation (from the choices of Verilog, SPICE, and Verilog-A), it is considered a *multiple view*.

PLI

A programming language interface (PLI) of Verilog HDL is a mechanism for interfacing Verilog programs with programs written in the C language. PLI also provides a mechanism for accessing internal databases of the simulator from the C program.

real data type

The Verilog or VHDL data type defined in IEEE Std 1264-1996 and Std 1364-2001.

resistance map file

An ASCII file that equates MOSFET "on" resistance to Verilog drive strength; the resistance map file contains the signal conversion data between a SPICE analog value to a Verilog digital value, and a Verilog digital value to a SPICE analog value.

SDF

A standard delay format (SDF) file stores the timing data generated by EDA tools for use in any stage of a design process. The data in the SDF file is represented in a tool-independent way and includes the following information: delay, timing check, timing constraint, incremental and absolute delay.

simv

A Verilog simulator command.

single view

In a given design, at a particular hierarchy, if there is only one view available for simulation (from the choices of Verilog, SPICE, and Verilog-A), it is considered a *single view*. A single view is automatically selected for simulation as it is the only view available.

SPEF

A standard parasitic extraction format (SPEF) file is an IEEE standard format. This file provides a standard median to pass parasitic information between EDA tools during any stage in the design process. This format contains actual net parasitic resistance and capacitance components.

SPICE netlist

In the present context, the term *SPICE netlist* is used in place of *transistor-level netlist*.

SPICE-top

The top level of the design hierarchy is described in a transistor-level netlist format.

SPICE view

In a given design, at a particular hierarchy, if a *SPICE* module is available and is used to simulate a particular block, it is considered a *SPICE view* for that block.

VCS

A Synopsys simulator for Verilog, VHDL, and mixed-HDL design descriptions.

Verilog dummy module

A module that is the Verilog *place holder* for a transistor block. A dummy module is an empty module containing only the module declaration and port declarations.

Verilog-top

The top level of the design hierarchy is described in Verilog RTL or gate-level netlist format.

Verilog view

In a given design, at a particular hierarchy, if a Verilog module is available and is used to simulate a particular block, it is considered a *Verilog view* for that block.

Verilog wrapper

A Verilog netlist comprising an empty module. Only the module name and port description are in the wrapper.

VHDL

VHSIC HDL

vhdlan

A VHDL analyzer command.

vlogan

A Verilog analyzer command.

VPD

An output format for VCS. VPD uses the VCD+ (value change dump) format.

wreal data type

A Verilog-AMS wire of type "real" which allows modules to exchange "real" values through ports. Also, a real net data type used in a Verilog wrapper module in the VHDL/Verilog-SPICE flow to interface a real VHDL port to a top-level SPICE net or connect a SPICE port to a top-level VHDL real net.

XMR

A feature that is extensively used in Verilog testbenches, and is referred to as a *cross-module reference* or Verilog hierarchical referencing. This feature enables simple probing into, or monitoring of, buried signals without requiring the signals to be routed to the top of the design for observation. No declaration of global signals in a package is required for this feature, nor is any modification of the original monitored code.